

LLM101n-BR

Vinicius Brasil Sanchez

CONSTRUINDO UM MODELO DE LINGUAGEM DO ZERO

Do bigrama ao Transformer multimodal, com medicao de cada afirmacao

Santo André

2026

Vinicius Brasil Sanchez

CONSTRUINDO UM MODELO DE LINGUAGEM DO ZERO

Do bigrama ao Transformer multimodal, com medicao de cada afirmacao

Material didatico em dezessete capitulos, com programas, exercicios e gabaritos, sobre a construcao integral de um modelo de linguagem autorregressivo sem o uso de bibliotecas de alto nivel. Organizado segundo o programa do curso LLM101n, de Andrej Karpathy e Eureka Labs.

Santo André
2026

RESUMO

Esta apostila documenta a construção completa de um modelo de linguagem do tipo *Transformer*, escrito integralmente do zero em Python, sem o uso de bibliotecas de alto nível para as partes conceituais. O percurso vai da contagem de pares de caracteres em um modelo bigrama até um modelo multimodal capaz de gerar imagens, passando por diferenciação automática, mecanismo de atenção, tokenização por *Byte Pair Encoding*, otimização, treinamento em precisão reduzida, paralelismo de dados, curadoria de *corpus*, cache de inferência, quantização, ajuste fino supervisionado, aprendizado por reforço e servimento em produção.

O texto adota uma restrição metodológica explícita: **nenhuma afirmação quantitativa é publicada sem medição**. Toda tabela de resultados provém da execução dos programas que acompanham o material, e os casos em que a medição contrariou a hipótese inicial do autor foram mantidos no texto, documentados como tal. Essa escolha responde a um objetivo pedagógico que se considera tão importante quanto o conteúdo técnico: demonstrar que uma explicação plausível e uma explicação correta são coisas distintas, e que apenas o experimento as separa.

A organização segue o programa do curso **LLM101n**, de Andrej Karpathy e da Eureka Labs, distribuído em dezessete capítulos. A exposição é feita em português; a terminologia técnica e os comentários de código permanecem em inglês, por serem a forma em que o leitor os encontrará na literatura e nas bibliotecas.

Palavras-chave: Modelos de linguagem. Redes neurais. *Transformer*. Processamento de linguagem natural. Aprendizado profundo. Metodologia experimental.

SUMARIO

APRESENTAÇÃO	17
A quem se destina	17
Como o material está organizado	17
Sobre os números apresentados	17
Reprodutibilidade	18
1 BIGRAM LANGUAGE MODEL	19
1.1 O que é um modelo de linguagem?	19
1.2 O modelo mais simples: o bigrama	20
1.3 Construindo o vocabulário	21
1.4 Aprendendo por contagem	21
1.5 Gerando nomes (sampling)	22
1.6 O modelo é bom? (a loss)	23
1.7 A virada de chave: o mesmo modelo como rede neural	24
1.8 Como a rede aprende: gradient descent	25
1.9 Regularização = smoothing, de novo	26
1.10 Resumo do capítulo	26
1.11 Exercícios	27
2 MICROGRAD (AUTOGRAD E BACKPROPAGATION)	29
2.1 O problema: como a rede sabe para que lado ir?	29
2.2 A regra da cadeia (chain rule), a chave de tudo	30
2.3 O grafo de computação	30
2.4 A classe Value	31
2.5 Cada operação conhece sua derivada local	31
2.6 A backward(): percorrer o grafo de trás para frente	32
2.7 De um número a uma rede neural	33

2.8 Treinando: o mesmo loop do Capítulo 1, agora "nosso"	34
2.9 "Então o PyTorch é isto?" — Sim.	34
2.10 Resumo do capítulo	35
2.11 Exercícios	35
3 N-GRAM MODEL (MLP EM PYTORCH)	38
3.1 O problema do bigrama: memória de um caractere só	38
3.2 Um dataset de verdade	39
3.3 Construindo o dataset de contexto -> alvo	39
3.4 A peça nova: embeddings	40
3.5 A arquitetura do MLP	40
3.6 A loss: cross_entropy	41
3.7 Treino com mini-batches	41
3.8 O conceito mais importante: treino / validação / teste	42
3.9 Rodando e os resultados	42
3.10 Resumo do capítulo	43
3.11 Exercícios	44
4 ATTENTION (O MECANISMO DE ATENÇÃO)	46
4.1 O que falta no MLP	46
4.2 Preparando o terreno: a forma dos dados	46
4.3 v1 — a média na mão (bag of words)	47
4.4 v2 — a mesma média, com uma multiplicação de matrizes	47
4.5 v3 — a mesma média, via softmax	48
4.6 v4 — self-attention de verdade: query, key e value	49
4.7 Dois detalhes que fazem o mecanismo funcionar	50
4.8 Atenção não sabe ordem: o embedding posicional	50
4.9 O resultado — e a descoberta honesta deste capítulo	51

4.10 Resumo do capítulo	52
4.11 Exercícios	53
5 TRANSFORMER	56
5.1 De onde partimos	56
5.2 Multi-head attention: várias relações em paralelo	56
5.3 Conexões residuais: um caminho livre para o gradiente	57
5.4 Layer normalization	58
5.5 O bloco: comunicação + computação	59
5.6 O modelo completo	60
5.7 Resultados	60
5.8 Resumo do capítulo	62
5.9 Exercícios	62
6 TOKENIZATION (BPE DO ZERO)	65
6.1 O problema	65
6.2 Antes do BPE: como texto vira número	66
6.3 O algoritmo BPE	67
6.4 encode e a importância da ordem	68
6.5 Rodando: o que o tokenizador aprendeu	68
6.6 Compressão só existe dentro do domínio	70
6.7 O imposto do português, medido	70
6.8 Resumo do capítulo	71
6.9 Exercícios	72
7 OPTIMIZATION	75
7.1 O que ficou pendente	75
7.2 Inicialização: o sinal precisa atravessar a rede	75
7.3 Da SGD ao AdamW	78

7.4 Agendamento da learning rate: warmup + cosine	79
7.5 Gradient clipping	80
7.6 Resultados: uma ablação	81
7.7 Resumo do capítulo	83
7.8 Exercícios	84
8 DEVICE (CPU E GPU)	88
8.1 Por que uma GPU ajuda	88
8.2 Onde está o ponto de virada	89
8.3 Transferir dados custa	90
8.4 O tamanho do batch importa mais do que parece	90
8.5 Código portátil: nunca crave .cuda()	91
8.6 Como medir GPU sem se enganar	91
8.7 E o nosso modelo, ganha algo?	93
8.8 Resumo do capítulo	95
8.9 Setup Gpu	96
8.10 Exercícios	98
9 PRECISION (TREINAR COM MENOS BITS)	101
9.1 Por que mexer nos bits	101
9.2 Anatomia de um float	101
9.3 Alcance e precisão, medidos	102
9.4 Onde 16 bits quebram	103
9.5 Por que o bf16 ganhou	103
9.6 O que medimos de verdade — e o que não se cumpriu	104
9.7 Loss scaling: o truque que salva o fp16	105
9.8 Precisão mista e os pesos mestres	107
9.9 Como se escreve na prática	108

9.10 Os limites deste capítulo, declarados	108
9.11 Resumo do capítulo	109
9.12 Exercícios	109
10 DISTRIBUTED (TREINAR EM VÁRIOS DISPOSITIVOS)	112
10.1 A ideia: paralelismo de dados	112
10.2 All-reduce: todos contribuem, todos recebem	112
10.3 Como o all-reduce funciona por dentro: o anel	113
10.4 Quanto custa comunicar	114
10.5 DDP: o gradiente é o mesmo do batch inteiro?	115
10.6 O cuidado que mais causa problema: batch efetivo	115
10.7 ZeRO: o desperdício que o DDP não resolve	116
10.8 Um problema real de infraestrutura, e a lição	117
10.9 Resumo do capítulo	118
10.10 Exercícios	119
11 DATASETS (A VIRADA PARA TEXTO DE VERDADE)	122
11.1 Por que os dados são o capítulo mais subestimado	122
11.2 O corpus: Machado de Assis	122
11.3 A divisão: onde quase todo mundo erra	123
11.4 Tokenização: o BPE aprende o domínio	125
11.5 Gravar em binário: uint16 e mmap	126
11.6 A mudança no formato do treino	126
11.7 Resultados: o modelo escreve prosa	127
11.8 Resumo do capítulo	129
11.9 Exercícios	129
12 INFERENCE I: KV-CACHE	133
12.1 Um modelo que faz a mesma conta 128 vezes	133

12.2 Por que dá para não repetir	134
12.3 A implementação: duas linhas	135
12.4 Armadilha 1 — a posição é absoluta	135
12.5 Armadilha 2 — a máscara	136
12.6 O teste que define se está certo	136
12.7 Prefill e decode: duas fases, perfis opostos	137
12.8 O preço: memória	137
12.9 O limite do contexto fixo	138
12.10 Resumo do capítulo	139
12.11 Exercícios	139
13 INFERENCE II: QUANTIZATION	142
13.1 Por que encolher os pesos	142
13.2 A aritmética, que é mais simples do que parece	143
13.3 Simétrica ou assimétrica — e uma armadilha que me pegou	143
13.4 Per-tensor vs per-channel, e uma métrica que mente	144
13.5 Quantos bits o modelo aguenta?	145
13.6 Quanto o modelo encolhe	146
13.7 E a velocidade? Piorou.	146
13.8 O que a prática faz além disto	147
13.9 Resumo do capítulo	148
13.10 Exercícios	149
14 FINETUNING I: SFT	152
14.1 O que falta a um modelo-base	152
14.2 Tokens que não são texto	153
14.3 Aumentar o vocabulário de um modelo já treinado	153
14.4 A máscara de loss	153

14.5 O resultado: o modelo aprende a parar	154
14.6 O erro que eu cometi montando os dados	155
14.7 A máscara importa? Menos do que dizem — e dá para saber quando	155
14.8 O que o SFT é, e o que não é	157
14.9 Resumo do capítulo	157
14.10 Exercícios	158
15 FINETUNING II: RL	161
15.1 Quando não há resposta certa para imitar	161
15.2 De onde vem a recompensa neste capítulo	161
15.3 O algoritmo cabe numa linha	162
15.4 A penalidade de KL	162
15.5 Duas recompensas, e uma delas é ruim de propósito	163
15.6 O experimento	163
15.7 O freio de KL não é um interruptor	164
15.8 Uma observação sobre orçamento, que quase me custou o capítulo	166
15.9 E o DPO?	166
15.10 Resumo do capítulo	167
15.11 Exercícios	167
16 DEPLOYMENT	170
16.1 Um modelo treinado não é um serviço	170
16.2 Streaming: a otimização que não acelera nada	170
16.3 O que acontece quando vários clientes chegam juntos	171
16.4 Batching: as requisições viajam juntas	172
16.5 O resultado	173
16.6 Um bug meu, e por que ele vale estar aqui	174
16.7 O que este capítulo não faz	174

16.8 Resumo do capítulo	175
16.9 Exercícios	175
17 MULTIMODAL	178
17.1 Por que um Transformer não come imagens	178
17.2 A ideia: aprender um alfabeto de pedaços de imagem	179
17.3 O truque que faz isso treinar	179
17.4 O resultado do VQ-VAE	180
17.5 A linha que é o capítulo inteiro	181
17.6 O modelo gerando imagens	181
17.7 O que a prática faz diferente	182
17.8 Resumo do capítulo	183
17.9 Exercícios	184
REFERÊNCIAS	187
APENDICE A — GABARITOS COMENTADOS	191
A.1 GABARITO DO CAPÍTULO 1 — BIGRAM LANGUAGE MODEL	192
A.1.1 E1 — Leitura de código	192
A.1.2 E2 — Mexa nos dados	192
A.1.3 E3 — O efeito da suavização	193
A.1.4 E4 — Temperatura no sampling	193
A.1.5 E5 — Igualando contagem e rede neural	194
A.1.6 E6 — NLL de um nome específico	194
A.1.7 E7 — Rumo ao trigrama	195
A.2 GABARITO DO CAPÍTULO 2 — MICROGRAD (AUTOGRAD E BACKPROPAGATION)	196
A.2.1 E1 — Leitura de código	196
A.2.2 E2 — Conferindo com diferença finita	196
A.2.3 E3 — Implementando log()	197

A.2.4 E4 — Por que zerar o gradiente	197
A.2.5 E5 — Bate com o PyTorch?	198
A.2.6 E6 — Arquitetura e learning rate	198
A.2.7 E7 — Visualizando o grafo	199
A.3 GABARITO DO CAPITULO 3 — N-GRAM MODEL (MLP EM PYTORCH)	200
A.3.1 E1 — Leitura de código	200
A.3.2 E2 — O efeito do contexto	200
A.3.3 E3 — Tamanho da camada oculta	201
A.3.4 E4 — Learning rate	202
A.3.5 E5 — Overfitting com dataset pequeno	202
A.3.6 E6 — Visualizando os embeddings	203
A.3.7 E7 — Contando os parâmetros	204
A.4 GABARITO DO CAPITULO 4 — ATTENTION (O MECANISMO DE ATENÇÃO)	205
A.4.1 E1 — Leitura de código	205
A.4.2 E2 — Sem a máscara causal	205
A.4.3 E3 — Sem o embedding posicional	206
A.4.4 E4 — Sem a escala $1/\sqrt{\text{head_size}}$	207
A.4.5 E5 — Tamanho do contexto	207
A.4.6 E6 — Inspeccionando os pesos	208
A.4.7 E7 — Multi-head attention	208
A.5 GABARITO DO CAPITULO 5 — TRANSFORMER	209
A.5.1 E1 — Leitura de código	209
A.5.2 E2 — Sem as conexões residuais	209
A.5.3 E3 — Sem LayerNorm	210
A.5.4 E4 — Profundidade e número de cabeças	211

A.5.5 E5 — Uma cabeça grande vs várias pequenas	212
A.5.6 E6 — Dropout	212
A.5.7 E7 — Contando os parâmetros	213
A.6 GABARITO DO CAPITULO 6 — TOKENIZATION (BPE DO ZERO)	215
A.6.1 E1 — Unicode e UTF-8	215
A.6.2 E2 — Tamanho do vocabulário	215
A.6.3 E3 — As primeiras fusões	216
A.6.4 E4 — A ordem das fusões importa	216
A.6.5 E5 — O imposto do português	217
A.6.6 E6 — Tokens fora do domínio	218
A.6.7 E7 — Integrando com o modelo	218
A.7 GABARITO DO CAPITULO 7 — OPTIMIZATION	220
A.7.1 E1 — Leitura de código	220
A.7.2 E2 — O warmup é necessário?	220
A.7.3 E3 — Calibrando o gradient clipping	221
A.7.4 E4 — O ganho certo para a GELU	222
A.7.5 E5 — A curva da learning rate	223
A.7.6 E6 — AdamW vs SGD no modelo de verdade	223
A.7.7 E7 — Por que o weight decay atrapalhou	224
A.7.8 E8 — Outros agendamentos	224
A.8 GABARITO DO CAPITULO 8 — DEVICE (CPU E GPU)	226
A.8.1 E1 — Latência vs vazão	226
A.8.2 E2 — Encontre o seu ponto de virada	226
A.8.3 E3 — A armadilha do .item()	227
A.8.4 E4 — Tamanho do batch	228
A.8.5 E5 — Meça errado de propósito	229

A.8.6 E6 — Operações que caem de volta na CPU	229
A.8.7 E7 — Quando vale a pena mudar para GPU?	230
A.9 GABARITO DO CAPITULO 9 — PRECISION (TREINAR COM MENOS BITS)	231
A.9.1 E1 — Contas com bits	231
A.9.2 E2 — Achando os limites na mão	231
A.9.3 E3 — O epsilon na prática	232
A.9.4 E4 — A janela do loss scaling	233
A.9.5 E5 — Melhorando o GradScaler	233
A.9.6 E6 — Seu hardware acelera 16 bits? (precisa de GPU)	234
A.9.7 E7 — Treino em precisão mista	234
A.10 GABARITO DO CAPITULO 10 — DISTRIBUTED (TREINAR EM VÁRIOS DISPOSITIVOS)	236
A.10.1 E1 — Entendendo o coletivo	236
A.10.2 E2 — Escalando o número de processos	237
A.10.3 E3 — Provoque um deadlock	238
A.10.4 E4 — A loss que engana	239
A.10.5 E5 — Batch efetivo e learning rate	240
A.10.6 E6 — Meça o ZeRO	240
A.10.7 E7 — Quando distribuir não compensa	241
A.11 GABARITO DO CAPITULO 11 — DATASETS (A VIRADA PARA TEXTO DE VERDADE)	243
A.11.1 Sobre o orçamento de treino — leia antes dos números	243
A.11.2 E1 — Por que essa ordem?	244
A.11.3 E2 — A divisão errada	245
A.11.4 E3 — O tokenizador é do domínio	246
A.11.5 E4 — Tamanho do contexto	246

A.11.6 E5 — Prever em todas as posições	249
A.11.7 E6 — Dados sintéticos e model collapse	250
A.11.8 E7 — Escalando o corpus	252
A.12 GABARITO DO CAPITULO 12 — INFERENCE I: KV-CACHE	255
A.12.1 E1 — O que dá e o que não dá para cachear	255
A.12.2 E2 — Quebrando a posição de propósito	256
A.12.3 E3 — Quebrando a máscara de propósito	256
A.12.4 E4 — Meça o ganho	258
A.12.5 E5 — A conta de memória	259
A.12.6 E6 — Cache com batch	260
A.12.7 E7 — Multi-query attention (desafio)	261
A.13 GABARITO DO CAPITULO 13 — INFERENCE II: QUANTIZATION	262
A.13.1 E1 — A aritmética	262
A.13.2 E2 — A armadilha do zero-point	262
A.13.3 E3 — A métrica que mente	263
A.13.4 E4 — Onde fica o precipício	264
A.13.5 E5 — Quantizando os embeddings também	265
A.13.6 E6 — O que a quantização faz com a velocidade	266
A.13.7 E7 — Quantização por grupos	266
A.14 GABARITO DO CAPITULO 14 — FINETUNING I: SFT	268
A.14.1 E1 — Tokens especiais e vocabulário	268
A.14.2 E2 — Quando a máscara importa	269
A.14.3 E3 — Dados de tamanho fixo	270
A.14.4 E4 — A learning rate do finetuning	271
A.14.5 E5 — Quantos exemplos bastam?	272
A.14.6 E6 — O alignment tax	273

A.14.7 E7 — Conversa com vários turnos	273
A.15 GABARITO DO CAPITULO 15 — FINETUNING II: RL	275
A.15.1 E1 — SFT e RL, lado a lado	275
A.15.2 E2 — Escreva uma recompensa ruim	276
A.15.3 E3 — O baseline importa mesmo?	276
A.15.4 E4 — Onde fica a faixa útil do β	277
A.15.5 E5 — A recompensa não é o objetivo	278
A.15.6 E6 — Conserte a recompensa	280
A.15.7 E7 — DPO (desafio)	280
A.16 GABARITO DO CAPITULO 16 — DEPLOYMENT	282
A.16.1 E1 — TTFT e latência	282
A.16.2 E2 — Meça o seu próprio TTFT	283
A.16.3 E3 — Por que threads não bastam	284
A.16.4 E4 — O tamanho do lote	285
A.16.5 E5 — A espera que custou caro	285
A.16.6 E6 — O p95 conta o que a média esconde	286
A.16.7 E7 — Backpressure (desafio)	287
A.17 GABARITO DO CAPITULO 17 — MULTIMODAL	289
A.17.1 E1 — Por que quantizar	289
A.17.2 E2 — O straight-through estimator	290
A.17.3 E3 — As duas perdas	291
A.17.4 E4 — Códigos mortos	292
A.17.5 E5 — O mesmo Transformer	292
A.17.6 E6 — Condicionar pelo dígito	294
A.17.7 E7 — Um modelo, duas modalidades (desafio)	294
APENDICE B — FONTES PRIMARIAS POR CAPITULO	296

B.1 Sobre o programa do curso	297
B.2 Sobre os dados	298

APRESENTAÇÃO

A quem se destina

O material pressupõe **Python básico**, noção elementar de programação e matemática de nível universitário inicial — derivadas parciais e multiplicação de matrizes. Não pressupõe qualquer contato prévio com aprendizado de máquina. Todo conceito da área é introduzido no ponto em que se torna necessário, e nunca antes.

Como o material está organizado

Cada capítulo é uma unidade fechada com quatro componentes:

Componente	Função
Texto expositivo	a teoria e os resultados medidos
Programas	a implementação, do zero, comentada
Exercícios	ao final de cada capítulo
Gabaritos	reunidos no Apêndice A, deliberadamente separados

A separação dos gabaritos é intencional. Um exercício cuja resposta está na página seguinte não é um exercício; nesta edição as soluções ocupam um apêndice, o que exige do leitor uma decisão consciente de consultá-las.

Os capítulos são cumulativos. O autógrafo escrito no Capítulo 2 é o mesmo usado no Capítulo 5; o *Transformer* do Capítulo 5 é o mesmo que, no Capítulo 17, recebe imagens sem uma única linha alterada. Essa continuidade é o argumento central do curso, e ler os capítulos fora de ordem a desfaz.

Sobre os números apresentados

Todas as medições foram obtidas em uma única máquina, sem GPU dedicada, e devem ser lidas como **ordens de grandeza e relações entre condições**, não como marcas absolutas. O que importa nas tabelas é a comparação entre as linhas — o ganho de um cache, o custo de uma quantização, a diferença entre duas taxas de aprendizado — e essa comparação é robusta mesmo quando o valor absoluto não é reproduzível em outro equipamento.

Duas advertências, ambas resultado de erros cometidos durante a redação e documentados nos capítulos onde ocorreram:

Conclusões dependem do orçamento de treino. Diversos resultados deste material se inverteram ao mudar o número de passos de otimização. Um experimento curto pode não apenas subestimar um efeito, mas apontá-lo na direção oposta.

Uma métrica que melhora não é um sistema que melhorou. Vários capítulos exibem casos em que o número escolhido para medir o sucesso subiu enquanto o comportamento de interesse piorava.

Reprodutibilidade

O repositório inclui um verificador (`tools/smoke_test.py`) que executa todos os programas do curso e reporta o estado de cada um. A última execução registrada percorreu os dezessete capítulos e setenta e oito programas sem falhas.

1 BIGRAM LANGUAGE MODEL

Objetivo de aprendizagem: entender o que significa "prever o próximo token" e treinar o modelo de linguagem mais simples possível — primeiro **por contagem**, depois como uma **rede neural** — e perceber que as duas abordagens são, no fundo, a mesma coisa.

Pré-requisitos deste capítulo: Python básico e a noção de que uma matriz é uma tabela de números. Nada além disso. Tudo o que for de probabilidade ou cálculo é introduzido aqui.

Arquivos:

- `bigram.py` — versão por contagem (*count-based*)
- `bigram_nn.py` — versão rede neural (*neural network*)
- `names.txt` — dataset de nomes
- `exercicios.md` — exercícios

1.1 O que é um modelo de linguagem?

Um **modelo de linguagem** (*language model*) é, essencialmente, uma máquina que responde a uma pergunta:

Dado o que já vi, o que vem em seguida?

Quando o seu celular sugere a próxima palavra enquanto você digita, é um modelo de linguagem trabalhando. Quando o ChatGPT escreve um texto, é o mesmo princípio, levado ao extremo: ele gera **um token de cada vez**, sempre perguntando "qual o próximo token mais provável, dado tudo que veio antes?".

Esse é o conceito central de todo o curso. Um LLM gigante e um modelo de brinquedo de uma linha fazem **a mesma tarefa** — prever o próximo token. A diferença está em quão bem eles fazem isso. Vamos começar pelo modelo de brinquedo.

1.1.1 Token? Caractere?

Um **token** é a "unidade" que o modelo manipula. Em modelos de verdade, um token costuma ser um pedaço de palavra (vamos construir um tokenizador no Capítulo 6). Aqui, para simplificar, **cada token é um único caractere** (uma letra). É o famoso *character-level language model*.

1.1.2 Nossa tarefa concreta: inventar nomes

Em vez de "escrever textos", vamos resolver um problema pequeno e divertido: **gerar nomes de pessoas que soam plausíveis**, mas que não existem necessariamente.

Para isso temos o arquivo `names.txt`, com um nome por linha:

```
ana
maria
joao
jose
pedro
...
```

O modelo vai olhar esses nomes reais e aprender os "padrões de letras" do português — que nomes costumam começar com certas letras, que q quase sempre é seguido de u, que muitos nomes terminam em a ou o, etc. Depois, ele inventa nomes novos seguindo esses padrões.

1.2 O modelo mais simples: o bigrama

A palavra **bigrama** (*bigram*) significa "dois elementos seguidos". Um modelo de bigrama faz a aposta mais ingênua possível:

Para prever o próximo caractere, **só olho o caractere atual**. Ignoro todo o resto.

É pouco contexto (no nome `maria`, ao prever a letra depois do segundo `a`, o modelo nem lembra que tinha um `m` no começo). Mas é o suficiente para aprender coisas úteis e é o ponto de partida perfeito.

1.2.1 Onde começa e onde termina um nome?

Um detalhe importante: o modelo precisa saber **quando começar** (qual a primeira letra provável de um nome?) e **quando parar** (a palavra acabou?). Resolvemos isso com um **token especial de fronteira**, que representamos pelo ponto `.`:

- Todo nome é tratado como `. + nome + .`
- O nome `ana` vira a sequência: `. a n a .`

Assim, o par `(. , a)` ensina "nomes podem começar com a", e o par `(a , .)` ensina "depois de a o nome pode acabar".

Listando todos os bigramas de `ana`:

```
.a  (começo -> a)
an  (a -> n)
na  (n -> a)
a.  (a -> fim)
```

1.3 Construindo o vocabulário

Antes de contar, precisamos transformar caracteres em números (computadores trabalham com números, não com letras). Criamos dois dicionários:

- `stoi` (*string to integer*): mapeia cada caractere para um índice.
- `itos` (*integer to string*): o caminho de volta.

```
chars = sorted(list(set(''.join(words)))) # ['a', 'b', 'c', ..., 'z']
stoi = {c: i + 1 for i, c in enumerate(chars)}
stoi['.'] = 0 # o token de fronteira fica no índice 0
itos = {i: c for c, i in stoi.items()}
vocab_size = len(itos) # 27 = 26 letras + '.'
```

Nosso **vocabulário** tem 27 tokens: as 26 letras de a a z mais o ..

1.4 Aprendendo por contagem

A ideia é direta: vamos **percorrer todos os nomes e contar**. Quantas vezes a letra a é seguida de n? Quantas vezes q é seguido de u? E assim por diante.

Guardamos tudo numa **matriz** `N` de tamanho 27×27 . A célula `N[i, j]` é "quantas vezes o token `j` apareceu logo depois do token `i`".

```
N = torch.zeros((vocab_size, vocab_size), dtype=torch.int32)
for w in words:
    chs = ['.'] + list(w) + ['.']
    for ch1, ch2 in zip(chs, chs[1:]): # cada par de caracteres vizinhos
        N[stoi[ch1], stoi[ch2]] += 1
```

`zip(chs, chs[1:])` é um truque comum em Python para percorrer pares vizinhos. Se `chs = ['. ', 'a', 'n', 'a', '. ']`, então `zip` produz `('. ', 'a'), ('a', 'n'), ('n', 'a'), ('a', '. ')`. Exatamente nossos bigramas.

Ao final, a **linha `i`** da matriz `N` é o "perfil" do token `i`: ela diz, dentre todas as vezes que `i` apareceu, o que costumou vir depois.

1.4.1 De contagens para probabilidades

Contagens são números absolutos ("o a foi seguido de n umas 120 vezes"). Para gerar nomes, queremos **probabilidades** ("dado que estou no a, qual a chance de o próximo ser n?"). Para isso, **normalizamos cada linha**: dividimos cada valor pela soma da linha, de modo que cada linha some 1.0.

```
P = (N + 1).float()
P = P / P.sum(dim=1, keepdim=True) # cada linha agora soma 1.0
```

Agora $P[i]$ é uma **distribuição de probabilidade**: 27 números entre 0 e 1 que somam 1, dizendo a probabilidade de cada token vir depois de i .

Probabilidade em uma frase: é só um número entre 0 (impossível) e 1 (certo) que mede a chance de algo acontecer. Se um dado honesto tem 6 faces, a probabilidade de cada face é $1/6 \approx 0,167$. Aqui, em vez de 6 faces, temos 27 "faces" (os tokens), e o dado é **viciado** de um jeito diferente para cada caractere atual.

1.4.2 Por que o + 1? (*model smoothing*)

Repare no $(N + 1)$. Alguns bigramas **nunca aparecem** no treino — talvez nenhum nome tenha j depois de q . A contagem seria 0, a probabilidade seria 0, e — como veremos na avaliação — $\log(0)$ é -infinito, o que quebra as contas. Somar 1 a todas as contagens garante que **nada** tenha probabilidade exatamente zero. Isso se chama **suavização de Laplace** (*Laplace / add-one smoothing*). Quanto maior o número somado, mais "uniforme" (chutômetro) o modelo fica.

1.5 Gerando nomes (sampling)

Temos a tabela P . Como inventar um nome? **Amostrando** (*sampling*), caractere a caractere:

1. Comece no token de fronteira . (índice 0).
2. Olhe a linha $P[ix]$ — a distribuição do próximo caractere.
3. **Sorteie** um caractere segundo essa distribuição.
4. Se sorteou o ., o nome acabou. Senão, anote a letra e volte ao passo 2 com ela.

```
g = torch.Generator().manual_seed(2147483647) # semente fixa -> resultado reprodutível

def sample_name():
    out = []
    ix = 0
    while True:
        p = P[ix]
        ix = torch.multinomial(p, num_samples=1, replacement=True, generator=g).item()
        if ix == 0:
            break
        out.append(itos[ix])
    return "".join(out)
```

`torch.multinomial(p, ...)` é o "sorteio viciado": dada a distribuição p , ele devolve um índice escolhido com aquela probabilidade. Tokens mais prováveis são sorteados mais vezes, mas os improváveis ainda têm chance — é isso que dá variedade aos nomes gerados.

`manual_seed` fixa o gerador de números aleatórios para que você obtenha sempre os mesmos resultados. Útil para estudar e depurar; em produção você deixaria aleatório de verdade.

Rode `python bigram.py`. Os nomes saem meio estranhos (mor, aya, kenadi...) porque o bigrama tem memória curtíssima — mas já são bem mais "pronunciáveis" do que

letras totalmente aleatórias. O modelo aprendeu **alguma** estrutura do português.

1.6 O modelo é bom? (a *loss*)

"Os nomes parecem ok" não é uma medida científica. Precisamos de **um número** que diga o quão bom o modelo é — de preferência, um número que possamos tentar **minimizar**. Esse número é a *loss* (função de perda/custo).

A ideia: um bom modelo atribui **alta probabilidade** aos bigramas que realmente acontecem nos dados reais. Então vamos medir, para cada bigrama do dataset, qual probabilidade o modelo deu a ele, e combinar tudo num só número.

1.6.1 Likelihood, log-likelihood e NLL

- A **likelihood** (verossimilhança) é o produto das probabilidades de todos os bigramas. Problema: multiplicar milhares de números entre 0 e 1 dá um valor minúsculo, que o computador arredonda para zero.
- Por isso usamos o **logaritmo**. Uma propriedade-chave: $\log(a \cdot b) = \log(a) + \log(b)$. Logo, em vez de **multiplicar** as probabilidades, podemos **somar** os seus logaritmos. Isso é a **log-likelihood**, numericamente estável.
- O log de um número entre 0 e 1 é **negativo** (ex.: $\log(0,1) \approx -2,3$). Para trabalharmos com um número **positivo que queremos minimizar**, pegamos o **negativo** e tiramos a **média**. Isso é a **negative log-likelihood (NLL)** — a nossa *loss*.

```
log_likelihood = 0.0
n = 0
for w in words:
    chs = ["."] + list(w) + ["."]
    for ch1, ch2 in zip(chs, chs[1:]):
        prob = P[stoi[ch1], stoi[ch2]]
        log_likelihood += torch.log(prob)
        n += 1
nll = -log_likelihood / n
```

Interpretação: quanto **menor** a NLL, melhor o modelo. O melhor valor possível seria 0 (o modelo acerta tudo com probabilidade 1 — impossível na prática). Um modelo que chuta uniformemente entre 27 tokens teria $\text{loss} \approx \log(27) \approx 3,30$. Nosso bigrama fica bem abaixo disso: ele realmente aprendeu algo.

Guarde esse valor de loss. No próximo passo, vamos treinar uma rede neural e ver que ela aprende **essencialmente a mesma tabela** — chegando a uma loss da mesma ordem (e idêntica, quando igualamos a suavização dos dois lados, como você verá no exercício E5).

1.7 A virada de chave: o mesmo modelo como rede neural

Até aqui, "treinar" foi só contar. Agora vem a ideia que sustenta o resto do curso: vamos obter **as mesmas probabilidades** sem contar — deixando uma **rede neural aprendê-las por otimização**. Parece um retrocesso (por que complicar?), mas é o contrário: contar só funciona para o bigrama. A abordagem de otimização vai escalar até o Transformer. Aqui a gente aprende no caso mais simples possível.

1.7.1 One-hot: transformando um índice num vetor

A rede não recebe o índice 5 direto; ela recebe um **vetor one-hot**: um vetor de 27 posições, todo zero, com um único 1 na posição do caractere. O caractere de índice 5 vira $[0, 0, 0, 0, 0, 1, 0, \dots, 0]$.

```
xenc = F.one_hot(xs, num_classes=vocab_size).float() # (num_exemplos, 27)
```

1.7.2 A "rede": uma única matriz de pesos W

Nosso modelo é uma só camada linear, representada por uma matriz de pesos w de 27×27 , com valores **aleatórios** no início:

```
W = torch.randn((vocab_size, vocab_size), generator=g, requires_grad=True)
```

O cálculo central é uma **multiplicação de matrizes** (*matmul*): $xenc @ W$.

Por que isso "seleciona uma linha"? Multiplicar um vetor one-hot (com o 1 na posição i) por W resulta exatamente na **linha i de W** . Ou seja: cada linha de W desempenha o papel que a linha de N tinha — ela guarda os "números brutos" do que vem depois do token i . A diferença é que esses números agora serão **aprendidos**, não contados.

1.7.3 De números brutos a probabilidades: o softmax

A saída de $xenc @ W$ são os **logits** — números reais quaisquer (podem ser negativos). Precisamos convertê-los em probabilidades (positivas, somando 1). Fazemos isso em dois passos, que juntos se chamam **softmax**:

```
logits = xenc @ W # números reais quaisquer (log-counts)
counts = logits.exp() # exp() deixa tudo positivo (equivale a N)
probs = counts / counts.sum(dim=1, keepdim=True) # normaliza -> soma 1
```

Repare na simetria com a versão por contagem: lá tínhamos N (positivo) e normalizávamos. Aqui, `logits.exp()` faz o papel de N (o `exp` garante valores positivos), e normalizamos igual. Por isso os logits são chamados de **log-counts**: exponenciados, viram contagens.

1.8 Como a rede aprende: gradient descent

No começo w é aleatório, então as probabilidades são lixo e a loss é alta. Treinar é **ajustar w aos poucos para baixar a loss**. O algoritmo é o *gradient descent* (descida do gradiente), e tem três passos repetidos muitas vezes:

```
for step in range(200):
    # 1) FORWARD: calcula as probabilidades e a loss a partir do W atual
    logits = xenc @ W
    counts = logits.exp()
    probs = counts / counts.sum(dim=1, keepdim=True)
    loss = -probs[torch.arange(num), ys].log().mean() + 0.01 * (W ** 2).mean()

    # 2) BACKWARD: descobre, para cada peso, se aumentá-lo sobe ou desce a loss
    W.grad = None
    loss.backward()

    # 3) UPDATE: empurra cada peso na direção que DIMINUI a loss
    W.data += -50 * W.grad
```

Vamos destrinchar cada passo.

Forward. Note `probs[torch.arange(num), ys]`: para cada exemplo, isso pega a probabilidade que o modelo atribuiu ao caractere **correto** (`ys` é o alvo). É exatamente a mesma NLL da Seção 6 — só que calculada de uma vez para todos os exemplos. O termo extra `0.01 * (W**2).mean()` é a **regularização** (próxima seção).

Backward — o gradiente. Aqui está a mágica. O **gradiente** é a derivada da loss em relação a cada peso: um número que diz "*se eu aumentar um tiquinho este peso, a loss sobe ou desce, e com que força?*". Calcular isso à mão para milhares de pesos seria inviável; o PyTorch faz **automaticamente** com `loss.backward()`, preenchendo `w.grad`. Esse mecanismo se chama **autograd / backpropagation** — e é tão central que o Capítulo 2 inteiro é dedicado a construí-lo do zero. Por ora, confie: depois do `backward()`, `w.grad` diz a "inclinação" da loss em cada direção.

`w.grad = None` antes de cada `backward()` zera os gradientes. Sem isso, o PyTorch **acumula** os gradientes de uma iteração na outra, e o treino dá errado.

Update. Para **descer** a loss, andamos na direção **oposta** ao gradiente — por isso o sinal de menos. O 50 é a *learning rate* (taxa de aprendizado): o tamanho do passo. Grande demais, o treino fica instável; pequeno demais, fica lento. (Ajustar a learning rate direito é tema do Capítulo 7.)

Rode `python bigram_nn.py` e veja a loss **caindo** a cada passo, estabilizando em torno de ~2,2. A versão por contagem deu ~2,4. "Perto, mas não igual" — e a explicação é exata e importante:

Por baixo do pano, os dois modelos buscam **a mesma tabela ótima** (o *MLE* — *maximum likelihood estimate*), que neste dataset tem loss $\approx 2,14$. A diferença que você vê entre os dois arquivos vem **só do quanto cada um foi suavizado**: o +1 da contagem puxa mais forte para o uniforme do que o 0.01 da rede. Iguale os dois botões e as losses se encontram (faça isso no exercício E5).

Ou seja, **suavização (+1) e regularização (L2) são o mesmo botão**, vestido de duas formas. Tirando esse botão, contagem e rede produzem o mesmo modelo.

Esse é o recado do capítulo, em uma frase:

Contar é um atalho que só funciona no bigrama. Otimizar uma rede por gradient descent é o método geral — e é o que vamos escalar até o Transformer.

1.9 Regularização = smoothing, de novo

Lembra do +1 que evitava probabilidade zero na versão por contagem? A rede tem um equivalente: o termo $0.01 * (w^{**2}).mean()$ na loss. Ele **penaliza pesos grandes**, empurrando w na direção do zero. E quando todos os pesos são iguais (zero), todos os logits são iguais, e o softmax dá uma distribuição **uniforme** — o máximo de suavização. Aumente esse 0.01 e os nomes gerados ficam mais "genéricos"; diminua e o modelo gruda mais nos dados. É o mesmo trade-off do +1, com outra roupa. Isso se chama **L2 regularization** (ou *weight decay*).

1.10 Resumo do capítulo

- Um **modelo de linguagem** prevê o próximo token. Tudo no curso é uma variação disso, em escala crescente.
- O **bigrama** prevê olhando só o caractere atual. Treinamos de duas formas:
- **por contagem**: monta a matriz N , normaliza para P ;
- **por rede neural**: uma matriz w , softmax, e gradient descent.
- As duas chegam à **mesma loss** — contar é um caso particular de otimizar.
- Conceitos que vão reaparecer o curso inteiro: **token, vocabulário, one-hot, logits, softmax, loss (NLL), gradient descent, backpropagation, learning rate, regularização/smoothing, sampling.**

1.10.1 O que vem no Capítulo 2

Neste capítulo, `loss.backward()` foi uma caixa-preta mágica. No **Capítulo 02 — Micrograd**, a gente abre a caixa: vamos construir um motor de **autograd** do zero, entendendo exatamente como o gradiente de cada peso é calculado pela **backpropagation**. É a peça que faz toda rede neural — do bigrama ao GPT-4 — conseguir aprender.

-> Antes de seguir, faça os exercícios.

1.11 Exercícios

Faça na ordem; os primeiros são de fixação, os últimos são mais desafiadores. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

1.11.1 E1 — Leitura de código (aquecimento)

Sem rodar, responda:

1. Por que `vocab_size` é 27, e não 26?
 2. No `bigram.py`, o que aconteceria se trocássemos $(N + 1)$ por N (sem suavização) ao calcular a `loss`? Por quê?
 3. Para que serve o `manual_seed`?
-

1.11.2 E2 — Mexa nos dados

Adicione 5 nomes ao final de `names.txt` (pode ser de amigos, ou nomes inventados). Rode `bigram.py` de novo. A `loss` subiu ou desceu? Os nomes gerados mudaram? Explique em 1-2 frases por que adicionar dados muda o modelo.

1.11.3 E3 — O efeito da suavização (*smoothing*)

No `bigram.py`, troque o $+ 1$ por $+ 0$, $+ 10$ e $+ 100$ (três execuções).

1. Como a **loss** muda em cada caso?
 2. Como os **nomes gerados** mudam (mais "criativos" ou mais "genéricos")?
 3. Explique: por que $+ 0$ pode dar problema?
-

1.11.4 E4 — Temperatura no sampling

No `bigram.py`, antes de amostrar, eleve a distribuição a uma potência e re-normalize: $p = P[ix] ** T$; $p = p / p.sum()$. Teste $T = 0.3$, $T = 1.0$ e $T = 3.0$.

1. Com T baixo, os nomes ficam mais "óbvios" ou mais "aleatórios"? E com T alto?
 2. Relacione isso com o conceito de **temperatura** que você já ouviu falar em LLMs.
-

1.11.5 E5 — Faça contagem e rede convergirem (a equivalência)

Rodando os dois arquivos como vêm, você acha a loss por contagem (~**2,4**) maior que a da rede (~**2,2**). Elas *deveriam* ser a mesma coisa — vamos provar igualando a "quantidade de suavização" dos dois lados:

1. No `bigram.py`, troque o `+ 1` por `+ 0.01`. Qual a nova loss por contagem? (Deve cair para ~2,15 — perto do MLE, o melhor possível.)
2. No `bigram_nn.py`, zere a regularização (troque `0.01 * w` por `0.0 * w`) e aumente os passos de 200 para 1000. Qual a loss pura da rede agora?
3. Os dois valores se encontram em torno de ~**2,1**? Explique, com suas palavras, por que `+1` (smoothing) e o termo `(w**2)` (regularização L2) são "o mesmo botão".

1.11.6 E6 — Avaliando um nome específico (desafio)

Escreva uma função `nll_de(nome)` que calcule a negative log-likelihood **de um único nome** segundo o modelo de contagem P. Use-a para responder:

1. Qual tem menor NLL (é mais "provável" segundo o modelo): ana ou xwq?
2. Teste com o seu próprio nome. Ele é "provável" para o modelo?

1.11.7 E7 — Rumo ao trigrama (desafio, antecipa o Cap. 3)

Modifique o modelo de contagem para um **trigrama**: prever o próximo caractere a partir dos **dois** caracteres anteriores.

1. Em vez de uma matriz 27×27, qual o formato da tabela de contagens agora?
2. A loss melhora em relação ao bigrama? Por quê?
3. Que novo problema de "dados esparsos" (muitos zeros) começa a aparecer? Como a suavização ajuda — e por que ela não resolve tudo? (Essa limitação é exatamente o que motiva o MLP do Capítulo 3.)

2 MICROGRAD (AUTOGRAD E BACKPROPAGATION)

Objetivo de aprendizagem: abrir a caixa-preta do `loss.backward()`. Vamos construir, do zero, um motor de **autograd** (diferenciação automática) e usá-lo para treinar uma rede neural de verdade — entendendo exatamente como o gradiente de cada peso é calculado pela **backpropagation**.

Pré-requisitos deste capítulo: o Capítulo 1 (você já viu *loss*, gradiente e gradient descent "de fora"). De matemática, só a ideia de **derivada** como "taxa de variação" — que revisamos já na Seção 1. Programação: vamos usar **classes** em Python; se você nunca criou uma, a Seção 4 explica o suficiente.

Arquivos:

- `micrograd.py` — o motor de autograd (a classe `Value`)
- `nn.py` — uma mini biblioteca de redes neurais sobre o `Value`
- `exercicios.md` — exercícios

2.1 O problema: como a rede sabe para que lado ir?

No Capítulo 1, treinar a rede foi isto:

```
loss.backward()           # <- mágica: de onde saem os gradientes?
W.data += -50 * W.grad
```

A segunda linha é simples: "ande na direção que diminui a *loss*". Mas ela depende de `W.grad` — o **gradiente** —, e esse número apareceu de graça quando chamamos `loss.backward()`. Este capítulo é sobre **fabricar esse número nós mesmos**.

2.1.1 Derivada em 30 segundos

A **derivada** de uma quantidade em relação a outra responde: "*se eu mexer um tiquinho nesta entrada, o quanto a saída muda, e para que lado?*"

Formalmente, é o limite

$$df/dx = \lim_{h \rightarrow 0} [f(x + h) - f(x)] / h$$

mas a intuição é o que importa: a derivada é a **inclinação** (positiva = "subindo", negativa = "descendo") e a **sensibilidade** (quão forte). Se $df/dx = 3$, aumentar x em 0.001 aumenta f em ~ 0.003 . Se $df/dx = -2$, aumentar x **diminui** f .

2.1.2 Gradiente = todas as derivadas de uma vez

Numa rede com milhares de pesos, o **gradiente** é só o conjunto de todas as derivadas da loss em relação a cada peso: $\partial \text{loss} / \partial w$ para cada w . Cada uma diz como mexer **aquele** peso para baixar a loss. É exatamente o w .grad do Capítulo 1.

O desafio: calcular isso à mão para milhares de pesos é inviável. A solução é a **backpropagation** — um jeito mecânico e barato de obter todas as derivadas de uma vez. E ela se apoia em uma única regra do cálculo: a **regra da cadeia**.

2.2 A regra da cadeia (chain rule), a chave de tudo

Imagine três engrenagens encaixadas. A primeira gira a segunda na proporção de 2:1; a segunda gira a terceira na proporção de 3:1. Quão rápido a terceira gira em relação à primeira? $2 \times 3 = 6$. Você **multiplica as taxas locais ao longo do caminho**.

Isso é a regra da cadeia. Se y depende de u , que depende de x :

$$dy/dx = (dy/du) \cdot (du/dx)$$

A backpropagation é só isto aplicado repetidamente: para saber o efeito de um peso lá no começo sobre a loss lá no fim, **multiplicamos as derivadas locais de cada passo no caminho de volta**. Cada operação só precisa saber a sua própria derivada local ("eu giro o próximo na proporção X"). Juntando o caminho todo, sai o gradiente.

A grande sacada: se cada operaçãozinha (+, *, tanh, ...) souber a sua derivada local, conseguimos compor essas peças em qualquer expressão gigante e ainda assim calcular todos os gradientes — só propagando de trás para frente.

2.3 O grafo de computação

Toda expressão pode ser desenhada como um **grafo**: as folhas são as entradas, e cada operação é um nó que combina resultados anteriores. Veja $d = a * b + c$:

```

a --+
  (*) --> e --+
b --+          (+) --> d
c -----+
```

A computação normal anda da esquerda para a direita (**forward pass**): calcula e , depois d . A backpropagation anda da direita para a esquerda (**backward pass**): parte de d , distribui o gradiente para e e c , depois de e para a e b .

Para fazer isso no código, cada valor precisa **lembrar de onde veio** — quais nós o geraram e por qual operação. É essa "memória" que vamos construir agora.

2.4 A classe Value

O coração do micrograd é uma classe que embrulha **um único número** e participa do grafo. Se você nunca usou classes: pense numa classe como um molde que junta dados (o número e seu gradiente) com comportamentos (somar, multiplicar...). Cada `Value` criado é um "objeto" feito nesse molde.

```
class Value:
    def __init__(self, data, _children=(), _op=""):
        self.data = data          # o número em si
        self.grad = 0.0          # d(loss)/d(este value); começa zerado
        self._backward = lambda: None # função que propaga o grad aos pais
        self._prev = set(_children) # de quais Values este nasceu
        self._op = _op           # rótulo da operação (debug)
```

Dois campos são "públicos" e te interessam: `data` (o valor) e `grad` (a derivada da loss em relação a ele). Os três com underline são a maquinaria interna do autograd: `_prev` guarda os "pais" no grafo, `_op` é só um rótulo, e `_backward` é uma função que sabe distribuir o gradiente deste nó para os pais.

2.5 Cada operação conhece sua derivada local

Aqui mora a inteligência. Ao sobrecarregar `+` e `*`, cada operação faz **duas** coisas: calcula o resultado (forward) e registra como o gradiente volta (backward).

2.5.1 Soma

```
def __add__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data + other.data, (self, other), "+")

    def _backward():
        self.grad += 1.0 * out.grad
        other.grad += 1.0 * out.grad
    out._backward = _backward
    return out
```

A derivada local da soma é **1 para cada parcela** ($d(a+b)/da = 1$). Então a soma apenas **repassa** o gradiente que chega (`out.grad`) igualzinho para os dois pais. É o "tubo de distribuição": a soma espalha o gradiente sem alterá-lo.

2.5.2 Multiplicação

```
def __mul__(self, other):
    other = other if isinstance(other, Value) else Value(other)
    out = Value(self.data * other.data, (self, other), ".*")

    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out
```

Aqui $d(a \cdot b)/da = b$ e $d(a \cdot b)/db = a$ — cada pai recebe o gradiente **multiplicado pelo valor do outro**. Faz sentido: se b é grande, mexer em a tem efeito grande.

2.5.3 Por que += e não =? (o detalhe que todo mundo erra)

Repare que **acumulamos** com +=. Se um mesmo Value é usado em dois lugares (ex.: $b = a + a$), o gradiente chega por **dois caminhos** e os efeitos **se somam** (é a regra da soma das derivadas). Usar = apagaria a primeira contribuição e daria o gradiente errado. Por isso, antes de cada backward, precisamos **zerar os gradientes** — o famoso `zero_grad()` / `w.grad = None` do Capítulo 1 reaparece aqui, agora com o motivo explícito.

2.5.4 Funções não-lineares: tanh, exp, relu

Uma rede que só soma e multiplica é, no fim, uma única conta linear — incapaz de aprender padrões curvos. Precisamos de uma **não-linearidade**. Usamos a tanh, que amassa qualquer número para o intervalo $(-1, 1)$:

```
def tanh(self):
    t = math.tanh(self.data)
    out = Value(t, (self,), "tanh")
    def _backward():
        self.grad += (1 - t**2) * out.grad      # d(tanh)/dx = 1 - tanh(x)^2
    out._backward = _backward
    return out
```

O padrão é sempre o mesmo: calcula o valor, e guarda em `_backward` a derivada local multiplicada pelo gradiente que vem de fora. `exp` e `relu` (veja o código) seguem a mesma receita. **É só isso que um framework de deep learning faz** — para centenas de operações, na GPU, mas com exatamente esta estrutura.

2.6 A `backward()`: percorrer o grafo de trás para frente

Com cada nó sabendo sua derivada local, falta orquestrar: chamar os `_backward()` na **ordem certa**. A regra: só posso processar um nó depois de já ter recebido todo o gradiente que vem "de fora" dele. Isso é uma **ordenação topológica** do grafo.

```
def backward(self):
    topo = []
    visited = set()
    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._prev:
                build_topo(child)
            topo.append(v)
    build_topo(self)

    self.grad = 1.0 # d(loss)/d(loss) = 1: a semente
    for node in reversed(topo):
        node._backward()
```

Dois detalhes:

1. **A semente `self.grad = 1.0`.** A derivada da loss em relação a ela mesma é 1. Esse 1.0 é a "gota" de gradiente que entra no topo e vai se distribuindo grafo abaixo pela regra da cadeia.
2. **`reversed(topo)`.** Construímos a ordem em que cada nó vem *depois* dos seus filhos; invertendo, processamos da loss para as folhas — o sentido do backward.

2.6.1 Veja funcionando

Rode `python micrograd.py`. Ele monta $L = \tanh(a \cdot b + c)$ e chama `L.backward()`:

```
L = Value(data=0.9993, grad=1.0000)
dL/da = -0.00402...
dL/db = 0.00268...
dL/dc = 0.00134...
```

Os gradientes são pequenos porque a `tanh` está **saturada** (entrada $a \cdot b + c = 4$, bem na parte "achatada" da curva, onde a inclinação é quase zero). Guarde esse efeito — "gradientes que somem em regiões saturadas" é um tema que volta no Capítulo 7.

2.7 De um número a uma rede neural

Um neurônio biológico recebe sinais, pondera e dispara. O artificial faz o mesmo: soma ponderada das entradas mais um *bias*, passada por uma não-linearidade. Com a classe `Value`, isso são poucas linhas (veja `nn.py`):

```
class Neuron(Module):
    def __init__(self, nin, nonlin=True):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(0.0)
        self.nonlin = nonlin
    def __call__(self, x):
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b) # w·x + b
        return act.tanh() if self.nonlin else act
```

Cada peso w e o b são **Values** — ou seja, **fazem parte do grafo** e terão gradiente automático. Empilhando neurônios viram uma **Layer**, e empilhando camadas vira um **MLP** (Multi-Layer Perceptron). Esse é, literalmente, o modelo do Capítulo 3.

2.8 Treinando: o mesmo loop do Capítulo 1, agora "nosso"

A demo em `nn.py` treina um MLP $3 \rightarrow 4 \rightarrow 4 \rightarrow 1$ para classificar 4 pontos. O laço de treino é idêntico em espírito ao do Capítulo 1 — forward, backward, update:

```
for step in range(100):
    ypred = [model(x) for x in xs]                # forward
    loss = sum((yp - yt)**2 for yp, yt in zip(ypred, ys)) # erro quadrático

    model.zero_grad()                             # zera (por causa do +=!)
    loss.backward()                                # NOSSA backpropagation

    for p in model.parameters():
        p.data -= 0.05 * p.grad                    # gradient descent
```

Rode python `nn.py`:

```
parametros no modelo: 41
step  0 | loss 3.092861
step 10 | loss 0.000186
...
loss final = 0.000000
previsoes (alvo entre parenteses):
+1.000 (+1)
-1.000 (-1)
-1.000 (-1)
+1.000 (+1)
```

A loss despenca e as 4 previsões batem com os alvos. **Treinamos uma rede neural com um motor de autograd que escrevemos do zero.** O `loss.backward()` deixou de ser mágica.

2.9 "Então o PyTorch é isto?" — Sim.

A diferença entre o nosso **Value** e o **Tensor** do PyTorch é **engenharia, não conceito**:

- PyTorch opera sobre **tensores** (arrays n-dimensionais), não um escalar por vez — por isso é milhares de vezes mais rápido (uma `matmul` de uma vez, na GPU).
- Tem muito mais operações, e roda em GPU.

Mas a alma é idêntica: cada operação registra sua derivada local, e um `backward()` percorre o grafo aplicando a regra da cadeia. Tanto que **os gradientes do nosso micrograd batem com os do PyTorch até a 6ª casa decimal** — você confirma isso no exercício E5. A partir daqui, voltamos a usar PyTorch sabendo exatamente o que ele faz por baixo.

2.10 Resumo do capítulo

- A **derivada** mede sensibilidade e direção; o **gradiente** reúne as derivadas da loss em relação a todos os pesos.
- A **regra da cadeia** permite obter o gradiente de qualquer expressão multiplicando derivadas locais ao longo do caminho de volta.
- Toda expressão é um **grafo de computação**; cada nó guarda como nasceu (`_prev`) e sua derivada local (`_backward`).
- **Backpropagation** = ordenar o grafo topologicamente e aplicar `_backward()` da loss até as folhas, semeando com `grad = 1.0`.
- Gradientes se **acumulam** (`+=`) -> por isso é preciso **zerar** antes de cada passo.
- Com isso construímos Neuron -> Layer -> MLP e treinamos uma rede de verdade. O `loss.backward()` do PyTorch é exatamente este mecanismo, vetorizado.

2.10.1 O que vem no Capítulo 3

Temos um motor de autograd e um MLP. No **Capítulo 03 — N-gram model** voltamos ao problema de gerar nomes do Capítulo 1, mas trocamos o bigrama por um **MLP** que olha *vários* caracteres de contexto — usando PyTorch de verdade, com `matmul`, embeddings e a função de ativação **GELU**. É a ponte do brinquedo para o modelo de verdade.

-> Antes de seguir, faça os exercícios.

2.11 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

2.11.1 E1 — Derivada na unha (aquecimento)

Sem código, só caneta:

1. Para $f = a * b$ com $a = 3$, $b = 4$, quanto vale df/da ? E df/db ?
 2. Para $d = a*b + c$, quanto vale dd/dc ? Por que a soma "não altera" o gradiente?
 3. Explique, com suas palavras, por que $\tanh$ saturada (entrada grande) produz gradiente quase zero.
-

2.11.2 E2 — Conferindo com diferença finita

A derivada é $[f(x+h) - f(x)] / h$ para h pequeno. Escolha uma expressão simples com `Value` (ex.: $L = a*b + c$), calcule `a.grad` via `backward()`, e depois compare com a estimativa numérica: recalcule L com a somado de $h = 1e-6$ e use a fórmula. Os dois valores batem? (Essa é a forma clássica de **testar** um autograd.)

2.11.3 E3 — Adicione uma operação nova

Implemente o método `log()` na classe `Value` (logaritmo natural).

1. Qual a derivada local de $\log(x)$? (Dica: $d(\ln x)/dx = 1/x$.)
 2. Escreva o `_backward` correspondente, seguindo o padrão de `exp()`.
 3. Teste comparando com `math.log` e a diferença finita do E2.
-

2.11.4 E4 — Por que zerar o gradiente?

No `nn.py`, **comente** a linha `model.zero_grad()` e rode de novo.

1. O que acontece com a `loss` ao longo dos passos?
 2. Explique usando o fato de que os gradientes são acumulados com `+=`.
 3. Relacione com o `w.grad = None` do Capítulo 1.
-

2.11.5 E5 — Bate com o PyTorch? (desafio)

Escreva um script que monte **a mesma expressão** com `Value` e com `torch.tensor(..., requires_grad=True)`, rode `backward()` nos dois e compare data e todos os `grad`.

1. Eles batem (com tolerância de $1e-6$)?

Troque uma operação (ex.: use `relu` em vez de `tanh`) e confirme que ainda batem.

A solução de referência está em
`solucoes/e5_check_vs_torch.py`.

2.11.6 E6 — Mexa na arquitetura e na learning rate

No `nn.py`:

1. Troque `MLP(3, [4, 4, 1])` por `MLP(3, [8, 1])` e depois `MLP(3, [16, 16, 1])`. Quantos parâmetros cada um tem? A loss final muda?
2. Varie a learning rate (`0.05`) para `0.5` e `0.001`. O que acontece com `0.5` (instável?) e com `0.001` (lento?)?
3. Troque a ativação `tanh` por `relu` nos neurônios. Treina melhor ou pior aqui?

2.11.7 E7 — Visualize o grafo (desafio, opcional)

Escreva uma função que percorra `_prev` recursivamente a partir de um `Value` e imprima o grafo (cada nó: seu `_op`, `data` e `grad`). Use-a numa expressão pequena para "ver" a estrutura que a `backward()` percorre. (Se conhecer a lib `graphviz`, desenhe de verdade — é opcional.)

3 N-GRAM MODEL (MLP EM PYTORCH)

Objetivo de aprendizagem: superar a memória curta do bigrama construindo um **MLP** (multi-layer perceptron) que olha **vários** caracteres de contexto. Pelo caminho, encontramos peças que vão aparecer no curso inteiro: **embeddings**, **matmul** em lote, a ativação **GELU**, **mini-batches**, **cross-entropy** e — o conceito mais importante de todos — a separação **treino / validação / teste**.

Pré-requisitos deste capítulo: Capítulos 1 (modelo de linguagem, softmax, loss) e 2 (como uma rede aprende por gradient descent). A partir daqui usamos **PyTorch de verdade** — a `backward()` que construímos à mão no Cap. 2 agora é a do framework, e sabemos exatamente o que ela faz.

Arquivos:

- `prepare_data.py` — prepara o dataset de nomes (fonte: IBGE)
- `mlp.py` — o modelo MLP completo (treino, avaliação e geração)
- `exercicios.md` — exercícios

3.1 O problema do bigrama: memória de um caractere só

No Capítulo 1, o bigrama previa o próximo caractere olhando **apenas o anterior**. Ao gerar o nome maria, na hora de decidir a letra depois do segundo a, ele já tinha esquecido o m, o r... tudo. Uma memória de um caractere só.

A consequência: nomes "pronunciáveis", mas sem estrutura de longo alcance. Para melhorar, precisamos de **mais contexto** — olhar os últimos k caracteres em vez de um só. A ideia ingênua seria um **trigrama** (dois caracteres anteriores), depois um quadrigrama, e assim por diante.

3.1.1 Por que contar não escala

Lembra que no Cap. 1 a contagem virou uma matriz 27×27 ? Para um trigrama, ela vira $27 \times 27 \times 27$. Para k caracteres de contexto, são $27^{(k+1)}$ células. Com $k = 10$, isso passa de **10^{20} células** — impossível de armazenar, e a maioria ficaria zerada (nenhum nome tem aquela combinação específica). É a **maldição da dimensionalidade**: mais contexto, com contagem, significa uma explosão de células vazias.

A saída é exatamente a do Cap. 1, levada a sério: em vez de **contar**, vamos **aprender** — com uma rede neural que comprime o contexto num espaço pequeno e denso. Essa é a arquitetura de Bengio et al. (2003), e é a base de tudo que vem depois.

3.2 Um dataset de verdade

Há uma mudança importante em relação aos capítulos anteriores: trocamos os 155 nomes de brinquedo por um dataset **real e grande** — os prenomes dos censos do **IBGE**, com mais de **64 mil nomes**.

```
maria, ana, joao, gabriel, lucas, pedro, mateus, jose, gustavo, vitoria, ...
```

O `prepare_data.py` baixa a lista do IBGE e **normaliza** cada nome: minúsculo, sem acento (josé -> jose), só letras a-z. Para reproduzir:

```
curl -L -o _ibge.csv https://raw.githubusercontent.com/datasets-br/prenomes/master/data/nomes-c
python prepare_data.py # gera names.txt
```

Por que isso importa tanto? Um MLP tem milhares de parâmetros. Com poucos dados (os 155 nomes do Cap. 1), ele simplesmente **decora** o dataset e não aprende nada generalizável — um problema chamado *overfitting*, que veremos de perto na Seção 8. Com dezenas de milhares de exemplos, ele é forçado a aprender **padrões** de verdade. O tamanho do dataset não é detalhe: é o que separa um modelo que funciona de um que só finge funcionar.

3.3 Construindo o dataset de contexto -> alvo

A janela de contexto tem tamanho fixo `block_size` (usamos **3**). Para cada posição do nome, o **input** são os 3 caracteres anteriores e o **alvo** é o caractere atual. Usamos o token de fronteira `.` (índice 0) para preencher o começo.

Veja como o nome `ana` é fatiado (`block_size = 3`):

```
contexto      -> alvo
. . .         -> a      (começo do nome)
. . a         -> n
. a n         -> a
a n a         -> .      (fim do nome)
```

No código, a janela "desliza": a cada passo descartamos o caractere mais antigo e anexamos o atual.

```
def build_dataset(words):
    X, Y = [], []
    for w in words:
        context = [0] * block_size # '...'
        for ch in w + ".":
            ix = stoi[ch]
            X.append(context)
            Y.append(ix)
            context = context[1:] + [ix] # desliza a janela
    return torch.tensor(X), torch.tensor(Y)
```

`X` tem formato `(N, 3)` (`N` contextos, 3 índices cada) e `Y` tem formato `(N, )`.

3.4 A peça nova: embeddings

Como dar os caracteres de input para a rede? No Cap. 1 usamos **one-hot** (um vetor de 27 posições com um único 1). Funciona, mas é desperdício: cada caractere fica isolado, sem relação com os outros.

A ideia melhor é o **embedding**: representar cada caractere por um vetor pequeno e **denso** de números reais — que a própria rede vai **aprender**. Guardamos isso numa tabela `C` de formato `(vocab_size, n_embd)`: uma linha por caractere, cada linha um vetor de `n_embd = 10` números.

```
C = torch.randn((vocab_size, n_embd))  # (27, 10): a tabela de embeddings
emb = C[X]                             # (N, 3, 10): "olha" o embedding de cada char
```

C[X] é indexação avançada do PyTorch. `X` tem os índices dos caracteres; `C[X]` substitui cada índice pela linha correspondente de `C`. Como `X` é `(N, 3)`, o resultado é `(N, 3, 10)`: para cada um dos `N` exemplos, os embeddings dos seus 3 caracteres de contexto. É o equivalente vetorizado e eficiente do "one-hot vezes matriz" do Cap. 1.

O bônus pedagógico: depois de treinar, caracteres com papéis parecidos (por exemplo, as vogais) tendem a ganhar embeddings próximos — a rede **descobre** estrutura sozinha. (Você explora isso no exercício E6.)

3.5 A arquitetura do MLP

Com os embeddings em mãos, o resto é uma rede de duas camadas:

```
def forward(X):
    emb = C[X]                                # (N, 3, 10)
    x = emb.view(emb.shape[0], -1)           # (N, 30): concatena o contexto num vetor só
    h = F.gelu(x @ W1 + b1)                  # (N, 200): camada oculta com GELU
    logits = h @ W2 + b2                     # (N, 27): pontuações do próximo caractere
    return logits
```

Três operações merecem atenção:

emb.view(N, -1) — **achatar o contexto**. A camada linear espera um vetor por exemplo, não uma matriz 3×10 . O `.view` reorganiza os mesmos números em `(N, 30)`, colando os 3 embeddings de 10 em um vetor de 30. (O `-1` diz ao PyTorch "calcule essa dimensão pra mim".) `view` não copia dados — é o conceito de *views/strides* do apêndice; só reinterpreta o mesmo bloco de memória.

x @ W1 + b1 — **a matmul**. O `@` é a **multiplicação de matrizes**: combina os 30 números de entrada nos 200 neurônios da camada oculta, de uma vez, para todo o batch. É a operação que domina o custo de qualquer rede neural — e o motivo de usarmos GPUs (Capítulo 8).

F.gelu(...) — a **não-linearidade**. Como vimos no Cap. 2, sem uma função não-linear a rede inteira colapsaria numa única transformação linear. Aqui usamos a **GELU** (*Gaussian Error Linear Unit*) no lugar da tanh: ela é a ativação padrão dos Transformers modernos (GPT, BERT). Na prática é uma versão "suave" do ReLU — deixa passar valores positivos, amortece os negativos sem zerá-los bruscamente.

3.6 A loss: cross_entropy

No Cap. 1 calculamos a loss em três passos manuais: softmax, pegar a probabilidade do alvo, tirar $-\log$ e a média. O PyTorch faz tudo isso de uma vez:

```
loss = F.cross_entropy(logits, Y)
```

`cross_entropy` é softmax + negative log-likelihood combinados. Usá-la em vez de fazer à mão não é só conveniência: a versão do PyTorch é **numericamente estável** (evita `exp()` de números grandes que estourariam) e mais rápida. É a loss padrão de todo modelo de classificação — e prever "qual o próximo entre 27 caracteres" é um problema de classificação.

3.7 Treino com mini-batches

Nosso dataset de treino tem **mais de 400 mil** exemplos. Calcular a loss sobre todos a cada passo seria lento. A solução é o **mini-batch**: a cada passo, sorteamos um punhado de exemplos (32) e damos um passo de gradiente baseado só neles.

```
for step in range(max_steps):
    ix = torch.randint(0, Xtr.shape[0], (batch_size,)) # 32 índices aleatórios
    logits = forward(Xtr[ix])
    loss = F.cross_entropy(logits, Ytr[ix])

    for p in parameters:
        p.grad = None
    loss.backward()

    lr = 0.1 if step < 15000 else 0.01 # learning rate decai no fim
    for p in parameters:
        p.data += -lr * p.grad
```

Duas observações:

- **O gradiente do mini-batch é "ruidoso"** (é uma estimativa, baseada em 32 exemplos, do gradiente verdadeiro). Por isso a loss impressa **pula** de um passo para outro — é normal e até útil: o ruído ajuda a escapar de mínimos ruins. O que importa é a tendência de queda, não cada valor.

- **O decaimento da learning rate** (de 0.1 para 0.01 no fim) é o truque de "passos largos no começo, passos finos para assentar". Inicialização e learning rate ganham um capítulo só pra eles (Cap. 7); aqui usamos valores razoáveis.

3.8 O conceito mais importante: treino / validação / teste

Aqui está a ideia que diferencia "decorar" de "aprender". Se medíssemos a qualidade do modelo **nos mesmos dados** em que ele treinou, um modelo que simplesmente **decorou** todos os nomes teria loss baixíssima — e seria inútil para nomes novos.

Por isso dividimos os dados em **três** partes:

Split	Proporção	Para que serve
treino	80%	ajustar os pesos (o modelo vê esses dados)
validação	10%	medir generalização e escolher hiperparâmetros
teste	10%	a prova final, usada uma vez só, no fim

```
random.shuffle(words)
n1, n2 = int(0.8*len(words)), int(0.9*len(words))
Xtr, Ytr = build_dataset(words[:n1])      # treino
Xdev, Ydev = build_dataset(words[n1:n2])  # validação
Xte, Yte = build_dataset(words[n2:])      # teste
```

A pergunta-chave: a loss de **validação** acompanha a de **treino**? Se a de treino despenca mas a de validação não acompanha (ou piora), o modelo está **decorando** em vez de generalizar — isso é o **overfitting**.

Veja na prática. O dataset grande deste capítulo dá treino e validação quase coladas (Seção 9) — sinal de generalização saudável. Se você rodar o **mesmo código** com os 155 nomes do Capítulo 1 (exercício E5), verá a loss de validação **disparar** enquanto a de treino cai: o retrato clássico do overfitting. Mesmo modelo, dataset pequeno — eis a diferença.

3.9 Rodando e os resultados

Rode python mlp.py. A saída (com a semente fixa do código):

```
treino: (410556, 3) | val: (51615, 3) | teste: (51504, 3)
parametros: 11897
step      0/20000 | loss 3.3753
```

```

step 2000/20000 | loss 2.2718
...
loss treino      = 1.9659
loss validacao   = 1.9666
loss teste       = 1.9740

```

Dois pontos a comemorar:

1. **Treino $\approx$ validação $\approx$ teste ($\sim 1,97$).** As três losses praticamente coincidem — o modelo **generaliza**. O que ele aprendeu nos 80% de treino vale igualmente para nomes que nunca viu.
2. **$\sim 1,97$ é bem melhor que o bigrama.** O bigrama do Capítulo 1 ficava em $\sim 2,4$ de loss. O MLP, com mais contexto, chega a $\sim 1,97$. **Mais contexto + capacidade de aprender = modelo melhor.** A tese do capítulo, confirmada por um número.

E os nomes gerados ficam mais variados e estruturados que os do bigrama:

```
nan, ven, jelia, maurisney, alemyr, mara, clesilenna, ...
```

Note que são nomes **novos** (não estão no dataset) e ainda assim plausíveis em português — exatamente o que queremos.

3.10 Resumo do capítulo

- O bigrama tem memória de **um** caractere; contar não escala para mais contexto (maldição da dimensionalidade). A saída é **aprender** com um MLP.
- **Embedding**: cada caractere vira um vetor denso e aprendível ($C[X]$), bem melhor que one-hot.
- O **MLP** concatena os embeddings do contexto (view), aplica uma camada oculta com **GELU** ($x @ w1 + b1$) e produz **logits** ($h @ w2 + b2$).
- **cross_entropy** = softmax + NLL, estável e padrão.
- **Mini-batches** tornam o treino viável em datasets grandes (loss "ruidosa" é normal); a **learning rate decai** para assentar no fim.
- **Treino / validação / teste**: a única forma honesta de medir se o modelo **generaliza** ou só **decora** (overfitting). Dataset grande $\rightarrow$ generalização.
- Resultado: loss $\sim 1,97$, **batendo o bigrama** ($\sim 2,4$), com nomes gerados melhores.

3.10.1 O que vem no Capítulo 4

O MLP ainda trata o contexto como um saco fixo de 3 caracteres concatenados. E se o modelo pudesse decidir **a quais** caracteres do passado prestar atenção, de forma flexível e dependente do conteúdo? Esse é o salto do **Capítulo 04 — Attention**, o mecanismo que torna os Transformers tão poderosos.

-> Antes de seguir, faça os exercícios.

3.11 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

3.11.1 E1 — Leitura de código (aquecimento)

Sem rodar, responda:

1. Por que `emb = C[X]` tem formato `(N, 3, 10)`? O que significa cada uma das três dimensões?
2. Por que precisamos do `.view(N, -1)` antes da primeira camada linear?
3. O que `F.cross_entropy(logits, Y)` faz, em termos do que fizemos "à mão" no Capítulo 1?

3.11.2 E2 — O efeito do contexto (`block_size`)

Rode `mlp.py` com `block_size = 1, 3 e 5` (três execuções).

1. Anote a loss de validação em cada caso. Mais contexto ajuda?
2. Com `block_size = 1`, o MLP vira essencialmente um bigrama "neural". A loss fica parecida com a do Capítulo 1 (~2,4)? Por quê?
3. O número de parâmetros muda com o `block_size`? Onde, no código, isso aparece?

3.11.3 E3 — Tamanho da camada oculta

Varie `n_hidden` em 50, 200 e 500.

1. Como muda o número de parâmetros e a loss de validação?
2. Camada maior sempre dá loss menor? Onde parece haver um retorno decrescente?

3.11.4 E4 — Learning rate

1. Troque a learning rate inicial (0.1) por 1.0 e por 0.001 . Descreva o que acontece com a curva de loss em cada caso (instável? lenta?).
2. Remova o decaimento (deixe $lr = 0.1$ o tempo todo). A loss final piora? Por que o decaimento ajuda no fim do treino?

3.11.5 E5 — Overfitting com dataset pequeno (o mais importante)

Copie os **155 nomes** do Capítulo 1 para um arquivo `names_pequeno.txt` e rode o MLP com ele (mude o nome do arquivo no código, ou use a solução pronta).

1. O que acontece com a loss de **treino** versus a de **validação**?
2. Por que o **mesmo modelo** generaliza com 64 mil nomes mas decora com 155?

Os nomes "gerados" passam a se parecer demais com nomes reais do dataset. Por quê isso é um **mau** sinal, e não um bom?

Solução de referência: `solucoes/e5_overfitting.py`.

3.11.6 E6 — Visualizando os embeddings (desafio)

Após treinar, a tabela `C` tem um vetor por caractere. Para "ver" o que a rede aprendeu, treine com `n_embd = 2` (assim cada caractere é um ponto no plano) e plote os 27 caracteres com `matplotlib`.

1. As **vogais** (a, e, i, o, u) ficam agrupadas? E outros grupos fazem sentido?
2. Por que reduzir para `n_embd = 2` piora a loss, mas ainda assim é útil para visualizar?

3.11.7 E7 — Conte os parâmetros na mão (desafio)

O código imprime `parametros: 11897`. Mostre, somando as partes (`C`, `w1`, `b1`, `w2`, `b2`), de onde vem esse número — em função de `vocab_size`, `block_size`, `n_embd` e `n_hidden`. Confira que sua fórmula reproduz o 11897.

4 ATTENTION (O MECANISMO DE ATENÇÃO)

Objetivo de aprendizagem: construir a **self-attention** do zero, em quatro versões equivalentes que vão da média mais burra até o mecanismo de verdade. Você vai entender **query/key/value**, a **máscara causal**, o fator de **escala** e o **embedding posicional** — e, no fim, descobrir experimentalmente por que atenção, sozinha, **não** basta.

Pré-requisitos: Capítulos 1-3. Em especial: softmax e loss (Cap. 1), como uma rede aprende (Cap. 2), embeddings e `matmul` em PyTorch (Cap. 3).

Arquivos:

- `attention.py` — o mecanismo em 4 versões, com prova de equivalência
- `model.py` — um modelo de linguagem com atenção, treinado nos nomes
- `exercicios.md` — exercícios

4.1 O que falta no MLP

O MLP do Capítulo 3 funciona, mas tem duas limitações incômodas:

1. **Ele amassa o contexto.** Os 3 embeddings são **concatenados** num vetor de 30 números e jogados numa camada linear. A posição 1 sempre entra nos mesmos "slots" de peso, a posição 2 nos seguintes, e assim por diante. O modelo não tem como dizer *"neste caso específico, o caractere 1 é o que importa; naquele outro, é o 3"*.
2. **O contexto é caro de aumentar.** Dobrar o contexto dobra o tamanho da primeira camada. Ir para 100 caracteres de contexto seria proibitivo.

O que queremos é diferente: um mecanismo em que cada posição **busca ativamente** a informação de que precisa nas posições anteriores — e essa busca depende do **conteúdo**, não só da posição. Isso é a **atenção**.

A ideia em uma frase: atenção é um mecanismo de **comunicação** — cada posição da sequência olha para as outras e traz de volta uma média **ponderada** delas, onde os pesos são calculados a partir do conteúdo.

Vamos construí-la em quatro versões, cada uma equivalente à anterior. Rode `python attention.py` para acompanhar.

4.2 Preparando o terreno: a forma dos dados

Trabalhamos com tensores de três dimensões: (B, T, C).

Dimensão	Nome	Significado
B	batch	quantas sequências processamos em paralelo
T	time	quantas posições tem cada sequência
C	channels	quantos números descrevem cada posição

```
B, T, C = 4, 8, 2
x = torch.randn(B, T, C)
```

A tarefa das próximas seções: para cada posição t , produzir um vetor que **combine informação das posições 0 até t** — nunca do futuro (isso é essencial, e a Seção 7 explica por quê).

4.3 v1 — a média na mão (*bag of words*)

A forma mais simples de "combinar o passado" é tirar a **média** dos vetores anteriores. Sem pesos, sem esperteza: todos contam igual.

```
xbow1 = torch.zeros((B, T, C))
for b in range(B):
    for t in range(T):
        xprev = x[b, : t + 1]
        xbow1[b, t] = xprev.mean(dim=0)  # tudo até a posição t
                                         # média ao longo do tempo
```

Isso se chama *bag of words* ("saco de palavras"): a informação de ordem é perdida, sobra só a mistura. É fraco, mas é o nosso ponto de partida — e, curiosamente, a estrutura desse cálculo já é a da atenção.

4.4 v2 — a mesma média, com uma multiplicação de matrizes

O loop acima é lento. Aqui entra um truque bonito: **essa média é uma multiplicação de matrizes**.

Monte uma matriz wei de $T \times T$ onde a linha t tem $1/(t+1)$ nas colunas $0 \dots t$ e zero no resto:

wei =		x =		wei @ x =
[1.00 0.00 0.00 0.00]		[x0]		[x0]
[0.50 0.50 0.00 0.00]	@	[x1]	=	[(x0+x1)/2]
[0.33 0.33 0.33 0.00]		[x2]		[(x0+x1+x2)/3]
[0.25 0.25 0.25 0.25]		[x3]		[(x0+...+x3)/4]

```
wei = torch.tril(torch.ones(T, T))          # triangular inferior: zera o futuro
wei = wei / wei.sum(dim=1, keepdim=True)    # normaliza cada linha para somar 1
xbow2 = wei @ x                             # (T,T) @ (B,T,C) -> (B,T,C)
```

Duas peças importantes aparecem aqui:

- **torch.tril** (*triangular lower*) é o que impede olhar para o futuro: as posições acima da diagonal são zero.
- **Cada linha soma 1**, então o resultado é uma média (uma combinação convexa) e não uma soma que explode.

O script confirma que `v1 == v2`:

```
v1 == v2 ? True
(diferença máxima v1 vs v2: 3.24e-08)
```

Sobre esse 3.24e-08: os dois resultados não são *bit a bit* idênticos, e isso é esperado. Números `float32` têm precisão finita, e somar na ordem do loop dá um arredondamento ligeiramente diferente de somar via `matmul`. Por isso comparamos com `torch.allclose(..., atol=1e-6)`: em ponto flutuante, "igual" significa "igual dentro de uma tolerância". Guarde isso — no Capítulo 9 a precisão numérica vira o assunto principal.

4.5 v3 — a mesma média, via softmax

Agora um passo que parece só uma complicação, mas é **a chave** para a versão final. Em vez de escrever as frações à mão, vamos partir de **zeros**, marcar o futuro com `-infinito` e aplicar **softmax**:

```
tril = torch.tril(torch.ones(T, T))
wei3 = torch.zeros((T, T))
wei3 = wei3.masked_fill(tril == 0, float("-inf")) # proíbe ver o futuro
wei3 = F.softmax(wei3, dim=-1)
xbow3 = wei3 @ x
```

Por que isso dá exatamente a mesma média? Porque o softmax exponencia e normaliza: $\exp(0) = 1$ nas posições permitidas e $\exp(-\infty) = 0$ no futuro. Uma linha com `k` uns e o resto zeros, normalizada, vira $1/k$ em cada posição permitida — a média uniforme. E o script confirma: `v2 == v3 ? True`.

Mas agora olhe o que ganhamos: aqueles zeros iniciais eram um *chute* de que todas as posições importam igual. Se em vez de zeros colocarmos **pontuações calculadas a partir do conteúdo**, o softmax as transforma em pesos que somam 1 — e a média deixa de ser uniforme. É isso que a atenção faz.

4.6 v4 — self-attention de verdade: query, key e value

Cada posição gera **três** vetores diferentes a partir do seu conteúdo, por três camadas lineares distintas:

Vetor	Pergunta que ele responde
query (q)	"o que eu estou procurando?"
key (k)	"o que eu tenho a oferecer?"
value (v)	"o que eu de fato entrego, se você me escolher"

A afinidade entre a posição t e a posição i é o **produto escalar** $q[t] \cdot k[i]$: alto quando o que t procura casa com o que i oferece. Calculamos todas as afinidades de uma vez com uma `matmul`:

```
k = key(x)      # (B, T, head_size)
q = query(x)    # (B, T, head_size)
v = value(x)    # (B, T, head_size)

wei = q @ k.transpose(-2, -1)          # (B, T, T): todas as afinidades
wei = wei * head_size ** -0.5          # escala (Seção 7)
wei = wei.masked_fill(tril == 0, float("-inf")) # máscara causal
wei = F.softmax(wei, dim=-1)

out = wei @ v                          # a média ponderada dos values
```

Por que `k.transpose(-2, -1)`? Para multiplicar q por k e obter uma matriz $T \times T$, precisamos que a dimensão de `head_size` seja contraída. `transpose(-2, -1)` troca as duas últimas dimensões de k , de (B, T, hs) para (B, hs, T) ; aí a `matmul (B, T, hs) @ (B, hs, T)` resulta em (B, T, T) . Usamos índices negativos para não mexer na dimensão de batch.

E por que separar `key` de `value`? Porque "*como eu sou encontrado*" e "*o que eu entrego*" são coisas diferentes. A `key` serve para o casamento (o endereço); a `value` é o conteúdo que trafega. Separá-las dá liberdade ao modelo.

Rodando, os pesos da primeira sequência ficam assim (linha = quem olha, coluna = quem é olhado):

```
[[1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00]
 [0.52 0.48 0.00 0.00 0.00 0.00 0.00 0.00]
 [0.33 0.30 0.37 0.00 0.00 0.00 0.00 0.00]
 [0.23 0.20 0.27 0.30 0.00 0.00 0.00 0.00]
 ...
 [0.10 0.16 0.08 0.07 0.08 0.25 0.10 0.15]]
```

Note as três propriedades: **triangular** (nada do futuro), **cada linha soma 1**, e — o ponto crucial — os pesos **não são mais uniformes**. Na última linha, a posição 5 recebe 0.25 enquanto a 3 recebe 0.07: o modelo está *escolhendo* onde olhar.

4.7 Dois detalhes que fazem o mecanismo funcionar

4.7.1 A escala $1/\sqrt{\text{head_size}}$

Sem ela, o produto escalar $q \cdot k$ cresce com a dimensão (é a soma de `head_size` produtos). Pontuações grandes fazem o softmax **saturar**: a distribuição vira quase um *one-hot*, a atenção "trava" numa única posição e — pior — o gradiente do softmax saturado é quase zero, então o modelo para de aprender (o mesmo efeito da *tanh* saturada que vimos no Cap. 2).

O `attention.py` demonstra na prática:

```
sem escala (valores grandes): tensor([0.80, 0.00, 0.16, 0.03])
com escala (divididos):      tensor([0.33, 0.17, 0.27, 0.22])
```

Dividir por $\sqrt{\text{head_size}}$ mantém a variância das pontuações em torno de 1, independentemente do tamanho da cabeça. É por isso que o artigo original se chama "*Attention Is All You Need*" e a fórmula aparece como $\text{softmax}(QK^T/\sqrt{d})V$.

4.7.2 A máscara causal

Estamos treinando um modelo que **prevê o próximo** caractere. Se a posição t pudesse ver a posição $t+1$, ela veria **a própria resposta** — a loss de treino despencaria e o modelo não aprenderia nada útil, porque na hora de gerar texto o futuro não existe. Isso se chama **vazamento** (*data leakage*).

A máscara triangular garante que cada posição só veja o passado e a si mesma. Modelos como o GPT são chamados de **decoder-only causais** exatamente por causa disso. (No exercício E2 você remove a máscara e vê o estrago.)

4.8 Atenção não sabe ordem: o embedding posicional

Aqui está uma propriedade contra-intuitiva: **a atenção é invariante a permutações**. Ela calcula afinidades entre pares de posições, mas nada no mecanismo diz *qual* vem antes. Se você embaralhasse os tokens de entrada, os pesos acompanhariam o embaralhamento e o resultado seria o mesmo conjunto — *ana* e *naa* seriam indistinguíveis.

Para uma linguagem, isso é fatal: ordem é tudo. A solução é **injetar a posição na entrada**, somando ao embedding do token um embedding que depende de *onde* ele está:

```
self.token_emb = nn.Embedding(vocab_size, n_embd) # o que é o token
self.pos_emb    = nn.Embedding(block_size, n_embd) # onde ele está

tok = self.token_emb(idx) # (B, T, n_embd)
pos = self.pos_emb(torch.arange(T)) # (T, n_embd)
```

$$x = \text{tok} + \text{pos}$$

$$\# \text{ conteúdo} + \text{ posição}$$

Somar (em vez de concatenar) é o padrão: mantém a dimensão e funciona bem na prática, porque a rede aprende a separar as duas contribuições. Aqui as posições são **aprendidas**; o artigo original usava senos e cossenos fixos, e modelos modernos como o Llama usam **RoPE** (no apêndice do curso). No exercício E3 você remove o + pos e mede o prejuízo.

4.9 O resultado — e a descoberta honesta deste capítulo

O model.py treina um modelo de linguagem com **uma cabeça de atenção**, contexto de **8** caracteres (contra 3 do MLP) e ~11,4k parâmetros — praticamente os mesmos ~11,9k do MLP do Capítulo 3, para a comparação ser justa.

Rode e compare:

Modelo	Contexto	Parâmetros	Loss validação
Bigrama (Cap. 1)	1	729	~2,4
MLP (Cap. 3)	3	11.897	1,967
Atenção	3	11.103	2,193
Atenção	8	11.363	2,099
Atenção + feedforward	8	33.255	1,913

Leia essa tabela com atenção, porque ela contradiz o que você provavelmente esperava:

1. Mais contexto ajuda a atenção. Passando de 3 para 8 caracteres, a loss cai de 2,193 para 2,099. O mecanismo consegue usar o contexto extra — e, diferente do MLP, sem explodir o número de parâmetros.

2. Mas a atenção sozinha perde do MLP (2,099 contra 1,967), mesmo com o mesmo orçamento de parâmetros. Isso **não** é bug. É a lição central do capítulo.

Por quê? Olhe o que a atenção realmente faz: $out = w_{ei} @ v$. Dados os pesos, isso é uma **média ponderada** dos values — uma operação essencialmente **linear**. A atenção é excelente em **comunicar** (mover informação entre posições), mas ela não **processa** essa

informação: falta o "pensar" não-linear em cada posição, que é justamente o que o MLP do Capítulo 3 fazia bem com a sua camada oculta e a GELU.

3. A prova: ligue `USE_FEEDFORWARD = True` no `model.py`. Isso adiciona apenas uma pequena rede por posição (expande, GELU, volta) depois da atenção:

```
self.net = nn.Sequential(
    nn.Linear(dim, 4 * dim),
    nn.GELU(),
    nn.Linear(4 * dim, dim),
)
```

A loss cai para **1,913** — melhor que o MLP. Note que o modelo fica maior (33k parâmetros), então não é uma comparação de orçamento igual; o ponto é **direcional** e é o recado do capítulo:

Atenção = comunicação. Feedforward = computação. O Transformer é a combinação dos dois, empilhada em camadas. Nenhum dos dois sozinho é suficiente.

Esse é exatamente o desenho do bloco do Transformer, e é o Capítulo 5.

Os nomes gerados, para constar, já saem bem plausíveis (do modelo só com atenção):

```
josoni, jovaldimar, spatie, cemindray, luedie, josmar, deseni, alcindilo, ...
```

Repare que a loss e a "beleza" dos nomes não andam perfeitamente juntas — a loss é a métrica objetiva e é nela que confiamos para comparar modelos.

4.10 Resumo do capítulo

- A atenção resolve as limitações do MLP: o modelo **escolhe** onde olhar, com base no **conteúdo**, e o contexto cresce sem explodir os parâmetros.
- Construímos o mecanismo em 4 versões equivalentes: média num loop -> matmul com tril -> softmax com máscara -inf -> **query/key/value**.
- **softmax(QKT/√d)V**: afinidades por produto escalar, escaladas por $1/\sqrt{d}$ (evita saturação do softmax), mascaradas (proíbe ver o futuro), normalizadas, e usadas para ponderar os **values**.
- A atenção é **invariante a permutações** -> precisa de **embedding posicional**.
- Comparação com precisão de ponto flutuante exige **tolerância** (`atol`), porque `float32` não é exato.
- **A descoberta honesta:** atenção sozinha não bate o MLP. Ela **comunica** mas não **computa**. Atenção + feedforward bate. Isso motiva o Transformer.

4.10.1 O que vem no Capítulo 5

Temos as duas metades: comunicação (atenção) e computação (feedforward). O **Capítulo 05 — Transformer** junta as duas num **bloco**, adiciona **múltiplas cabeças** (várias relações em paralelo), **conexões residuais** e **layer normalization** — e empilha tudo em profundidade. O resultado é a arquitetura do GPT-2.

-> Antes de seguir, faça os exercícios.

4.11 Exercícios

Faça na ordem. Soluções comentadas em solucoes/ — **tente antes de olhar**.

4.11.1 E1 — Leitura de código (aquecimento)

Sem rodar, responda:

1. O que a matriz `tril` (triangular inferior) impede? O que aconteceria com o **treino** se o modelo pudesse ver o futuro?
2. Por que dividimos as pontuações por `sqrt(head_size)`?
3. Qual a diferença de papel entre **query**, **key** e **value**?

4.11.2 E2 — Sem a máscara causal (vazamento de dados)

No `model.py`, comente a linha do `masked_fill` (a máscara causal) e treine de novo.

1. A loss de treino cai mais rápido? Ela fica **artificialmente** boa?
2. Gere nomes com esse modelo. A qualidade acompanha a loss?
3. Explique por que remover a máscara é **trapaça**: o modelo passa a usar a resposta para prever a resposta.

4.11.3 E3 — Sem o embedding posicional

Remova o `+ pos` do `forward` (use apenas `tok`) e treine de novo.

1. A loss piora? Quanto?
2. **Por quê?** Sem posição, a atenção vê o contexto como um *conjunto* e não como uma *sequência* — ou seja, `ana` e `naa` ficariam indistinguíveis. Explique com suas palavras por que isso atrapalha na hora de prever o próximo caractere.

4.11.4 E4 — Sem a escala `sqrt(head_size)`

Remova o fator `* k.shape[-1]**-0.5`.

1. O que acontece com a loss no começo do treino?
2. Imprima os pesos de atenção (`wei`) de um batch. Eles ficam mais "concentrados" (perto de 0 e 1) do que com a escala?
3. Relacione com o experimento do fim do `attention.py`: softmax de valores grandes satura, e gradiente saturado quase não aprende.

4.11.5 E5 — Tamanho do contexto (`block_size`)

Treine com `block_size = 3, 8 e 16`.

1. Anote a loss de validação. Mais contexto sempre ajuda?
2. Para nomes (palavras curtas, ~7 letras), faz sentido um contexto de 16? O que o embedding posicional aprende nas posições que quase nunca são usadas?
3. Como o custo de computação da atenção cresce com o `block_size`? (Dica: a matriz de afinidades é $\tau \times \tau$.)

4.11.6 E6 — Inspeccionando o que o modelo olha (desafio)

Faça o modelo devolver também os pesos `wei` e, para um nome específico (ex.: o contexto `. . . . . a n a`), imprima a linha da última posição.

1. Em quais posições o modelo mais presta atenção ao prever o próximo caractere?
2. Você provavelmente vai achar **muito peso nos tokens de preenchimento** `.`, e não nas letras. Antes de chamar isso de bug, pense: o softmax **obriga** os pesos a somarem 1. Se o modelo não precisa trazer informação de nenhuma posição específica, onde ele "estaciona" esse peso? (Esse fenômeno tem nome em LLMs grandes: *attention sink*.)

A distribuição é claramente **não uniforme** — compare com o *bag of words* da Seção 3, onde todos os pesos eram iguais. O que isso mostra sobre o mecanismo?

Solução de referência: `solucoes/e6_inspeccionar_atencao.py`.

4.11.7 E7 — Multi-head attention (desafio, antecipa o Cap. 5)

Uma cabeça só aprende **um** tipo de relação. Implemente **multi-head attention**: crie `n_heads` cabeças com `head_size = n_embd // n_heads`, rode todas em paralelo e **concatene** as saídas.

1. Com o mesmo total de parâmetros, 4 cabeças pequenas vão melhor que 1 grande?
2. Por que ter várias cabeças pode ajudar? (Pense: uma cabeça olha a vogal anterior, outra o começo da palavra...)
3. Essa é exatamente a peça que abre o Capítulo 5 — o Transformer.

5 TRANSFORMER

Objetivo de aprendizagem: montar um **Transformer completo**, no estilo do **GPT-2**. Vamos juntar a atenção do Capítulo 4 com o feedforward, adicionar as três peças que faltam — **multi-head attention**, **conexões residuais** e **layer normalization** — e empilhar tudo em profundidade.

Pré-requisitos: Capítulos 1-4. Em especial a self-attention do Cap. 4 (esta aula é a continuação direta dela).

Arquivos:

- `layernorm.py` — LayerNorm do zero, verificada contra o PyTorch
- `transformer.py` — o Transformer completo, treinado nos nomes
- `exercicios.md` — exercícios

5.1 De onde partimos

O Capítulo 4 terminou com uma conclusão precisa: a atenção **comunica** (move informação entre posições) mas não **computa** (não processa essa informação de forma não-linear). Juntando atenção + feedforward, a loss caiu de 2,099 para 1,913.

Este capítulo transforma esse par em uma **arquitetura** de verdade. Faltam três peças, e nenhuma delas é decorativa — cada uma resolve um problema concreto que aparece quando tentamos empilhar camadas.

Peça	Problema que resolve
Multi-head attention	uma cabeça só aprende um tipo de relação
Conexões residuais	o gradiente se degrada ao atravessar muitas camadas
Layer normalization	as ativações saem de escala e o treino desestabiliza

5.2 Multi-head attention: várias relações em paralelo

Uma cabeça de atenção produz **um** conjunto de pesos — ou seja, aprende **um** critério de "onde olhar". Mas numa linguagem há vários critérios úteis ao mesmo tempo: a última consoante, a vogal anterior, o começo da palavra...

A solução é rodar **várias cabeças em paralelo** e concatenar o que cada uma trouxe:

```

class MultiHeadAttention(nn.Module):
    def __init__(self, n_head, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(n_head)])
        self.proj = nn.Linear(head_size * n_head, n_embd)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)  # (B, T, n_embd)
        return self.proj(out)

```

Dois detalhes importantes:

O truque da dimensão. Definimos `head_size = n_embd // n_head`. Com `n_embd = 64` e `n_head = 4`, cada cabeça tem dimensão 16, e a concatenação de 4×16 volta a 64. Ou seja: **quatro cabeças pequenas custam o mesmo que uma cabeça grande** — ganhamos diversidade de graça. Não é um trade-off de custo, é um trade-off de "uma relação rica" contra "várias relações mais simples", e na prática o segundo ganha.

A projeção final (`self.proj`). Depois da concatenação, cada cabeça ocupa a sua fatia do vetor, isolada das outras. A camada linear de projeção **mistura** essas fatias, deixando o modelo combinar o que as diferentes cabeças descobriram. Sem ela, as cabeças ficariam em compartimentos estanques.

5.3 Conexões residuais: um caminho livre para o gradiente

Esta é a ideia mais simples e mais poderosa do capítulo. Em vez de

```
x = sublayer(x)  # a camada SUBSTITUI o que veio antes
```

escrevemos

```
x = x + sublayer(x)  # a camada ACRESCENTA algo ao que veio antes
```

A diferença parece cosmética. Não é.

Na ida (forward): a sub-camada não precisa mais produzir a representação inteira — ela só precisa calcular uma **correção** ao que já estava lá. Aprender um ajuste é bem mais fácil que reconstruir tudo.

Na volta (backward): lembre do Capítulo 2 — a derivada de uma soma **distribui** o gradiente igualmente para as duas parcelas. Então o `x +` cria uma **rodovia** pela qual o gradiente flui da loss até as primeiras camadas **sem ser multiplicado por nada**. Sem esse caminho, o gradiente precisa atravessar todas as sub-camadas, sendo multiplicado a cada passo — e depois de muitas multiplicações ele encolhe até virar quase zero (o problema do *vanishing gradient*).

É por isso que residuais são o que **viabiliza a profundidade**. A ideia vem das ResNets (2015) e é o motivo de conseguirmos treinar redes com dezenas ou centenas de camadas.

E isso não é só teoria — nós **medimos**. A solução do exercício E2 (solucoes/e2_ablacoes.py) treina o mesmo modelo com e sem residuais, em duas profundidades:

Configuração	Loss validação	Penalidade por remover
3 blocos, residual ligado	1,903	—
3 blocos, residual desligado	2,197	+0,294
6 blocos, residual ligado	1,887	—
6 blocos, residual desligado	2,773	+0,886

Leia a coluna da direita: com 3 blocos, remover os residuais custa +0,29 de loss. Com 6 blocos, custa +0,89 — **três vezes mais**. A penalidade **cresce com a profundidade**, exatamente como a teoria do gradiente prevê. E note o detalhe mais revelador: **sem** residuais, ir de 3 para 6 blocos *piora* o modelo (2,197 -> 2,773) — mais profundidade vira um estorvo. **Com** residuais, ir de 3 para 6 melhora (1,903 -> 1,887).

Em uma frase: **sem conexões residuais, profundidade atrapalha; com elas, ajuda**. Foi essa peça de duas linhas que destravou o deep learning profundo.

5.4 Layer normalization

Ao empilhar camadas, as ativações tendem a sair de escala — crescem ou encolhem progressivamente, e o treino fica instável. A **LayerNorm** conserta isso normalizando cada vetor de ativações: subtrai a média e divide pelo desvio padrão.

Implementada do zero (veja layernorm.py):

```
class LayerNorm(nn.Module):
    def __init__(self, dim, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.gamma = nn.Parameter(torch.ones(dim))    # ganho aprendido
        self.beta = nn.Parameter(torch.zeros(dim))    # deslocamento aprendido

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        return self.gamma * (x - mean) / torch.sqrt(var + self.eps) + self.beta
```

Três observações:

- **dim=-1**: normalizamos ao longo das **features**, dentro de cada posição. Cada posição de cada sequência é normalizada com os seus próprios números.
- **gamma e beta** são aprendidos: se a normalização não for boa para alguma feature, a rede pode desfazê-la. Normalizar é uma *sugestão*, não uma imposição.
- **eps** evita divisão por zero quando a variância é minúscula.

Rodando `python layernorm.py`, confirmamos que a nossa implementação bate com a do PyTorch e vemos o efeito:

```
=== nossa LayerNorm vs nn.LayerNorm ===
resultados batem? True
diferença máxima: 4.77e-07

=== efeito da normalizacao ===
ANTES : media = +39.858 | desvio = 14.524
DEPOIS: media = -0.000 | desvio = 1.000
```

5.4.1 Por que "Layer" e não "Batch"?

A **BatchNorm** normaliza usando estatísticas do *batch* — ou seja, o resultado de um exemplo **depende dos outros exemplos** processados junto. Para um modelo de linguagem isso é um problema sério: na hora de gerar texto processamos uma sequência sozinha, e a saída não pode depender de quem estava no batch durante o treino.

A LayerNorm não tem esse problema, e o `layernorm.py` prova:

```
=== independencia do batch ===
processar sozinho == processar no batch? True
```

É por isso que Transformers usam LayerNorm.

5.5 O bloco: comunicação + computação

Agora juntamos tudo. Um **bloco** do Transformer é:

```
class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ff = FeedForward(n_embd)
        self.ln1 = LayerNorm(n_embd)
        self.ln2 = LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))    # comunicação entre posições
        x = x + self.ff(self.ln2(x))    # computação dentro de cada posição
        return x
```

Leia o forward como duas etapas: primeiro as posições **conversam** (atenção), depois cada uma **pensa** sozinha sobre o que ouviu (feedforward). Cada etapa vem embrulhada em LayerNorm (antes) e residual (a soma).

5.5.1 Pre-norm vs post-norm

Note que a LayerNorm vem **antes** da sub-camada ($\text{sublayer}(\ln(x))$), e não depois. O artigo original de 2017 fazia o contrário (*post-norm*: $\ln(x + \text{sublayer}(x))$), mas descobriu-se que o **pre-norm** treina de forma bem mais estável — porque assim o caminho residual fica completamente **limpo**, sem nenhuma normalização no meio. O gradiente atravessa a rede sem obstáculo algum. GPT-2 e praticamente todos os modelos modernos usam pre-norm, e é o que fazemos aqui.

5.6 O modelo completo

Empilhar blocos e coroar com uma norma final e a camada de saída:

```
class Transformer(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, n_embd)
        self.pos_emb = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head) for _ in range(n_layer)])
        self.ln_f = LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx):
        B, T = idx.shape
        x = self.token_emb(idx) + self.pos_emb(torch.arange(T))
        x = self.blocks(x)
        x = self.ln_f(x)
        x = x[:, -1, :]
        return self.lm_head(x)
```

Isto é um GPT. A arquitetura acima é, em estrutura, a mesma do GPT-2 — e a do GPT-3, e a da maioria dos LLMs em produção. A diferença entre este arquivo e um modelo de fronteira é **escala** (número de blocos, dimensão, tamanho do contexto, volume de dados) e um punhado de refinamentos de engenharia. Não é outra ideia; é a mesma ideia, muito maior.

Para comparar: nosso modelo tem **153.499 parâmetros**, 3 blocos e 4 cabeças. O GPT-2 "small" tem 124 **milhões**, 12 blocos e 12 cabeças, com $n_embd = 768$ e contexto de 1024 tokens. Mesmo desenho.

5.7 Resultados

Rodando `python transformer.py` (cerca de 6 minutos na CPU):

```

parametros: 153499 (3 blocos, 4 cabecas, n_embd=64)
step      0/15000 | loss 3.5052 | 0s
step  1500/15000 | loss 2.4018 | 34s
...
treino concluido em 355s

loss treino      = 1.7910
loss validacao   = 1.8114
loss teste       = 1.8199

```

Comparando com todos os modelos do curso, na **mesma tarefa e mesma métrica**:

Capítulo	Modelo	Contexto	Parâmetros	Loss validação
01	Bigrama (contagem)	1	729	~2,4
03	MLP	3	11.897	1,967
04	Atenção (1 cabeça)	8	11.363	2,099
04	Atenção + feedforward	8	33.255	1,913
05	Transformer	8	153.499	1,811

O Transformer é o **melhor modelo do curso até aqui**, e a trajetória completa é bonita: saímos de 2,4 (contando pares de letras) para 1,811 — construindo cada peça à mão.

Uma ressalva de honestidade: este modelo tem 153 mil parâmetros contra 33 mil do "atenção + feedforward" do Capítulo 4, então **não é uma comparação de orçamento igual**. Parte do ganho vem simplesmente de ser maior. O que a tabela mostra com segurança é que a arquitetura **escala bem** — colocar mais capacidade nesse desenho converte em ganho real, e é exatamente essa propriedade que permite chegar aos modelos de fronteira. Se você quiser a comparação controlada, o exercício E4 varia profundidade e cabeças mantendo o resto fixo.

Note também que treino (1,791) e validação (1,811) continuam próximos: o modelo cresceu bastante, mas o dataset de 64 mil nomes ainda o mantém honesto — sem *overfitting* relevante.

E os nomes gerados ficaram convincentes:

```

rilme, jandir, ludo, raiude, natta, waldes, valdinia, makyane, nerandila

```

Compare com os do bigrama do Capítulo 1 (cexzma, zktahwelo, iczisqctoujkwptedo) — a diferença é evidente.

5.8 Resumo do capítulo

- **Multi-head attention:** várias cabeças de dimensão $n_embed // n_head$ em paralelo, concatenadas e **projetadas** — várias relações pelo mesmo custo de uma.
- **Conexões residuais** ($x + sublayer(x)$): a sub-camada aprende uma *correção*, e o gradiente ganha um caminho direto até o início da rede. É o que viabiliza profundidade.
- **LayerNorm:** normaliza cada posição ao longo das features, com γ/β aprendidos. Independente do batch (diferente da BatchNorm) — essencial para geração.
- **Bloco** = comunicação (atenção) + computação (feedforward), cada uma com **pre-norm** e residual. O pre-norm mantém o caminho residual limpo.
- **Transformer** = embeddings + N blocos + norma final + lm_head . **Isto é a arquitetura do GPT-2**; o resto é escala.
- Resultado: **1,811** de loss — o melhor do curso, contra 1,967 do MLP e ~2,4 do bigrama.

5.8.1 O que vem no Capítulo 6

Temos a arquitetura. Mas ainda estamos alimentando o modelo com **um caractere por token**, o que é um desperdício: sequências ficam longas e o modelo gasta capacidade aprendendo a montar palavras a partir de letras. Modelos reais usam **tokens de sub-palavra**. No **Capítulo 06 — Tokenization** vamos construir um tokenizador **BPE** (*byte pair encoding*) do zero, o mesmo algoritmo que o GPT usa.

-> Antes de seguir, faça os exercícios.

5.9 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

Aviso de tempo: os exercícios que treinam modelos levam alguns minutos cada na CPU. Para experimentar mais rápido, reduza `max_steps` (ex.: 3000) — as *comparações relativas* continuam válidas, só não compare com os números absolutos da apostila.

5.9.1 E1 — Leitura de código (aquecimento)

Sem rodar, responda:

1. No Block, por que escrevemos `x = x + self.sa(self.ln1(x))` e não `x = self.sa(self.ln1(x))`? O que a soma acrescenta?
2. Por que `head_size = n_embd // n_head`? O que aconteceria com o custo se cada cabeça tivesse `head_size = n_embd`?
3. Qual a diferença entre o que a **atenção** faz e o que o **feedforward** faz dentro de um bloco?

5.9.2 E2 — Sem as conexões residuais

No Block, troque as duas linhas por `x = self.sa(self.ln1(x))` e `x = self.ff(self.ln2(x))` (sem o `x +`).

1. A loss final piora? Quanto?
2. Aumente `n_layer` para 6 **sem** os residuais e depois **com**. A diferença cresce com a profundidade?
3. Explique o papel do residual como "caminho livre" para o gradiente.

Solução de referência: `solucoes/e2_ablacoes.py`.

5.9.3 E3 — Sem LayerNorm

Remova as LayerNorms do bloco (aplique as sub-camadas direto em `x`).

1. O treino fica instável? A loss oscila mais entre passos?
2. Tente também aumentar a learning rate para `3e-3` sem LayerNorm. O que acontece?
3. Por que normalizar as ativações ajuda a estabilizar o treino?

5.9.4 E4 — Profundidade e número de cabeças

Treine com:

- `n_layer = 1, 3 e 6` (mantendo `n_head = 4`)
- `n_head = 1, 4 e 8` (mantendo `n_layer = 3`)

1. Anote parâmetros e loss de validação em cada caso.
2. Mais profundidade sempre ajuda? Onde começa o retorno decrescente?

3. Com o **mesmo** `n_embd`, mudar `n_head` altera o número de parâmetros? Por quê? (Dica: olhe `head_size = n_embd // n_head`.)

5.9.5 E5 — Uma cabeça grande vs várias pequenas

Compare `n_head = 1` (uma cabeça de 64) com `n_head = 8` (oito de 8), ambos com `n_embd = 64` — ou seja, praticamente o mesmo número de parâmetros.

1. Qual vai melhor?
2. Por que várias cabeças pequenas podem ganhar de uma grande? (Pense: cada cabeça pode se especializar numa relação diferente.)
3. Existe um limite? O que acontece com `n_head = 64` (cabeças de dimensão 1)?

5.9.6 E6 — Dropout (regularização, desafio)

O GPT-2 usa **dropout**: durante o treino, zera aleatoriamente uma fração das ativações, o que dificulta a memorização. Adicione `nn.Dropout(0.1)` na saída da projeção da atenção e no fim do feedforward.

1. Com nosso dataset (64 mil nomes), treino e validação já andam juntos. O dropout ajuda, atrapalha ou é indiferente aqui?
2. Em que situação o dropout seria claramente útil? (Releia a lição do Cap. 3 sobre *overfitting*.)
3. Por que o dropout deve ser desligado na avaliação (`model.eval()`)?

5.9.7 E7 — Conte os parâmetros (desafio)

O código informa **153.499** parâmetros. Deduza esse número somando as partes: embeddings (token e posição), por bloco (atenção: `k,q,v,proj`; feedforward: duas camadas; duas LayerNorms), a LayerNorm final e a `lm_head`.

1. Escreva a fórmula em função de `vocab_size`, `block_size`, `n_embd`, `n_head` e `n_layer`.
2. Confira que ela reproduz 153.499.
3. Qual parte domina o total? Se você dobrar `n_embd`, o número de parâmetros dobra ou quadruplica?

6 TOKENIZATION (BPE DO ZERO)

Objetivo de aprendizagem: construir um tokenizador **BPE** (*byte pair encoding*) do zero — o mesmo algoritmo que o GPT usa para transformar texto em tokens. No caminho, entender **Unicode** e **UTF-8**, por que tokenizadores sérios começam pelos **bytes**, e descobrir (medindo) por que escrever em português custa mais caro numa API de LLM.

Pré-requisitos: Capítulos 1-5. Este capítulo não usa PyTorch nem treina redes — é Python puro, e roda em segundos. Mas é uma das peças mais consequentes de um LLM.

Arquivos:

- `unicode_utf8.py` — os fundamentos: caractere -> code point -> bytes
- `bpe.py` — o tokenizador BPE completo (treino, encode, decode)
- `exercicios.md` — exercícios

6.1 O problema

Até aqui usamos **um token por caractere**: 27 tokens (26 letras + fronteira). É simples e funciona, mas tem um custo sério. Considere as duas alternativas óbvias:

Abordagem	Vocabulário	Problema
Um token por caractere	~27 (ou ~150 mil, com Unicode)	sequências longuíssimas
Um token por palavra	centenas de milhares	palavras novas viram "desconhecido"

Por que sequências longas são ruins? Lembre do Capítulo 4: a atenção compara todas as posições com todas as outras, então o custo cresce com T^2 no tamanho do contexto. Se cada letra é um token, "informação" gasta 10 posições. Dobrar o número de tokens por texto **quadruplica** o custo da atenção — e reduz quanto texto real cabe na janela de contexto.

Por que vocabulário de palavras é ruim? Além do tamanho, existe o problema de palavras nunca vistas (*out-of-vocabulary*). O que o modelo faz com "criptomoeda" se ela não estava no vocabulário? E com um nome próprio novo? E com uma palavra em japonês?

O BPE resolve os dois de uma vez, com uma ideia simples: **comece pelos bytes e vá juntando os pares mais frequentes**.

6.2 Antes do BPE: como texto vira número

Precisamos de três níveis bem claros. Rode `python unicode_utf8.py`.

6.2.1 Nível 1 — code point

O **Unicode** atribui um número (*code point*) a cada caractere existente:

```
'a'          -> code point    97   (U+0061)
'é'          -> code point    233  (U+00E9)
chr(26085)   -> code point  26085  (U+65E5)   # ideograma japonês
chr(128640)  -> code point 128640  (U+1F680)  # emoji de foguete
```

Nota sobre esta apostila: os dois últimos exemplos aparecem como `chr(...)` porque as fontes deste PDF não cobrem ideogramas nem emoji. No terminal, ao rodar `unicode_utf8.py`, você vê os caracteres de verdade. A limitação é da fonte do documento, não do código — e é, ela mesma, um exemplo do assunto do capítulo.

São ~150 mil caracteres definidos. Um token por caractere daria um vocabulário gigantesco — e ainda assim incompleto, porque o Unicode cresce.

6.2.2 Nível 2 — bytes (UTF-8)

Code points são conceitos; para gravar em memória usamos uma **codificação**. O UTF-8 usa um número **variável** de bytes:

```
'a'          -> 1 byte(s): [97]
'é'          -> 2 byte(s): [195 169]
chr(26085)   -> 3 byte(s): [230 151 165]   # ideograma japonês
chr(128640)  -> 4 byte(s): [240 159 154 128] # emoji
```

6.2.3 O imposto do português

Aqui aparece um fato que nos afeta diretamente: **caracteres acentuados custam 2 bytes**.

acao	4 bytes		ação	6 bytes (+2)
informacao	10 bytes		informação	12 bytes (+2)
coracao	7 bytes		coração	9 bytes (+2)

A frase "A ação começa às três horas." tem 28 caracteres mas **33 bytes**. Guarde isso — vamos medir a consequência prática na Seção 7.

6.2.4 Nível 3 — por que começar pelos bytes

Existem apenas **256** valores possíveis de byte. Isso dá um vocabulário base que é ao mesmo tempo **pequeno** e **completo**: qualquer texto, de qualquer idioma, mais emoji, mais código-fonte, é representável como bytes. **Nunca existe um token "desconhecido"**.

Essa garantia é o alicerce do BPE.

6.3 O algoritmo BPE

A ideia inteira em três linhas:

1. Comece com um token por byte (256 tokens).
2. Encontre o **par de tokens vizinhos mais frequente** no corpus.
3. Funda esse par num **token novo**. Repita.

Um exemplo à mão, com o texto "aa ab aa ab":

inicial:	a a _ a b _ a a _ a b	o par ('a', 'a') aparece 2x -> vira o token X
depois:	X _ a b _ X _ a b	o par ('a', 'b') aparece 2x -> vira o token Y
depois:	X _ Y _ X _ Y	nada mais se repete

Cada fusão **encurta** a sequência e **acrescenta** um token ao vocabulário. Você controla o quanto quer disso pelo `vocab_size`.

6.3.1 As duas funções que fazem tudo

```
def get_stats(ids):
    """Conta quantas vezes cada par de tokens vizinhos aparece."""
    counts = {}
    for pair in zip(ids, ids[1:]):
        counts[pair] = counts.get(pair, 0) + 1
    return counts

def merge(ids, pair, new_id):
    """Substitui toda ocorrência de `pair` pelo token `new_id`."""
    out = []
    i = 0
    while i < len(ids):
        if i < len(ids) - 1 and ids[i] == pair[0] and ids[i + 1] == pair[1]:
            out.append(new_id)
            i += 2          # salta os DOIS tokens fundidos
        else:
            out.append(ids[i])
            i += 1
    return out
```

O `i += 2` importa: depois de fundir um par, pulamos os dois elementos consumidos. Sem isso, em "aaa" a mesma letra do meio participaria de duas fusões.

6.3.2 O laço de treino

```
ids = list(text.encode("utf-8"))          # começa nos bytes crus
self.vocab = {i: bytes([i]) for i in range(256)}

for k in range(vocab_size - 256):
    stats = get_stats(ids)
    pair = max(stats, key=stats.get)      # o par mais frequente
    new_id = 256 + k
    ids = merge(ids, pair, new_id)
    self.merges[pair] = new_id
    self.vocab[new_id] = self.vocab[pair[0]] + self.vocab[pair[1]]
```

Note a última linha: o significado de um token novo é a **concatenação dos bytes dos seus dois pais**. É assim que o vocabulário se constrói recursivamente — um token pode ser feito de tokens que também foram fundidos antes.

6.4 encode e a importância da ordem

Para tokenizar um texto novo, aplicamos as fusões aprendidas. Mas **na ordem em que foram aprendidas**:

```
def encode(self, text):
    ids = list(text.encode("utf-8"))
    while len(ids) >= 2:
        stats = get_stats(ids)
        # entre os pares presentes, escolhe o que foi aprendido MAIS CEDO
        pair = min(stats, key=lambda p: self.merges.get(p, float("inf")))
        if pair not in self.merges:
            break
        ids = merge(ids, pair, self.merges[pair])
    return ids
```

Por que a ordem é obrigatória? Porque as fusões são **dependentes**: a fusão nº 7 do nosso treino foi ('on', '\n') -> 'on\n', e o token 'on' só existe porque a fusão nº 5 o criou. Aplicar a nº 7 antes da nº 5 seria impossível — o token nem existiria. O `min(...)` com o índice da fusão garante essa disciplina. (No exercício E4 você quebra isso de propósito e observa o resultado.)

O decode é o caminho inverso, e tem um detalhe:

```
def decode(self, ids):
    bs = b"".join(self.vocab[i] for i in ids)
    return bs.decode("utf-8", errors="replace")
```

O `errors="replace"` é necessário porque **um token pode conter meio caractere**. Se 'é' são os bytes [195, 169] e o tokenizador nunca aprendeu a juntá-los, cada byte é um token — e isolado, nenhum deles é UTF-8 válido. Sem o `errors="replace"`, o programa quebraria.

6.5 Rodando: o que o tokenizador aprendeu

Rodando `python bpe.py` (leva ~8 segundos, com `vocab_size = 512` sobre 150 mil caracteres de nomes):

```
fusao    1: (97, 110) -> 256 ('an'), 4608x
fusao    2: (97, 10)  -> 257 ('a\n'), 4239x
fusao    3: (101, 10) -> 258 ('e\n'), 3878x
fusao    4: (101, 108) -> 259 ('el'), 3405x
fusao    5: (111, 110) -> 260 ('on'), 2610x

bytes originais : 150000
tokens depois   : 66936
```

```
taxa de compressao: 2.24x
```

As fusões contam uma história. 'an' é a primeira porque é a sequência mais comum em nomes brasileiros. Várias fusões iniciais terminam em \n ('a\n', 'e\n', 'on\n') porque nosso arquivo tem **um nome por linha** — o modelo está aprendendo *terminações de nome*, e o fim de linha faz parte do padrão.

E os tokens mais longos que ele criou são reveladores:

```
token 334 = 'ilson\n'
token 348 = 'erson\n'
token 374 = 'isson\n'
token 412 = 'ilton\n'
token 463 = 'ilane\n'
token 362 = 'iana\n'
```

Ninguém ensinou morfologia ao algoritmo. Ele descobriu sozinho, apenas contando pares, que -ilson, -erson, -ilton e -iana são sufixos produtivos de nomes brasileiros. É estatística pura virando estrutura linguística.

6.5.1 A propriedade que não pode falhar

Um tokenizador precisa ser **reversível**: `decode(encode(x)) == x`, sempre, para qualquer entrada. Testamos casos difíceis de propósito:

```
[OK ] 'maria eduarda'          13 bytes ->  7 tokens (1.86x)
[OK ] 'josé da conceição'      20 bytes -> 16 tokens (1.25x)
[OK ] 'A ação começa às três.' 27 bytes -> 27 tokens (1.00x)
[OK ] 'Olá! ' + japones + emoji 17 bytes -> 17 tokens (1.00x)
[OK ] ''                        0 bytes ->  0 tokens
[OK ] 'xyzkw'                   5 bytes ->  5 tokens (1.00x)

TODOS os round-trips passaram? True
```

Repare em duas coisas. Primeiro: **japonês e emoji funcionam**, embora não existisse nada parecido no treino — é a garantia dos bytes em ação. Segundo: a compressão nesses casos é 1.00x, ou seja, **nenhuma**. O tokenizador não comprime o que nunca viu; ele só não quebra.

6.6 Compressão só existe dentro do domínio

Texto	Compressão
Nomes usados no treino	2,24x
Nomes novos (não vistos)	2,17x
Frase em português corrido	1,20x
Japonês + emoji	1,00x

A compressão se sustenta em nomes novos (2,17x contra 2,24x) — ele aprendeu o *padrão*, não decorou a lista. Mas cai para 1,20x numa frase comum, porque ele foi treinado em **nomes**, não em prosa.

Regra prática: tokenizador e dados andam juntos. Um tokenizador é um modelo estatístico do seu corpus — treiná-lo no domínio errado desperdiça vocabulário.

6.7 O imposto do português, medido

Aqui a teoria vira dinheiro. Veja como a frase "A informação sobre a ação de coração está na página." é fatiada pelo nosso tokenizador de nomes:

```
A |   | in | f | or | ma | <0xC3> | <0xA7> | <0xC3> | <0xA3> | o |   | s | o | b | re | ...
```

São **50 tokens** para 60 bytes, e **16 deles são fragmentos de byte** (<0xNN>) — todos vindos de caracteres acentuados. O *ção* sozinho consome 5 tokens (<0xC3> <0xA7> <0xC3> <0xA3> o) para representar 3 caracteres.

A solução do exercício E5 treina um segundo tokenizador, do **mesmo tamanho**, em texto português de verdade (as próprias apostilas deste curso). Ele aprende:

```
token 371 = 'ação '
token 317 = 'ção '
token 471 = 'são '
token 485 = 'não '
token 287 = 'ão '
```

E a mesma frase passa a ser fatiada assim:

```
A | in | form | ação | so | b | r | e | a | ação | de | c | or | ação | est | á | ...
```

25 tokens em vez de 50. Medindo em três frases:

Tokenizador	Total de tokens
-------------	-----------------

Treinado em nomes (sem acento)	135
Treinado em português	64

53% menos tokens para o mesmo texto, com vocabulário do mesmo tamanho.

Isso tem consequência direta: APIs de LLM cobram **por token**, e a janela de contexto é medida **em tokens**. Um tokenizador treinado majoritariamente em inglês — como os dos modelos comerciais — faz o português custar mais caro e ocupar mais contexto pelo mesmo conteúdo. Não é impressão: é aritmética de bytes.

6.8 Resumo do capítulo

- Tokenizar é escolher a **unidade** do modelo. Caracteres dão sequências longas (custo T^2 na atenção); palavras dão vocabulário enorme e o problema de palavras novas.
- **Unicode** dá um número a cada caractere; **UTF-8** grava esse número em 1-4 bytes. Acentos custam 2 bytes — o "imposto do português".
- Começar pelos **bytes** (256 tokens) garante que **qualquer** texto é representável: nunca há "desconhecido".
- **BPE**: funda repetidamente o par de tokens vizinhos mais frequente. `get_stats` + `merge` são o algoritmo inteiro.
- O encode deve aplicar as fusões **na ordem em que foram aprendidas**, porque elas são dependentes entre si.
- decode precisa de `errors="replace"`: um token pode conter meio caractere.
- **Reversibilidade** (`decode(encode(x)) == x`) é obrigatória — verificamos com acentos, japonês, emoji e string vazia.
- Compressão medida: **2,24x** no domínio, **2,17x** fora dele, **1,00x** em texto totalmente estranho. Tokenizador é um modelo do corpus.
- Treinar o tokenizador no idioma certo economizou **53% dos tokens** — com impacto direto em custo de API e contexto útil.

6.8.1 O que vem no Capítulo 7

Já temos arquitetura (Cap. 5) e entrada (Cap. 6). Falta treinar **bem**. No **Capítulo 07 — Optimization** vamos olhar o que até agora tratamos com valores "razoáveis" escolhidos na mão: **inicialização** dos pesos, o otimizador **AdamW** por dentro, agendamento da **learning rate** e *gradient clipping*. É a diferença entre um treino que funciona e um treino que funciona bem.

-> Antes de seguir, faça os exercícios.

6.9 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

6.9.1 E1 — Unicode e UTF-8 (aquecimento)

Sem rodar código:

1. Quantos **bytes** ocupa "pão" em UTF-8? E "pao"? Por quê?
2. `ord("ç")` vale 231. Isso significa que "ç" cabe em 1 byte? Explique.
3. Por que um tokenizador que começa pelos **bytes** nunca encontra um caractere "desconhecido"?

6.9.2 E2 — O tamanho do vocabulário

Rode `bpe.py` com `VOCAB_SIZE = 300, 512 e 1024`.

1. Como muda a **taxa de compressão** em cada caso?
2. O ganho é linear? Onde começa o retorno decrescente?
3. Qual o custo de um vocabulário maior? (Pense na camada de saída do modelo: a `lm_head` do Capítulo 5 tem `n_embd × vocab_size` parâmetros.)

6.9.3 E3 — As primeiras fusões

Olhe a lista de fusões que o `bpe.py` imprime (as 10 primeiras).

1. Por que `('a', 'n') -> 'an'` é a primeira? O que isso diz sobre nomes brasileiros?

2. Várias fusões iniciais terminam em `\n` (fim de linha), como `'a\n'` e `'on\n'`. Por quê? O que isso revela sobre o formato do nosso arquivo de treino?
3. Entre os tokens mais longos aparecem `'ilson'`, `'erson'`, `'ilton'`. O tokenizador descobriu **sufixos de nomes** sem ninguém ensinar. Como?

6.9.4 E4 — A ordem das fusões importa

No método `encode`, trocamos a escolha do par por `min(stats, key=...)` usando a ordem de aprendizado. Experimente trocar por uma escolha arbitrária (por exemplo, o primeiro par encontrado que esteja em `self.merges`).

1. O `decode(encode(x)) == x` continua valendo?
2. A compressão fica pior? Por quê a ordem de aprendizado é a ordem correta?
3. Dica: uma fusão tardia pode usar um token criado por uma fusão anterior. O que acontece se você aplicar a tardia primeiro?

6.9.5 E5 — O imposto do português (importante)

O `bpe.py` mostra que `"A informação sobre a ação..."` é fatiada com 16 tokens de fragmentos de byte (`<0xNN>`), porque o tokenizador foi treinado em nomes **sem** acento.

1. Treine um BPE **em texto português com acentos** e verifique se ele aprende `'ção'` (ou `'ão'`) como um token só. Use como corpus os próprios `README.md` do curso.
2. Compare a contagem de tokens da mesma frase nos dois tokenizadores.

Isso tem consequência prática real: APIs de LLM cobram **por token**. Se o tokenizador foi treinado majoritariamente em inglês, escrever em português custa mais. Estime esse sobrecusto a partir dos seus números.

Solução de referência: `solucoes/e5_bpe_portugues.py`.

6.9.6 E6 — Tokens fora do domínio

Note que o caso com japonês e emoji (veja `bpe.py`) teve compressão 1.00x — nenhuma.

1. Por que o tokenizador não comprime nada nesse caso?

2. Ele ainda consegue **representar** o texto? (Rode e confirme.) Por que isso é uma garantia importante?
3. O que aconteceria com um tokenizador baseado em **palavras** ao receber uma palavra em japonês?

6.9.7 E7 — Integrando com o modelo (desafio, conecta ao Cap. 5)

Nosso Transformer do Capítulo 5 usa 27 tokens (um por letra). Adapte-o para usar o tokenizador BPE deste capítulo.

1. O que muda em `vocab_size` e, por consequência, no número de parâmetros?
2. Com tokens maiores, cada posição carrega mais informação. Um `block_size` de 8 passa a cobrir mais texto — quanto mais, aproximadamente, dada a taxa de compressão?
3. A loss fica comparável à do Capítulo 5? **Cuidado:** ela **não** é diretamente comparável. Explique por que uma loss por *token* com vocabulário diferente não pode ser comparada diretamente. (Dica: prever entre 27 opções e prever entre 512 é uma tarefa de dificuldade diferente.)

7 OPTIMIZATION

Objetivo de aprendizagem: abrir a caixa dos valores que até agora escolhemos "na mão". Vamos entender **inicialização** de pesos, implementar o **AdamW do zero** (e verificar contra o PyTorch), e aplicar **warmup + cosine decay** e **gradient clipping** — medindo quanto se ganha **sem tocar na arquitetura**.

Pré-requisitos: Capítulos 1-6. Especialmente o gradient descent do Cap. 2 e o Transformer do Cap. 5, que é o modelo que vamos treinar melhor aqui.

Arquivos:

- `initialization.py` — de onde vem o $(5/3)/\sqrt{\text{fan_in}}$ do Capítulo 3
- `optimizers.py` — SGD -> momentum -> **AdamW do zero**, verificado
- `train_tuned.py` — baseline vs afinado, comparação controlada
- `exercicios.md` — exercícios

7.1 O que ficou pendente

Ao longo do curso deixei três dívidas explícitas:

Onde	O que apareceu sem explicação
Cap. 3	<code>w1 = randn(...)</code> * $(5/3)$ / <code>fan_in**0.5</code> — de onde vem esse $5/3$?
Cap. 3	<code>lr = 0.1 if step < 15000 else 0.01</code> — por que decair?
Cap. 5	<code>torch.optim.AdamW(...)</code> — o que ele faz por dentro?

Este capítulo paga as três. E no fim mede o efeito prático: **a mesma arquitetura do Capítulo 5, treinada melhor, chega onde?**

7.2 Inicialização: o sinal precisa atravessar a rede

Uma rede profunda é uma pilha de multiplicações. Cada camada multiplica a escala do sinal por algum fator. Se esse fator for consistentemente menor que 1, o sinal **encolhe** a cada camada até desaparecer; se for maior que 1, ele **cresce** até saturar ou explodir.

Rodando `python initialization.py`, medimos o desvio padrão das ativações ao longo de 8 camadas (tanh, dimensão 256), variando só o **ganho** da inicialização $\text{randn} * \text{ganho} / \sqrt{\text{fan_in}}$:

ganho	c1	c2	c4	c6	c8
0,50	0,415	0,201	0,049	0,012	0,003
1,00	0,628	0,486	0,358	0,295	0,254
5/3	0,759	0,694	0,658	0,650	0,653
3,00	0,863	0,840	0,837	0,836	0,838

Leia linha por linha:

- **ganho 0,50** — o sinal morre. Na oitava camada, o desvio padrão é 0,003: as camadas finais recebem praticamente ruído zero, e o gradiente que volta é minúsculo.
- **ganho 1,00** — ainda encolhe (0,628 -> 0,254). Por quê? Porque a tanh **comprime**: $|\tanh(x)| < |x|$ sempre. Ganho 1 não compensa essa perda.
- **ganho 5/3** — estável (0,759 -> 0,653). Este é exatamente o valor que cancela a compressão da tanh. **É o 5/3 do Capítulo 3**. Não era mágica: é o *ganho de Kaiming* tabelado para essa ativação.
- **ganho 3,00** — as ativações grudam perto de 0,84, ou seja, a tanh **saturou**. Lembre do Capítulo 2: tanh saturada tem derivada ~ 0 . O gradiente morre por um caminho diferente, mas morre.

7.2.1 Por que $\sqrt{\text{fan_in}}$?

Um neurônio soma `fan_in` produtos independentes. A variância de uma soma de n termos independentes cresce com n , então o **desvio padrão** cresce com $\sqrt{n}$. Dividir por $\sqrt{\text{fan_in}}$ cancela exatamente esse crescimento — e por isso a inicialização funciona igual para camadas de qualquer tamanho.

7.2.2 O ganho depende da ativação

Ativação	ganho	std na camada 1	std na camada 8
tanh	1,000	0,627	0,255
tanh	1,667 (5/3)	0,758	0,653
relu	1,000	0,584	0,056
relu	1,414 ($\sqrt{2}$)	0,825	0,920

A ReLU zera metade dos valores, cortando metade da variância; $\sqrt{2}$ recompõe. A regra geral:

```
W ~ randn * ganho /  $\sqrt{\text{fan\_in}}$       # inicialização de Kaiming
```

7.2.3 E por que o Capítulo 5 funcionou sem esse cuidado?

Boa pergunta — e a resposta é a **LayerNorm**. Ela re-normaliza as ativações a cada bloco, então o erro de escala não se acumula com a profundidade. Medindo:

ganho	std na camada 8 sem LayerNorm	com LayerNorm
0,50	0,0030	0,4151
1,00	0,2548	0,6271
3,00	0,8381	0,8634

Com LayerNorm, até o ganho catastrófico de 0,50 sobrevive. Isso não dispensa inicializar bem — o começo do treino ainda melhora —, mas explica por que o Transformer do Capítulo 5 treinou sem nenhum ajuste especial. A LayerNorm estava fazendo esse trabalho.

7.3 Da SGD ao AdamW

7.3.1 O problema: uma learning rate não serve para todos

Imagine dois parâmetros: um cujo gradiente típico é 10, outro cujo gradiente típico é 0,001. Com uma learning rate única, você escolhe entre dois males: pequena o bastante para o primeiro não divergir (e então o segundo praticamente não anda), ou grande o bastante para o segundo andar (e então o primeiro explode).

O `optimizers.py` monta esse cenário de propósito (uma loss com termos de escalas 1000x diferentes) e mede quantos passos cada otimizador precisa:

SGD	: nao chegou em 3000	termo grande 0.0e+00	termo pequeno 5.1e-03
SGD + momentum	: nao chegou em 3000	termo grande 0.0e+00	termo pequeno 5.1e-03
AdamW (nosso)	: 711 passos	termo grande 9.4e-04	termo pequeno 2.6e-14

Olhe as duas últimas colunas. As duas versões de SGD resolvem o termo **grande** (chega a zero) e **empacam** no pequeno — por isso param no mesmo lugar. O gargalo não é o momentum: é a learning rate única. O AdamW resolve os dois na mesma corrida.

7.3.2 A escada, degrau por degrau

SGD — anda na direção do gradiente:

```
p -= lr * p.grad
```

Momentum — acumula uma "velocidade". Se o gradiente aponta sempre para o mesmo lado, a velocidade cresce; se ele oscila, as oscilações se cancelam:

```
v = beta * v + (1 - beta) * p.grad
p -= lr * v
```

Adam — a ideia nova: guardar também a média móvel do gradiente **ao quadrado** (v), que estima *quão grande o gradiente daquele parâmetro costuma ser*, e **dividir** o passo por isso. O resultado é um passo cujo tamanho efetivo fica na escala da learning rate, qualquer que seja a escala do gradiente:

```
m = beta1 * m + (1 - beta1) * g          # "para onde ir"
v = beta2 * v + (1 - beta2) * g**2        # "quão grande costuma ser"
p -= lr * m_hat / (sqrt(v_hat) + eps)
```

7.3.3 A correção de viés

m e v começam em **zero**. No primeiro passo, $m = (1 - \beta_1) \cdot g = 0,1 \cdot g$ — dez vezes menor que o gradiente real. As médias móveis **subestimam** no início. A correção compensa exatamente isso:

```
m_hat = m / (1 - beta1 ** t)      # t = número do passo
v_hat = v / (1 - beta2 ** t)
```

No passo 1 com $\beta_1 = 0,9$, o divisor é $1 - 0,9 = 0,1$, o que multiplica m por 10 e recupera a escala. Conforme t cresce, $\beta_1 t \rightarrow 0$ e a correção desaparece sozinha.

7.3.4 O "W": weight decay desacoplado

Weight decay é a regularização L2 que já vimos no Capítulo 1: empurrar os pesos para zero. O Adam original somava esse termo **ao gradiente** — e aí ele passava pela divisão por $\sqrt{v}$, o que distorce a regularização (parâmetros com gradiente grande recebiam menos decay). O **AdamW** aplica o decay **direto no parâmetro**, fora do caminho adaptativo:

```
if self.wd != 0:
    p.mul_(1 - self.lr * self.wd)      # decay direto, desacoplado
# ... só depois o passo adaptativo
```

7.3.5 A verificação

Nossa implementação bate com a do PyTorch:

```
=== nosso AdamW vs torch.optim.AdamW (20 passos) ===
passo 1: diferenca maxima = 0.00e+00
passo 5: diferenca maxima = 2.98e-08
passo 20: diferenca maxima = 8.94e-08

resultados batem (atol=1e-6)? True
```

Como no Capítulo 2 (autograd) e no Capítulo 5 (LayerNorm): construímos do zero, e conferimos contra a implementação de referência.

7.4 Agendamento da learning rate: warmup + cosine

Duas ideias combinadas, ambas com motivo concreto.

Warmup (subir a lr do zero ao pico, nos primeiros passos). O motivo está na seção anterior: no começo, m e v do Adam quase não têm informação — são estimativas ruins baseadas em pouquíssimos gradientes. Dar passos grandes com base em estimativas ruins é como correr no escuro. O warmup deixa as médias móveis se estabelecerem primeiro.

Cosine decay (descer suavemente até uma fração do pico). No começo do treino, você está longe do mínimo: passos largos são bons. No fim, você está ajustando detalhes: passos largos passam do ponto. Decair resolve isso — é a versão suave e sem "degraus" do $lr = 0.1$ if $step < 15000$ else 0.01 que usei no Capítulo 3.

```
def lr_agendada(step):
    if step < WARMUP_STEPS:
```

```

    return base_lr * (step + 1) / WARMUP_STEPS          # sobe linearmente
    progresso = (step - WARMUP_STEPS) / (max_steps - WARMUP_STEPS)
    cos = 0.5 * (1 + math.cos(math.pi * progresso))      # vai de 1 a 0
    return base_lr * (MIN_LR_FRAC + (1 - MIN_LR_FRAC) * cos)

```

Um detalhe com consequência: o cosseno precisa saber `max_steps` de **antemão**. Se você quiser continuar um treino depois de terminado, o agendamento já chegou ao fim e não há como "esticá-lo" sem estragar a curva. É um incômodo real na prática (e assunto do exercício E7).

7.5 Gradient clipping

De vez em quando um mini-batch produz um gradiente muito maior que o normal — um dado atípico, ou uma região ruim da superfície de erro. Um único passo gigante pode desfazer muito progresso.

O **gradient clipping** limita a **norma** total do gradiente:

```
torch.nn.utils.clip_grad_norm_(modelo.parameters(), GRAD_CLIP)
```

O ponto crucial: ele reescala, mas **não muda a direção**. Se a norma passa do limite, todos os gradientes são multiplicados pelo mesmo fator, de modo que a norma final seja exatamente o limite. A informação de "para onde ir" é preservada; só o tamanho do passo é contido. É uma rede de segurança barata contra picos.

7.5.1 Como escolher o limite (aprendido do jeito difícil)

A primeira versão deste capítulo usava `GRAD_CLIP = 1.0`, e o resultado foi este:

```
clipados: 14841/15000 passos (99%)
```

99% dos passos cortados. Isso não é clipping — é normalização. Com o limite abaixo da norma típica, todo gradiente era reescalado para exatamente 1,0, o que transforma o algoritmo em outra coisa (uma espécie de *normalized gradient descent*), em vez de proteger contra picos raros.

O erro foi meu, e a lição é prática e generalizável:

Meça antes de escolher. O limite de clipping tem que ficar **acima** da norma típica do gradiente, para capturar apenas as exceções. Rode alguns passos, olhe a distribuição das normas e escolha um limite algumas vezes maior que a mediana.

No nosso caso, a norma média medida é **~1,23** e a máxima **~1,94**. Um limite de `1.0` corta tudo; um limite de `3.0` corta só o que realmente foge do padrão. O `train_tuned.py` agora **mede a norma sempre** (mesmo com o clipping desligado) e informa quantos passos foram cortados — exatamente para você poder calibrar.

7.6 Resultados: uma ablação

7.6.1 O método

Para saber quanto cada técnica contribui, não basta ligar todas e comparar com o baseline — se o resultado mudar, você não sabe a quem atribuir. O procedimento correto é a **ablação**: ligar **uma técnica por vez**, mantendo absolutamente todo o resto igual (mesma arquitetura, mesmos dados, mesma semente, mesmo número de passos). É o mesmo método que usamos no Capítulo 5 para medir o valor das conexões residuais.

As quatro técnicas em teste, mais a combinação de todas:

Técnica	O que muda em relação ao baseline
agendamento	warmup de 500 passos + cosine decay até 10% do pico
clipping	norma do gradiente limitada a 3,0
weight decay	0,1 em vez do default 0,01
init escalada	projeções residuais encolhidas por $1/\sqrt{2 \cdot n_layer}$

7.6.2 Os números

Rodando `python train_tuned.py` (~33 minutos na CPU, seis treinos):

Configuração	treino	validação	teste	vs baseline
1. baseline (Cap. 5)	1,7910	1,8114	1,8199	—
2. só agendamento	1,7541	1,7760	1,7816	+0,0355
3. só clipping	1,7910	1,8114	1,8199	0,0000
4. só weight decay 0,1	1,8608	1,8646	1,8727	-0,0532
5. só init escalada	1,7993	1,8187	1,8234	-0,0073

6. tudo junto	1,8063	1,8110	1,8183	+0,0004
---------------	--------	--------	--------	---------

Este resultado é mais instrutivo do que um "afinamos e melhorou". Vamos por partes.

O agendamento ganha, e é o único que ganha. +0,0355 de melhoria — e o valor absoluto de **1,7760** é o **melhor modelo do curso até aqui**, superando o Transformer do Capítulo 5. Isto sem tocar em uma linha da arquitetura. Vale registrar que dos quatro "cuidados de otimização" testados, só este entregou o que prometia.

O clipping é exatamente neutro — e olhe o quanto isso é informativo: as três colunas de loss são **idênticas** às do baseline, até a quarta casa. O motivo está no relatório do próprio script: `clipados: 0/15000 (0%)`. A norma máxima do gradiente em todo o treino foi **1,937**, e o limite era 3,0 — o corte nunca disparou. Ou seja, o clipping aqui é um seguro que nunca foi acionado. Isso não o torna inútil em geral: em modelos e batches maiores, picos de gradiente acontecem, e um único passo descontrolado pode custar horas de treino. Mas neste modelo ele não faz nada, e é honesto dizer isso. (Note também que os números idênticos **validam a medição**: se eles tivessem divergido com zero cortes, haveria um bug em algum lugar.)

O weight decay alto atrapalha — e muito. -0,0532 é o **maior efeito de toda a tabela**, e é negativo. A explicação está no Capítulo 3: weight decay é uma arma contra *overfitting*, e nós **medimos** que este modelo não está decorando (treino 1,791 vs validação 1,811 — praticamente iguais). Sem *overfitting* para combater, a regularização forte só restringe a capacidade do modelo sem nada em troca. É remédio para uma doença que o paciente não tem.

A init escalada atrapalha um pouco (-0,0073). Ela foi desenhada para modelos **profundos**: com 12 ou mais blocos, as contribuições somadas ao caminho residual se acumulam e precisam ser contidas. Com **3 blocos**, esse acúmulo é pequeno, e encolher as projeções por $1/\sqrt{6} \approx 0,41$ só faz o modelo começar mais fraco sem necessidade.

E **"tudo junto" dá quase zero** (+0,0004). Agora a razão é transparente: o ganho do agendamento (+0,0355) é praticamente anulado pelas perdas do weight decay (-0,0532) e da init escalada (-0,0073). As contribuições **não se somam** — elas se cancelam.

A lição central deste capítulo: "melhores práticas" não são aditivas nem universais. Cada uma dessas técnicas existe para resolver um problema específico — *overfitting*, instabilidade em redes profundas, picos de gradiente. Aplicá-la quando o problema **não existe** custa desempenho. A única forma de saber é **medir uma por vez**.

E é por isso que a melhor configuração encontrada não é a que tem mais recursos ligados, e sim **baseline + agendamento**: 1,7760.

7.6.3 A inicialização escalada (truque do GPT-2)

Vale entender a técnica que testamos, mesmo tendo saído perdedora aqui. Cada bloco **soma** sua contribuição ao caminho residual (Cap. 5). Com N blocos, essas contribuições se acumulam e o sinal cresce com a profundidade. O GPT-2 compensa encolhendo a inicialização das camadas que *escrevem* no residual, por $1/\sqrt{2 \cdot n_layer}$:

```
escala = (2 * n_layer) ** -0.5
for b in self.blocks:
    for camada in (b.proj, b.ff_out):      # as 2 que escrevem no residual
        camada.weight.mul_(escala)
```

São duas escritas por bloco (atenção e feedforward), daí o $2 \cdot n_layer$. Num modelo de 12 blocos o fator é $1/\sqrt{24} \approx 0,20$, e aí ele importa. No nosso, de 3 blocos, atrapalha — o que ilustra exatamente a lição acima.

7.7 Resumo do capítulo

- **Inicialização:** $\text{randn} * \text{ganho} / \sqrt{\text{fan_in}}$. O $\sqrt{\text{fan_in}}$ neutraliza o tamanho da camada; o **ganho** corrige o efeito da ativação (1 linear, $5/3$ tanh, $\sqrt{2}$ ReLU). Medimos: ganho errado faz o sinal morrer (0,003) ou saturar.
- **LayerNorm** reduz a sensibilidade à inicialização — foi por isso que o Cap. 5 funcionou sem cuidado especial.
- **Adam** guarda duas médias móveis: do gradiente (m , direção) e do seu quadrado (v , escala). Dividir por $\sqrt{v}$ dá um passo adaptado a cada parâmetro — indispensável quando as escalas variam muito.
- **Correção de viés** conserta a subestimação inicial de m e v , e se apaga sozinha.
- **AdamW** aplica weight decay **direto no parâmetro**, fora do caminho adaptativo.
- **Warmup** protege os primeiros passos (quando m e v são estimativas ruins); **cosine decay** troca passos largos por finos conforme o treino avança.
- **Gradient clipping** limita a norma sem mudar a direção — rede de segurança contra picos. **Calibre o limite acima da norma típica**, senão você normaliza todo passo em vez de cortar exceções (foi o erro documentado na Seção 5).
- Nossa implementação de AdamW **bate com a do PyTorch** (8,9e-08).
- **A ablação mudou a conclusão do capítulo.** Das quatro técnicas testadas uma por vez, só o **agendamento** ajudou (+0,0355, chegando a **1,7760** — o melhor modelo do curso). O clipping foi neutro (nunca disparou), e o **weight decay alto** (-0,0532) e a **init escalada** (-0,0073) **pioraram** — cada um por aplicar uma solução a um problema que este modelo não tem.

- **Boas práticas não são aditivas nem universais.** "Tudo junto" deu quase zero porque os ganhos e as perdas se cancelaram. Medir uma por vez é a única forma de saber.

7.7.1 O que vem no Capítulo 8

Fecha aqui a **Fase II**: temos arquitetura, tokenizador e treino afinado. A Fase III ataca outra dimensão: **velocidade**. No **Capítulo 08 — Device** saímos da CPU e levamos o treino para a **GPU**, entendendo por que a diferença é de ordens de grandeza e o que exatamente muda no código.

-> Antes de seguir, faça os exercícios.

7.8 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

Aviso de tempo: os exercícios que treinam o Transformer levam ~6 minutos cada na CPU. Reduza `max_steps` (ex.: 4000) para experimentar mais rápido — as *comparações relativas* seguem válidas, só não compare com os números absolutos da apostila.

7.8.1 E1 — Leitura de código (aquecimento)

Sem rodar:

1. No nosso AdamW, o que a **correção de viés** ($m / (1 - \text{beta1}^{*t})$) faz? Por que ela importa mais nos primeiros passos e vira irrelevante depois?
2. Qual a diferença entre somar o *weight decay* ao gradiente (Adam original) e aplicá-lo direto no parâmetro (AdamW)? Por que a segunda forma é melhor?
3. Por que `initialization.py` divide por `sqrt(fan_in)` e não por `fan_in`?

7.8.2 E2 — O warmup é necessário?

No `train_tuned.py`, coloque `WARMUP_STEPS = 0` e treine.

1. A loss dos primeiros passos fica pior? O treino chega ao mesmo lugar no fim?
2. Olhe a **norma do gradiente** nos primeiros passos (o script reporta média e máxima). Por que o começo do treino é o momento mais instável?
3. Explique o warmup em uma frase: por que começar com passos pequenos ajuda um otimizador adaptativo como o Adam? (Dica: nos primeiros passos, as médias

móveis m e v têm pouquíssima informação.)

7.8.3 E3 — Calibrando o gradient clipping

O script informa a **norma média** do gradiente e **quantos passos foram clipados**. Use essas duas informações.

1. Com o `GRAD_CLIP = 3.0` do código, que percentual dos passos é cortado? Isso é coerente com "cortar apenas as exceções"?
2. Reduza para `1.0` e depois `0.1`. Em que ponto o clipping deixa de ser uma rede de segurança e passa a ser uma **normalização** de todo passo? (A apostila conta que a primeira versão deste capítulo cortava 99% dos passos — o erro está documentado na Seção 5.)
3. Aumente para `100` (nunca corta). A loss piora? Se não piorar, o que isso diz sobre a necessidade de clipping **neste** modelo?
4. O clipping altera a **direção** do gradiente ou apenas o seu **tamanho**? Por que essa distinção importa?

7.8.4 E4 — O ganho certo para a GELU

O `initialization.py` mostra o ganho correto para $\tanh$ ($5/3$) e ReLU ($\sqrt{2}$). Nosso Transformer usa **GELU**.

1. Adapte a função rodar para aceitar GELU e descubra empiricamente qual ganho mantém o desvio padrão estável ao longo das 8 camadas.
2. O valor ficou mais perto do da ReLU ou do da $\tanh$? Isso faz sentido, dado que a GELU é uma versão suave da ReLU?
3. Compare com `torch.nn.init.calculate_gain` — ela tem entrada para GELU? Se não, o que isso sugere na prática?

7.8.5 E5 — A curva da learning rate

Treine com `base_lr = 1e-4, 3e-4, 1e-3, 3e-3` e `1e-2` (reduza `max_steps` para 3000).

1. Monte a tabela de loss de validação. O formato é de um "U"? Onde está o mínimo?
2. O que acontece com `1e-2` — o treino diverge ou só fica ruim?

3. **Você provavelmente vai achar um mínimo em $3e-3$, maior que o $1e-3$ da apostila.** Isso contradiz o capítulo? (Dica: a apostila treina 15.000 passos, e este exercício 3.000. Com pouco orçamento, compensa andar mais rápido.)

Por que a melhor learning rate depende do **tamanho do batch**? (Pense: com batch maior, o gradiente é menos ruidoso.)

Solução de referência: `solucoes/e5_curva_lr.py`.

7.8.6 E6 — AdamW vs SGD no modelo de verdade

O `optimizers.py` compara os otimizadores num problema de brinquedo. Faça a comparação no Transformer: troque o `torch.optim.AdamW` por `torch.optim.SGD` (com e sem momentum).

1. Com a **mesma** learning rate, o SGD chega perto?
2. Ajuste a learning rate do SGD para o melhor valor que encontrar. Ele empata com o AdamW?
3. Por que redes com muitos parâmetros de escalas diferentes (embeddings, pesos de atenção, ganhos de LayerNorm) favorecem otimizadores adaptativos?

7.8.7 E7 — Por que o weight decay atrapalhou? (importante)

A ablação da apostila mostra que `weight_decay = 0.1` **piorou** o modelo em -0,0532 — o maior efeito da tabela, e negativo.

1. Varie o weight decay: 0.0, 0.01, 0.1, 0.5. Monte a tabela de loss de validação. Existe um valor melhor que o default 0.01?
2. Compare a loss de **treino** com a de **validação** em cada caso. Quando o weight decay aumenta, as duas sobem juntas ou a de treino sobe mais? O que isso indica?
3. **A pergunta que importa:** weight decay combate *overfitting*. Olhe os números do baseline (treino 1,791 vs validação 1,811). Existe overfitting para combater? Se não existe, o que a regularização forte está fazendo com a capacidade do modelo?
4. Em que cenário você **esperaria** que `weight_decay = 0.1` ajudasse? (Dica: releia a solução do E5 do Capítulo 3, com 155 nomes.)

7.8.8 E8 — Outro agendamento (desafio)

Implemente e compare três agendamentos, todos com o mesmo warmup:

- **constante** (o baseline)
- **cosseno** (o nosso)
- **linear** (decai em linha reta até MIN_LR_FRAC)
- **degraus** (divide por 10 em 50% e 80% do treino)

1. Qual vence no nosso problema? A diferença é grande?
2. O cosseno é o padrão em treinos de LLM. Pelos seus números, isso se justifica ou é convenção?
3. Um detalhe importante: o agendamento depende de saber max_steps de antemão. Que problema isso cria se você quiser **continuar** um treino já terminado?

8 DEVICE (CPU E GPU)

Objetivo de aprendizagem: entender **por que** a GPU acelera o treino de redes neurais — e, mais importante, **quando ela não acelera**. Vamos medir o ponto de virada, o custo de transferir dados, o efeito do tamanho do batch, e escrever código que roda em qualquer dispositivo sem alteração.

Pré-requisitos: Capítulos 1-7. Este capítulo abre a **Fase III** do curso, sobre velocidade e escala.

Arquivos:

- `SETUP-GPU.md` — instalação nos três caminhos (NVIDIA / AMD-Intel / sem GPU)
- `device.py` — detecção portátil de dispositivo
- `benchmark.py` — matmul, transferência e batch: CPU vs GPU
- `train_device.py` — treino nos dois dispositivos, em três tamanhos
- `exercicios.md` — exercícios

Sobre o hardware desta apostila: os números foram medidos numa **AMD Radeon RX 7600** via **DirectML**, contra uma CPU de desktop. Numa NVIDIA com CUDA os ganhos tendem a ser **maiores** (o CUDA é mais maduro e otimizado). Os *padrões* que os números revelam, porém, valem para qualquer GPU — e é neles que você deve prestar atenção, não nos valores absolutos.

8.1 Por que uma GPU ajuda

CPU e GPU resolvem problemas diferentes, e a diferença está no desenho:

Característica	CPU	GPU
Núcleos	poucos (4-32), muito rápidos	milhares, individualmente lentos
Otimizada para	latência — terminar <i>uma</i> tarefa rápido	vazão — terminar <i>muitas</i> tarefas por segundo
Cada núcleo	complexo (predição de desvio, cache grande, execução fora de ordem)	simples
Boa em	lógica sequencial, desvios, tarefas variadas	a mesma operação sobre muitos dados

Uma rede neural é, computacionalmente, quase só **multiplicação de matrizes** — e uma matmul é o caso ideal para GPU: milhares de multiplicações independentes, todas iguais, sem desvios. É por isso que a GPU domina deep learning, e não por ser "mais rápida"

em algum sentido geral.

A consequência prática — e é o fio condutor deste capítulo:

A GPU só ganha quando há **trabalho suficiente para ocupar os seus milhares de núcleos**. Para trabalho pequeno, o custo fixo de acioná-la é maior que o cálculo, e ela **perde** da CPU.

8.2 Onde está o ponto de virada

Rodando `python benchmark.py`, multiplicando matrizes quadradas de tamanhos crescentes:

Tamanho	CPU (ms)	GPU (ms)	Speedup	GFLOP/s CPU	GFLOP/s GPU
128	0,02	0,03	0,75x	223	167
256	0,11	0,04	2,90x	296	860
512	0,83	0,10	8,73x	322	2.808
1024	6,25	0,41	15,26x	343	5.240
2048	50,40	3,56	14,14x	341	4.820
4096	393,40	25,02	15,72x	349	5.493

Leia as **duas últimas colunas** — elas contam a história melhor que o speedup:

A CPU satura. Ela entrega ~220-350 GFLOP/s e não passa disso, independentemente do tamanho da matriz. Já está usando tudo o que tem desde matrizes pequenas.

A GPU precisa ser preenchida. Ela começa em 167 GFLOP/s (pior que a CPU!) e sobe até ~5.500 conforme o trabalho cresce — **33 vezes** a própria vazão inicial. Os milhares de núcleos só rendem quando há trabalho para todos.

E em **128×128 a GPU perde** (0,75x). Não é anomalia: é o custo fixo de lançar a operação e coordenar o dispositivo dominando um cálculo minúsculo. Esse é o dado mais importante da tabela, porque contradiz a intuição de que "GPU é sempre mais rápido".

8.3 Transferir dados custa

Os dados nascem na memória da CPU e precisam ir para a memória da GPU. Isso não é grátis:

Tamanho	MB	CPU->GPU	GPU->CPU	Vazão
256×256	0,3	0,07 ms	0,17 ms	4,0 GB/s
1024×1024	4,2	0,72 ms	1,04 ms	5,8 GB/s
2048×2048	16,8	2,79 ms	3,97 ms	6,0 GB/s

Compare com a Seção 2: transferir uma matriz 1024×1024 custa **0,72 ms**, e multiplicar duas delas na GPU custa **0,41 ms**. Ou seja, **mover o dado custa mais que a conta**. Se você transferir a cada operação, joga fora todo o ganho — e é isso que faz muita gente concluir que "a GPU não ajudou".

Note também que a volta (GPU->CPU) é consistentemente mais lenta que a ida, e que a vazão (~6 GB/s) fica muito abaixo da banda interna da GPU: o gargalo é o barramento PCIe entre os dois mundos, não a memória da placa.

A regra prática que decorre disso:

Mova os dados para a GPU uma vez e deixe-os lá. Todo `.cpu()`, `.item()` ou `print` de um tensor dentro do laço de treino força uma transferência e uma sincronização — a CPU para e espera a GPU terminar. É um dos gargalos mais comuns em código de iniciante.

No `train_device.py` fazemos isso explicitamente:

```
# Move os dados UMA VEZ, fora do laço
Xd, Yd = Xtr.to(device), Ytr.to(device)
```

8.4 O tamanho do batch importa mais do que parece

Aqui um experimento que isola o efeito: processar **exatamente a mesma quantidade** de vetores, mudando só como eles são entregues.

Batch	Chamadas	Tempo total (ms)
1	4.096	62,80
16	256	4,40

64	64	1,07
256	16	0,39
1.024	4	0,26
4.096	1	0,37

Processar de a **um** custa 62,80 ms; processar tudo de uma vez custa 0,26 ms — uma diferença de **240 vezes** para o mesmo número de multiplicações.

O total de multiplicações é **idêntico** em todas as linhas. A diferença é só o empacotamento — e ela é enorme. Entregar de a um deixa a GPU quase ociosa e paga o custo fixo de lançamento a cada chamada; entregar tudo de uma vez usa a máquina inteira.

É por isso que, ao mudar de CPU para GPU, **aumentar o batch** costuma ser o primeiro ajuste a fazer.

8.5 Código portátil: nunca crave `.cuda()`

O jeito errado é espalhar `.cuda()` pelo código: aí ele só roda em NVIDIA, e quebra em todo lugar mais. O jeito certo é detectar o dispositivo uma vez e usar a variável:

```
def pegar_device(verbose=True):
    if torch.cuda.is_available():                # 1. NVIDIA
        return torch.device("cuda"), "CUDA"
    try:                                         # 2. AMD/Intel no Windows
        import torch_directml
        if torch_directml.device_count() > 0:
            return torch_directml.device(), "DirectML"
    except ImportError:
        pass
    return torch.device("cpu"), "CPU"           # 3. sempre funciona
```

Depois, todo tensor e todo módulo vai para lá:

```
modelo = Modelo().to(dev)
x = x.to(dev)
```

Com isso, os scripts deste capítulo rodam sem alteração em NVIDIA, AMD, Intel ou CPU. Veja `SETUP-GPU.md` para a instalação de cada caminho.

8.6 Como medir GPU sem se enganar

Esta seção existe porque **eu errei a medição na primeira tentativa**, e o erro é tão comum que vale mais que um aviso.

8.6.1 Problema 1: execução assíncrona

Chamadas para a GPU são **assíncronas**. Quando você escreve `c = a @ b`, o Python **não** espera o cálculo: ele enfileira a operação e devolve o controle imediatamente. Se você cronometrar assim:

```
t0 = time.perf_counter()
c = a @ b
t = time.perf_counter() - t0    # só ENFILEIRA
                                # mede microssegundos!
```

...você mede o tempo de *enfileirar*, não de *executar*, e conclui que a GPU é absurdamente rápida. Em CUDA a solução é `torch.cuda.synchronize()`. O DirectML não expõe essa função, então é preciso forçar a fila a esvaziar de outra forma.

8.6.2 Problema 2: sincronizar da forma errada

Minha primeira versão criava **outro** tensor e o copiava para a CPU, esperando que isso esvaziasse a fila. Não é confiável. O resultado foram speedups sem sentido — não monotônicos:

512	2.26	3.64	0.62x	<- GPU "mais lenta" que em 256
1024	6.42	0.35	18.17x	<- e depois "18x mais rápida"

Uma curva assim é sinal de medição contaminada, não de comportamento real do hardware. A forma correta é **tocar no próprio resultado**, o que obriga o dispositivo a produzi-lo:

```
def drenar(saida, device):
    if device.type == "cuda":
        torch.cuda.synchronize()
    elif device.type != "cpu" and torch.is_tensor(saida):
        saida.cpu()    # lê o RESULTADO -> a fila tem de esvaziar
```

8.6.3 Problema 3: média em vez de mínimo

Mesmo sincronizando certo, sobrava ruído. A correção final foi usar o **mínimo de várias rodadas** em vez da média:

O tempo real de uma operação é um **piso**. Qualquer medição acima dele foi contaminada por outra coisa disputando a máquina — o sistema operacional, outro processo, o próprio Python. A média incorpora esse ruído; **o mínimo o descarta**.

Com as três correções (drenar pelo resultado, aquecer antes, mínimo de 3 rodadas), a tabela da Seção 2 ficou monotônica e coerente. Sem elas, eu teria publicado números errados com aparência de medição.

8.6.4 Problema 4: o primeiro uso é sempre mais lento

A primeira chamada aloca buffers e compila kernels para aquele formato de tensor. Por isso todo benchmark começa com **aquecimento** — algumas execuções descartadas antes de medir.

8.7 E o nosso modelo, ganha algo?

Agora a pergunta prática. O `train_device.py` treina o Transformer do Capítulo 5 nos dois dispositivos, em três tamanhos, medindo o tempo por passo:

Modelo	Parâmetros	Batch	CPU (ms/passo)	GPU (ms/passo)	Speedup
pequeno (o do Cap. 5)	153.499	64	12,8	42,7	0,30x
médio	3.172.379	256	206,9	78,0	2,65x
grande	18.937.883	512	1.916,8	280,9	6,82x

O nosso modelo é 3,3x MAIS LENTO na GPU. Esse é o resultado mais útil do capítulo, e ele é contra-intuitivo.

O motivo já está na Seção 2. Com `n_embd = 64` e `batch = 64`, as `matmuls` do nosso modelo são da ordem de 64×64 a 512×64 — exatamente a faixa onde a GPU perde. Somando o custo de lançar dezenas de kernels minúsculos por passo (mais o `lerp` do AdamW, que cai de volta na CPU — veja `SETUP-GPU.md`), o resultado é pior que rodar na CPU.

Conforme o modelo cresce, a conta se inverte: **2,65x** com 3,2 milhões de parâmetros e **6,82x** com 18,9 milhões. E a tendência continuaria: quanto maior o modelo, maior a vantagem — que é exatamente o motivo de LLMs de verdade serem treinados em GPU.

A recomendação honesta para este curso: continue rodando os capítulos na CPU. O nosso modelo é pequeno demais para se beneficiar, e a CPU é mais simples de usar. A GPU passa a valer a pena quando você escalar — e agora você sabe medir a partir de quando.

8.7.1 Quanto custa uma operação não suportada

Ao rodar na GPU aparece um aviso: o AdamW usa `aten::lerp`, que o DirectML não implementa, e essa parte **volta a executar na CPU** a cada passo. Quanto isso custa? A solução do exercício E6 mede, no modelo médio:

Otimizador	GPU (ms/ passo)	Speedup	Operações em fallback
AdamW	75,2	2,65x	aten::lerp.Scalar_out
SGD + momentum	32,7	5,94x	nenhuma
SGD puro	30,9	6,23x	nenhuma

Trocar o otimizador por um que não usa lerp **mais que dobra** o ganho: de 2,65x para 5,94x. Aquela única operação custava ~42 ms por passo — **mais da metade** do tempo total.

Duas conclusões:

1. **Os speedups desta apostila são um piso, não um teto.** Com um backend completo (CUDA) ou um otimizador sem lacunas, a mesma placa daria mais.
2. **Lacunas de backend não aparecem como erro — aparecem como lentidão silenciosa.** O código roda, o resultado está certo, e você simplesmente perde metade da performance sem saber. Rode uma vez com `python -W always::UserWarning` e leia os avisos.

8.7.2 Um efeito colateral: a GPU dá o mesmo resultado?

Modelo	Loss CPU	Loss GPU	Diferença
pequeno	2,3270	2,3270	4,77e-07
médio	2,2308	2,2308	2,38e-07
grande	2,1605	2,1520	8,50e-03

Nos dois primeiros, a diferença é de arredondamento puro (~1e-07): a GPU usa kernels diferentes e soma os números em outra **ordem**, e em float32 a ordem da soma altera o último dígito. É o mesmo efeito que vimos no Capítulo 4, ao comparar as versões da atenção.

Mas no modelo **grande** a diferença salta para 8,5e-03 — quatro ordens de grandeza maior. Isso **não** é um bug, e a explicação importa:

Treinar é um processo **caótico**. Uma diferença de 1e-07 no gradiente do primeiro passo muda levemente o peso atualizado, o que muda o gradiente do passo seguinte, e assim por diante. Ao longo de 105 passos, esse desvio microscópico é

amplificado. Quanto maior o modelo, mais operações por passo e mais rápido o desvio cresce.

A consequência prática é relevante e pouco divulgada: **não existe reprodutibilidade bit-exata entre dispositivos diferentes.** Mesmo código, mesma semente, GPUs distintas — trajetórias distintas. Por isso, ao comparar resultados entre máquinas, compare **estatísticas** (loss final, média de várias sementes), nunca valores exatos.

8.8 Resumo do capítulo

- **CPU otimiza latência; GPU otimiza vazão.** A GPU tem milhares de núcleos simples, e só rende quando há trabalho para todos.
- Medindo matmul: a **CPU satura** em ~340 GFLOP/s; a **GPU escala** de 167 até ~5.500. Em matrizes pequenas (128×128) a **GPU perde**.
- **Transferir custa.** Mova os dados uma vez e deixe-os na GPU. `.item()` e `.cpu()` dentro do laço de treino forçam sincronização e viram gargalo.
- **Batch grande é essencial na GPU.** O mesmo trabalho total, entregue em pedaços pequenos, desperdiça a máquina.
- **Escreva código portátil** (`pegar_device()`), nunca `.cuda()` cravado.
- **Medir GPU é sujeito a erro:** execução assíncrona, sincronização mal feita, ruído e aquecimento. Errei os três primeiros na primeira tentativa e documentei a correção.
- **O nosso modelo é 3,3x mais lento na GPU** (0,30x), e só passa a ganhar em escala: 2,65x com 3,2 M de parâmetros, 6,82x com 18,9 M. Para este curso, continue na CPU.
- **Backends imaturos têm lacunas silenciosas:** o AdamW usa uma operação que o DirectML não implementa e que volta para a CPU a cada passo — sem erro, só lentidão. Rode com `-w always::UserWarning` para descobrir.
- **Não há reprodutibilidade bit-exata entre dispositivos.** O treino é caótico: 1e-07 de diferença no primeiro passo virou 8,5e-03 depois de 105 passos no modelo grande. Compare estatísticas, não valores exatos.

8.8.1 O que vem no Capítulo 9

Já usamos a GPU. O próximo passo é usá-la **melhor**: no **Capítulo 09 — Precision** vamos treinar com **menos bits por número** (fp16, bf16), o que dobra a vazão e reduz o uso de memória — e entender por que isso funciona sem estragar o treino, e quando estraga.

-> Antes de seguir, faça os exercícios.

8.9 Setup Gpu

O código do capítulo (`device.py`) detecta o dispositivo automaticamente, então os scripts **rodam sem alteração** em qualquer um dos casos abaixo. O que muda é a instalação.

Descubra primeiro que placa você tem:

```
# Windows
wmic path win32_VideoController get name
# Linux
lspci | grep -i vga
```

8.9.1 Caminho 1 — NVIDIA (CUDA)

O mais simples e o melhor suportado. Instale a build de CUDA do PyTorch (a versão do `cu###` depende do seu driver — consulte <https://pytorch.org/get-started/locally/>):

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
```

Verifique:

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

Deve imprimir `True` e o nome da placa. Nada mais é necessário — `device.py` já encontra.

8.9.2 Caminho 2 — AMD ou Intel no Windows (DirectML)

Placas AMD e Intel **não** rodam CUDA. No Windows, a alternativa é o **DirectML**, que funciona sobre qualquer GPU compatível com DirectX 12.

Atenção à versão do Python. O `torch-directml` não tem distribuição para todas as versões. No momento em que este capítulo foi escrito, ele exigia **Python 3.10-3.12** (não funciona em 3.13+). Se o seu Python principal for mais novo, crie um ambiente separado só para isso.

```
# Windows PowerShell — use um Python 3.12 (ajuste o caminho)
py -3.12 -m venv C:\dml312
C:\dml312\Scripts\python.exe -m pip install torch-directml
```

Cuidado com caminho longo: o Windows limita caminhos a 260 caracteres, e a instalação do numpy cria caminhos profundos. Se der `WinError 206`, crie o ambiente num diretório curto (`C:\dml312`) em vez de dentro de pastas aninhadas.

Verifique:

```
C:\dml312\Scripts\python.exe -c "import torch_directml; print(torch_directml.device_count(), torch_directml.device())"
```

Deve imprimir `1 privateuseone:0` (ou mais dispositivos).

Limitações honestas do DirectML:

- É mais lento que CUDA em hardware equivalente e recebe menos otimização.
- **A cobertura de operações é boa, mas não é total** — e as lacunas aparecem em lugares inesperados. Todas as operações do *modelo* funcionam (embedding, softmax, GELU, masked_fill, LayerNorm, cross_entropy, multinomial e o backward), mas o **AdamW** usa `aten::lerp`, que **não** é implementada: o PyTorch avisa e executa essa parte na CPU.

UserWarning: The operator 'aten::lerp.Scalar_out' is not currently supported on the DML backend and will fall back to run on the CPU.

O treino continua correto, mas parte do passo do otimizador atravessa a fronteira CPU<->GPU a cada iteração — o que come parte do ganho. Ou seja: **os speedups que medimos neste capítulo são um piso**, não o teto do que a placa poderia dar com um backend completo.

Como detectar isso no seu caso: rode com `python -W always::UserWarning seu_script.py` e procure por "fall back to run on the CPU". Um aviso desses dentro do laço de treino é um gargalo silencioso.

- Não expõe `torch.cuda.synchronize()`; para medir tempo é preciso forçar a sincronização de outra forma (ver a Seção 6 da apostila).

Alternativa no Linux: placas AMD suportam **ROCm**, que é bem mais completo que o DirectML. Se você usa Linux com uma AMD recente, prefira ROCm: <https://pytorch.org/get-started/locally/>

8.9.3 Caminho 3 — sem GPU (CPU)

Nada a instalar: é a instalação padrão do curso. Os scripts detectam a ausência de GPU e rodam na CPU.

Você perde as comparações de velocidade, mas **todo o conteúdo conceitual do capítulo continua acessível** — inclusive porque uma das lições é justamente que, para modelos pequenos, a CPU **ganha**.

Se quiser experimentar GPU sem ter uma, o **Google Colab** oferece GPU gratuita: suba os arquivos do capítulo e rode lá.

8.9.4 Como saber que funcionou

```
python device.py
```

Saída esperada (exemplo, no caminho DirectML):

```
dispositivo: DirectML -> privateuseone:0 (1 disponível)
rotulo: DirectML
torch: 2.4.1+cpu
tensor de teste criado em privateuseone:0: soma = -1.2345
```

O +cpu na versão do torch é normal no DirectML: o pacote base é a build de CPU, e o DirectML entra como um backend adicional.

8.10 Exercícios

Faça na ordem. Soluções comentadas em solucoes/ — **tente antes de olhar.**

Sem GPU? Os exercícios E1, E3 e E7 funcionam na CPU. Os demais precisam de GPU — veja `SETUP-GPU.md`, ou use o Google Colab (GPU gratuita).

8.10.1 E1 — Latência vs vazão (aquecimento)

Sem rodar código:

1. Por que a GPU **perde** da CPU numa matmul de 128×128 ?
2. Na tabela da Seção 2, a CPU fica em ~340 GFLOP/s em todos os tamanhos, enquanto a GPU sobe de 167 a 5.500. Explique a diferença em termos de "núcleos ocupados".
3. Uma tarefa que precisa de muitos if e desvios roda melhor em CPU ou GPU? Por quê?

8.10.2 E2 — Encontre o SEU ponto de virada

Rode `python benchmark.py` na sua máquina.

1. A partir de qual tamanho de matriz a sua GPU passa a ganhar?
2. Qual o speedup máximo que você observa? Ele estabiliza?
3. Compare os GFLOP/s de pico da sua GPU com a especificação oficial da placa. Você chega perto? (Difícilmente — e a diferença é o que os capítulos 9 e 10 atacam.)

8.10.3 E3 — A armadilha do `.item()`

No laço de treino do `train_device.py`, adicione uma leitura do valor da loss **a cada passo**:

```
historico.append(loss.item())      # força transferência + sincronização
```

1. Quanto o tempo por passo aumenta? Meça com o modelo **médio**, na GPU.
2. Por que essa linha é tão cara? (Duas coisas acontecem: transferência e sincronização.)
3. Como registrar a loss sem pagar esse preço a cada passo? (Dica: a cada quantos passos você realmente precisa dela?)

8.10.4 E4 — Tamanho do batch na GPU

Com o modelo **médio** na GPU, varie o batch: 32, 128, 512, 2048.

1. O tempo por passo cresce proporcionalmente ao batch, ou mais devagar?
2. Calcule o **tempo por exemplo** (tempo do passo ÷ batch). Onde ele é menor?
3. Aumente o batch até dar erro de memória. Qual o limite da sua placa?
4. Cuidado conceitual: batch maior processa mais exemplos por segundo, mas **cada passo dá um passo só** de gradiente. Isso significa que o treino inteiro fica mais rápido? (Releia o E5 do Capítulo 7 sobre a relação entre batch e learning rate.)

8.10.5 E5 — Meça errado de propósito

Na função `cronometrar` do `benchmark.py`, remova a chamada a `drenar()` (tanto a de aquecimento quanto a final).

1. Que speedups a GPU passa a "apresentar"? Eles fazem sentido físico?
2. Por que a medição da CPU **não** é afetada por essa remoção?
3. Esse é um erro que se detecta olhando o quê? (Dica: o que a curva de speedup faz quando a medição está contaminada?)

8.10.6 E6 — Operações que caem de volta na CPU (desafio)

Ao rodar `train_device.py` no DirectML, aparece um aviso: o AdamW usa `aten::lerp`, que não é implementada, e essa parte executa na CPU.

1. Rode com `python -W always::UserWarning train_device.py` e liste **todas** as operações que caem de volta.

2. Troque o `torch.optim.AdamW` por `torch.optim.SGD` (que não usa `lerp`). O aviso desaparece? O speedup da GPU **melhora**?
 3. Se melhorar: o que isso diz sobre os números da apostila? (Eles são um piso ou um teto do que a placa pode dar?)
 4. Em CUDA esse problema não existe. Que lição geral você tira sobre depender de um backend menos maduro?
-

8.10.7 E7 — Quando vale a pena mudar para GPU? (desafio)

Junte os dados que você mediu.

1. Usando o tempo por passo dos três tamanhos, estime a partir de **quantos parâmetros** (ou de que dimensão `n_embd`) a GPU passa a valer a pena na sua máquina.
2. Considere o custo total: se treinar na GPU exige instalar um ambiente separado e depurar operações não suportadas, a partir de que ponto o ganho justifica o trabalho?
3. Para o modelo **pequeno** deste curso, qual dispositivo você recomendaria? Justifique com números — e note que essa é a resposta contra-intuitiva do capítulo.

9 PRECISION (TREINAR COM MENOS BITS)

Objetivo de aprendizagem: entender o que são fp32, fp16 e bf16 **no nível dos bits**, por que treinar em 16 bits quebra o modelo de duas formas distintas (*overflow* e *underflow*), e como a **precisão mista** e o **loss scaling** resolvem isso. Vamos medir tudo — inclusive descobrir que, neste hardware, metade da promessa não se cumpre.

Pré-requisitos: Capítulos 1-8. Em especial o Capítulo 8 (dispositivos) e a metodologia de medição que estabelecemos lá.

Arquivos:

- `floats.py` — anatomia de um float: bits, alcance, precisão, limites
- `precision_bench.py` — velocidade e memória por precisão
- `loss_scaling.py` — o problema medido e o **GradScaler do zero**
- `device.py` — detecção de dispositivo (do Capítulo 8)
- `exercicios.md` — exercícios

9.1 Por que mexer nos bits

Duas razões, e é importante separá-las porque **elas não vêm juntas**:

1. **Memória.** Um número de 16 bits ocupa metade de um de 32. Isso permite modelo maior, batch maior ou contexto mais longo na mesma placa.
2. **Velocidade.** Se o hardware tiver unidades dedicadas a 16 bits (como os *tensor cores* da NVIDIA), a mesma matmul roda várias vezes mais rápido.

O benefício 1 é garantido pela aritmética. O benefício 2 **depende do hardware e do backend** — e a Seção 6 mostra um caso real em que ele simplesmente não aparece.

Em treino de LLM, memória costuma ser o recurso mais escasso. Um modelo de 1 bilhão de parâmetros em fp32 ocupa 4 GB só de pesos — e, com AdamW, mais uns 12 GB de gradientes e momentos (*m* e *v*), antes de qualquer ativação. Cortar isso pela metade não é um luxo de otimização; é o que decide se o treino cabe na máquina.

9.2 Anatomia de um float

Um número de ponto flutuante tem três partes:

[sinal | expoente | mantissa]

sinal : 1 bit – positivo ou negativo

expoente : a ESCALA (ordem de grandeza) -> mais bits = mais ALCANCE
 mantissa : os DÍGITOS significativos -> mais bits = mais PRECISÃO

Os três formatos que importam:

Formato	Sinal	Expoente	Mantissa	Total
fp32 (float)	1	8	23	32 bits
fp16 (half)	1	5	10	16 bits
bf16 (bfloat16)	1	8	7	16 bits

Repare no detalhe que decide tudo: o **bf16 tem os mesmos 8 bits de expoente do fp32**. Ele sacrifica precisão para manter o alcance. O fp16 faz o contrário — guarda mais dígitos, mas num intervalo estreito.

Rodando `python floats.py`, os bits de 3.14159:

```
fp32: 0 10000000 10010010000111111010000
fp16: 0 10000 1001001000
bf16: 0 10000000 1001001
```

O expoente do bf16 é **idêntico** ao do fp32 (10000000); a mantissa é o pedaço truncado.

9.3 Alcance e precisão, medidos

formato	maior	menor normal	epsilon
fp32	3.403e+38	1.175e-38	1.192e-07
fp16	6.550e+04	6.104e-05	9.766e-04
bf16	3.390e+38	1.175e-38	7.812e-03

Três leituras:

- **O fp16 estoura em 65.504.** Qualquer valor maior vira inf.
- **O bf16 alcança 3,4e38** — o mesmo do fp32, como esperado pelos 8 bits de expoente.
- **Em troca, o epsilon do bf16 é 8x pior:** 7,8e-03 contra 9,8e-04.

O *epsilon* é a menor diferença relativa que o formato distingue. Um epsilon de 7,8e-03 significa cerca de **2 casas decimais significativas** — parece pouquíssimo para treinar uma rede. E ainda assim o bf16 é o formato preferido. A Seção 5 explica por quê.

Um cuidado ao medir precisão: comparando um único valor, fp16 e bf16 podem dar exatamente o mesmo erro (basta que os bits extras do fp16 sejam zeros). Por

isso o `floats.py` mede vários valores e reporta a **média**: erro relativo médio de **1,54e-04** (fp16) contra **8,67e-04** (bf16) — o bf16 é ~6x pior, coerente com os 3 bits menos de mantissa ($2^3 = 8x$).

9.4 Onde 16 bits quebram

9.4.1 Overflow: números grandes demais

```
1e+04 -> fp16    10000.0 | bf16    9.984e+03
6e+04 -> fp16    60000.0 | bf16    5.990e+04
7e+04 -> fp16      inf | bf16    7.014e+04
1e+05 -> fp16      inf | bf16    9.984e+04
```

Passando de 65.504, o fp16 devolve `inf`. Ativações e valores intermediários da loss alcançam essa faixa com facilidade em redes grandes.

9.4.2 Underflow: números pequenos demais

```
1e-04 -> fp16    1.000e-04 | bf16    1.001e-04
1e-06 -> fp16    1.013e-06 | bf16    9.984e-07
1e-08 -> fp16    0.000e+00 | bf16    1.001e-08   <- virou ZERO
1e-10 -> fp16    0.000e+00 | bf16    1.000e-10   <- virou ZERO
```

Este é o mais perigoso dos dois, e a razão é sutil: **um gradiente que vira zero não atualiza o peso — e o treino trava sem dar erro nenhum**. Não há exceção, não há NaN na loss. O modelo simplesmente aprende menos, e você não sabe por quê.

9.5 Por que o bf16 ganhou

Gradientes típicos de uma rede treinando ficam entre $1e-4$ e $1e-8$. Cruzando isso com os limites de cada formato:

Gradiente	fp16	bf16
1e-03	1,000e-03	9,995e-04
1e-05	1,001e-05	1,001e-05
1e-07	1,192e-07	1,001e-07
1e-08	0 (zero)	1,001e-08
1e-09	0 (zero)	9,968e-10

Aqui está a resposta inteira:

Para treinar, alcance importa mais que precisão. A direção aproximada do gradiente é útil; um gradiente zerado não é. O bf16 troca dígitos significativos — que o gradient descent tolera bem, porque ele dá milhares de passos pequenos — por alcance, que é o que impede o número de desaparecer.

É por isso que TPUs, A100 e H100 adotaram o bf16 como formato padrão de treino, e por que o fp16 precisa de uma engrenagem extra (Seção 7).

9.6 O que medimos de verdade — e o que não se cumpriu

Aqui o capítulo entrega um resultado que contraria o discurso comum. Multiplicando matrizes 1024×1024:

Dispositivo	fp32	fp16	bf16	Ganho vs fp32
CPU	5,35 ms	1.951 ms	2.401 ms	~0,003x (centenas de vezes pior!)
Radeon RX 7600 (DirectML)	0,40 ms	0,41 ms	n/d	0,98x (nenhum)

Na CPU, 16 bits é catastróficamente mais lento. Não é erro de medição: CPUs não têm unidades de aritmética de 16 bits para matmul, e o PyTorch emula a operação convertendo elemento por elemento. O resultado é da ordem de **350 a 450 vezes** pior que o fp32.

Sobre a imprecisão desse número: esse caso patológico tem variância alta — entre execuções eu medi de 1.950 a 4.017 ms. Não vale perseguir o valor exato; o que importa é a ordem de grandeza, e ela é inequívoca. Já os números da GPU são estáveis e reproduzíveis (0,98-0,99x em todas as execuções).

Na Radeon via DirectML, o ganho é zero. O fp16 roda exatamente na mesma velocidade que o fp32 (0,98x). O hardware RDNA3 tem, em teoria, aritmética de 16 bits em taxa dobrada, mas o DirectML não despacha para esses kernels.

E o bf16 **não existe** nesta placa — pior, ele **aborta o processo**:

```
[F730 10:46:20.000000000 dml_util.cc:118] Invalid or unsupported data type BFloat16.
```

O [F] é *fatal*: não é uma exceção Python que se possa capturar com try/except, é o processo morrendo. Por isso o `precision_bench.py` consulta uma lista de suporte conhecido **antes** de tentar, em vez de tentar e tratar o erro:

```
def suportado(device, dtype):
    if device.type == "privateuseone" and dtype == torch.bfloat16:
        return False, "DirectML aborta o processo com bfloat16"
    return True, ""
```

A conclusão honesta: neste hardware, precisão reduzida compra **memória, não velocidade**. O benefício de velocidade exige hardware e software preparados — na prática, NVIDIA com CUDA e tensor cores. Se você mediu ganho zero na sua máquina, não é porque a técnica não funciona: é porque o seu caminho de execução não a implementa.

O benefício de memória, esse sim, é garantido:

Formato	Bytes/parâmetro	Modelo de 1B parâmetros (só pesos)
fp32	4	4,0 GB
fp16	2	2,0 GB
bf16	2	2,0 GB

9.7 Loss scaling: o truque que salva o fp16

O problema medido em `loss_scaling.py`, com uma rede pequena de pesos pequenos:

```
total de gradientes: 26896
zerados em fp32:      0 (  0.0%)
zerados em fp16: 13541 ( 50.3%)  <- perdidos por underflow

magnitude mediana dos gradientes (fp32): 2.93e-08
menor valor normal do fp16:              6.10e-05
```

Metade dos gradientes vira zero. Metade do modelo, efetivamente, não aprende.

A solução se apoia numa propriedade elementar da derivada — ela é **linear**:

$$d(k \cdot L)/dw = k \cdot dL/dw$$

Ou seja: se multiplicarmos a **loss** por um fator k grande antes do backward, **todos** os gradientes saem multiplicados por k , longe da zona de underflow. Depois dividimos por k antes de atualizar os pesos. O resultado final é o mesmo do fp32 — só os números intermediários passaram a ser representáveis.

```
loss × 1024  ->  backward  ->  gradientes × 1024  ->  ÷ 1024  ->  usar
```

Funciona? Medindo:

Escala	Zerados	%	infs	Situação
1	13.541	50,3%	0	perde muito gradiente

16	1.026	3,8%	0	perde muito gradiente
256	68	0,3%	0	ainda perde um pouco
1.024	14	0,1%	0	ok
65.536	0	0,0%	0	ok
4.194.304	0	0,0%	26.640	overflow
67.108.864	0	0,0%	26.896	overflow

Existe uma **janela**: escala pequena não resolve o underflow, escala grande causa overflow. E — o ponto crucial — **a janela se move durante o treino**, porque a magnitude dos gradientes muda conforme o modelo aprende.

9.7.1 O GradScaler dinâmico

Por isso ninguém usa escala fixa. A solução é um controle de realimentação:

```
def passo(self, parametros):
    achou_inf = any(not torch.isfinite(p.grad).all() for p in parametros ...)
    if achou_inf:
        self.escala = max(1.0, self.escala / self.fator)    # recua
        self.descartados += 1
        return False                                       # DESCARTA o passo
    for p in parametros:
        p.grad /= self.escala                             # desescala
    self.bons_seguidos += 1
    if self.bons_seguidos >= self.intervalo:
        self.escala *= self.fator                          # ousa mais
    return True
```

A lógica: **apareceu inf, recua e joga o passo fora; passaram N passos limpos, arrisca uma escala maior**. O algoritmo persegue continuamente a maior escala que ainda não estoura.

Rodando a simulação (com gradientes que crescem de propósito):

```
passo    escala    aplicado?
5        131072     sim
10       262144     sim
15       524288     sim
20      1048576     sim
21       524288  DESCARTADO
25       524288     sim

passos descartados: 1/25
```

Ele sobe até 1.048.576, estoura, recua para 524.288 e segue. **Descartar passos é aceitável** — com a escala bem ajustada isso acontece em menos de 1% das iterações.

Na prática você usa `torch.amp.GradScaler`, que implementa exatamente este algoritmo. Mas agora você sabe o que ele faz e por quê.

9.8 Precisão mista e os pesos mestres

"Precisão mista" **não** significa converter o modelo todo para 16 bits. Significa escolher, operação por operação:

Em 16 bits (rápido, e o erro não acumula)	Em 32 bits (onde a precisão importa)
matmul e convolução — onde está ~95% do tempo	a cópia mestra dos pesos
—	somas longas: softmax, LayerNorm, loss
—	o estado do otimizador (m e v)

O ganho vem da coluna da esquerda; a estabilidade, da direita.

9.8.1 Por que os pesos mestres ficam em fp32

Este é o ponto menos intuitivo, e o `precision_bench.py` demonstra. Somando uma atualização típica de $1e-4$ a um peso de valor 1.0 , cem vezes:

```
peso = 1.0, atualizacao = 1e-04, aplicada 100 vezes:
fp32: 1.010002 (esperado 1.010000, erro 1.66e-06)
fp16: 1.000000 (esperado 1.010000, erro 1.00e-02)
bf16: 1.000000 (esperado 1.010000, erro 1.00e-02)
```

Leia com atenção: em 16 bits o peso **não se moveu nem um pouco** depois de cem atualizações. Não é um erro de arredondamento acumulado — é que cada soma individual foi arredondada de volta para 1.0 , todas as cem vezes.

Em 16 bits, uma atualização pequena somada a um peso de ordem 1 cai **abaixo do epsilon do formato** e é arredondada para fora — o peso simplesmente não muda. Repetido por milhares de passos, o treino estagna.

É por isso que a economia de memória real fica **abaixo** dos 50% que a aritmética ingênua sugere: você mantém uma cópia fp32 dos pesos além da cópia de trabalho em 16 bits.

9.9 Como se escreve na prática

Com fp16 — precisa de scaler:

```
scaler = torch.amp.GradScaler(device="cuda")
for x, y in dados:
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        loss = F.cross_entropy(modelo(x), y)      # matmuls em fp16
    opt.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()                 # escala o backward
    scaler.step(opt)                               # desescala e aplica
    scaler.update()                               # ajusta a escala
```

Com bf16 — não precisa de scaler:

```
for x, y in dados:
    with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
        loss = F.cross_entropy(modelo(x), y)
    opt.zero_grad(set_to_none=True)
    loss.backward()
    opt.step()
```

A segunda versão é mais simples, e é por isso que o bf16 virou padrão onde o hardware o suporta: **ele elimina uma engrenagem inteira do treino.**

9.10 Os limites deste capítulo, declarados

Sendo explícito sobre o que foi verificado e o que não pôde ser:

Dispositivo	fp16	bf16	autocast
CPU	[OK] funciona (mas centenas de vezes mais lento)	[OK] funciona	[OK] funciona
Radeon RX 7600 (DirectML)	[OK] funciona, ganho 0,98x	[nao] aborta o processo	[nao] não implementado
NVIDIA + CUDA	não testado aqui	não testado aqui	não testado aqui

O que **foi medido**: a anatomia dos formatos, os limites de alcance e precisão, o underflow de gradientes (50,3%), a janela do loss scaling, o comportamento do scaler dinâmico, a estagnação dos pesos em 16 bits, e a velocidade em CPU e nesta GPU.

O que **não foi medido**: o ganho de velocidade em tensor cores, o treino completo em bf16 na GPU, e o fp8 (que exige hardware classe H100). Onde eu não medi, não afirmo.

9.11 Resumo do capítulo

- Um float é [sinal | expoente | mantissa]. **Expoente = alcance; mantissa = precisão.**
- **fp16:** 5 bits de expoente -> estoura em 65.504 e zera abaixo de $\sim 6e-5$. **bf16:** 8 bits (como o fp32) -> mesmo alcance, mas epsilon 8x pior.
- Duas falhas distintas: **overflow** (vira inf) e **underflow** (vira zero). O segundo é pior porque é **silencioso** — o treino trava sem erro.
- **Para treinar, alcance > precisão.** Por isso o bf16 é o padrão do hardware moderno.
- **Loss scaling:** multiplica a loss por k, gradientes saem $\times k$, divide antes de atualizar. Existe uma **janela** de escalas seguras, e ela se move -> use um scaler **dinâmico** (implementamos um do zero).
- **Precisão mista** = 16 bits nas matmuls, 32 bits nos pesos mestres, somas longas e estado do otimizador.
- **Medido:** 50,3% dos gradientes zeram em fp16; o loss scaling recupera 100% deles.
- **Medido, e contra o discurso comum:** neste hardware, 16 bits **não acelera nada** (CPU centenas de vezes mais lenta; DirectML 0,98x). O ganho de memória é real; o de velocidade depende de hardware e backend.

9.11.1 O que vem no Capítulo 10

Fecha a trilogia "Need for Speed": no **Capítulo 10 — Distributed** vamos treinar em **vários dispositivos ao mesmo tempo**, entendendo o all-reduce que sincroniza os gradientes. Como esta máquina tem uma GPU só, verificaremos a **mecânica** com múltiplos processos na CPU — e o limite será declarado, como aqui.

-> Antes de seguir, faça os exercícios.

9.12 Exercícios

Faça na ordem. Soluções comentadas em solucoes/ — **tente antes de olhar.**

Os exercícios E1 a E5 rodam **na CPU** e não precisam de GPU. O E6 e o E7 precisam de GPU para a parte de velocidade.

9.12.1 E1 — Contas com bits (aquecimento)

Sem rodar código:

1. O bf16 tem 8 bits de expoente e 7 de mantissa; o fp16 tem 5 e 10. Qual dos dois representa o número 100000? E qual representa 0,000000001 (1e-9)?
2. Quantos valores distintos um formato com 7 bits de mantissa consegue representar **entre 1 e 2**? E com 10 bits? (Dica: é 2^{bits} .)
3. Por que dobrar os bits de mantissa **não** dobra a precisão, mas a multiplica por muito mais?

9.12.2 E2 — Ache o limite na mão

Escreva um laço que, começando em 1,0, multiplique por 2 até que o valor vire inf em fp16.

1. Em que potência de 2 isso acontece? Confere com `torch.finfo(torch.float16).max`?
2. Faça o mesmo dividindo por 2 até chegar a zero. Em que potência? Compare com `.tiny` — por que dá para ir **abaixo** do `.tiny` antes de zerar? (Pesquise "números subnormais".)
3. Repita para bf16 e explique a diferença.

9.12.3 E3 — O epsilon na prática

1. Em fp16, calcule $1.0 + x$ para $x = 1e-2, 1e-3, 1e-4$. A partir de qual valor a soma deixa de mudar o resultado?
2. Repita em bf16. O limite é maior ou menor? Por quê?
3. Relacione com a Seção 4 da apostila: por que os **pesos mestres** precisam ficar em fp32?

9.12.4 E4 — A janela do loss scaling

No `loss_scaling.py`, mexa no modelinho para que os gradientes fiquem **maiores** (aumente o `p.mul_(0.02)` para `p.mul_(0.5)`).

1. A porcentagem de gradientes zerados em fp16 muda?
2. A janela de escalas seguras se move? Para cima ou para baixo?
3. Isso justifica o scaler **dinâmico**? Explique com os seus números.

9.12.5 E5 — Melhore o GradScaler

O `GradScalerSimple`s da apostila reduz a escala pela metade a cada overflow e a dobra a cada N passos bons.

1. Qual o problema de um fator muito grande (ex.: 16)? E muito pequeno (ex.: 1,01)?
2. Implemente uma variante que, ao detectar overflow, **não** reduza abaixo de um piso configurável. Por que um piso é útil?
3. O `torch.amp.GradScaler` real usa `growth_interval=2000` por padrão, não 100. Por que um intervalo tão longo?

9.12.6 E6 — Seu hardware acelera 16 bits? (precisa de GPU)

Rode `precision_bench.py` na sua máquina.

1. O `fp16` é mais rápido que o `fp32` na sua GPU? Quanto?
2. **Cuidado com a conclusão:** na máquina desta apostila o ganho foi **nulo** (0,99x). Se o seu também for, isso significa que precisão reduzida é inútil? (Dica: releia a Seção 6 — há dois benefícios distintos.)
3. Se você tem NVIDIA: procure "tensor cores" na especificação da sua placa. O ganho que você mediu é compatível com o prometido?

9.12.7 E7 — Treine de verdade em precisão mista (desafio)

Pegue o `transformer.py` do Capítulo 5 e adapte para usar `torch.autocast`.

1. Com `bfloat16` (sem scaler): a loss final fica igual à do `fp32`? Compare com os números do Capítulo 5 (validação 1,811).
2. Com `float16` **sem** scaler: o treino degrada? Meça.
3. Com `float16` **com** `torch.amp.GradScaler`: recupera a qualidade?
4. Meça o **pico de memória** nos três casos (use `torch.cuda.max_memory_allocated()` se tiver NVIDIA). A economia chega aos 50% teóricos? Por que não?

10 DISTRIBUTED (TREINAR EM VÁRIOS DISPOSITIVOS)

Objetivo de aprendizagem: entender como N processos treinam **o mesmo modelo** ao mesmo tempo. Vamos implementar o **ring all-reduce** do zero, provar que o **DDP** reproduz exatamente o gradiente do batch inteiro, e ver como o **ZeRO** elimina a duplicação de memória que o DDP deixa passar.

Pré-requisitos: Capítulos 1-9. Fecha a Fase III ("Need for Speed").

Arquivos:

- `dist_utils.py` — infraestrutura (rendezvous e rede)
- `allreduce.py` — a primitiva, **implementada à mão** e verificada
- `ddp_train.py` — treino com DDP, com prova de equivalência
- `zero_memory.py` — a conta de memória e o ZeRO medido
- `exercicios.md` — exercícios

Sobre o hardware: esta máquina tem **uma** GPU, então não é possível medir ganho real de velocidade com múltiplas GPUs. O que **é** verificável — e é o conteúdo conceitual do capítulo — roda com **múltiplos processos na CPU** usando o backend `gloo`: a mecânica do all-reduce, a equivalência matemática do DDP e o fatiamento do ZeRO. Tudo neste capítulo foi executado de verdade; o que não foi medido está declarado na Seção 8.

10.1 A ideia: paralelismo de dados

O modelo cabe numa placa, mas os dados são muitos. A solução mais usada:

1. Todo processo tem uma **cópia completa e idêntica** do modelo.
2. Cada um recebe um **pedaço diferente** do batch.
3. Cada um calcula gradientes sobre o seu pedaço.
4. Um **all-reduce** faz a **média** dos gradientes.
5. Todos aplicam a mesma atualização -> os modelos continuam idênticos.

O passo 4 é o coração de tudo. Sem ele, os modelos divergiriam e você teria N modelos ruins em vez de um bom.

10.2 All-reduce: todos contribuem, todos recebem

```
t = torch.tensor([float(rank + 1)]) # rank 0 tem 1, rank 1 tem 2, ...
dist.all_reduce(t, op=dist.ReduceOp.SUM)
```

Com 4 processos:

```
[rank 0] tinha 1, depois do all_reduce SUM: 10 (esperado 10) OK
[rank 1] tinha 2, depois do all_reduce SUM: 10 (esperado 10) OK
[rank 2] tinha 3, depois do all_reduce SUM: 10 (esperado 10) OK
[rank 3] tinha 4, depois do all_reduce SUM: 10 (esperado 10) OK
```

Todos terminam com o mesmo valor. Não é "o rank 0 calcula e avisa os outros" — é uma operação coletiva em que cada participante entra com um valor e sai com o resultado combinado. É essa simetria que mantém os modelos sincronizados.

Para gradientes, queremos a **média**, que é SUM seguido de divisão:

```
dist.all_reduce(grad, op=dist.ReduceOp.SUM)
grad /= world_size
```

```
[rank 0] gradiente local 0.10 -> medio 0.2500
[rank 1] gradiente local 0.20 -> medio 0.2500
[rank 2] gradiente local 0.30 -> medio 0.2500
[rank 3] gradiente local 0.40 -> medio 0.2500
```

10.3 Como o all-reduce funciona por dentro: o anel

A implementação ingênua seria: todos mandam para o rank 0, ele soma e devolve. **Não escala.** O rank 0 recebe $(N-1) \times \text{tamanho}$ e devolve $(N-1) \times \text{tamanho}$ — ele vira um gargalo que **piora** conforme N cresce.

A solução usada na prática é o **ring all-reduce**, em que os processos formam um anel e cada um só fala com os vizinhos. Ele tem duas fases, e a chave é **fatiar o tensor em N pedaços**:

Fase 1 — reduce-scatter. Em $N-1$ passos, cada processo passa um pedaço ao vizinho da direita e soma o que recebe da esquerda. Ao final, cada processo tem **um** pedaço completamente somado (pedaços diferentes em cada um).

Fase 2 — all-gather. Em mais $N-1$ passos, esses pedaços já prontos circulam pelo anel até todos terem todos.

O ganho: **cada processo envia e recebe sempre a mesma quantidade** ($\sim 2 \times$ o tamanho do tensor), **independentemente de N** . É por isso que o ring all-reduce escala para centenas de nós.

Implementamos do zero em `allreduce.py`:

```
for passo in range(world_size - 1):
    idx_envio = (rank - passo) % world_size
    idx_recebe = (rank - passo - 1) % world_size
    recebido = torch.zeros_like(pedacos[idx_recebe])
    if rank % 2 == 0:
        dist.send(pedacos[idx_envio].contiguous(), dst=direita)
        dist.recv(recebido, src=esquerda)
    else:
```

```
dist.recv(recebido, src=esquerda)
dist.send(pedacos[idx_envio].contiguous(), dst=direita)
pedacos[idx_recebe] += recebido
```

O detalhe do par/ímpar: se todos os processos enviassem primeiro e o buffer do sistema enchesse, todos ficariam bloqueados esperando alguém receber — um **deadlock**. Alternar a ordem entre pares e ímpares garante que sempre há alguém pronto para receber.

E funciona:

```
[rank 0] nosso == PyTorch? True (primeiros 3: nosso [600.0, 604.0, 608.0], pytorch [600.0, 604.0, 608.0])
[rank 1] nosso == PyTorch? True
[rank 2] nosso == PyTorch? True
[rank 3] nosso == PyTorch? True
```

Nossa implementação bate com a do PyTorch — a quarta verificação contra a biblioteca de referência no curso, depois do autograd (Cap. 2), da LayerNorm (Cap. 5) e do AdamW (Cap. 7).

10.4 Quanto custa comunicar

Medido com 4 processos na mesma máquina:

Elementos	MB	Tempo	Vazão
10 mil	0,04	2,90 ms	14 MB/s
1 milhão	4,0	12,88 ms	310 MB/s
10 milhões	40,0	107,83 ms	371 MB/s

Note que tensores pequenos têm vazão péssima (14 MB/s): o custo fixo por operação domina — o mesmo padrão que vimos na GPU no Capítulo 8. Por isso frameworks **agrupam** gradientes em *buckets* antes de comunicar, em vez de fazer um all-reduce por parâmetro.

E aqui está o limite fundamental do treino distribuído:

A cada passo, **todo o gradiente do modelo atravessa a rede**. Um modelo de 1 bilhão de parâmetros gera **4 GB de gradientes por passo**. Se a comunicação demorar mais que o cálculo, acrescentar máquinas **para de ajudar**. É por isso que existem NVLink e InfiniBand — e por que a topologia da rede é parte do projeto do treino.

10.5 DDP: o gradiente é o mesmo do batch inteiro?

Esta é a propriedade que justifica tudo. O `DistributedDataParallel` dispara o `all_reduce` automaticamente durante o `backward()`. A pergunta: o resultado é **exatamente** o mesmo de treinar com o batch inteiro num processo só?

O `ddp_train.py` compara diretamente — calcula o gradiente de referência com batch 256 num processo, e o gradiente do DDP com 4 processos de 64 cada:

```
batch total 256, dividido em 4 pedacos de 64
[rank 0] diferenca maxima vs batch unico: 1.12e-08 OK
[rank 1] diferenca maxima vs batch unico: 1.12e-08 OK
[rank 2] diferenca maxima vs batch unico: 1.12e-08 OK
[rank 3] diferenca maxima vs batch unico: 1.12e-08 OK
```

São iguais, a menos de arredondamento. O DDP não aproxima nada: ele reproduz a matemática exata do batch inteiro.

E ao longo do treino, os modelos continuam sincronizados:

```
[rank 0] loss final 2.1846 | pesos identicos entre ranks: True
[rank 1] loss final 2.3620 | pesos identicos entre ranks: True
[rank 2] loss final 2.1699 | pesos identicos entre ranks: True
[rank 3] loss final 2.3078 | pesos identicos entre ranks: True
```

Repare que as losses são diferentes! Cada rank imprime a loss do **seu pedaço** do batch. Para registrar a loss real do treino é preciso fazer `all_reduce` dela também — e esquecer disso é um erro comum, que faz o gráfico de treino parecer mais ruidoso do que é.

10.6 O cuidado que mais causa problema: batch efetivo

Com 4 processos de 64 exemplos cada, o batch **efetivo** é **256**, não 64. Isso muda um hiperparâmetro importante:

- Gradiente de batch maior é **menos ruidoso**.
- Logo, tolera (e costuma pedir) **learning rate maior**.
- Regra prática usual: multiplicar a lr por $\sqrt{\text{world_size}}$, ou linearmente com warmup mais longo.

Trocar 1 GPU por 8 **sem tocar na learning rate** é um erro clássico: cada passo fica mais rápido, mas o treino precisa de **mais passos** para chegar ao mesmo lugar — e o ganho evapora. Releia o exercício E5 do Capítulo 7: a melhor learning rate depende do tamanho do batch.

10.7 ZeRO: o desperdício que o DDP não resolve

O DDP resolve **velocidade**, não **memória**. Cada processo guarda uma cópia completa de tudo. Fazendo a conta, por parâmetro, num treino com AdamW em fp32:

Componente	Bytes/param
pesos	4
gradientes	4
AdamW: momento m	4
AdamW: momento v	4
TOTAL	16 — em cada processo

Um modelo de 1 bilhão de parâmetros pede **16 GB por GPU**, e esse número **não diminui** por mais GPUs que você acrescenta.

O **ZeRO** (*Zero Redundancy Optimizer*) elimina a duplicação em três estágios:

Processos	DDP	ZeRO-1	ZeRO-2	ZeRO-3
1	16,0 GB	16,0 GB	16,0 GB	16,0 GB
4	16,0 GB	10,0 GB	7,0 GB	4,0 GB
8	16,0 GB	9,0 GB	5,5 GB	2,0 GB
64	16,0 GB	8,1 GB	4,2 GB	0,25 GB

- **ZeRO-1** fatia o estado do otimizador (m, v)
- **ZeRO-2** fatia também os gradientes
- **ZeRO-3** fatia também os pesos, buscados sob demanda durante o forward

Leia a linha de 64 processos: o DDP continua exigindo 16 GB por GPU — o mesmo de rodar sozinho. O ZeRO-3 pede 0,25 GB. É a diferença entre "não cabe" e "cabe folgado".

O preço é comunicação. O ZeRO-3 precisa buscar pesos de outros processos durante o forward e o backward. Troca-se memória por banda — e o estágio certo depende de qual é o seu gargalo.

10.7.1 Medido de verdade

O `zero_memory.py` usa o `ZeroRedundancyOptimizer` do PyTorch (ZeRO-1) num modelo de 1.067.040 parâmetros, com 4 processos:

```
[rank 0] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.23 MB (3.8x menor)
[rank 1] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.11 MB (4.1x menor)
[rank 2] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.10 MB (4.1x menor)
[rank 3] estado do otimizador: AdamW normal 8.54 MB | ZeRO-1 2.10 MB (4.1x menor)
```

Cada processo guarda ~1/4 do estado, exatamente a promessa. (Os pesos e gradientes seguem duplicados — é o que os estágios 2 e 3 atacam.)

10.8 Um problema real de infraestrutura, e a lição

Este capítulo quase não aconteceu, e a razão vale mais que o código.

Ao rodar os primeiros testes, os processos **travavam para sempre** no `init_process_group` — sem erro, sem mensagem, sem timeout. Só um programa pendurado.

A causa levou um tempo para ser encontrada:

```
hostname: SANCHEZPC1
resolve para: 54.232.189.113 <- um IP PÚBLICO, não um endereço local
```

Mesmo rodando tudo numa máquina só, o gloo abre sockets TCP entre os processos, e ele escolhe a interface **resolvendo o hostname da máquina**. Nesta máquina o hostname resolvia para um IP público (por causa de um adaptador virtual), e o Windows recusava a conexão com o erro 10049 — "endereço não é válido no contexto".

A correção é dizer explicitamente qual interface usar:

```
GLOO_SOCKET_IFNAME=Ethernet python allreduce.py
```

E o `dist_utils.py` faz isso **automaticamente**: detecta que o hostname não resolve para um endereço local, procura um adaptador com IP privado e configura a variável.

A lição: em treino distribuído, o modo de falha mais comum não é o erro — é o **travamento silencioso**. Rendezvous que não completa, interface errada, um processo que morreu e deixou os outros esperando no `barrier`. Antes de suspeitar do seu código, verifique a rede. E sempre rode com timeout.

10.8.1 O que não foi possível medir aqui

Sendo explícito, como no Capítulo 9:

Item

Verificado

Como

Mecânica do all-reduce	[OK]	4 processos, CPU, backend gloo
Ring all-reduce do zero	[OK]	comparado com o do PyTorch
Equivalência do DDP	[OK]	gradiente vs batch único: 1,12e-08
Sincronização dos pesos	[OK]	all_gather após 50 passos
ZeRO-1	[OK]	ZeroRedundancyOptimizer, 4 processos
Speedup real com N GPUs	[nao]	a máquina tem uma GPU só
Backend NCCL	[nao]	exige GPU NVIDIA
ZeRO-2 / ZeRO-3	[nao]	tabela é aritmética; não medidos

10.9 Resumo do capítulo

- **Paralelismo de dados:** cópias idênticas do modelo, pedaços diferentes do batch, gradientes combinados por **all-reduce**.
- **All-reduce** é coletivo: todos contribuem e todos recebem o mesmo resultado.
- O **ring all-reduce** faz cada processo trocar sempre a mesma quantidade de dados, independentemente de N — por isso escala. Implementamos do zero e bate com o PyTorch.
- **Comunicação é o gargalo:** todo o gradiente atravessa a rede a cada passo. Tensores pequenos têm vazão péssima -> frameworks agrupam em *buckets*.
- **DDP reproduz exatamente** o gradiente do batch inteiro (verificado: 1,12e-08), e mantém os pesos idênticos entre processos.
- **Batch efetivo** = batch local × world_size -> **ajuste a learning rate**.
- **ZeRO** elimina a duplicação do estado: com 64 processos, 16 GB por GPU (DDP) contra 0,25 GB (ZeRO-3). Medimos o ZeRO-1: 4,1x menos estado por processo.
- **Falhas em distribuído são silenciosas.** O maior obstáculo aqui foi um hostname que resolvia para um IP público — e o sintoma era o programa pendurado.

10.9.1 O que vem no Capítulo 11

Fecha a Fase III e começa a Fase IV. E é a virada que muda o curso: no **Capítulo 11** — **Datasets** trocamos os nomes por **texto de verdade**. O modelo deixa de gerar nomes e passa a gerar prosa — e o nosso benchmark de 1,776 será aposentado, porque a tarefa muda.

-> Antes de seguir, faça os exercícios.

10.10 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

Todos os exercícios rodam na CPU, com múltiplos processos. Não é preciso ter GPU nem mais de uma máquina.

Se algo travar: rode `python dist_utils.py` para diagnosticar a rede, e sempre use `timeout` ao experimentar (`timeout 60 python ...` no Linux/Git Bash). Travamento silencioso é o modo de falha normal aqui — veja a Seção 8 da apostila.

10.10.1 E1 — Entendendo o coletivo (aquecimento)

Sem rodar:

1. Com 4 processos onde o rank i tem o valor i , qual o resultado do `all_reduce` com SUM em cada rank? E com MAX?
2. Qual a diferença entre `all_reduce` e `reduce`? E entre `all_gather` e `gather`?
3. Por que o broadcast **não** é suficiente para sincronizar gradientes? (Pense: quem teria o valor correto para transmitir?)

10.10.2 E2 — Escalando o número de processos

Rode `python allreduce.py N` com $N = 2, 4$ e 8 .

1. O tempo do all-reduce cresce com N ? Meça com **três tamanhos** de tensor — 10 mil, 1 milhão e 10 milhões de elementos — antes de responder. A resposta é a mesma nos três?
2. Compare com o que aconteceria no esquema ingênuo (todos -> rank 0 -> todos): esse tempo cresceria **como** em função de N ?
3. Sua máquina tem quantos núcleos? A partir de qual N os processos passam a competir por CPU em vez de somar? (Rode `python dist_utils.py` para ver.)

10.10.3 E3 — Provoque um deadlock (importante)

No `ring_allreduce_manual`, remova a alternância par/ímpar — faça **todos** os processos chamarem `dist.send` antes de `dist.recv`.

1. **Preveja antes de rodar.** A explicação clássica do deadlock em `send/recv` é o **buffer de socket**: o sistema operacional aceita mensagens pequenas e devolve o controle na hora, então só as grandes travariam. Sob essa teoria, o que deveria acontecer com um tensor de 100 elementos? E com um de 5 milhões?
2. Agora **meça**, com os dois tamanhos — e também com um tensor de **um único elemento**. O resultado bate com a sua previsão? (Rode cada caso com `timeout`; o objetivo aqui é justamente travar o programa.)
3. Se a previsão falhou, o que isso diz sobre aplicar um modelo mental correto *em geral* a uma implementação específica? E sobre um backend que falha **sempre** em vez de falhar só às vezes — isso é um defeito ou um recurso?

10.10.4 E4 — A loss que engana

No `ddp_train.py`, os ranks imprimem losses diferentes (2,18 / 2,36 / 2,17 / 2,31).

1. Por quê, se os pesos são idênticos?
2. Implemente o registro **correto** da loss: faça `all_reduce` dela e divida por `world_size`. Os valores passam a bater entre os ranks?
3. Se você registrasse só a loss do rank 0 num gráfico, o que aconteceria com a aparência da curva?

Solução de referência: `solucoes/e4_loss_correta.py`.

10.10.5 E5 — Batch efetivo e learning rate

Treine o `ddp_train.py` com 1, 2 e 4 processos, mantendo o batch **por processo** fixo.

1. O batch efetivo muda. A loss após 50 passos melhora, piora ou fica igual?
2. Agora ajuste a learning rate por $\sqrt{\text{world_size}}$ e repita. Muda a conclusão?
3. Por que "mais GPUs" pode deixar o treino **mais lento em número de passos** se você não mexer na learning rate?

10.10.6 E6 — Meça o ZeRO nos três estágios

O `zero_memory.py` mede o ZeRO-1 de verdade e calcula os estágios 2 e 3 por aritmética.

1. Confirme a conta do ZeRO-1: com N processos, o estado medido é mesmo $\sim 1/N$?
2. A tabela assume AdamW em fp32. Refaça a conta para **SGD sem momentum** (que não guarda estado). O ZeRO ainda ajuda? Quanto?
3. E se combinássemos com o Capítulo 9 (bf16 nos pesos)? Recalcule a linha de 8 processos.

10.10.7 E7 — Quando distribuir *não* compensa (desafio)

Junte o que você mediu neste capítulo e no Capítulo 8.

1. Estime: para o modelo pequeno do curso (153 mil parâmetros), quanto tempo levaria o all-reduce dos gradientes por passo? Compare com o tempo de cálculo de um passo (12,8 ms na CPU, do Capítulo 8).
2. Distribuir esse modelo em 4 processos aceleraria ou atrasaria o treino?
3. Formule a regra geral: qual a relação entre **tempo de cálculo** e **tempo de comunicação** que decide se vale a pena distribuir?

11 DATASETS (A VIRADA PARA TEXTO DE VERDADE)

Objetivo de aprendizagem: montar o **pipeline de dados** completo de um LLM — baixar, limpar, dividir, tokenizar e gravar em formato eficiente. E, com isso, fazer a maior mudança do curso: o modelo deixa de gerar **nomes** e passa a gerar **prosa em português**.

Pré-requisitos: Capítulos 1-10. Em especial o BPE do Capítulo 6, que usamos aqui.

Arquivos:

- `prepare_data.py` — o pipeline completo (baixar -> limpar -> dividir -> tokenizar -> gravar)
- `dataset.py` — carregamento com `mmap` e formação de batches
- `train_text.py` — o Transformer treinado em prosa
- `bpe.py` — o tokenizador do Capítulo 6
- `exercicios.md` — exercícios

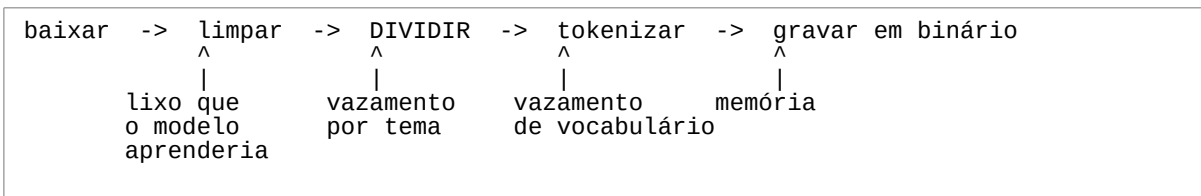
Aviso sobre a métrica: a loss deste capítulo **não é comparável** com o 1,776 dos capítulos anteriores. Mudou a tarefa (prosa em vez de nomes), mudou a unidade (tokens BPE em vez de caracteres) e mudou o vocabulário (1024 em vez de 27). Prever entre 1024 opções é muito mais difícil que entre 27. O benchmark antigo está **aposentado**; este capítulo estabelece o novo ponto de partida.

11.1 Por que os dados são o capítulo mais subestimado

Nos capítulos 1 a 10 o dataset era uma lista de nomes que cabia num tensor. Isso permitiu focar na arquitetura — mas escondeu que, num LLM de verdade, **a maior parte do trabalho de engenharia está nos dados**, não no modelo.

O modelo do GPT-3 cabe em algumas centenas de linhas. O corpus dele tem ~570 GB filtrados a partir de muito mais. A proporção de esforço é essa mesma.

Este capítulo monta o pipeline inteiro, e cada etapa tem uma armadilha:



11.2 O corpus: Machado de Assis

Escolhemos as obras de **Machado de Assis** (1839-1908), obtidas do **Project Gutenberg**. Três razões:

1. **É português literário de verdade** — prosa complexa, vocabulário rico, pontuação variada. Bem mais difícil (e mais interessante) que nomes.
2. **A licença é limpa.** Machado morreu em 1908; a obra está em domínio público. Num curso que ensina a treinar modelos, ignorar a procedência dos dados seria um mau exemplo — a origem e a licença do corpus são parte do trabalho.
3. **É cultural e linguisticamente nosso**, o que combina com um curso em português.

Dom Casmurro	410037	->	378215	chars	(7.8% removido)
Memórias Postumas de Bras Cubas	396533	->	363830	chars	(8.2% removido)
Quincas Borba	484321	->	453033	chars	(6.5% removido)
Esau e Jacó	444833	->	414801	chars	(6.8% removido)
Memorial de Aires	316580	->	284854	chars	(10.0% removido)

11.2.1 Limpeza: o que aqueles 7-10% eram

Os arquivos do Gutenberg vêm com cabeçalho de licença, créditos e metadados **em inglês**. Deixar isso no corpus ensinaria o modelo a escrever termos de licença americanos no meio de um romance brasileiro — lixo competindo com o texto que interessa. O `prepare_data.py` corta pelos marcadores:

```

inicio = re.search(r'""\s*\s*START OF TH[EIS] PROJECT GUTENBERG EBOOK.*?\s*\s*"', texto)
fim     = re.search(r'""\s*\s*END OF TH[EIS] PROJECT GUTENBERG EBOOK.*?\s*\s*"', texto)

```

11.2.2 Deduplicação

Corpus real tem repetição: sumários, cabeçalhos, trechos citados. Texto duplicado faz o modelo **memorizar** em vez de generalizar, e contamina a avaliação se a mesma passagem cair nos dois lados da divisão. Removemos parágrafos idênticos (com mais de 40 caracteres, para não apagar repetições legítimas como "— Sim.>").

11.3 A divisão: onde quase todo mundo erra

Duas decisões que parecem detalhes e não são.

11.3.1 Dívida por documento, não por linha sorteada

O jeito errado, e muito comum: juntar tudo, embaralhar os parágrafos, separar 15% para validação. Parece razoável e é **vazamento**.

Se você embaralhar os parágrafos de *Dom Casmurro*, o modelo treina no capítulo inteiro e é avaliado em frases **do mesmo capítulo** — mesmos personagens, mesmo assunto, mesmo estilo, às vezes as mesmas construções. A loss de validação fica boa e **não significa**

nada.

O jeito certo é separar por **obra inteira**:

```
treino: 4 obras, 1609699 chars (Dom Casmurro, Brás Cubas, Quincas Borba, Esaú e Jacó)
val   : 'Memorial de Aires', 284854 chars
proporcao: 15.0% para validacao
```

Memorial de Aires é um livro que o modelo **nunca vê**. Avaliar nele mede o que a gente quer medir: o modelo aprendeu **português e o estilo de Machado**, ou decorou frases?

11.3.2 Quanto isso muda na prática — medido

A solução do exercício E2 quantifica o vazamento sem precisar treinar nada: conta quantas sequências de N tokens do conjunto de validação **já aparecem literalmente** no treino.

n-grama	Divisão correta	Divisão errada	Razão
3 tokens	26,74%	31,10%	1,2x
5 tokens	2,82%	4,86%	1,7x
8 tokens	0,24%	0,59%	2,4x
12 tokens	0,13%	0,16%	1,3x

Os n-gramas **curtos** aparecem muito nos dois casos — isso é só a língua portuguesa ("de que a", "não se"), não vazamento. Os **longos** é que denunciam: uma sequência específica de 8 tokens aparecer nos dois lados não acontece por acaso.

O efeito é consistente (a divisão errada perde em todos os tamanhos), mas seja honesto sobre a **magnitude**: aqui ele é **moderado**, não dramático. Duas razões, e ambas ensinam algo:

1. Nosso embaralhamento é de blocos de 512 tokens. Trechos diferentes do mesmo livro ainda são texto diferente. Com embaralhamento por **frase**, ou com documentos duplicados, o vazamento seria bem maior.
2. Este corpus é de **um autor só**. Machado escreve parecido em todos os livros, então até a divisão honesta tem sobreposição real de estilo. Isso é informação legítima.

Em corpus da web — onde o mesmo artigo aparece copiado em dezenas de sites — o efeito é muito maior. É por isso que **deduplicação** é passo obrigatório nos pipelines de verdade, e por que laboratórios publicam quanto do benchmark vazou para o treino.

11.3.3 Dívida ANTES de tokenizar

Sutil e importante: o tokenizador é **treinado nos dados**. Se ele vir o texto de validação, ele aprende o vocabulário de lá — nomes de personagens, expressões específicas — e a avaliação fica otimista. É vazamento por outro caminho.

```
tok.train(texto_treino[:BYTES_TREINO_TOKENIZADOR], VOCAB_SIZE)
# ~~~~~ só o treino, nunca o de validação
```

11.4 Tokenização: o BPE aprende o domínio

Usamos o BPE do Capítulo 6, com vocabulário de 1024. Veja o que ele aprendeu deste corpus:

```
tokens mais longos: 'José Dias ', 'ima Justin', 'que não ', 'minha mã',
                   'Capitú, ', 'os olhos ', 'da minha ', 'Capitú '
```

Ele descobriu sozinho os **personagens de Dom Casmurro** — José Dias, Capitú, Prima Justina — e os transformou em tokens únicos. Ninguém disse a ele que são nomes; ele apenas notou que essas sequências de bytes se repetem muito.

Isso é ótimo para este corpus e revelador sobre a natureza dos tokenizadores:

Um tokenizador é um modelo estatístico do corpus em que foi treinado. Ele comprime muito bem o que conhece, e mal o que não conhece. Um tokenizador treinado em Machado gastaria muitos tokens num artigo de medicina — e vice-versa. Foi a mesma lição do Capítulo 6, agora com consequência prática.

Medindo a compressão no corpus inteiro:

Split	Caracteres	Tokens	Compressão
treino (4 obras)	1.609.699	621.134	2,64x
validação (1 obra)	284.854	108.628	2,68x

A compressão se sustenta no texto de validação (2,68x contra 2,64x) — o tokenizador aprendeu o **padrão** do português literário, não decorou as frases do treino. Compare com o Capítulo 6, onde a compressão caiu de 2,24x para 1,20x ao sair do domínio: aqui não cai, porque validação e treino são o mesmo domínio (prosa do mesmo autor), só que textos diferentes.

Detalhe de implementação: o BPE em Python puro é lento, então o treinamos num subconjunto de 300 mil caracteres (34 segundos) e o **aplicamos** ao corpus inteiro. E a tokenização é feita parágrafo a parágrafo, não no texto todo de uma vez — cada chamada ao **encode** percorre a sequência repetidamente, então textos curtos são muito mais rápidos. Foram 67s para o treino e 11s para a validação.

11.5 Gravar em binário: uint16 e mmap

Duas decisões de formato que parecem prematuras e não são.

Por que uint16? O vocabulário tem 1024 tokens, então cada token cabe em 16 bits. Guardar em int64 (o padrão do PyTorch) gastaria **4x mais espaço** para representar exatamente a mesma informação.

Por que binário puro, sem cabeçalho? Porque permite `np.memmap`:

```
dados = np.memmap("treino.bin", dtype=np.uint16, mode="r")
```

O `memmap` **não lê o arquivo** — ele mapeia o arquivo no espaço de endereçamento, e o sistema operacional traz do disco apenas as páginas efetivamente tocadas. Medindo:

```
np.fromfile (carrega tudo):      0.5 ms, 1.2 MB na RAM
np.memmap   (so' mapeia)  :      0.4 ms, ~0 MB na RAM
```

E a formação de batches é rápida o bastante para nunca ser o gargalo:

batch × block	Tempo	Vazão
32 × 64	0,89 ms	2,29 M tokens/s
64 × 128	1,65 ms	4,96 M tokens/s
128 × 256	3,36 ms	9,76 M tokens/s

Se o carregamento de dados for mais lento que o passo de treino, a GPU fica **esperando** — e você paga por uma placa ociosa. É por isso que frameworks usam `DataLoader` com workers paralelos e *prefetch*. Aqui, como cada batch é só a leitura de fatias contíguas, o custo é desprezível perto do forward/backward.

Com um corpus deste tamanho a diferença é irrelevante. A questão é o que acontece com 100 GB: o `fromfile` simplesmente não roda, e o `memmap` continua funcionando igual. Escrever assim desde o início custa nada e evita reescrever o pipeline depois.

11.6 A mudança no formato do treino

Aqui há uma diferença importante em relação aos capítulos 3-7.

Antes: cada exemplo era uma janela de contexto com preenchimento, e havia **um** alvo (o próximo caractere). Um batch de 64 exemplos gerava 64 previsões.

Agora: o texto é um **fluxo contínuo**, e cada posição tem um alvo. O truque é elegante:

```
x = tokens[i      : i + block_size]      # entrada
y = tokens[i + 1 : i + block_size + 1]  # alvo = a entrada DESLOCADA DE UM
```

O alvo é o próprio texto deslocado de um token. E o modelo prevê em **todas** as posições:

```
logits = self.lm(self.ln(x))              # (B, T, vocab)
loss = F.cross_entropy(logits.view(-1, vocab_size), alvos.view(-1))
```

Com `batch_size=32` e `block_size=128`, isso dá **4.096 previsões por batch** em vez de 32 — o sinal de treino fica 128x maior **pelo mesmo custo de forward**. É assim que LLMs são treinados de verdade.

E note o que isso significa: não existe rótulo humano nenhum aqui. O texto já é a resposta. É por isso que se chama aprendizado **auto-supervisionado**, e é por isso que dá para treinar com a internet inteira sem ninguém anotar nada.

11.7 Resultados: o modelo escreve prosa

O `train_text.py` treina um Transformer de **2.196.352 parâmetros** (4 blocos, 6 cabeças, `n_embd=192`, contexto de **128 tokens**) por 3.000 passos — cerca de 18 minutos na CPU.

```
passo    0 | treino 7.1026 | val 7.1043 |    3s
passo  500 | treino 4.3011 | val 4.5850 |   193s
passo 1000 | treino 3.5819 | val 4.0891 |   374s
passo 1500 | treino 3.2023 | val 3.9380 |   552s
passo 2000 | treino 2.9862 | val 3.9176 |   730s
passo 2500 | treino 2.8253 | val 3.8969 |   911s
```

```
FINAL: treino 2.7569 | validacao 3.9450
perplexidade de validacao: 51.7
```

A **perplexidade** é $\exp(\text{loss})$, e tem leitura intuitiva: é o número médio de opções entre as quais o modelo está "em dúvida" a cada token. Um modelo que chutasse uniformemente entre 1024 tokens teria perplexidade 1024. O nosso está em **51,7** — ele reduziu o espaço de dúvida em 20 vezes.

11.7.1 O overfitting é real, e a conta explica

Repare no que aconteceu a partir do passo 1500: a loss de **treino** continua caindo (3,20 -> 2,76) enquanto a de **validação** praticamente estaciona (3,94 -> 3,95). Isso é *overfitting*, o mesmo fenômeno do Capítulo 3 — e desta vez a causa é fácil de calcular:

Grandeza	Valor
Parâmetros do modelo	2.196.352

Tokens de treino	621.134
Tokens por parâmetro	0,28

Temos **mais parâmetros que tokens de treino**. A regra prática mais conhecida (*Chinchilla*, DeepMind 2022) sugere cerca de **20 tokens por parâmetro** para um treino equilibrado — estamos ~70x abaixo disso. Com essa proporção, memorizar é mais fácil que generalizar, e o modelo faz o que é mais fácil.

Duas saídas, e é útil saber qual escolher: menos parâmetros (modelo menor) ou mais dados. O exercício E7 explora a segunda — as obras completas de Machado no Gutenberg são 12, não 5.

11.7.2 O que ele escreve

Este é o ponto do capítulo. Com temperatura 0,8:

de cós. Leito de Baptista tinha
de fazer um grande e outro lado da beleza da devida; era a nossa gente;
a dissesse a figura de seis velhos, eu represso para os dias de
roupa da liberdade de Fulher e adiante nas caras, mas os homens, a primeira
mão; a filha não é só um collegio, o imperiogro, nem as alegres
de uma boca philosophias na vespera á mãe que a filha não aliás
instituição. Um collo jura

Não é Machado, e boa parte não faz sentido semântico — esperado para 2,2 milhões de parâmetros treinados em 1,6 MB de texto. Mas olhe **o que já está lá**:

- **é português**, com concordância local plausível ("a filha não é só um colégio")
- **pontuação correta**: vírgulas, ponto e vírgula, pontos finais nos lugares certos
- **estrutura de diálogo**, com o travessão que Machado usa (--Sim, ...)
- **quebras de parágrafo** e ritmo de frase
- **a ortografia de época** do corpus (beleza, collegio, philosophias) — ele aprendeu a escrever como se escrevia em 1880, porque é isso que os dados ensinam

Compare com o que o mesmo curso gerava antes:

Capítulo	Saída
01 (bigrama)	cexzma, zktahwelo
05 (Transformer)	jandir, valdinia
11 (prosa)	frases em português, com pontuação e diálogo

O modelo não ficou "mais inteligente" — a arquitetura é a mesma do Capítulo 5, só um pouco maior. **Os dados é que mudaram.** É a demonstração mais direta possível da tese deste capítulo.

11.8 Resumo do capítulo

- **Dados são a maior parte do trabalho** de um LLM, e o capítulo mais subestimado.
- **Procedência importa:** usamos domínio público (Machado de Assis, Gutenberg) e declaramos a fonte. Num curso sobre treinar modelos, isso é parte do ofício.
- **Limpeza:** cabeçalhos de licença em inglês seriam aprendidos como texto — 7-10% do arquivo era isso.
- **Deduplicação** evita memorização e contaminação da avaliação.
- **Divida por documento, não por linha sorteada** — senão o modelo é avaliado em texto quase idêntico ao que treinou.
- **Divida antes de tokenizar** — o tokenizador aprende com os dados, e vazaria também.
- **O BPE aprende o domínio:** descobriu 'José Dias ' e 'Capitú, ' como tokens únicos.
- **uint16 + mmap:** 4x menos espaço e leitura sem carregar na RAM — requisito, não otimização, quando o corpus cresce.
- **Prever em todas as posições** multiplica o sinal de treino por `block_size` de graça.

11.8.1 O que vem no Capítulo 12

Temos um modelo que escreve prosa — e que é **lento** para gerar, porque recalcula tudo a cada token novo. No **Capítulo 12 — Inference I: KV-cache** vamos guardar as chaves e valores já calculados e acelerar a geração em várias vezes, sem mudar uma vírgula do que o modelo produz.

-> Antes de seguir, faça os exercícios.

11.9 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar.**

Rode `python prepare_data.py` uma vez antes dos exercícios (ele baixa e prepara o corpus; os livros ficam em cache, então rodar de novo é rápido).

11.9.1 E1 — Por que essa ordem? (aquecimento)

Sem rodar:

1. Por que o tokenizador é treinado **só** no texto de treino? O que exatamente vazaria se ele visse o texto de validação?
2. Por que dividimos **por obra** e não sorteando parágrafos? Dê um exemplo concreto do que o modelo "veria de graça" na divisão errada.
3. Por que gravamos os tokens como `uint16` e não `int64`? Quanto de espaço isso poupa?

11.9.2 E2 — Faça a divisão errada de propósito (importante)

Modifique o `prepare_data.py` para juntar **todas as 5 obras**, embaralhar os parágrafos e separar 15% aleatoriamente para validação.

1. Treine o modelo com essa divisão. A loss de validação fica **melhor** ou pior?
2. Ela ficou melhor? Isso significa que o modelo é melhor? Explique o que aconteceu.
3. Esse é um dos erros mais comuns em ML aplicado — e ele faz o modelo parecer bom no relatório e falhar na prática. Como você detectaria isso numa revisão de código?

Solução de referência: `solucoes/e2_vazamento.py`.

11.9.3 E3 — O tokenizador é do domínio

O BPE treinado neste corpus aprendeu tokens como `'José Dias '`, `'Capitú, '` e `'minha mã'`.

1. Liste os 20 tokens mais longos aprendidos. Quantos são nomes de personagens?
 2. Isso é bom ou ruim? (Pense: e se você usasse este tokenizador para tokenizar um texto de medicina?)
 3. Treine um BPE no corpus de **nomes** do Capítulo 6 e tokenize um trecho de Machado com ele. Quantos tokens a mais são necessários?
-

11.9.4 E4 — Tamanho do contexto (o mais instrutivo do capítulo)

No `train_text.py`, varie o `block_size`: 32, 128 e 256.

1. **Preveja primeiro.** Prosa tem dependências longas (concordância, referência a personagens) que nomes de 7 letras não têm. Qual `block_size` deve vencer? Agora meça, com **poucos passos** (400 serve) — bateu com a previsão?
2. **Meça de novo com o orçamento cheio** (3.000 passos). O ranking é o mesmo? Olhe também a diferença entre a loss de treino e a de validação em cada configuração.
3. O custo da atenção cresce com T^2 . Dobrar o contexto quadruplica o tempo por passo?
4. Com o número de **passos** fixo, quantas vezes cada configuração passa pelo corpus inteiro? (Conta: $\text{batch} \times \text{block} \times \text{passos} \div \text{tokens do corpus}$.) O que isso explica sobre o que você viu no item 2?

11.9.5 E5 — Prever em todas as posições

Este capítulo mudou o modelo para prever em **todas** as posições, não só na última.

1. Quantas previsões por batch o modelo faz agora, contra antes? (Use `batch_size` e `block_size`.)
2. Modifique o `forward` para usar só a última posição (como nos capítulos 5-7) e treine com o mesmo orçamento de passos. A loss final é pior? Muito?
3. O custo do `forward` mudou? Se não mudou, o que isso diz sobre "aproveitar o cálculo"?

11.9.6 E6 — Dados sintéticos (desafio)

O syllabus deste capítulo inclui **geração de dados sintéticos**. Use o modelo treinado para gerar 200 KB de texto, e depois treine um **segundo** modelo só com esse texto gerado.

1. O segundo modelo chega perto do primeiro? Ou fica pior?
2. Esse fenômeno tem nome na literatura (*model collapse*). Explique com os seus números por que treinar em dados gerados pelo próprio modelo degrada a qualidade.
3. Em que situações dados sintéticos **ajudam** de verdade? (Dica: pense em tarefas onde é possível **verificar** a resposta, como matemática ou código.)

11.9.7 E7 — Escalando o corpus (desafio)

O `prepare_data.py` usa 5 obras. A lista completa de Machado no Project Gutenberg tem 12.

1. Acrescente mais obras e refaça o preparo. Como muda a loss de validação?
2. Mantenha o modelo do mesmo tamanho. A partir de quanto texto o ganho satura?
3. Relacione com o Capítulo 3: lá, aumentar os dados de 155 para 64 mil nomes resolveu o *overfitting*. A mesma lógica vale aqui? Compare as losses de treino e validação.

12 INFERENCE I: KV-CACHE

Objetivo de aprendizagem: entender por que gerar texto é **desperdício** do jeito ingênuo, e construir o KV-cache — a otimização que torna a inferência de LLMs viável. E, junto com ela, aprender a distinção entre **prefill** e **decode**, que organiza tudo o que vem depois.

Pré-requisitos: Capítulos 4 (atenção), 5 (Transformer) e 11 (o modelo treinado).

Arquivos:

- `modelo.py` — o GPT do Capítulo 11 com KV-cache, e o teste de equivalência
- `gerar.py` — escreve texto usando o cache
- `benchmark_cache.py` — quanto acelera e quanto de memória custa
- `exercicios.md` — exercícios

Este capítulo não treina nada. Ele usa os pesos que o Capítulo 11 salvou em `modelo.pt` — que está **versionado no repositório** (8,6 MB), então tudo aqui roda logo após o clone, sem esperar os 18 minutos de treino do Cap. 11.

Nenhum número de qualidade muda neste capítulo — e é exatamente esse o ponto: a otimização precisa ser **invisível na saída** e visível só no relógio.

12.1 Um modelo que faz a mesma conta 128 vezes

Volte ao `gerar()` do Capítulo 11:

```
for _ in range(n_tokens):
    recorte = idx[:, -block_size:]      # todo o contexto
    logits, _ = self(recorte)          # forward COMPLETO
    logits = logits[:, -1, :]           # ... e joga fora tudo menos a última posição
    ...
```

Leia de novo a terceira e a quarta linha. O modelo processa **todas** as posições do contexto e usa **uma**.

E na iteração seguinte ele faz tudo de novo — com o mesmo contexto de antes, mais um token. As posições 0 a 126 são recalculadas do zero, produzindo exatamente os mesmos números que produziram um instante atrás.

Para gerar 128 tokens com contexto 128, o modelo executa:

Caminho	Posições processadas
Trabalho útil	128 (uma por token gerado)
Trabalho realizado	$1 + 2 + 3 + \dots + 128 = 8.256$

Mais de 98% do cálculo é jogado fora. Não é uma ineficiência de implementação — é o algoritmo, escrito da forma mais direta possível.

12.1.1 E quanto isso vale no relógio? Menos do que você espera

Guarde este número antes de seguir, porque ele evita uma conclusão errada. Eliminando 98,4% do trabalho, o ganho medido neste modelo é de **1,93x** — não 60x.

Tokens gerados	Ingênuo	Com cache	Speedup
16	0,070 s	0,045 s	1,53x
128	0,906 s	0,469 s	1,93x

O motivo aparece na Seção 7: existe um **custo fixo por chamada** (~3,3 ms aqui) que os dois caminhos pagam igualmente, uma vez por token gerado. O cache elimina o trabalho *dentro* da chamada — que num modelo de 2,2 M parâmetros na CPU é a menor parte da conta.

Isso não torna o KV-cache dispensável; torna o **capítulo honesto**. A técnica é indispensável em escala real, onde o cálculo domina. Aqui ela é ensinável e o ganho é modesto. É a mesma lição do Capítulo 8: otimizar o que não domina não acelera.

12.2 Por que dá para não repetir

A pergunta certa não é "como acelero a atenção?", e sim: **o que exatamente muda quando um token novo chega?**

Lembre do Capítulo 4. Para cada posição, a atenção calcula três vetores:

q = x @ Wq	query : "o que eu procuro"
k = x @ Wk	key : "o que eu ofereço"
v = x @ Wv	value : "o que eu entrego"

Repare que k e v da posição 3 dependem **só** do token na posição 3 e da sua posição. Não dependem de nada que venha depois — porque a máscara causal garante que a informação só flui do passado para o futuro.

Quando o token 4 chega, as chaves e valores das posições 0 a 3 são exatamente os mesmos de antes. Recalculá-los é repetir uma conta cujo resultado já temos.

Então guardamos. É isso o **KV-cache**: uma lista, por camada, com todos os k e v já calculados.

O que **não** dá para cachear é o q: a cada passo há uma query nova, a do token atual, e é ela que faz a pergunta ao passado inteiro.

12.2.1 O que sobra para calcular

Com o cache, gerar um token exige:

1. calcular q, k, v **do token novo** — uma posição, não T
2. **concatenar** o k e o v novos ao cache
3. atenção: uma query contra T chaves
4. feedforward, também de uma posição só

O custo por token deixa de crescer com o comprimento do texto já gerado. Na prática ele cresce um pouco — o passo 3 lê um cache cada vez maior — mas sai de "reprocessar tudo" para "ler o que está guardado".

12.3 A implementação: duas linhas

O bloco inteiro muda em duas linhas, dentro do forward:

```
if cache is not None:
    k_ant, v_ant = cache
    k = torch.cat((k_ant, k), dim=2)    # dim=2 é a dimensão do tempo
    v = torch.cat((v_ant, v), dim=2)
```

O k e o v calculados nesta chamada valem só para o token novo. Concatenados ao que estava guardado, viram a sequência completa — e a atenção prossegue como sempre.

O formato do cache, por camada:

```
(B, n_head, T, head_size)
|      |      |      +--- tamanho de cada cabeça (n_embd / n_head)
|      |      +-----+----- quantos tokens já vimos <- só esta dimensão cresce
|      +-----+-----+----- cabeças de atenção
+-----+-----+----- batch
```

Simples assim. **E é aqui que aparecem os dois erros que quase todo mundo comete.**

12.4 Armadilha 1 — a posição é absoluta

No modo *decode* passamos **um** token ao modelo. É tentador escrever:

```
x = self.te(idx) + self.pe(torch.arange(T))    # T == 1 -> posição 0
```

Errado. Esse token não está na posição 0 — está na posição "*quantos tokens já vieram antes*". O embedding posicional precisa refletir isso:

```
ja_vistos = 0 if cache is None else cache[0][0].shape[2]
pos = torch.arange(ja_vistos, ja_vistos + T)
x = self.te(idx) + self.pe(pos)
```

Por que este bug é perigoso: ele **não quebra nada**. Não há exceção, não há NaN, não há erro de formato. O modelo continua gerando texto de aparência plausível, só que um pouco pior — porque todo token acha que é o primeiro da frase. Você só descobre comparando com a saída de referência.

12.5 Armadilha 2 — a máscara

A máscara causal impede que uma posição olhe para o futuro. No modo *decode*, quantas posições futuras existem? **Nenhuma** — o token atual é o último.

A única query é a do token novo, e ela **pode** olhar para todas as chaves do cache. Isso é precisamente o que a última linha da matriz triangular já dizia: toda de uns.

Aplicar `tril[:1, :T]` aqui seria um desastre: essa é a **primeira** linha da matriz, que só permite olhar a posição 0. O modelo passaria a enxergar apenas o primeiro token.

```
if T == T_total: # prefill: máscara normal
    w = w.masked_fill(self.tril[:T, :T] == 0, float("-inf"))
# decode (T == 1): nada a mascarar
```

Os dois erros têm a mesma assinatura: **degradam em silêncio**. É o motivo de a próxima seção existir.

12.6 O teste que define se está certo

Um KV-cache não é uma aproximação. Não é uma troca de qualidade por velocidade. É a **mesma conta**, reorganizada para não repetir trabalho.

Logo, o critério é binário:

Com a mesma semente, o texto gerado com cache tem de ser **idêntico**, token por token, ao gerado sem cache.

```
$ python modelo.py
ingenue      : [0, 310, 115, 10, 673, 521, 97, 496, 78, 656, 895, 702]
com cache    : [0, 310, 115, 10, 673, 521, 97, 496, 78, 656, 895, 702]
idênticos    : True
```

Se der `False`, não existe "quase certo": há um bug, e provavelmente é um dos dois das seções 4 e 5.

Uma nota sobre ponto flutuante. Aqui a igualdade é exata porque as operações acontecem na mesma ordem. Em modelos maiores, ou na GPU, a concatenação pode mudar a ordem de somas dentro de um kernel e produzir diferenças na última casa decimal. Nesse caso o critério vira "logits iguais dentro da tolerância de `float32`" — mas a exigência continua a mesma em espírito: a diferença tem de ser explicável por arredondamento, nunca por lógica.

12.7 Prefill e decode: duas fases, perfis opostos

Com o cache, a geração se separa naturalmente em duas fases — e essa distinção organiza tudo o que se faz em inferência de verdade.

Prefill processa o prompt inteiro de uma vez e monta o cache. É **uma** passada com T posições: uma matmul grande, que ocupa bem o processador.

Decode produz um token de cada vez. Para cada token, é preciso **ler todos os pesos do modelo** da memória — e usá-los para uma única posição.

Fase	Tokens por passada	Gargalo
Prefill	muitos	cálculo — matmuls grandes saturam as unidades
Decode	um	memória — lê todos os pesos para produzir 1 token

É a dicotomia latência/vazão do Capítulo 8, reaparecendo num lugar diferente. E ela explica o comportamento que todo mundo já viu num chat: o modelo demora um instante para "começar" (prefill do seu prompt) e depois cospe palavras num ritmo constante (decode).

12.8 O preço: memória

O cache não é grátis. Ele guarda, **por camada**:

$$2 \text{ (K e V)} \times \text{batch} \times \text{n_head} \times T \times \text{head_size} \times \text{bytes_por_número}$$

Para o modelo deste curso é irrelevante. Para um modelo de verdade, não:

Modelo	Contexto	Batch	Cache
7B (32 camadas, 32 cabeças, hs 128), bf16	4.096	1	~2 GB
idem	4.096	32	~68 GB
idem	32.768	32	~550 GB

Os **pesos** desse modelo ocupam ~14 GB. Com batch 32 e contexto longo, o cache passa dos pesos — muitas vezes.

É por isso que servir um LLM é caro de um jeito que treinar não é. O cache cresce com o número de usuários simultâneos **vezes** o comprimento de cada conversa. Uma GPU que caberia o modelo inteiro não consegue atender 32 pessoas ao mesmo tempo.

Boa parte da engenharia de inferência moderna existe para atacar exatamente essa tabela:

Técnica	O que faz
Multi-query attention (MQA)	uma só cabeça de K/V para todas as de Q — divide o cache por n_head
Grouped-query attention (GQA)	meio-termo: grupos de cabeças compartilham K/V
Paged attention (vLLM)	aloca o cache em páginas, como memória virtual, sem desperdício
Quantização do cache	guarda K/V em 8 bits em vez de 16

O Capítulo 13 trata da última.

12.9 O limite do contexto fixo

Este modelo tem embeddings posicionais aprendidos para 128 posições. Quando o texto passa disso, é preciso descartar o começo — e aí **as posições de todos os tokens mudam**, o que invalida o cache inteiro:

```
if cache[0][0].shape[2] >= self.block_size:
    logits, cache = self(idx[:, -self.block_size:]) # recomeça do zero
```

Toda vez que a janela desliza, você paga um prefill completo. É um custo real, e é consequência de uma escolha de arquitetura: **embeddings posicionais absolutos e aprendidos**.

As alternativas modernas (RoPE, ALiBi) codificam posição de forma **relativa**, e com elas a janela desliza sem invalidar o cache. Não é detalhe de otimização — é uma das razões pelas quais quase nenhum modelo atual usa o nn.Embedding posicional que construímos no Capítulo 4.

12.10 Resumo do capítulo

- Gerar texto do jeito ingênuo desperdiça **mais de 98%** do cálculo: cada token novo reprocessa todo o contexto anterior.
- As **chaves** e **valores** de uma posição não mudam quando tokens novos chegam — a máscara causal garante isso. Então dá para guardá-los.
- A implementação são **duas linhas** de concatenação, mais duas armadilhas que degradam em silêncio: a **posição absoluta** e a **máscara no decode**.
- O critério de correção é **saída idêntica**, não "parecida". Um cache que muda o texto está errado, não aproximado.
- **Prefill** é limitado por cálculo; **decode**, por memória. Reconhecer isso é o começo da engenharia de inferência.
- O cache custa **memória**, e em escala real ele **passa dos pesos** — o que faz da inferência um problema econômico diferente do treino.

12.10.1 Próximo capítulo

Capítulo 13 — Inference II: Quantization. Se o gargalo do decode é ler os pesos da memória, a saída óbvia é **encolher os pesos**. Vamos guardar o modelo em 8 bits — e medir o que isso custa em qualidade.

12.11 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

Estes exercícios usam o checkpoint do Capítulo 11 (`../11-datasets/modelo.pt`). Se ele não existir, rode antes `cd ../11-datasets && python train_text.py`.

12.11.1 E1 — O que dá e o que não dá para cachear (aquecimento)

Sem rodar:

1. Por que k e v de uma posição não mudam quando um token novo chega, mas q precisa ser recalculado a cada passo?
2. Se o modelo **não** tivesse máscara causal (atenção bidirecional, como no BERT), o KV-cache continuaria válido? Justifique.
3. Gerar 128 tokens com contexto 128 processa quantas posições no caminho ingênuo? E no caminho com cache? Qual a fração de trabalho descartado no

primeiro?

12.11.2 E2 — Quebre a posição de propósito (importante)

No `forward` do `modelo.py`, troque o cálculo da posição por `torch.arange(T)` — o bug descrito na Seção 4 da apostila.

1. O programa levanta alguma exceção? O texto continua saindo?
2. Compare com a saída de referência (`gerar_ingenuo`). A partir de qual token elas divergem, e por quê **daquele** token em diante?
3. Leia os dois textos. Se você não tivesse a referência para comparar, conseguiria dizer qual está errado? O que isso diz sobre depurar otimizações de inferência?

12.11.3 E3 — Quebre a máscara de propósito (importante)

Agora aplique a máscara também no modo `decode`: troque a condição por `w = w.masked_fill(self.tril[:T, :T_total] == 0, float("-inf"))`, sem a guarda `if T == T_total`.

1. O que acontece com o texto? Descreva o padrão que aparece.
2. Explique a partir da matriz triangular: qual linha `tril[:1, :T_total]` seleciona, e o que ela permite o token enxergar?
3. Este bug é mais fácil ou mais difícil de notar que o do E2? Por quê?

12.11.4 E4 — Meça o ganho

Rode o `benchmark_cache.py` com a máquina ociosa.

1. Como o **ms por token** evolui com o comprimento, nos dois caminhos? Faça o gráfico mentalmente antes de olhar a tabela.
2. O `speedup` cresce, satura ou cai conforme o texto fica mais longo? Por quê?
3. Para **um único** token gerado, o cache ajuda? Existe comprimento em que ele atrapalha?

12.11.5 E5 — A conta de memória

1. Escreva a fórmula do tamanho do cache em função de n_{layer} , n_{head} , head_size , τ , batch e bytes por número. Confira contra `bytes_do_cache()`.
2. Para um modelo de 7B (32 camadas, 32 cabeças, head_size 128) em bf16, com contexto 8.192: quanto ocupa o cache por usuário? E quantos usuários simultâneos cabem numa placa de 80 GB, descontando os ~14 GB dos pesos?
3. O cache cresce com $n_{\text{layer}} \times n_{\text{head}} \times \text{head_size}$. Note que $n_{\text{head}} \times \text{head_size} = n_{\text{embd}}$. Reescreva a fórmula em função de n_{embd} — o que isso diz sobre qual decisão de arquitetura controla o custo de servir o modelo?

12.11.6 E6 — Cache com batch

O `gerar_com_cache` gera uma sequência por vez. Faça-o gerar B sequências em paralelo.

1. O que muda na forma do cache? E no código?
2. Meça: gerar 32 sequências de 64 tokens em paralelo custa quanto, comparado a 32 gerações separadas?
3. Onde está o problema quando as sequências têm **comprimentos diferentes** (umas terminam antes)? Descreva pelo menos uma estratégia para lidar com isso.

12.11.7 E7 — Multi-query attention (desafio)

A MQA usa **uma só** cabeça de K/V, compartilhada por todas as cabeças de Q. O cache cai por um fator de n_{head} .

1. Implemente. Dica: k e v passam a ter forma $(B, 1, \tau, \text{hs})$, e o *broadcast* faz o resto na multiplicação com q .
2. Quanto o cache encolhe no modelo deste curso? E num 7B?
3. **Cuidado com a conclusão.** Você pode implementar a MQA sobre os pesos já treinados, mas o modelo foi treinado com 6 cabeças de K/V independentes. Medindo a loss antes e depois, o que você espera? O que isso diz sobre quando a MQA precisa entrar no projeto?

13 INFERENCE II: QUANTIZATION

Objetivo de aprendizagem: encolher o modelo trocando `float32` por inteiros de 8, 4 ou menos bits — implementando a aritmética do zero — e medir as **três** coisas que importam: tamanho, qualidade e velocidade. Uma delas não vai melhorar, e entender por que é metade do capítulo.

Pré-requisitos: Capítulos 9 (precisão), 11 (o modelo treinado) e 12 (por que o decode é limitado por memória).

O checkpoint do Capítulo 11 está **versionado** — tudo aqui roda logo após o clone.

Arquivos:

- `quantizacao.py` — a aritmética do zero: escala, zero-point, per-channel
- `quantizar_modelo.py` — aplica ao modelo do Cap. 11 e mede
- `exercicios.md` — exercícios

13.1 Por que encolher os pesos

O Capítulo 12 estabeleceu que a fase de **decode** é limitada por **memória**: para produzir *um* token é preciso ler *todos* os pesos do modelo. O cálculo é pequeno; o tráfego de memória é tudo.

Quando o gargalo é ler bytes, a intervenção óbvia é ter menos bytes para ler.

Formato	Bytes por peso	Um 7B ocupa
<code>float32</code>	4	28 GB
<code>bfloat16</code>	2	14 GB
<code>int8</code>	1	7 GB
<code>int4</code>	0,5	3,5 GB

A diferença entre 14 GB e 3,5 GB é a diferença entre "precisa de uma placa de datacenter" e "roda no seu notebook". É por isso que quantização deixou de ser assunto de especialista.

13.2 A aritmética, que é mais simples do que parece

Quantizar é achar uma **escala** que mapeie o intervalo dos seus floats no intervalo dos inteiros, e depois arredondar.

```
qmax = 127                                # int8 vai de -128 a 127
escala = x.abs().max() / qmax
q = torch.round(x / escala)                # agora cabe em 8 bits
```

E voltar é uma multiplicação:

```
x_recuperado = q * escala
```

Rodando em um tensor de verdade:

```
original      : +0.1808  -0.0700  -0.3596  -0.9152  +0.6258
int8          :      11      -4      -21      -54      +37
de volta      : +0.1871  -0.0680  -0.3572  -0.9184  +0.6293

escala = max|x| / 127 = 0,017008
erro relativo: 0,49%
```

Meio por cento de erro, e o tensor ocupa **um quarto** do espaço. É esse o negócio que a quantização oferece.

13.3 Simétrica ou assimétrica — e uma armadilha que me pegou

A versão acima é **simétrica**: uma escala só, e o zero do float é o zero do inteiro. Funciona bem quando os dados são centrados em zero.

A **assimétrica** acrescenta um *zero-point*, o inteiro que representa o float 0.0:

```
escala = (xmax - xmin) / (qmax - qmin)
zero = round(qmin - xmin / escala)
q = round(x / escala + zero)
```

Ela aproveita o intervalo inteiro quando os dados não são centrados. Medindo:

Distribuição	Simétrica	Assimétrica
normal (centrada em 0)	0,782%	0,724%
só positiva (pós-ReLU)	0,595%	0,277%
deslocada (média 5)	0,358%	0,177%

Pesos de rede costumam ser centrados em zero — a simétrica basta, e sai mais barata (não há zero-point para somar em cada operação). **Ativações**, depois de ReLU ou GELU, não

são; aí a assimétrica ganha o dobro.

13.3.1 A armadilha: o zero-point pode não caber

A primeira versão deste capítulo media, para a distribuição de média 5, **4,16%** de erro na assimétrica contra 0,36% na simétrica. Doze vezes pior, exatamente onde ela deveria ganhar.

A causa, encontrada imprimindo os números intermediários:

```
dados em [1,24 ; 8,63], escala = 0,02898
zero-point IDEAL = -170,7
intervalo de int8 = [-128, 127] -> não cabe
```

Quando o intervalo dos dados **não contém o zero**, o zero-point necessário cai fora do que o int8 representa. O `clamp` o força a -128, o mapeamento inteiro se desloca, e os valores grandes saturam no teto.

A correção é padrão — PyTorch, TFLite e ONNX fazem o mesmo — e consiste em **estender o intervalo para incluir o zero**:

```
xmin = min(xmin, 0)
xmax = max(xmax, 0)
```

Custa um pouco de resolução (parte do intervalo passa a representar valores que não ocorrem) e garante duas coisas: o zero-point é representável, e o float `0.0` tem um inteiro exato — o que importa para *padding* e para máscaras.

13.4 Per-tensor vs per-channel, e uma métrica que mente

Até aqui usamos **uma** escala para a matriz inteira. A alternativa é uma escala **por linha** (por canal de saída).

Numa matriz 8×64 onde uma linha tem valores 100x maiores que as outras:

Métrica	Per-tensor	Per-channel
erro na matriz inteira	1,32%	0,50%
erro nas 7 linhas normais	48,43%	0,58%
erro na linha grande	0,50%	0,50%

Olhe a primeira linha da tabela e depois a segunda. Pelo erro global, o per-tensor parece aceitável — 1,32%. Ele não é: as sete linhas normais estão com **48% de erro**,

praticamente destruídas.

A métrica global mentiu porque é normalizada pela norma da matriz, e essa norma é dominada pela linha grande. O erro está todo nas linhas pequenas, que quase não contribuem para o denominador.

A lição vale além da quantização. Um erro médio baixo pode esconder um subconjunto completamente arruinado. Sempre que houver heterogeneidade de escala nos dados, desagregue a métrica antes de concluir.

O mecanismo: uma escala única é refém do **maior** valor da matriz. A linha 100x maior define a escala, e as outras sete passam a caber em pouquíssimos níveis inteiros — perto de zero, viram zero.

O per-channel custa **28 bytes** neste exemplo e resolve. É por isso que praticamente toda quantização de peso na prática é per-channel.

13.5 Quantos bits o modelo aguenta?

Erro de reconstrução do peso **não é** erro do modelo. A única forma de saber é medir a loss. Aplicando ao modelo do Capítulo 11:

Bits	Loss de validação	Piora	Perplexidade
float32	3,9372	—	51,3
int8	3,9371	-0,0001	51,3
int6	3,9393	+0,0021	51,4
int4	4,0008	+0,0636	54,6
int3	4,4029	+0,4657	81,7
int2	7,4670	+3,5298	1.749

int8 é de graça. A loss não piorou — variou na quarta casa decimal, dentro do ruído. Quatro vezes menos memória, zero de custo em qualidade.

int4 custa 0,06 de loss e ainda é perfeitamente utilizável. É por isso que modelos de 4 bits são hoje o padrão para rodar LLM em hardware modesto.

Entre 4 e 3 bits está o precipício. A piora salta de 0,06 para 0,47 — quase oito vezes. E int2 destrói o modelo: perplexidade 1.749 contra 51.

Note que o erro de *reconstrução do peso* cresce suavemente (0,69% -> 2,84% -> 12,6% -> 29,3%), enquanto a *loss do modelo* tem um joelho abrupto. Um não prevê o outro. Meça o que você quer saber.

13.6 Quanto o modelo encolhe

Onde estão os parâmetros	Quantidade	Fração
camadas Linear	1.966.080	90%
embeddings e LayerNorm	230.272	10%

Configuração	Tamanho	Redução
tudo em float32	8,79 MB	—
Linear em int8	2,89 MB	3,04x
Linear em int4	1,90 MB	4,61x
tudo em int8	2,20 MB	4,00x

Quantizar só as camadas Linear dá **3,04x**, não 4x — e a conta fecha exatamente: $0,9/4 + 0,1 = 0,325$, ou seja 3,08x. Os 10% que ficam em float32 explicam toda a diferença.

(!) **A linha do int4 é teórica.** Ela supõe **empacotamento**: dois pesos por byte. A implementação deste capítulo não empacota — guarda cada valor num `int8` mesmo usando só 4 bits de resolução. Empacotar é trabalho de serialização, não de quantização, e formatos como o GGUF fazem isso. O número está na tabela como promessa, não como medição, e vale saber qual é qual.

13.7 E a velocidade? Piorou.

Configuração	Gerar 64 tokens
float32	0,126 s
int8	0,174 s (0,72x)

O modelo quantizado ficou mais lento, e não é bug.

Esta implementação guarda o peso em `int8` e o **reconstrói em `float32`** a cada forward. Ou seja: ela paga a desquantização e depois faz exatamente a mesma `matmul` de antes. A economia de memória é real; a de velocidade não existe, porque a conta não mudou.

Para a quantização acelerar de verdade é preciso um **kernel** que multiplique `int8 × int8` acumulando em `int32`, sem passar por `float`. Isso não se escreve em PyTorch puro — é o que `llama.cpp`, `bitsandbytes` e o ONNX Runtime fornecem.

"O modelo ficou 4x menor" e "o modelo ficou mais rápido" são afirmações independentes. A primeira decorre da representação. A segunda exige que alguém tenha escrito o kernel.

É a terceira vez que o curso encontra a mesma forma de erro: no Capítulo 8 a GPU era mais lenta para modelo pequeno, no Capítulo 12 o KV-cache rendeu 1,9x em vez de 60x, e aqui a quantização deixa mais lento. **Otimização é uma hipótese até ser medida.**

13.8 O que a prática faz além disto

O que construímos é **round-to-nearest** (RTN) per-channel: a linha de base honesta. Os métodos usados em produção atacam duas limitações dela.

Escolher melhor o arredondamento. RTN arredonda cada peso isoladamente. **GPTQ** e **AWQ** usam um conjunto pequeno de dados de calibração para escolher os arredondamentos que menos alteram a **saída** da camada — não o peso. É a diferença entre minimizar o erro no que você guarda e no que você produz.

Lidar com outliers de ativação. Quantizar pesos é fácil; quantizar **ativações** é difícil, porque em Transformers algumas dimensões têm valores muito maiores que as demais — e você já viu, na Seção 4, o que um outlier faz com uma escala compartilhada. **LLM.int8()** resolve separando essas dimensões e calculando-as em `float16`; **SmoothQuant** redistribui a magnitude entre ativação e peso antes de quantizar.

Os dois problemas são a Seção 4 deste capítulo em escala real.

13.8.1 Quantização por grupos: o meio-termo que todo mundo usa

Entre o per-tensor (1 escala) e o per-channel (uma por linha) existe o esquema adotado por GGUF e GPTQ: **uma escala a cada G valores consecutivos** da linha, tipicamente $G = 64$ ou 128 .

Numa matriz 256×256 onde 32 colunas têm valores 50x maiores, tudo em **int4**:

Esquema	Erro global	Erro nas colunas normais	bits/peso reais
---------	-------------	--------------------------	-----------------

per-channel	11,23%	100,00%	4,06
grupos de 128	10,51%	65,90%	4,12
grupos de 64	10,13%	38,57%	4,25
grupos de 32	9,93%	9,69%	4,50

Repare que o **erro global quase não muda** — e as colunas normais, que são 87,5% da matriz, saem de *completamente apagadas* para 9,7% de erro. É a Seção 4 outra vez: a métrica agregada esconde exatamente o efeito que interessa.

E há um custo que raramente aparece nas discussões:

```
bits_reais = bits + (bits_da_escala / G)
```

Com int4 e $G = 64$, guardando a escala em fp16, cada peso ocupa **4,25 bits** — não 4. Com $G = 32$, **4,5 bits**. É esse compromisso que os nomes dos formatos GGUF codificam: Q4_0, Q4_K_M e afins diferem no tamanho do grupo e em quantos bits gastam com metadados.

13.9 Resumo do capítulo

- Quantizar é achar uma **escala**, dividir e arredondar. A aritmética cabe em cinco linhas.
- **Simétrica** para pesos (centrados em zero), **assimétrica** para ativações — e o intervalo precisa **conter o zero**, ou o zero-point não cabe no inteiro.
- **Per-channel**, sempre. E cuidado com métricas globais: elas escondem subconjuntos destruídos.
- **int8 é de graça** neste modelo (-0,0001 de loss). **int4 custa 0,06**. Entre 4 e 3 bits há um precipício.
- Erro no peso **não prevê** erro no modelo. Meça a loss.
- **Encolher não é acelerar**. Sem kernel inteiro, a quantização economiza memória e custa tempo.

13.9.1 Próximo capítulo

Capítulo 14 — Finetuning I: SFT. Até aqui o modelo só sabe continuar texto. O *supervised finetuning* é o que transforma um continuador de texto em algo que responde ao que você pede.

13.10 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar.**

Estes exercícios usam o checkpoint do Capítulo 11 (`../11-datasets/modelo.pt`).

13.10.1 E1 — A aritmética (aquecimento)

Sem rodar:

1. Um tensor tem valores em $[-3, 3]$. Qual a escala simétrica para `int8`? Qual inteiro representa o float 1,5? E que float se recupera dele?
 2. Por que a quantização simétrica usa $[-127, 127]$ em vez de $[-128, 127]$, se o `int8` comporta o -128?
 3. Um tensor tem todos os valores em $[10, 11]$. Qual o erro relativo esperado com quantização **simétrica** de 8 bits? E por que a assimétrica ajudaria tanto aqui?
-

13.10.2 E2 — Reproduza a armadilha do zero-point (importante)

No `quantizacao.py`, remova as duas linhas que estendem o intervalo para incluir o zero (`xmin = min(xmin, 0)` e `xmax = max(xmax, 0)`).

1. Meça o erro da assimétrica para uma distribuição `randn(1000) + 5`. Compare com a simétrica. Qual ganha?
 2. Imprima o *zero-point ideal* (`qmin - xmin/escala`) antes do `clamp`. Ele cabe em `int8`?
 3. Quantos valores saturam no teto (`q == 127`)? Explique a cadeia causal completa: do zero-point fora do intervalo até o erro medido.
-

13.10.3 E3 — A métrica que mente (importante)

Reproduza a matriz 8×64 com uma linha 100x maior.

1. Meça o erro **global** e o erro **só nas 7 linhas normais**, com per-tensor. Por que os dois números são tão diferentes?
2. A norma usada em erro_relativo é dominada por quê? Escreva a conta.
3. Proponha **outra** métrica que teria denunciado o problema sem precisar desagregar por linha. (Dica: pense em erro relativo *por elemento*.)

13.10.4 E4 — Onde fica o precipício

Rode o `quantizar_modelo.py`.

1. Faça a tabela de duas colunas: *erro de reconstrução do peso* (do `quantizacao.py`) e *piora da loss* (do `quantizar_modelo.py`), para 8, 6, 4, 3 e 2 bits. Uma prevê a outra?
2. Entre quais bits está o joelho? O que acontece com a perplexidade ali?
3. Se você tivesse que escolher um número de bits para servir este modelo, qual escolheria? Justifique com os dois eixos (qualidade e tamanho), não com um só.

13.10.5 E5 — Quantize os embeddings também

O `quantizar_modelo.py` tem o argumento `incluir_embeddings`.

1. Ative-o e meça: quanto o modelo encolhe a mais? Quanto a loss piora?
2. Neste modelo os embeddings são 10% dos parâmetros. Num modelo de 7B são ~2%. Em qual dos dois vale mais a pena quantizá-los, e por quê a resposta **não** é "no que tem a maior fração"?
3. A camada de saída (lm) mapeia `n_embd -> vocab_size`. Ela é um `nn.Linear`, então já está sendo quantizada. Há algum motivo para tratá-la com mais cuidado que as outras?

13.10.6 E6 — Meça o que a memória realmente faz

1. Meça o tempo de gerar 64 tokens com `float32` e com `int8`. Qual é mais rápido?
2. Explique, a partir do `forward` da `LinearQuantizada`, exatamente onde o tempo extra é gasto.
3. Escreva o que precisaria mudar para que a quantização de fato acelerasse. Por que isso não se faz em PyTorch puro?

13.10.7 E7 — Quantização por grupos (desafio)

Entre o per-tensor (1 escala) e o per-channel (uma por linha) existe o meio-termo usado por GGUF e GPTQ: uma escala a cada **G** valores consecutivos da linha (tipicamente $G = 64$ ou 128).

1. Implemente. Para uma matriz (saida, entrada), isso significa entrada/ G escalas por linha.
2. Meça o erro de reconstrução com $G = 256$ (equivalente ao per-channel de uma linha de 256), 128, 64 e 32. Quanto se ganha?
3. Faça a conta do **custo real em bits por peso**: com int4 e $G = 64$, quantos bits cada peso ocupa de fato, contando as escalas em float16? Compare com os 4 bits nominais.

14 FINETUNING I: SFT

Objetivo de aprendizagem: transformar um modelo que **continua texto** num modelo que **responde e para**. Implementar o *supervised finetuning* do zero: tokens especiais, expansão de vocabulário, máscara de loss — e medir a mudança com um número binário, não com uma impressão de leitura.

Pré-requisitos: Capítulos 11 (o modelo treinado) e 12 (geração).

Arquivos:

- `preparar_sft.py` — monta o dataset de instrução a partir do corpus
- `sft.py` — o finetuning: expande o vocabulário e treina com máscara
- `avaliar.py` — a medição que define se funcionou
- `exercicios.md` — exercícios

Uma promessa que este capítulo não faz. Um modelo de 2,2 M parâmetros treinado em 1,6 MB de Machado **não vira um assistente**. Ele não vai responder perguntas, não vai seguir instruções complexas, não vai ficar inteligente. Fingir o contrário estragaria o capítulo.

O que ele **consegue** aprender é exatamente o mecanismo que separa um modelo-base de um modelo de chat — e que dá para medir com honestidade nesta escala.

14.1 O que falta a um modelo-base

O modelo do Capítulo 11 sabe uma coisa: dado um texto, prever o próximo token. Peça a ele que continue "Havia em mim" e ele continua. Continua, e continua, e continua — até você mandar parar.

Ele não sabe terminar. Não é uma limitação de qualidade; é que a tarefa de pré-treino nunca tem um fim. O corpus é um fluxo contínuo de texto, e o modelo aprendeu a modelar esse fluxo.

Um modelo de chat faz três coisas que este não faz:

Capacidade	Modelo-base	Modelo de chat
Distingue quem falou	não	sim (papéis)
Sabe onde a resposta começa	não	sim
Sabe onde a resposta termina	não	sim

A terceira é a mais fácil de medir e a mais fundamental. É a que este capítulo persegue.

14.2 Tokens que não são texto

Para marcar as fronteiras, precisamos de símbolos que o texto **não possa produzir**:

```
PEDIDO, RESPOSTA, FIM = 1024, 1025, 1026
```

O vocabulário do Capítulo 11 vai de 0 a 1023. Esses três ids são novos, e nenhuma sequência de caracteres do corpus os gera.

Por que não usar uma string, tipo `###RESPOSTA###`? Porque o próprio texto poderia contê-la. Um token especial é uma garantia estrutural: não existe entrada do usuário que produza o id 1025. Quando você ouvir falar de *prompt injection* via delimitadores, é exatamente essa a diferença.

O formato de cada exemplo:

```
<|pedido|> {contexto} <|resposta|> {continuação} <|fim|>
```

14.3 Aumentar o vocabulário de um modelo já treinado

O modelo tem 1.024 saídas e precisa de 1.027. Isso significa esticar **duas** matrizes: a tabela de embeddings (entrada) e a camada final (saída).

A parte mecânica é trivial. A decisão que importa é: **com o que preencher as linhas novas?**

```
te.weight.data[:velho] = m.te.weight.data          # o que já existia
te.weight.data[velho:] = m.te.weight.data.mean(0)   # os tokens novos
```

Zeros parecem naturais e são ruins na camada de saída. Um logit zero para o token novo não significa "neutro" — significa um valor específico numa distribuição já treinada, e ele compete de igual para igual com os tokens mais improváveis. A rede leva tempo para separá-lo.

Inicializar com a **média das linhas existentes** faz o token novo nascer como "um token médio": ele herda a escala e a estatística do que já está lá, e se afasta dali conforme aprende. É o que se faz na prática, e é mais estável.

Custo total: **1.155 parâmetros a mais** — 2.196.352 -> 2.197.507. Um aumento de 0,05% que muda o que o modelo é capaz de expressar.

14.4 A máscara de loss

Cada exemplo tem duas partes com papéis diferentes:

```
<|pedido|> {contexto} <|resposta|> {continuação} <|fim|>
\_____ vem do usuário _____/ \___ o modelo produz ___/
```

O modelo não precisa aprender a escrever o pedido — o pedido chega pronto. Então a loss é calculada **só na resposta**:

```
Y[i, :] = IGNORAR # -100 em tudo
Y[i, ini_resposta:fim] = seq[ini_resposta+1:] # exceto na resposta
...
F.cross_entropy(logits, alvos, ignore_index=-100)
```

O `ignore_index=-100` é do PyTorch e sai de graça: aquelas posições simplesmente não entram na média.

Quanto isso muda o resultado é a Seção 7 — e a resposta não é a que eu esperava.

14.5 O resultado: o modelo aprende a parar

600 passos, $lr = 3e-4$ (três vezes menor que a do pré-treino: não queremos destruir o que já existe). Menos de cinco minutos.

Avaliação em 40 pedidos da validação, com orçamento de 120 tokens:

Modelo	Taxa de parada	Comprimento mediano
modelo-base	0%	nunca para
SFT (com máscara)	100%	34 tokens

0% para 100%. E o zero não é "o modelo erra sempre" — é que o token `<|fim|>` **não existe** no vocabulário do modelo-base. Ele não tem como acertar.

Essa é a distância que o SFT atravessa: não uma melhora gradual, mas uma capacidade que antes não estava lá.

14.5.1 Por que medir isto, e não "a qualidade das respostas"

A tentação é gerar duas amostras, ler, e declarar que melhorou. Numa escala destas isso não significa nada: o modelo escreve mal antes e depois.

E o Capítulo 12 já provou o argumento da forma mais dura possível — lá, um bug rebaixou o Transformer ao nível de um bigrama, e **não dava para perceber lendo o texto**.

Uma taxa de parada é binária, objetiva e não depende de julgamento. Prefira sempre a métrica que não precisa da sua opinião.

14.6 O erro que eu cometi montando os dados

A primeira versão deste capítulo usava resposta de **tamanho fixo**: 40 tokens, sempre. O resultado:

Modelo	Taxa de parada	Comprimento mediano
SFT (com máscara)	100%	40
SFT (sem máscara)	100%	40

Cem por cento nos dois, mediana cravada em 40. Parecia um sucesso.

Era um temporizador. Todos os 8.000 exemplos tinham exatamente 66 tokens, então a única regra consistente nos dados era *posicional*: "pare na posição 66". O modelo aprendeu a contar, não a concluir — e aprendeu perfeitamente, porque era a regra que estava lá.

É o modo de falha mais característico do SFT. O modelo aprende a regularidade que os seus dados de fato contêm, não a que você pretendia ensinar. Se todas as suas respostas começam com "Claro!", ele aprende a dizer "Claro!". Se todas têm o mesmo tamanho, ele aprende o tamanho.

A correção: a resposta agora vai até o primeiro **fim de frase** depois de um mínimo. Os comprimentos passam a variar de 39 a 82 tokens, com 44 valores distintos — e parar exige olhar o texto.

14.7 A máscara importa? Menos do que dizem — e dá para saber quando

Todo tutorial de SFT recomenda mascarar o prompt. Medindo, com pedido de 24 tokens e resposta de ~34:

Variante	Loss na resposta	Taxa de parada
com máscara	4,0030	100%
sem máscara	3,9995	100%

Nenhuma diferença. A versão sem máscara é até marginalmente melhor, dentro do ruído.

Eu tinha escrito, antes de medir, que treinar no pedido "gasta capacidade com uma tarefa que ninguém pediu". A medição não sustenta isso — nesta configuração.

Mas um resultado nulo não encerra a questão; ele a reformula. **Sob que condição a máscara passaria a importar?**

A hipótese vem da própria definição: a loss sem máscara é a **média sobre todas as posições**. Com o pedido curto, ele é uma fração pequena e quase não dilui o sinal da resposta. Com o pedido **longo**, ele domina a média.

Medindo, com `e2_quando_a_mascara_importa.py`:

Pedido/resposta	% de pedido	Penalidade de não mascarar
24 / ~34	42%	-0,0066 (mascarar é pior)
64 / ~20	72%	+0,0136
104 / ~12	87%	+0,0165

O sinal **inverte** conforme o pedido cresce, como a hipótese previa. Repetindo a configuração de pedido longo com **seis sementes**: 6 de 6 na mesma direção, média **+0,0199 ± 0,0112**.

(!) **Mas olhe a magnitude**: 0,0199 sobre uma loss de 3,95 é **0,5%**. O efeito é real e pequeno. "Estatisticamente detectável" e "importante na prática" são coisas diferentes, e este número é o primeiro sem ser o segundo — nesta escala.

Por que isso importa na prática: em SFT de verdade a proporção é justamente a desfavorável. Um prompt de sistema com instruções longas, mais o histórico da conversa, mais a pergunta — tudo isso costuma ser muito maior que a resposta. A recomendação padrão está certa **no regime em que foi formulada**, e este capítulo não reproduz esse regime por acidente de escala.

14.7.1 E um erro meu que virou a parte mais útil do exercício

Ao analisar as primeiras três sementes, eu comparei a diferença entre variantes (0,0175) com a variação da **mesma** variante entre sementes (0,0348) e concluí: *"o ruído supera o efeito, a tendência não está estabelecida"*.

O critério estava errado para o desenho do experimento. As duas variantes rodam com a **mesma semente** — é um experimento **pareado**. Nele, a variação que a semente causa afeta os dois lados e **se cancela na diferença**. Comparar o efeito com a variação absoluta joga fora exatamente a vantagem de ter pareado.

O mesmo dado, os dois critérios:

Critério	Números	Veredito
não pareado	ruído 0,0348 > efeito 0,0199	"não estabelecido"

pareado

6/6 na mesma direção, média > desvio

estabelecido

Conclusões opostas, decididas só pela estatística escolhida.

É o segundo caso deste curso em que a **medição estava certa e a leitura não**. O outro foi o Capítulo 13, onde uma métrica global marcava 1,3% de erro enquanto 48% das linhas estavam destruídas.

Escolher a métrica, e escolher o teste, são decisões tão determinantes quanto rodar o experimento — e passam despercebidas porque parecem detalhes técnicos depois do trabalho "de verdade".

14.8 O que o SFT é, e o que não é

Vale terminar com a distinção que o capítulo inteiro sustenta:

O SFT não acrescenta conhecimento. Ele muda o que o modelo faz com o conhecimento que já tem.

O modelo depois do finetuning não sabe nada de Machado que não soubesse antes — são os mesmos pesos, ajustados por 600 passos com learning rate pequena. O que mudou foi o **comportamento**: ele agora reconhece um formato, sabe onde a sua vez começa e sabe onde ela termina.

É por isso que finetuning não conserta um modelo-base ruim, e é por isso que "vou fazer finetuning para ensinar os meus dados ao modelo" quase sempre decepciona. Conhecimento entra no pré-treino, ou entra pelo contexto. O SFT molda a **forma**.

14.9 Resumo do capítulo

- Um modelo-base **não sabe terminar** — a tarefa de pré-treino nunca tem fim.
- **Tokens especiais** funcionam como delimitadores porque o texto não pode produzi-los.
- Aumentar o vocabulário exige esticar embeddings e camada de saída; inicialize as linhas novas com a **média**, não com zeros.
- A **máscara de loss** sai de graça com `ignore_index=-100` — e importa quando o pedido domina as posições, não sempre.
- Meça com um número **binário** (taxa de parada), não lendo amostras.
- **Cuidado com regularidades acidentais nos dados**: se todas as respostas têm o mesmo tamanho, o modelo aprende o tamanho.
- SFT muda **comportamento**, não conhecimento.

14.9.1 Próximo capítulo

Capítulo 15 — Finetuning II: RL. O SFT ensina o modelo a imitar respostas que alguém escreveu. E quando não há resposta certa para imitar — só respostas melhores e piores? Aí é preciso otimizar contra uma **preferência**, e não contra um alvo.

14.10 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

Rode `python preparar_sft.py` e `python sft.py` antes dos exercícios.
Juntos levam menos de seis minutos.

14.10.1 E1 — Tokens especiais e vocabulário (aquecimento)

Sem rodar:

1. Por que `<|fim|>` precisa ser um **id novo** em vez de uma string como `####FIM####`? Descreva um caso concreto em que a string falharia.
 2. Ao aumentar o vocabulário de 1.024 para 1.027, quantos parâmetros novos aparecem? Faça a conta com `n_embd = 192`, lembrando das **duas** matrizes e do bias.
 3. As linhas novas são inicializadas com a **média** das existentes. O que aconteceria com zeros na camada de saída? E com valores aleatórios grandes?
-

14.10.2 E2 — Quando a máscara importa (importante)

O capítulo mediu que mascarar o pedido **não faz diferença** com pedido de 24 tokens e resposta de ~34.

1. Antes de rodar: escreva por que você espera que a máscara importe, e o que na sua explicação depende do **tamanho relativo** do pedido.
 2. Rode `solucoes/e2_quando_a_mascara_importa.py`, que varia a proporção. A partir de qual fração de pedido a penalidade aparece?
 3. Um resultado nulo ("não faz diferença") e um resultado negativo ("faz diferença ao contrário") pedem reações diferentes. Qual dos dois você obteve no caso de pedido curto, e o que cada um autorizaria você a concluir?
-

14.10.3 E3 — Quebre os dados de propósito (importante)

Volte o `preparar_sft.py` para resposta de **tamanho fixo** (era `TAM_RESPOSTA = 40`).

1. Treine e meça a taxa de parada e o comprimento **mediano**. O que você vê?
2. A taxa de parada é 100% nos dois casos. Isso significa que a versão de tamanho fixo é igualmente boa? Que métrica separa as duas?
3. Generalize: se todas as respostas do seu conjunto de SFT começassem com "Claro!", o que o modelo aprenderia? Dê mais dois exemplos de regularidade accidental que apareceriam num conjunto real.

14.10.4 E4 — A learning rate do finetuning

O `sft.py` usa `lr = 3e-4`, três vezes menor que a do pré-treino.

1. Rode com `1e-3` (a do pré-treino) e com `3e-5`. Meça a loss na resposta e a taxa de parada nas três.
2. Com learning rate alta, o que acontece com a capacidade do modelo de escrever português comum? (Sugestão: gere texto **sem** o formato de instrução e compare com o modelo-base.)
3. O fenômeno tem nome — *catastrophic forgetting*. Explique por que finetuning com poucos dados e learning rate alta o provoca.

14.10.5 E5 — Quantos exemplos bastam?

1. Treine com 500, 2.000 e 8.000 exemplos, mantendo os passos fixos. Como muda a taxa de parada? E a loss?
2. Compare com o Capítulo 11, onde mais dados sempre ajudaram. Por que a curva aqui é diferente?
3. Na literatura de SFT há resultados mostrando que **mil exemplos bem escolhidos** batem dezenas de milhares aleatórios. O que a sua medição sugere sobre o porquê?

14.10.6 E6 — O modelo esqueceu de escrever?

1. Meça a loss do modelo **depois do SFT** no conjunto de validação **original** do Capítulo 11 (texto corrido, sem formato de instrução). Compare com o modelo-base.

2. O SFT melhorou, piorou ou não mexeu na capacidade de modelar português comum?
 3. Este é o *alignment tax*: o custo, na capacidade original, de ensinar um comportamento novo. Ele é inevitável? O que reduziria o custo?
-

14.10.7 E7 — Um formato de conversa de verdade (desafio)

O formato deste capítulo tem um turno só. Modelos de chat têm vários.

1. Estenda o formato para `<|pedido|> A <|resposta|> B <|pedido|> C <|resposta|> D <|fim|>`. O que muda na máscara?
2. Onde entra o `<|fim|>` — no fim de cada resposta ou só no fim da conversa? Qual das duas escolhas permite a geração parar no turno certo?
3. Modelos reais usam um token de fim **por turno** (`<|im_end|>` e afins) e param nele. Implemente assim e verifique que a geração para no fim do primeiro turno, não da conversa inteira.

15 FINETUNING II: RL

Objetivo de aprendizagem: otimizar um modelo contra uma **pontuação** em vez de contra respostas prontas. Implementar policy gradient do zero, entender a penalidade de KL — e ver, medido, o que acontece quando a recompensa mede a coisa errada.

Pré-requisitos: Capítulos 12 (geração) e 14 (o modelo com formato de instrução).

Arquivos:

- `recompensa.py` — duas recompensas, uma boa e uma **mal especificada de propósito**
- `reinforce.py` — o policy gradient, com penalidade de KL
- `experimento.py` — as três configurações que definem o capítulo
- `exercicios.md` — exercícios

15.1 Quando não há resposta certa para imitar

O Capítulo 14 ensinou por **imitação**: existiam respostas escritas, e o modelo aprendeu a reproduzi-las. A loss media o quanto ele acertava o texto de referência.

Muita coisa que se quer de um modelo não tem texto de referência. "Seja mais útil." "Não invente fatos." "Escreva no tom certo." Não existe *a* resposta correta para comparar — existem respostas melhores e piores.

Quando é assim, o que sobra é uma **pontuação**: o modelo gera, alguém (ou algo) avalia, e os gradientes empurram na direção do que pontuou bem.

É essa a diferença entre SFT e RL, e ela é de **fonte de sinal**, não de algoritmo. SFT tem um alvo; RL tem um juiz.

15.2 De onde vem a recompensa neste capítulo

Em RLHF de verdade, o juiz é um **modelo de recompensa** treinado em preferências humanas: mostram-se duas respostas a uma pessoa, ela escolhe, e um modelo aprende a prever a escolha.

Aqui não há gente para coletar preferências. Então usamos recompensas **calculáveis por programa** — e a escolha não é só de conveniência.

O Capítulo 11 estabeleceu, medindo, quando dados gerados pelo próprio modelo ajudam: **quando existe um verificador externo**. Uma recompensa programática é

exatamente isso — objetiva, reproduzível, e imune ao modelo se auto-elogiar.

É também o regime que mais cresce na prática: RL sobre matemática, código e tarefas com resposta verificável, onde ninguém precisa opinar porque dá para conferir.

15.3 O algoritmo cabe numa linha

O **REINFORCE** é o policy gradient mais simples que funciona:

```
loss = -(R - baseline) * log_prob(resposta_gerada)
```

Se a recompensa ficou **acima da média** do batch, a loss empurra para **aumentar** a probabilidade daquela resposta. Se ficou abaixo, para diminuir. É só isso.

O **baseline** — a média do batch — não muda o gradiente esperado; ele reduz a **variância**. Sem baseline, toda resposta com recompensa positiva é reforçada, inclusive as ruins, e o aprendizado fica lento e instável.

O PPO, usado no RLHF clássico, é o REINFORCE mais três coisas: um *critic* que estima o baseline, *clipping* para limitar o tamanho da atualização, e várias épocas por amostra. Complexidade que existe para estabilizar, não para mudar a ideia.

15.4 A penalidade de KL

Otimizar uma pontuação sem restrição leva o modelo para onde a pontuação for maior — e esse lugar pode não ter nada a ver com o que você queria. A defesa padrão é amarrar a política ao modelo de partida:

```
R_efetiva = R - β · log( P_política / P_referência )
```

O termo é a divergência de KL estimada na trajetória gerada. Quanto mais a política se afasta da referência, mais ela é penalizada.

O modelo de referência é o SFT do Capítulo 14, **congelado**. Ele é a memória do que o modelo era antes de começar a perseguir pontos.

15.5 Duas recompensas, e uma delas é ruim de propósito

Recompensa	O que se quer dizer	O que ela mede
comprimento	"responda com ~30 tokens"	a mesma coisa — não há atalho
pontos	"escreva frases bem pontuadas"	a fração de tokens que são pontuação

As duas coincidem em texto normal. É por isso que a segunda *parece* razoável quando você a escreve.

Testando à mão, antes de qualquer treino:

Sequência	comprimento	pontos
30 tokens, sem pontos	1,00	0,00
30 tokens, metade pontos	1,00	0,50
30 tokens, só pontos	1,00	1,00

Uma resposta que é **só pontuação** tira nota máxima em "escreva frases bem pontuadas".

A recompensa não tem bug. Ela mede exatamente o que foi escrito. O problema é que o que foi escrito não é o que se queria — e é assim que toda recompensa mal especificada se parece **antes** de alguém otimizá-la: razoável.

15.6 O experimento

Três configurações, 400 passos cada. E **três** números por configuração, porque um só engana:

Configuração	Recompensa	KL	Loss em Machado
comprimento (boa), com freio	0,591 -> 0,743	2,75	4,1701
pontos (má), SEM freio	0,058 -> 0,996	14,87	4,8211

pontos (má), com freio	0,058 -> 0,958	11,63	4,3963
(partida, sem RL)	—	0,00	4,0784

E as respostas que saem:

```
comprimento, com freio : 'é a busão não me lembra-me para que um bom silencio.
                        Para que seriam da vida... De ficar corte...'
pontos, SEM freio      : '.'
pontos, com freio      : '.'
```

15.6.1 Como ler, e a ordem importa

A recompensa sobe nas três. Isso não é informação — é o que o algoritmo faz. Uma recompensa que *não* sobe indica bug, não sucesso. Reportar só essa coluna é a forma mais comum de fingir que um treino de RL funcionou.

A loss em Machado é o juiz, e o RL nunca a observa. Ela mede se o modelo continua sendo um modelo de português enquanto persegue pontos.

A KL diz o quanto a política se afastou. Ela é o mecanismo; a loss real é a consequência.

15.6.2 O que a segunda linha mostra

Recompensa **0,996** — praticamente perfeita. E o modelo responde '.' a qualquer coisa que você pergunte.

Ele não trapaceou. Ele **otimizou exatamente o que estava escrito**, e foi extremamente bem nisso. O erro foi meu, na especificação — e nenhuma quantidade de RL bem implementado conserta uma recompensa que mede a coisa errada.

Este é o argumento inteiro do capítulo, e vale além dele: **toda métrica é uma medida do que você quer, nunca o que você quer**. Enquanto ninguém otimiza a medida, a diferença não aparece. O RL é uma máquina de encontrar essa diferença.

15.7 O freio de KL não é um interruptor

Eu escrevi, antes de medir, que a terceira configuração mostraria o freio **segurando** o hacking.

Ela mostra o freio **perdendo mais devagar**. Com $\beta = 0,02$, a recompensa ainda chega a 0,958 e a resposta ainda é '.'. O que muda é o custo: +0,32 de loss em vez de +0,74 — o estrago cai pela metade, e é só.

Métrica	Sem freio	Com $\beta = 0,02$
recompensa final	0,996	0,958
KL	14,87	11,63
custo em português	+0,74	+0,32

A penalidade de KL **compra tempo e limita o dano**. Não substitui uma recompensa correta.

15.7.1 E a faixa útil é um ponto só

Varrendo o β com `e4_dial_do_kl.py`:

β	Recompensa	KL	Custo em português	Regime
0,00	0,058 -> 0,996	14,87	+0,743	hackeia por completo
0,02	0,058 -> 0,958	11,63	+0,318	hackeia quase igual
0,10	0,055 -> 0,218	1,41	+0,005	aprende sem custo
0,50	0,053 -> 0,064	0,31	+0,003	mal se move
2,00	0,053 -> 0,055	0,23	+0,002	congelado

Só $\beta = 0,10$ tem as duas coisas: recompensa subindo e custo indistinguível de zero. Um fator de cinco para baixo e o modelo hackeia; um fator de cinco para cima e ele paralisa.

E note a armadilha das duas últimas linhas: em $\beta = 0,5$ o custo é **menor** que em $\beta = 0,10$. Pela coluna do custo, seria a melhor configuração — mas a recompensa subiu 0,011 em 400 passos. O modelo não foi protegido; foi impedido de aprender.

Custo zero é o que você obtém não treinando. Uma métrica de segurança que melhora quando o sistema para de funcionar não está medindo segurança.

15.8 Uma observação sobre orçamento, que quase me custou o capítulo

A primeira versão deste experimento usava **150 passos e $lr = 1e-5$** . O resultado:

Orçamento	Recompensa	KL	Loss real
pontos, sem freio (150 passos)	0,052 -> 0,163	2,02	4,1229
pontos, sem freio (400 passos)	0,058 -> 0,996	14,87	4,8211

Com o orçamento curto, o texto ainda parecia português e o efeito parecia sutil. Eu quase publiquei aquela tabela como "demonstração de reward hacking". **Não era** — era um empurrãozinho na direção certa, e teria dado ao leitor a impressão errada de que o fenômeno é ameno.

É o terceiro caso deste curso com a mesma forma. No Capítulo 11, orçamento curto **inverteu** a resposta de dois exercícios. No Capítulo 14, uma learning rate pequena demais fez parecer que o SFT não instalava o comportamento.

Quando um fenômeno não aparece, a primeira pergunta não é "será que ele não existe?". É "eu dei orçamento para ele aparecer?".

15.9 E o DPO?

O RLHF clássico tem três estágios: SFT -> treinar um modelo de recompensa -> otimizar com PPO. É caro e instável.

O **DPO** (*Direct Preference Optimization*) mostra que, para o caso de preferências pareadas, dá para pular o modelo de recompensa e o RL: existe uma loss supervisionada que tem o mesmo ótimo. Você treina direto nos pares "esta resposta é melhor que aquela", com algo que se parece com uma cross-entropy.

Por que o capítulo não usa DPO, então? Porque **ele precisa de pares de preferência**, e aqui não temos. Com recompensa programática — o regime de matemática, código e tarefas verificáveis — o policy gradient continua sendo o caminho, e é o que a prática atual usa.

E a lição da Seção 6 não muda com o algoritmo: DPO otimiza a preferência que você deu, com a mesma obediência literal.

15.10 Resumo do capítulo

- RL entra quando **não há resposta certa para imitar** — só respostas melhores e piores.
- O **REINFORCE** é uma linha: $-(R - \text{baseline}) * \log_prob$. O baseline reduz variância.
- Recompensa **verificável por programa** é o regime honesto quando não há gente para opinar — e o que mais cresce na prática.
- **Reportar só a recompensa não prova nada.** Ela sempre sobe. É preciso uma métrica que o RL não observe.
- **Reward hacking não é trapaça:** é o modelo otimizando exatamente o que foi escrito. O erro está na especificação.
- A **penalidade de KL é um dial**, não um interruptor. Limita o dano; não conserta a recompensa.
- Se um fenômeno não aparece, **confira o orçamento antes de concluir que ele não existe.**

15.10.1 Próximo capítulo

Capítulo 16 — Deployment. O modelo está treinado, afinado e alinhado. Falta a parte que separa um experimento de um produto: servi-lo para mais de uma pessoa ao mesmo tempo, sem que a latência exploda.

15.11 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar.**

Este capítulo usa o modelo do Capítulo 14. Se `../14-sft/modelo_sft.pt` não existir, rode antes `cd ../14-sft && python preparar_sft.py` && `python sft.py`.

[tempo] Os experimentos de RL são **lentos**: cada configuração de 400 passos leva ~50 min, porque cada passo gera 48 tokens com dois modelos (política e referência). Planeje.

15.11.1 E1 — SFT e RL, lado a lado (aquecimento)

Sem rodar:

1. O Capítulo 14 treinou com `cross_entropy` contra respostas prontas. Aqui a loss é $-(R - \text{baseline}) * \log_prob$. Descreva, em uma frase cada, **de onde vem o sinal**

nos dois casos.

2. Por que o RL precisa **gerar** durante o treino, e o SFT não? O que isso custa?
3. O baseline é a média da recompensa no batch. Ele não muda o gradiente esperado — só a variância. Explique por quê. (Dica: qual é o valor esperado de $R - \text{média}(R)$?)

15.11.2 E2 — Escreva uma recompensa ruim (importante)

O capítulo usa duas: comprimento (bem especificada) e pontos (mal especificada).

1. Escreva uma **terceira** recompensa que pareça razoável e seja hackeável. Antes de rodar, descreva **como** você espera que o modelo a hackeie.
2. Rode com $\beta=0.0$ e 400 passos. O modelo hackeou do jeito que você previu, ou achou outro caminho?
3. Toda recompensa da Seção 5 da apostila tem a forma "medir X como proxy de Y". Para a sua, qual é o X e qual é o Y — e onde eles deixam de coincidir?

15.11.3 E3 — O baseline importa mesmo? (importante)

O `reinforce.py` aceita `--sem-baseline`.

1. Rode a recompensa comprimento com e sem baseline, mesma semente. Compare a curva de recompensa e a variância dela entre passos.
2. Sem baseline, **toda** resposta com recompensa positiva é reforçada — inclusive as ruins. Explique por que isso não impede o aprendizado, só o torna mais lento.
3. Se a sua recompensa fosse sempre negativa (digamos, de -1 a 0), o que aconteceria sem baseline? E isso sugere o quê sobre normalizar recompensas?

15.11.4 E4 — Onde fica a faixa útil do β

Rode `solucoes/e4_dial_do_kl.py`, que varre o β .

1. Faça a tabela de β contra três colunas: recompensa final, KL e **custo em português**. Onde está a faixa em que a recompensa sobe e o custo é pequeno?
2. Entre $\beta = 0,02$ e $\beta = 0,10$ há um fator de cinco. O que muda no comportamento do modelo nessa faixa? Por que a transição é tão abrupta?

3. Com β grande o custo vai a zero. Isso é sucesso? Olhe também a coluna da recompensa antes de responder.

15.11.5 E5 — A recompensa não é o objetivo

1. Para as três configurações do `experimento.py`, ordene os modelos por **recompensa** e depois por **loss em Machado**. As ordens coincidem?
2. Imagine que você só reportasse a recompensa num relatório. Que conclusão o leitor tiraria? E ela seria falsa em que sentido exatamente?
3. Proponha **duas** métricas que um projeto real de RLHF deveria acompanhar além da recompensa, e diga o que cada uma detectaria que a recompensa não detecta.

15.11.6 E6 — Conserte a recompensa

O problema de pontos não é o β — é a especificação.

1. Reescreva `muitos_pontos` para capturar melhor a intenção original ("escreva frases completas e bem pontuadas"). Pense no que distingue pontuação **útil** de pontuação **repetida**.
2. Rode com `beta=0.0`. A sua versão resiste sem freio nenhum?
3. Se resistiu: existe alguma recompensa que resista a qualquer otimização? Se não resistiu: o que isso diz sobre a estratégia de "escrever uma recompensa melhor"?

15.11.7 E7 — DPO (desafio)

O DPO troca o RL por uma loss supervisionada sobre **pares** de preferência.

1. Gere pares com o próprio modelo: para cada pedido, amostra duas respostas e rotule a de maior recompensa como preferida. Você acabou de construir um dataset de preferências sintético.
2. Implemente a loss do DPO e treine. Compare com o REINFORCE em recompensa, KL e custo.
3. **A pergunta que importa:** o DPO evita o reward hacking? Justifique pelo que ele otimiza — e note que os seus pares foram rotulados pela **mesma** recompensa ruim.

16 DEPLOYMENT

Objetivo de aprendizagem: servir o modelo por HTTP e descobrir, medindo, as duas decisões que definem um serviço de LLM — **streaming** (o que o usuário sente) e **batching** (o que o servidor entrega). São coisas diferentes, e otimizar uma não otimiza a outra.

Pré-requisitos: Capítulos 12 (KV-cache, prefill vs decode) e 13 (quantização).

Arquivos:

- `servidor.py` — servidor HTTP com streaming, biblioteca padrão apenas
- `carga.py` — mede TTFT, latência e vazão sob concorrência
- `servidor_batch.py` — o mesmo servidor, agrupando requisições
- `exercicios.md` — exercícios

Sem framework, de propósito. Nada de FastAPI ou uvicorn — só `http.server`. O assunto do capítulo são as decisões de serviço, e um framework as esconde atrás de decoradores. Você vai escrever o Transfer-Encoding: chunked na mão.

16.1 Um modelo treinado não é um serviço

Até aqui o modelo viveu dentro de scripts. Servir é outra coisa, e as perguntas mudam:

No treino você pergunta	Servindo você pergunta
a loss caiu?	quanto tempo até o usuário ver algo?
quanto tempo leva uma época?	quantos usuários simultâneos cabem?
cabe na memória?	o que acontece quando dez chegam juntos?

Nenhuma dessas aparece numa curva de treino, e é por isso que este capítulo mede coisas que nenhum capítulo anterior mediu.

16.2 Streaming: a otimização que não acelera nada

O servidor tem dois endpoints que produzem **o mesmo texto**:

POST /gerar	devolve a resposta inteira, de uma vez
POST /gerar/stream	devolve token por token, conforme são produzidos

O que muda é quando o primeiro byte chega:

Endpoint	TTFT	Total
/gerar	0,115 s	0,115 s
/gerar/stream	0,079 s	0,125 s

O **total é o mesmo** — tem de ser, o servidor faz o mesmo trabalho. O que muda é o **TTFT** (*time to first token*), e é ele que o usuário sente como "travou" ou "respondeu".

16.2.1 Por que isso é possível

Porque a geração é **autorregressiva**: os tokens existem um a um, em ordem. O gerar do servidor é um **gerador Python** — ele não devolve o texto pronto, cede cada token no instante em que existe:

```
for t in gerar_tokens(ids, n):
    pedaco = decodificar([t]).encode("utf-8")
    self.wfile.write(f"{len(pedaco):X}\r\n".encode() + pedaco + b"\r\n")
    self.wfile.flush()           # sem isto, o SO junta tudo e não há streaming
```

Aquele `flush()` é o capítulo inteiro em uma linha. Sem ele o sistema operacional acumula os bytes num buffer e entrega tudo junto — o código parece streaming e não é.

Um modelo que produzisse a resposta inteira de uma vez não teria o que transmitir aos poucos. O streaming é grátis porque a arquitetura já é sequencial.

16.2.2 (!) E a magnitude aqui é pequena, por um motivo

A diferença medida é de 0,04 s — uns 30%. É pouco, e a causa é que **este modelo é rápido demais para o streaming brilhar**: 40 tokens saem em 0,12 s, então o TTFT já era baixo e boa parte dele é custo de HTTP, não de geração.

O ganho do streaming é proporcional ao **tempo total de geração**. Num modelo de verdade, 500 tokens a 30 tok/s levam ~17 segundos — e o TTFT cai de **17 s para ~0,3 s**. É a diferença entre um sistema que parece travado e um que parece instantâneo.

Este capítulo mede o **mecanismo**. A magnitude vem da escala.

16.3 O que acontece quando vários clientes chegam juntos

O servidor usa `ThreadingHTTPServer`, então cada requisição roda numa thread. Parece que vai escalar. Medindo:

Clientes	Latência mediana	p95	Vazão total
----------	------------------	-----	-------------

1	0,15 s	0,15 s	268 tok/s
2	0,20 s	0,27 s	301 tok/s
4	0,32 s	0,50 s	320 tok/s
8	0,54 s	0,96 s	334 tok/s

Oito vezes mais clientes, 25% mais trabalho entregue. E a latência cresce 3,6x, com o p95 quase dobrando a mediana — há fila, e quem chega por último paga.

A causa está no `servidor.py`: existe uma **trava** em volta da geração, então as requisições se **revezam** no modelo. Os 25% de ganho vêm das partes que rodam fora da trava — HTTP, tokenização, JSON.

E vale ser preciso: **a trava não está errada**. Sem ela, várias threads chamariam o modelo ao mesmo tempo e disputariam os mesmos núcleos, o que costuma sair pior. O problema é outro — requisições concorrentes estão sendo tratadas como **independentes** quando poderiam viajar **juntas**.

16.4 Batching: as requisições viajam juntas

A solução vem direto do Capítulo 12: o decode é limitado por **memória**. Ler os pesos do modelo custa o mesmo para 1 ou para 16 sequências — então processar 16 juntas custa quase o mesmo que processar 1.

Lá isso foi medido: gerar 16 sequências em paralelo custou **9,1x menos** que 16 gerações separadas. Aqui é a mesma economia, aplicada a requisições de usuários diferentes.

O desenho inverte quem manda:

```

cliente A --\                               /--> tokens de A
cliente B ---+--> [fila] --> laço -----+----> tokens de B
cliente C --/                               \--> tokens de C
                        (1 thread)

```

Um **único thread** é dono do modelo. As requisições HTTP não geram nada — elas põem um pedido numa fila e esperam os tokens voltarem. O thread-dono junta quem estiver esperando, gera **um token para todos de uma vez**, distribui, e repete.

E quem termina **sai do lote na hora**, sem esperar os outros:

```

for p in [p for p in ativos if p.restantes <= 0]:
    p.saida.put(None)
ativos = [p for p in ativos if p.restantes > 0]

```

É isso que torna o batching **contínuo** em vez de estático — e é o que vLLM e TGI fazem, com muito mais cuidado.

16.5 O resultado

Vazão total (tokens/s):

Cientes	Simple (trava)	Com batching	Ganho
1	268	306	1,14x
2	301	434	1,44x
4	320	585	1,83x
8	334	755	2,26x

Latência (mediana / p95):

Cientes	Simple (trava)	Com batching
1	0,15 s / 0,15 s	0,13 s / 0,13 s
2	0,20 s / 0,27 s	0,18 s / 0,18 s
4	0,32 s / 0,50 s	0,27 s / 0,27 s
8	0,54 s / 0,96 s	0,42 s / 0,42 s

2,26x mais vazão com 8 clientes — e latência menor. Não houve troca: o batching ganhou nos dois eixos.

16.5.1 O detalhe mais bonito da tabela

Olhe o **p95** das duas colunas.

Com a trava, ele é quase o dobro da mediana (0,96 contra 0,54). Isso é uma **fila**: a maioria é atendida rápido e alguns esperam muito, porque chegaram por último.

Com batching, **p95 = mediana**. Todos avançam **no mesmo passo** — cada iteração produz um token para todo mundo. Não existe "último da fila", então não existe cauda.

Um p95 igual à mediana é a assinatura de um sistema onde ninguém espera pelos outros. Vale procurar por isso em qualquer serviço que você meça.

16.6 Um bug meu, e por que ele vale estar aqui

A primeira versão do `servidor_batch.py` era **mais lenta** que o servidor simples: 0,62 s de latência contra 0,15 s, e 293 tok/s contra 334.

A causa era uma linha:

```
ativos.append(FILA.get(timeout=None if not ativos else ESPERA_MS / 1000))
```

Ela esperava 8 ms por novas requisições **a cada passo de decode**. Com 40 tokens, isso somava **320 ms de espera pura** por requisição. O ganho do agrupamento existia e estava sendo inteiramente consumido pela espera.

A correção é uma regra que vale para qualquer laço de serviço:

Nunca bloqueie quando há trabalho a fazer. Bloqueie só quando estiver ocioso; quando houver fila, drene sem esperar.

```
if not ativos:
    ativos.append(FILA.get())                # ocioso: pode bloquear
while len(ativos) < LOTE_MAX:
    try:
        ativos.append(FILA.get_nowait())    # ocupado: nunca espera
    except queue.Empty:
        break
```

Vale registrar o que quase aconteceu: eu tinha uma tabela mostrando batching **pior** que a versão simples. Ela era verdadeira e teria ensinado a coisa errada, porque a causa não era o batching — era a minha espera de 8 ms.

16.7 O que este capítulo não faz

Um servidor de produção tem várias coisas que aqui foram simplificadas de propósito, e convém saber quais:

Simplificação	O que a produção faz
todas as sequências recortadas no mesmo comprimento	<i>padding</i> com máscara, ou paged attention
KV-cache recalculado a cada passo do lote	cache persistente por sequência
sem limite de fila, sem timeout de admissão	<i>backpressure</i> — recusar quando cheio é melhor que atender mal

sem autenticação, sem limite de taxa	ambos, sempre
um processo, uma máquina	réplicas atrás de um balanceador

A que mais dói é a primeira: recortar no menor comprimento é correto aqui porque todas as requisições do teste começam parecidas, e seria errado com prompts de tamanhos diferentes.

Este capítulo mostra o mecanismo do batching, não uma implementação de produção. A diferença entre os dois é o assunto de bibliotecas inteiras.

16.8 Resumo do capítulo

- **Streaming** não acelera nada e muda tudo: mesmo total, TTFT muito menor. O ganho é proporcional ao tempo de geração — pequeno aqui, enorme num modelo real.
 - O `flush()` depois de cada token é o que separa streaming de streaming aparente.
 - **Threads não escalam** um serviço de LLM: com a trava, 8 clientes rendem 25% mais vazão e 3,6x mais latência.
 - **Batching escala**, porque o decode é limitado por memória — 2,26x de vazão e latência menor.
 - **p95 = mediana** é a assinatura de um sistema sem fila. Meça o p95, não só a média.
 - Num laço de serviço, **nunca bloqueie quando há trabalho** — foi o que fez o meu batching ficar mais lento que o servidor simples.
-

16.8.1 Próximo capítulo

Capítulo 17 — Multimodal. O último. Até aqui o modelo só viu tokens de texto. E se a mesma arquitetura pudesse receber uma imagem?

16.9 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar.**

Estes exercícios precisam de **dois terminais**: um com o servidor de pé, outro rodando o cliente. Suba o servidor com `python servidor.py` e meça com `python carga.py`.

16.9.1 E1 — TTFT e latência (aquecimento)

Sem rodar:

1. O `/gerar` e o `/gerar/stream` produzem o mesmo texto e levam o mesmo tempo total. Então o que exatamente o streaming melhora? Descreva em termos do que o usuário observa.
2. Por que o streaming é possível num LLM e não seria, digamos, num classificador de imagens que devolve uma etiqueta?
3. O servidor chama `self.wfile.flush()` depois de cada token. O que aconteceria sem essa linha, e por que o código continuaria "parecendo" streaming?

16.9.2 E2 — Meça o seu próprio TTFT (importante)

1. Rode o `carga.py` e anote TTFT e total nos dois endpoints. Qual a razão entre eles?
2. Agora peça **200 tokens** em vez de 40 (`TOKENS = 200` no `carga.py`). O que acontece com a razão TTFT/total nos dois casos? Explique a tendência.
3. Extrapole: para um modelo que gera 30 tokens/s e uma resposta de 500 tokens, quanto seria o TTFT com e sem streaming? Isso muda a sua opinião sobre a magnitude medida no capítulo?

16.9.3 E3 — Por que threads não bastam (importante)

1. Rode `carga.py` contra o `servidor.py` e monte a tabela de vazão por número de clientes. A vazão cresce proporcionalmente? Quanto?
2. Remova a trava (`TRAVA`) do `servidor.py` e meça de novo. A vazão melhorou? (Cuidado: o resultado pode surpreender — meça antes de opinar.)
3. Explique, usando o que o Capítulo 12 estabeleceu sobre o decode, por que threads independentes não resolvem — e por que agrupar resolve.

16.9.4 E4 — O tamanho do lote

No `servidor_batch.py`, o `LOTE_MAX` é 16.

1. Meça a vazão com `--lote 1`, `--lote 4` e `--lote 16`, sempre com 8 clientes. Como ela varia?
2. `--lote 1` deveria reproduzir o comportamento do servidor sem batching. Reproduz? Se não, o que mais mudou entre os dois servidores?

3. A partir de que tamanho de lote o ganho satura? O que limita — e a resposta é a mesma do E4 do Capítulo 8?

16.9.5 E5 — A espera que custou caro

O capítulo relata um bug: esperar 8 ms por novos pedidos **a cada passo** de decode.

1. Reintroduza a espera e meça. Quanto ela custa por requisição de 40 tokens? A conta bate com 40×8 ms?
2. Existe algum cenário em que essa espera **ajudaria**? (Dica: pense em requisições que chegam espaçadas em vez de todas juntas.)
3. Formule a regra geral que a correção segue, e dê um exemplo de outro sistema onde ela se aplica.

16.9.6 E6 — O p95 conta o que a média esconde

1. Compare mediana e p95 nos dois servidores, com 8 clientes. Por que eles são diferentes num caso e iguais no outro?
2. Um serviço tem latência média de 200 ms e p99 de 4 s. Descreva o que o usuário experimenta — e por que reportar só a média seria enganoso.
3. Que outra métrica de cauda você acompanharia num serviço de LLM, além do p95 de latência? (Pense no que mais pode ter cauda longa.)

16.9.7 E7 — Backpressure (desafio)

O servidor deste capítulo aceita requisições sem limite: a fila cresce indefinidamente.

1. Meça o que acontece com 64 clientes simultâneos. A latência ainda é utilizável?
2. Implemente um limite: se a fila passar de N pedidos, responda **HTTP 429** em vez de enfileirar. Escolha o N a partir da sua medição do item 1.
3. Argumente: por que recusar uma requisição pode ser **melhor** que atendê-la devagar? Pense do ponto de vista do cliente que já está sendo atendido.

17 MULTIMODAL

Objetivo de aprendizagem: fazer o **mesmo Transformer** dos capítulos anteriores receber imagens. Construir um VQ-VAE do zero para transformar pixels em **tokens discretos**, e então descobrir que o modelo de texto funciona sem nenhuma alteração.

Pré-requisitos: Capítulos 5 (Transformer), 6 (tokenização) e 11 (pipeline de dados).

Arquivos:

- `preparar_dados.py` — baixa o MNIST
- `vqvae.py` — o VQ-VAE do zero: encoder, codebook, decoder
- `gerar_imagens.py` — o GPT dos capítulos de texto, gerando imagens
- `exercicios.md` — exercícios

17.1 Por que um Transformer não come imagens

Um Transformer recebe **tokens**: inteiros de um vocabulário finito. Uma imagem é o oposto — números contínuos, e muitos deles.

O MNIST é pequeno e já mostra o problema:

Grandeza	Valor
pixels por imagem	784 (28×28)
valores por pixel	256
contexto do modelo do Cap. 11	128 tokens

Passar pixel por pixel daria **784 tokens para uma única imagem** — seis vezes o contexto inteiro do nosso modelo. E seria desperdício: pixels vizinhos são quase sempre parecidos.

É exatamente o problema que o Capítulo 6 resolveu para texto. Tokenizar byte a byte funciona e é ruim; o BPE aprendeu pedaços maiores e recorrentes.

O VQ-VAE é o BPE das imagens.

17.2 A ideia: aprender um alfabeto de pedaços de imagem

```
imagem --[encoder]--> vetores contínuos --[quantização]--> INTEIROS
                                     |
imagem' <--[decoder]-- vetores do codebook <-----+----->
```

O **encoder** é uma pilha de convoluções que reduz 28×28 para um mapa de 7×7 , onde cada posição é um vetor de 32 números.

A peça nova é a **quantização**. Existe um *codebook*: K vetores aprendidos. Cada vetor que sai do encoder é substituído pelo **mais próximo** do codebook — e o índice desse vetor é o token.

```
dist = (plano.pow(2).sum(1, keepdim=True)
        - 2 * plano @ self.codebook.weight.t()
        + self.codebook.weight.pow(2).sum(1))
indices = dist.argmin(1) # <- o token
```

O resultado, medido:

Medida	Valor
pixels	784
tokens	49
compressão	16x menos posições

E uma imagem vira literalmente isto:

```
[24, 72, 64, 64, 24, 24, 72, 64, 72, 33, 71, 35, 35, 72, 64, 28, 28, 119, 119,
11, 72, 64, 43, 106, 71, 17, 52, 72, 64, 64, 35, 19, 11, 64, 72, 64, 64, 101,
92, 3, 64, 72, 95, 43, 77, 4, 64, 24, 78]
```

Uma lista de inteiros. Indistinguível, para o Transformer, de uma frase tokenizada.

17.3 O truque que faz isso treinar

`argmin` não tem derivada. Escolher o vizinho mais próximo é uma operação discreta — o gradiente morre ali, e o encoder nunca aprenderia.

A solução tem nome (*straight-through estimator*) e cabe numa linha:

```
z_q = z + (z_q - z).detach()
```

No **forward**, isso vale exatamente `z_q` — o vetor quantizado, que é o que o decoder deve receber. No **backward**, o termo `.detach()` não propaga nada, então o gradiente chega

em z como se a quantização fosse a identidade.

É **aproximado**: fingimos que a quantização não existe na hora de calcular gradientes. E funciona bem o bastante para o encoder aprender a produzir vetores que quantizam bem.

17.3.1 As duas perdas

O codebook também precisa aprender. São duas parcelas, e as duas são necessárias:

```
perda_codebook = F.mse_loss(z_q, z.detach()) # puxa o codebook para o encoder
perda_commit   = F.mse_loss(z, z_q.detach()) # puxa o encoder para o codebook
perda = perda_codebook + 0.25 * perda_commit
```

Sem a segunda — o *commitment loss* — o encoder foge livremente pelo espaço latente e o codebook nunca o alcança. Ela é o que obriga o encoder a **se comprometer** com os códigos que existem.

17.4 O resultado do VQ-VAE

1.500 passos, menos de dois minutos:

Medida	Valor
erro de reconstrução (validação)	0,0039
códigos usados	93 de 128 (73%)
compressão	16x

Original e reconstruída, em arte ASCII:

```
%@@@@@#####* :          #@@@@@#*+*#***=
:   :::: @@=         : :::-%@-
      @@-           :  .%@-
      :@@:          :  .%@:
      %@:           :  .@@.
      :@@:          :  -@@:
      #@@.          :  *@@.
      .%@=          :  %@*
      :@@@:         :  :@@@.
      =@%           :  +%#.
```

Reconhecível, com as bordas mais grosseiras — é o preço de descrever a imagem com 49 símbolos de um alfabeto de 128.

17.4.1 (!) 35 códigos nunca foram usados

Esse é o defeito clássico do VQ-VAE, e tem nome: **colapso do codebook**. Um código que nunca é o mais próximo de nada nunca recebe gradiente — e nunca se move para uma região onde seria útil. Ele morre no lugar onde foi inicializado.

É um problema de **realimentação**: para ser escolhido, o código precisa estar perto de alguma coisa; para chegar perto de alguma coisa, precisa ser escolhido. Quem começa mal fica mal.

Soluções conhecidas (o E4 mede uma): reinicializar códigos mortos sobre vetores do batch, usar médias móveis exponenciais em vez de gradiente, ou adicionar ruído no começo do treino.

17.5 A linha que é o capítulo inteiro

```
from modelo import GPT
```

É a **mesma classe** que escreveu prosa de Machado nos capítulos 11 a 15. Não há versão "visual" dela. Não há camada nova, nem atenção especial para imagens.

```
cfg = {"vocab_size": 128, "block_size": 49,
      "n_embd": 128, "n_head": 4, "n_layer": 4}
m = GPT(cfg)
```

O que mudou foi só o que entra:

Capítulo	Entrada	Tokenizador	Vocabulário	Tokens por amostra
Capítulo 11	texto	BPE	1.024	128
Capítulo 17	imagem	VQ-VAE	128	49

A arquitetura não sabe a diferença. Ela recebe inteiros e aprende a prever o próximo — e "o próximo" pode ser a próxima palavra ou o próximo pedaço de imagem.

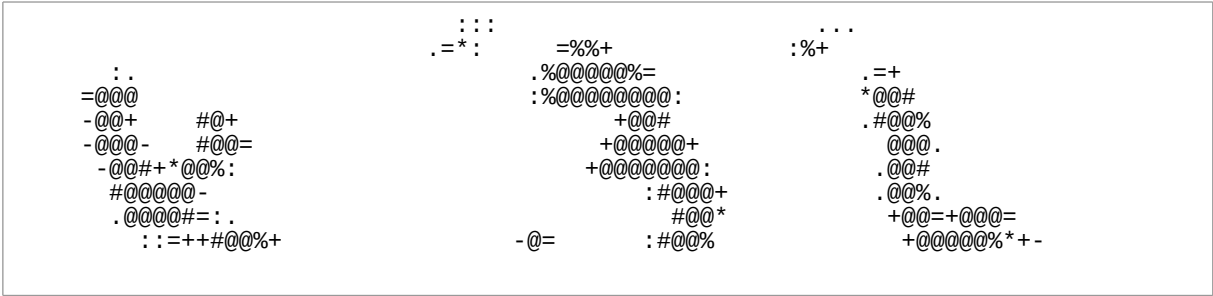
É por isso que modelos multimodais existem. Uma vez que tudo vira token, o mesmo modelo serve. A parte difícil não é a arquitetura — é o tokenizador de cada modalidade.

17.6 O modelo gerando imagens

830.976 parâmetros, 2.000 passos, 4,4 minutos sobre 2,9 milhões de tokens:

Medida	Valor
loss de validação	1,3599
perplexidade	3,9 entre 128 tokens

Três imagens geradas do zero, token por token — nenhuma delas estava no dataset:



O processo é **idêntico** ao dos capítulos de texto: amostra um token, realimenta, repete 49 vezes. Depois o decoder do VQ-VAE transforma a sequência em pixels.

17.6.1 E vale olhar com honestidade

Os traços parecem dígitos, e vários não são dígito nenhum. É o esperado: **0,8 M de parâmetros** treinados em **2,9 M de tokens** — a mesma conta desfavorável do Capítulo 11.

Este capítulo demonstra o **mecanismo**, não a qualidade. Um modelo que gera imagens convincentes por este caminho existe (o DALL·E original fazia exatamente isto) e é três ordens de grandeza maior.

17.7 O que a prática faz diferente

O caminho deste capítulo — VQ-VAE mais Transformer autorregressivo — é real e foi o estado da arte por um tempo. O que mudou desde então:

Abordagem	Como difere
VQ-GAN	acrescenta uma loss adversária ao decoder; reconstruções muito mais nítidas
Difusão	abandona os tokens discretos: aprende a remover ruído de forma iterativa, em espaço contínuo

ViT / encoders contínuos

para *entender* imagens (não gerar), pula a quantização e alimenta o Transformer com vetores contínuos de patches

E há uma distinção que vale carregar: **gerar** e **entender** imagens pedem coisas diferentes.

Para gerar, você precisa de uma saída discreta que o modelo possa amostrar — daí a quantização. Para entender (responder perguntas sobre uma foto, por exemplo), basta projetar os patches direto no espaço de embeddings do modelo de linguagem, sem codebook nenhum. É o que fazem os modelos de visão-linguagem atuais.

17.8 Resumo do capítulo

- Um Transformer come **tokens**. Imagens são contínuas e longas — 784 pixels contra 128 de contexto.
 - O **VQ-VAE é o BPE das imagens**: aprende um alfabeto de pedaços e comprime 784 pixels em 49 tokens.
 - `argmin` não tem derivada; o **straight-through estimator** finge que a quantização é a identidade no backward. Aproximado, e funciona.
 - O **commitment loss** é o que obriga o encoder a se comprometer com os códigos que existem.
 - **Códigos mortos** são a falha clássica: 35 de 128 nunca foram usados. É um problema de realimentação — quem começa mal fica mal.
 - `from modelo import GPT` — a mesma classe dos capítulos de texto, gerando imagens. Uma vez que tudo vira token, a arquitetura não sabe a diferença.
-

17.8.1 Fim do curso

Você começou contando pares de letras no Capítulo 1 e terminou com o mesmo Transformer recebendo texto e imagem, servido por HTTP com batching.

Nada disso veio de uma biblioteca: o autograd, a atenção, o tokenizador, o otimizador, o KV-cache, a quantização e o VQ-VAE foram escritos aqui, peça por peça.

E o hábito que este curso tentou instalar não é nenhuma das técnicas — é o de **medir antes de concluir**. Os gabaritos guardam, ao lado das respostas certas, uma boa coleção de previsões minhas que a medição desmentiu. Elas estão lá de propósito.

17.9 Exercícios

Faça na ordem. Soluções comentadas em `solucoes/` — **tente antes de olhar**.

Rode `python preparar_dados.py` e `python vqvae.py` antes. Juntos levam ~3 minutos.

17.9.1 E1 — Por que quantizar (aquecimento)

Sem rodar:

1. Uma imagem 28×28 vira 49 tokens. Qual a taxa de compressão em **posições**? E se você contasse em **bits**, considerando que cada pixel é um byte e cada token cabe em 7 bits?
2. Por que não alimentar o Transformer com os vetores contínuos do encoder, pulando a quantização? O que se perderia? (Dica: pense no que a camada de saída do modelo produz.)
3. O capítulo chama o VQ-VAE de "BPE das imagens". Em que a analogia é boa, e em que ela quebra?

17.9.2 E2 — O straight-through estimator (importante)

1. Remova a linha `z_q = z + (z_q - z).detach()` e passe `z_q` direto para o decoder. Treine 300 passos. O que acontece com a perda de reconstrução, e por quê?
2. Escreva o que o `.detach()` faz no forward e no backward, separadamente. Por que a expressão vale `z_q` num e `z` no outro?
3. O estimador é **aproximado** — o gradiente que o encoder recebe não é o gradiente verdadeiro. Por que isso não impede o treino de funcionar?

17.9.3 E3 — As duas perdas (importante)

O quantizador tem `perda_codebook` e `perda_commit`.

1. Treine com `BETA_COMP = 0` (sem commitment loss). Meça o erro de reconstrução e o número de códigos usados. O que piora?
2. Treine com `BETA_COMP = 2.0`. O que acontece agora, e por quê?
3. Explique o papel de cada perda em uma frase, e diga o que aconteceria removendo a **outra** (a `perda_codebook`).

17.9.4 E4 — Códigos mortos

O capítulo mede 93 códigos usados de 128.

1. Varie o tamanho do codebook (`--codebook 32, 64, 256, 512`) e monte a tabela de códigos usados e erro de reconstrução. A fração usada cresce ou cai com o tamanho?
2. Implemente a correção mais simples: a cada N passos, reinicialize os códigos não usados sobre vetores aleatórios do batch atual. Quantos códigos passam a ser usados?
3. A correção melhorou o **erro de reconstrução**, ou só a estatística de uso? Se só a estatística, isso ainda vale a pena? Justifique.

17.9.5 E5 — O mesmo Transformer

1. No `gerar_imagens.py`, a config do GPT é `{vocab_size: 128, block_size: 49, ...}`. Compare com a do Capítulo 11. Quantas linhas de código do modelo precisaram mudar para ele aceitar imagens?
2. A perplexidade medida é 3,9 entre 128 tokens. Compare com a do Capítulo 11 (51,7 entre 1.024). O modelo de imagens é "melhor"? O que torna a comparação difícil?
3. O modelo gera as 49 posições em ordem de varredura (esquerda para a direita, de cima para baixo). Isso é natural para texto e arbitrário para imagem. Que problema essa ordem cria, e o que você tentaria no lugar?

17.9.6 E6 — Condicionar pelo dígito

O MNIST tem rótulos, e o `preparar_dados.py` já os salva.

1. Acrescente o rótulo como **primeiro token** da sequência (reserve 10 ids novos, como o Capítulo 14 fez com os tokens especiais). Retreine.
 2. Gere dez imagens, uma por dígito, começando cada sequência pelo rótulo correspondente. O modelo obedece?
 3. Você acabou de construir um gerador **condicional** — o mesmo mecanismo de um "texto -> imagem", com um vocabulário de condição de tamanho 10 em vez de uma frase. O que mudaria para condicionar por uma legenda de verdade?
-

17.9.7 E7 — Um modelo, duas modalidades (desafio)

1. Junte os vocabulários: tokens de texto do Capítulo 11 (ids 0-1023) e tokens de imagem deste capítulo (ids 1024-1151). Treine **um** GPT em sequências dos dois tipos.
2. Meça a loss em cada modalidade separadamente e compare com os modelos especializados. Houve interferência? Em qual direção?
3. **A pergunta que importa:** um modelo treinado nas duas modalidades separadamente, sem nenhum exemplo *pareado*, aprende alguma relação entre elas? Justifique pelo que o objetivo de treino recompensa — e diga o que seria preciso para que aprendesse.

REFERÊNCIAS

Lista elaborada segundo a **ABNT NBR 6023:2018**, em ordem alfabética de sobrenome.

Uma observação de honestidade acadêmica: o corpo desta apostila é expositivo e não traz citações diretas no formato autor-data. As obras abaixo são as **fontes primárias** de cada técnica apresentada — os trabalhos em que os métodos foram originalmente propostos e a partir dos quais o texto foi escrito. O Apêndice B associa cada capítulo às suas fontes.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6024**: informação e documentação: numeração progressiva das seções de um documento: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6027**: informação e documentação: sumário: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

ASSIS, Machado de. **Dom Casmurro**. Rio de Janeiro: Garnier, 1899. Texto digital disponível em: Project Gutenberg, ebook n. 55752.

ASSIS, Machado de. **Memórias póstumas de Brás Cubas**. Rio de Janeiro: Tipografia Nacional, 1881. Texto digital disponível em: Project Gutenberg, ebook n. 54829.

ASSIS, Machado de. **Quincas Borba**. Rio de Janeiro: Garnier, 1891. Texto digital disponível em: Project Gutenberg, ebook n. 55682.

BA, Jimmy Lei; KIROS, Jamie Ryan; HINTON, Geoffrey E. Layer normalization. **arXiv preprint arXiv:1607.06450**, 2016.

BAHDANAU, Dzmitry; CHO, Kyunghyun; BENGIO, Yoshua. Neural machine translation by jointly learning to align and translate. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 3., 2015, San Diego. **Proceedings [...]**, 2015.

BENGIO, Yoshua; DUCHARME, Réjean; VINCENT, Pascal; JAUVIN, Christian. A neural probabilistic language model. **Journal of Machine Learning Research**, v. 3, p. 1137-1155, 2003.

BENGIO, Yoshua; LÉONARD, Nicholas; COURVILLE, Aaron. Estimating or propagating gradients through stochastic neurons for conditional computation. **arXiv preprint arXiv:1308.3432**, 2013.

BROWN, Tom B. et al. Language models are few-shot learners. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 33., 2020. **Proceedings [...]**, 2020.

DEEPSEEK-AI. DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning. **arXiv preprint arXiv:2501.12948**, 2025.

DETTMERS, Tim; LEWIS, Mike; BELKADA, Younes; ZETTLEMOYER, Luke. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 35., 2022. **Proceedings** [...], 2022.

DOSOVITSKIY, Alexey et al. An image is worth 16x16 words: transformers for image recognition at scale. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 9.,

2021. **Proceedings** [...], 2021.

ESSER, Patrick; ROMBACH, Robin; OMMER, Björn. Taming transformers for high-resolution image synthesis. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2021. **Proceedings** [...], 2021.

FRANTAR, Elias; ASHKBOOS, Saleh; HOEFLER, Torsten; ALISTARH, Dan. GPTQ: accurate post-training quantization for generative pre-trained transformers. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 11., 2023. **Proceedings** [...], 2023.

GAGE, Philip. A new algorithm for data compression. **The C Users Journal**, v. 12, n. 2, p. 23-38, 1994.

GLOROT, Xavier; BENGIO, Yoshua. Understanding the difficulty of training deep feedforward neural networks. In: INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE AND STATISTICS (AISTATS), 13., 2010. **Proceedings** [...], 2010. p. 249-256.

HE, Kaiming; ZHANG, Xiangyu; REN, Shaoqing; SUN, Jian. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In: IEEE INTERNATIONAL CONFERENCE ON COMPUTER VISION (ICCV), 2015. **Proceedings** [...], 2015. p. 1026-1034.

HO, Jonathan; JAIN, Ajay; ABBEEL, Pieter. Denoising diffusion probabilistic models. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 33., 2020. **Proceedings** [...], 2020.

HOFFMANN, Jordan et al. Training compute-optimal large language models. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 35., 2022. **Proceedings** [...], 2022.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Nomes no Brasil**: censo demográfico

2010. Rio de Janeiro: IBGE, 2016. Disponível em: <https://censo2010.ibge.gov.br/nomes/>.

JACOB, Benoît et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: IEEE/CVF CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2018. **Proceedings** [...], 2018. p. 2704-2713.

JURAFSKY, Daniel; MARTIN, James H. **Speech and language processing**: an introduction to natural language processing, computational linguistics, and speech recognition. 3. ed. (rascunho). Stanford: [s. n.], 2024.

KAPLAN, Jared et al. Scaling laws for neural language models. **arXiv preprint arXiv:2001.08361**, 2020.

KARPATY, Andrej. **Neural networks**: zero to hero. 2023. Disponível em: <https://karpathy.ai/zero-to-hero.html>.

KARPATY, Andrej. **LLM101n**: let's build a storyteller. Eureka Labs, 2024. Disponível em: <https://github.com/karpathy/LLM101n>.

KINGMA, Diederik P.; BA, Jimmy. Adam: a method for stochastic optimization. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 3., 2015, San Diego. **Proceedings [...]**, 2015.

KWON, Woosuk et al. Efficient memory management for large language model serving with PagedAttention. In: ACM SYMPOSIUM ON OPERATING SYSTEMS PRINCIPLES (SOSP), 29., 2023. **Proceedings [...]**, 2023.

LECUN, Yann; CORTES, Corinna; BURGESS, Christopher J. C. **The MNIST database of handwritten digits**. 1998. Disponível em: <http://yann.lecun.com/exdb/mnist/>.

LI, Shen et al. PyTorch distributed: experiences on accelerating data parallel training. **Proceedings of the VLDB Endowment**, v. 13, n. 12, p. 3005-3018, 2020.

LOSHCHILOV, Ilya; HUTTER, Frank. Decoupled weight decay regularization. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 7., 2019. **Proceedings [...]**, 2019.

MICIKEVICIUS, Paulius et al. Mixed precision training. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 6., 2018. **Proceedings [...]**, 2018.

OORD, Aäron van den; VINYALS, Oriol; KAVUKCUOGLU, Koray. Neural discrete representation learning. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 30., 2017. **Proceedings [...]**, 2017.

OUYANG, Long et al. Training language models to follow instructions with human feedback. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 35., 2022. **Proceedings [...]**, 2022.

PASZKE, Adam et al. PyTorch: an imperative style, high-performance deep learning library. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 32., 2019. **Proceedings [...]**, 2019.

POPE, Reiner et al. Efficiently scaling transformer inference. In: CONFERENCE ON MACHINE LEARNING AND SYSTEMS (MLSys), 2023. **Proceedings [...]**, 2023.

RADFORD, Alec et al. **Improving language understanding by generative pre-training**. São Francisco: OpenAI, 2018.

RADFORD, Alec et al. **Language models are unsupervised multitask learners**. São Francisco: OpenAI, 2019.

RAJBHANDARI, Samyam; RASLEY, Jeff; RUWASE, Olatunji; HE, Yuxiong. ZeRO: memory optimizations toward training trillion parameter models. In: INTERNATIONAL CONFERENCE FOR HIGH PERFORMANCE COMPUTING, NETWORKING,

STORAGE AND ANALYSIS (SC), 2020. **Proceedings** [...], 2020.

RAMESH, Aditya et al. Zero-shot text-to-image generation. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML), 38., 2021. **Proceedings** [...], 2021.

SCHULMAN, John et al. Proximal policy optimization algorithms. **arXiv preprint arXiv:1707.06347**, 2017.

SENNRICH, Rico; HADDOW, Barry; BIRCH, Alexandra. Neural machine translation of rare words with subword units. In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS (ACL), 54., 2016, Berlim. **Proceedings** [...], 2016. p. 1715-1725.

SHANNON, Claude E. A mathematical theory of communication. **The Bell System Technical Journal**, v. 27, n. 3, p. 379-423, 1948.

SHAZEER, Noam. Fast transformer decoding: one write-head is all you need. **arXiv preprint arXiv:1911.02150**, 2019.

STIENNON, Nisan et al. Learning to summarize from human feedback. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 33., 2020. **Proceedings** [...], 2020.

VASWANI, Ashish et al. Attention is all you need. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS (NeurIPS), 30., 2017, Long Beach. **Proceedings** [...], 2017. p. 5998-6008.

WEI, Jason et al. Finetuned language models are zero-shot learners. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR), 10., 2022. **Proceedings** [...], 2022.

WILLIAMS, Ronald J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. **Machine Learning**, v. 8, n. 3-4, p. 229-256, 1992.

XIONG, Ruibin et al. On layer normalization in the transformer architecture. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING (ICML), 37., 2020. **Proceedings** [...], 2020.

YU, Gyeong-In et al. Orca: a distributed serving system for transformer-based generative models. In: USENIX SYMPOSIUM ON OPERATING SYSTEMS DESIGN AND IMPLEMENTATION (OSDI), 16.,

2022. **Proceedings** [...], 2022.

APENDICE A — GABARITOS COMENTADOS

Os gabaritos foram reunidos em apêndice, e não ao final de cada capítulo, por uma razão pedagógica: um exercício cuja resposta está na página seguinte não é um exercício. Consultar este apêndice deve ser uma decisão, não um acidente de leitura.

Todos os números aqui apresentados vêm de execução. Onde a medição contrariou a hipótese inicial do autor, o texto registra as duas coisas — a previsão e o resultado —, porque o erro documentado ensina mais que o acerto silencioso.

A.1 GABARITO DO CAPÍTULO 1 — BIGRAM LANGUAGE MODEL

Respostas discursivas. Os **números** vêm de `gabarito.py`, que roda os experimentos; rode-o e compare com os seus resultados.

A.1.1 E1 — Leitura de código

1. Por que `vocab_size` é 27, e não 26? São as 26 letras a-z **mais** o token especial de fronteira `.`, que marca início e fim de nome. Sem ele o modelo não saberia com que letra começar nem quando parar.

2. O que aconteceria trocando $(N + 1)$ por N no cálculo da loss? Bigramas nunca vistos teriam contagem 0, logo probabilidade 0, e $\log(0) = -\text{infinito}$ — a loss viraria infinita.

Um detalhe que o `gabarito.py` mostra e que vale entender: com este dataset, avaliando **nos mesmos dados do treino**, a loss com `+0` sai finita (2,1473). Isso porque todo bigrama avaliado foi, por definição, visto no treino. O infinito aparece assim que você avalia um nome **novo** que contenha um par inédito. É por isso que se suaviza mesmo quando "parece funcionar".

3. Para que serve o `manual_seed`? Fixa o gerador de números aleatórios, tornando a execução **reprodutível** — você e eu vemos os mesmos nomes gerados. Útil para estudar e depurar; em produção você deixaria aleatório.

A.1.2 E2 — Mexa nos dados

Medido (`gabarito.py`):

Dataset	Loss
155 nomes originais	2,3832
+ 5 nomes estranhos (zwqk, xyzabc...)	2,3982 (+0,0150)
+ 5 nomes típicos (ana, maria...)	2,3750 (-0,0081)

A resposta correta é "depende", e é isso que o exercício quer ensinar. Não existe "mais dados = loss menor". O que existe é: dados que **seguem o padrão** que o modelo já capturou reduzem a loss; dados com padrões **raros** a aumentam, porque o modelo precisa gastar probabilidade em bigramas incomuns.

Se você acrescentou nomes de amigos (portugueses, típicos), provavelmente viu a loss cair um pouco. Se acrescentou nomes inventados esquisitos, viu subir.

A.1.3 E3 — O efeito da suavização

Suavização	Loss
+0	2,1473
+1	2,3832
+10	2,8674
+100	3,2114

1. A loss **cresce** monotonicamente com a suavização. Suavizar afasta o modelo dos dados observados.

2. Os nomes gerados ficam mais **genéricos e aleatórios** conforme a suavização aumenta — a distribuição tende ao uniforme, e o modelo "esquece" o que aprendeu. Com +0 eles grudam mais nos padrões do dataset.

3. Ver E1.2 acima: +0 produz probabilidade zero para pares não vistos, e o $\log(0)$ quebra a conta assim que você sai dos dados de treino.

A.1.4 E4 — Temperatura no sampling

Medido, com a mesma semente:

T	Nomes gerados
0,3	celia, lo, la, ria, ca
1,0	cexzma, ogosuraro, zktahwelo, imjtta
3,0	cexzm, loglkurkicczktyh, mvlzimjttainrlkf

1. T **baixo** -> nomes mais "óbvios", curtos e repetitivos: a temperatura concentra a massa de probabilidade nas opções mais prováveis. T **alto** -> mais aleatórios e

impronunciáveis: ela achata a distribuição.

2. É exatamente o parâmetro temperature das APIs de LLM. Note a forma usada: $p^{**}(1/T)$. Com $T < 1$ o expoente é maior que 1, o que **acentua** as diferenças entre as probabilidades; com $T > 1$ ele as **achata**. Nos modelos modernos a mesma operação é aplicada aos *logits* antes do softmax (dividir os logits por T), o que é matematicamente equivalente.

A.1.5 E5 — Igualando contagem e rede neural

Configuração	Loss
Contagem +1	2,3832
Contagem +0,01	2,1518
Rede, reg=0.01, 200 passos	2,1774
Rede, reg=0.0, 1000 passos	2,1600

1 e 2. Com a suavização **igualada** dos dois lados, os valores convergem para a mesma faixa (~2,15-2,18).

3. É a confirmação da tese do capítulo: contar e otimizar chegam ao **mesmo modelo**. A diferença que se via na apostila (2,38 contra 2,21) vinha só do fato de que o +1 é uma suavização **muito mais forte** que o reg=0.01. São o mesmo botão, em escalas diferentes.

A.1.6 E6 — NLL de um nome específico

Solução em `e6_nll_de_nome.py`. Resultados:

```
ana      -> NLL = 1.7871
maria    -> NLL = 1.7626
vinicius -> NLL = 2.4613
xwq      -> NLL = 3.6696
kffkz    -> NLL = 3.4179
```

1. ana tem NLL bem menor que xwq — o modelo considera ana muito mais provável.

2. Nomes com bigramas comuns em português (ma, ri, an) pontuam bem. Nomes com sequências raras (xw, kz) pontuam mal. vinicius fica no meio: tem vi e ni comuns, mas ci seguido de us é menos frequente no dataset.

A.1.7 E7 — Rumo ao trigrama

Modelo	Suavização	Loss
Bigrama	+0, 01	2,1518
Trigrama	+1, 0	2,3449
Trigrama	+0, 01	1,1523

1. A tabela passa de $27 \times 27 = 729$ para $27 \times 27 \times 27 = \text{**19.683**}$ células.

2. Com suavização comparável (+0, 01), a loss **melhora muito**: 2,15 -> 1,15. Mais contexto ajuda de verdade.

Atenção ao efeito da suavização aqui: com +1, o trigrama fica *pior* que o bigrama (2,34 contra 2,15). Isso não é o trigrama sendo ruim — é que somar 1 a **19.683** células injeta muito mais "massa uniforme" do que somar 1 a 729. Quanto maior a tabela, mais destrutiva é a mesma suavização. Comparar modelos com suavização fixa é comparar coisas diferentes.

3. O problema novo é a **esparsidade**: a maioria das células nunca é preenchida. A suavização evita o $\log(0)$, mas **não inventa informação** — para combinações nunca vistas o modelo apenas chuta uniformemente. E isso piora exponencialmente: com k caracteres de contexto são $27^{(k+1)}$ células.

É exatamente essa parede que motiva o **Capítulo 3**: em vez de contar células, o MLP aprende uma representação **densa e compartilhada**, onde contextos parecidos se apoiam mutuamente.

A.2 GABARITO DO CAPÍTULO 2 — MICROGRAD (AUTOGRAD E BACKPROPAGATION)

Respostas discursivas. Os **números** vêm de `gabarito.py`.

A.2.1 E1 — Leitura de código

1. Para $f = a * b$ com $a = 3$, $b = 4$: $df/da = b = 4$ e $df/db = a = 3$. A derivada em relação a um fator é o **outro** fator.

2. Para $d = a*b + c$: $dd/dc = 1$. A soma não altera o gradiente porque a derivada de uma soma em relação a cada parcela é 1 — ela apenas **distribui** o gradiente que chega, sem escalá-lo. É por isso que, no código, o `_backward` da soma é `self.grad += 1.0 * out.grad`.

3. Por que **tanh saturada tem gradiente quase zero**: a derivada é $1 - \tanh(x)^2$. Para x grande, $\tanh(x) \rightarrow \pm 1$, logo $\tanh(x)^2 \rightarrow 1$ e a derivada $\rightarrow 0$. Nas regiões extremas a curva é praticamente horizontal — mexer na entrada quase não muda a saída, e o gradiente morre. É o mesmo efeito que reaparece no softmax do Capítulo 4 (daí o fator $1/\sqrt{d}$) e na inicialização do Capítulo 7.

A.2.2 E2 — Conferindo com diferença finita

Variável	Autograd	Diferença finita	Erro relativo
a	-0,00402285	-0,00402285	5,97e-10
b	+0,00268190	+0,00268190	5,97e-10
c	+0,00134095	+0,00134095	5,97e-10

Batem. Dois detalhes que valem a pena:

- Usamos a **diferença central**, $(f(x+h) - f(x-h)) / 2h$, e não a ingênua $(f(x+h) - f(x)) / h$. A central tem erro $O(h^2)$ em vez de $O(h)$ — bem mais precisa pelo mesmo custo.
- Existe um **h ótimo**: grande demais e a aproximação da derivada é ruim; pequeno demais e o cancelamento em ponto flutuante domina (você subtrai dois números quase iguais). Por volta de $1e-6$ costuma ser um bom meio-termo em `float64`.

Esta é a forma clássica de **testar** um autograd, e é o que bibliotecas de verdade usam nos seus testes (`torch.autograd.gradcheck` faz exatamente isso).

A.2.3 E3 — Implementando `log()`

1. A derivada local é $d(\ln x)/dx = 1/x$.
2. Seguindo o padrão de sempre — derivada local vezes o gradiente que vem de fora:

```
def log(self):
    out = Value(math.log(self.data), (self,), "log")
    def _backward():
        self.grad += (1.0 / self.data) * out.grad
    out._backward = _backward
    return out
```

3. Verificado: `log(3.0) = 1,098612` (bate com `math.log`), e $d(\log)/dx = 0,333333$ contra `0,333333` da diferença finita — erro de $4,66e-11$.

Cuidado que o exercício não pede mas vale saber: `log` só é definido para $x > 0$. Numa implementação séria você trataria $x \leq 0$ explicitamente, em vez de deixar o `math.log` levantar exceção no meio de um treino.

A.2.4 E4 — Por que zerar o gradiente

Medido, 20 passos:

Configuração	passo 0	passo 5	passo 19
com <code>zero_grad</code>	3,0929	0,0006	0,0001
sem <code>zero_grad</code>	3,0929	0,7659	0,6158

1 e 2. Leia os números, não a intuição. Sem o `zero_grad` o treino **não explode neste caso — ele estagna**. A loss cai até $\sim 0,6$ e trava, enquanto com `zero_grad` chega a 0,0001.

Por que estagna em vez de divergir? Os gradientes são acumulados com `+=`, então o gradiente do passo N é a soma de todos os anteriores. Isso produz dois efeitos que se combatem: o passo fica **grande demais** (empurra para divergir) mas também **desatualizado** — carrega direções calculadas para pesos que já mudaram. Aqui o modelo entra num vai-e-vem e não converge. Com learning rate maior, ou em outros problemas, o mesmo bug realmente diverge.

3. É exatamente o `w.grad = None` do Capítulo 1, pelo mesmo motivo.

A lição que mais importa: o resultado fica errado de um jeito que **não se parece com um erro**. Não há exceção, não há NaN — só um treino pior. Bugs assim são os mais caros de encontrar, e é por isso que se compara sempre contra uma referência.

A.2.5 E5 — Bate com o PyTorch?

Solução em `e5_check_vs_torch.py`. Resultado: ALL MATCH: True, com diferença máxima de **8,94e-08**.

2. Trocando tanh por relu, continuam batendo — o mecanismo é o mesmo, só muda a derivada local registrada por aquela operação.

A.2.6 E6 — Arquitetura e learning rate

Arquitetura	Parâmetros	Loss final	Situação
[4, 4, 1]	41	0,000000	convergiu
[8, 1]	41	0,000000	convergiu
[16, 16, 1]	353	1,38e+147	DIVERGIU

Learning rate	Loss final	Situação
0,5	1,80e+248	DIVERGIU
0,05	0,000000	boa
0,001	0,136863	lenta demais

1. A primeira resposta contraria o senso comum. [4, 4, 1] e [8, 1] convergem — e, por coincidência, têm exatamente os **mesmos 41 parâmetros**. Mas [16, 16, 1], com 353 parâmetros, **diverge** com a mesma learning rate.

Rede maior não é automaticamente melhor. Com mais parâmetros, a soma dos gradientes fica maior, e uma lr que servia para a rede pequena passa a ser grande demais. **A learning rate adequada depende do tamanho do modelo** — que é justamente o que motiva a inicialização escalada e o agendamento do Capítulo 7.

2. A curva em U clássica: 0,5 diverge, 0,05 converge, 0,001 quase não anda no orçamento de 100 passos. O Capítulo 7 mede isso de forma sistemática (exercício E5 de lá).

3. Trocar `tanh` por `relu` neste problema faz pouca diferença: com 4 pontos de dados, qualquer não-linearidade razoável resolve. A diferença entre ativações aparece em redes profundas, e é o assunto da Seção 2 do Capítulo 7.

A.2.7 E7 — Visualizando o grafo

Implementação em `gabarito.py`. Para `(a*b + c).tanh()`:

```
Value(data=0.9993, grad=1.0000) [tanh]
+-- Value(data=4.0000, grad=0.0013) [+]
+-- Value(data=10.0000, grad=0.0013)
+-- Value(data=-6.0000, grad=0.0013) [*]
+-- Value(data=2.0000, grad=-0.0040)
+-- Value(data=-3.0000, grad=0.0027)
```

Leia de cima para baixo: a raiz é a saída, e cada nível mostra os "pais" que a produziram. A `backward()` percorre exatamente esta estrutura, aplicando a regra da cadeia.

Repare em como o gradiente se comporta:

- O `tanh` na raiz tem `grad = 1.0` (é a semente, $d(\text{saída})/d(\text{saída}) = 1$).
- A **soma** distribui `0,0013` igualmente para `c` e para o produto — sem alterar nada.
- A **multiplicação** escala: `a` recebe $0,0013 \times (-3) = -0,0040$ e `b` recebe $0,0013 \times 2 = 0,0027$. Cada um multiplicado pelo valor do outro.

E note que todos os gradientes são pequenos ($\sim 1e-3$): a `tanh` está saturada em $x = 4$, o que confirma o que a resposta do E1.3 previu.

A.3 GABARITO DO CAPÍTULO 3 — N-GRAM MODEL (MLP EM PYTORCH)

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Sobre o orçamento: o gabarito treina **4.000 passos** (a apostila usa 20.000) para rodar em poucos minutos. Os valores absolutos ficam piores que os da apostila, mas as **comparações entre configurações** — que é o que os exercícios pedem — continuam válidas. Onde essa diferença muda a conclusão, está dito explicitamente (veja o E4).

A.3.1 E1 — Leitura de código

1. Por que `emb = c[X]` tem formato `(N, 3, 10)`? `x` tem formato `(N, 3)`: `N` exemplos, 3 índices de caractere cada. `c[X]` substitui cada índice pela linha correspondente de `c`, que tem 10 números. Então: `N` exemplos $\times$ 3 posições de contexto $\times$ 10 dimensões de embedding.

2. Por que o `.view(N, -1)` antes da camada linear? A camada linear espera **um vetor** por exemplo, não uma matriz 3×10 . O `view` reorganiza os mesmos números em `(N, 30)`, colando os três embeddings num vetor só. O `-1` diz ao PyTorch "calcule esta dimensão". Note que `view` **não copia** dados — apenas reinterpreta o mesmo bloco de memória.

3. O que `F.cross_entropy(logits, y)` faz? Exatamente o que fizemos à mão no Capítulo 1: **softmax** seguido de **negative log-likelihood**. A diferença é que a versão do PyTorch é numericamente estável (evita `exp()` de números grandes, que estouraria) e mais rápida.

A.3.2 E2 — O efeito do contexto

<code>block_size</code>	Parâmetros	Treino	Validação
1	7.897	2,3498	2,3401
3	11.897	2,0508	2,0452
5	15.897	2,0165	2,0114

1. Mais contexto ajuda: a validação cai de 2,34 para 2,01.

2. Com `block_size = 1` o MLP vira um **bigrama neural — ele só vê o caractere anterior, como no Capítulo 1. E de fato a loss (2,34) cai na faixa do bigrama daquele capítulo (~2,4). É uma boa confirmação de que os dois modelos são a mesma coisa quando o contexto**

é o mesmo.

3. Sim, os parâmetros crescem: a primeira camada tem $(n_embd \times block_size) \times n_hidden$ pesos. Cada caractere a mais de contexto acrescenta $n_embd \times n_hidden = 2.000$ pesos.

É exatamente esse custo linear que motiva a **atenção** do Capítulo 4: lá, aumentar o contexto **não** aumenta o número de parâmetros.

A.3.3 E3 — Tamanho da camada oculta

n_hidden	Parâmetros	Treino	Validação
50	3.197	2,1128	2,1023
200	11.897	2,0508	2,0452
500	29.297	2,0287	2,0271

1. Mais neurônios, mais parâmetros, loss menor.

2. **Retorno decrescente, e claro:** de 50 para 200 (parâmetros $\times 3,7$) a validação melhora 0,057. De 200 para 500 (parâmetros $\times 2,5$) melhora só 0,018 — três vezes menos ganho. Cada parâmetro novo contribui menos que o anterior.

A.3.4 E4 — Learning rate

lr	Validação	Situação
0,001	2,4547	lenta demais
0,01	2,2162	
0,1	2,0452	<- o valor da apostila
1,0	2,0019	melhor aqui
10,0	—	divergiu
50,0	—	divergiu

1. Esta resposta provavelmente contraria o que você esperava. A curva é em U, mas o mínimo **não está no 0,1 que a apostila usa** — neste orçamento de 4.000 passos, $lr = 1,0$ vai melhor. A divergência só aparece em 10,0.

Isso **não** significa que a apostila esteja errada. Ela treina 20.000 passos, e com orçamento maior uma learning rate menor tem tempo de refinar e chega mais longe. A conclusão correta:

A melhor learning rate depende do orçamento de passos. Ela não é uma propriedade do modelo. O mesmo fenômeno reaparece no Capítulo 7 (exercício E5), onde $3e-3$ ganha de $1e-3$ justamente porque o experimento treina menos passos que a apostila.

2. Removendo o decaimento com a melhor lr: 2,0915 sem contra **2,0019** com — o decaimento vale **0,09** de loss. Passos menores no fim permitem "assentar" no mínimo em vez de ficar pulando em torno dele.

A.3.5 E5 — Overfitting com dataset pequeno

Solução em `e5_overfitting.py`:

```
=== DATASET PEQUENO (155 nomes) ===
loss treino      = 0.8001
loss validacao   = 6.5086
```

1. Um abismo: treino 0,80 contra validação 6,51.

2. O mesmo modelo generaliza com 64 mil nomes e decora com 155 porque o número de **parâmetros** (11.897) é maior que o número de **exemplos de treino** (880). Com essa proporção, memorizar é mais fácil que aprender o padrão — e o modelo faz o que é mais fácil. (É a mesma conta que reaparece no Capítulo 11: 0,28 token por parâmetro.)

3. Os nomes gerados se parecerem demais com nomes reais do dataset é **mau** sinal porque significa que o modelo está **reproduzindo**, não **generalizando**. Um bom modelo gera nomes que *poderiam* estar na lista mas não estão. Qualidade aparente na saída pode esconder memorização — por isso se mede na validação, não no olho.

A.3.6 E6 — Visualizando os embeddings

Com `n_embd = 2` (treino 2,2954 / validação 2,2873 — pior que com 10, como esperado):

Distância média	Valor
vogal <-> vogal	0,732
vogal <-> consoante	1,173
razão	1,60x

1. **As vogais ficam agrupadas.** Elas estão 1,6x mais próximas entre si do que das consoantes. A rede descobriu sozinha que vogais têm papel parecido — **só por prever o próximo caractere**. Ninguém ensinou fonética a ela.

Este é um dos resultados mais bonitos do curso: estrutura linguística emergindo de pura estatística de sequência.

2. Reduzir para `n_embd = 2` piora a loss (menos capacidade por caractere), mas é o único jeito de desenhar os embeddings num plano. É um trade-off comum em interpretabilidade: sacrifica-se desempenho para poder **ver**. (Em modelos de verdade usa-se PCA ou t-SNE para projetar embeddings de centenas de dimensões em 2, sem retreinar.)

A.3.7 E7 — Contando os parâmetros

Componente	Cálculo
C (embeddings)	27270 × 10
W1	306.000 × 200
b1	20000
W2	206.400 × 27
b2	2727
Total	11.897

1. Fórmula geral:

$$\text{vocab} \times \text{n_embd} + (\text{n_embd} \times \text{block_size}) \times \text{n_hidden} + \text{n_hidden} + \text{n_hidden} \times \text{vocab} + \text{vocab}$$

2. Confere exatamente com o 11897 que o código informa.

3. **Quem domina:** w1 (6.000) e w2 (5.400) somam **96%** do total. As duas matrizes grandes são o modelo; embeddings e vieses são detalhes.

Consequências:

- Dobrar n_embd quase dobra o total (ele aparece em C e em w1).
- Dobrar n_hidden também (aparece em w1 e em w2).
- Dobrar block_size só afeta w1 — cresce menos que os outros dois.

A.4 GABARITO DO CAPÍTULO 4 — ATTENTION (O MECANISMO DE ATENÇÃO)

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Orçamento: 4.000 passos por configuração (a apostila usa 20.000). Valores absolutos ficam piores; as comparações continuam válidas.

A.4.1 E1 — Leitura de código

1. O que a `tril` impede, e o que aconteceria sem ela no treino? Ela impede que uma posição veja o **futuro**. Sem ela, ao prever o token seguinte a posição poderia consultar justamente a resposta — a loss de treino despencaria e o modelo não aprenderia nada útil, porque na hora de gerar texto o futuro não existe. É *data leakage*.

(O exercício E2 mostra que isso tem uma sutileza importante — veja abaixo.)

2. Por que dividir por $\sqrt{\text{head_size}}$? O produto escalar $q \cdot k$ é a soma de `head_size` produtos, então a sua magnitude cresce com a dimensão. Pontuações grandes fazem o softmax **saturar**: a distribuição vira quase um one-hot e o gradiente morre. Dividir por $\sqrt{\text{head_size}}$ mantém a variância em torno de 1 independentemente do tamanho da cabeça.

3. Query, key e value:

- **query:** "o que eu estou procurando?"
- **key:** "o que eu tenho a oferecer?" (o endereço pelo qual sou encontrado)
- **value:** "o que eu de fato entrego, se você me escolher" (o conteúdo)

Separar *key* de *value* dá liberdade ao modelo: como sou encontrado e o que eu entrego são coisas diferentes.

A.4.2 E2 — Sem a máscara causal

Este exercício tem uma armadilha que vale mais que a pergunta.

Configuração	Treino	Validação
1 camada, máscara ON	2,1589	2,1517
1 camada, máscara OFF	2,1589	2,1517

2 camadas, máscara ON	2,2238	2,2122
2 camadas, máscara OFF	2,1614	2,1513

Com uma camada, remover a máscara não muda absolutamente nada. Os números são idênticos até a quarta casa. Não é erro de medição.

O motivo é estrutural: este modelo usa **apenas a última posição** para prever, e a última linha da matriz triangular **já é toda de uns**. Para a posição que importa, a máscara nunca fez diferença — ela só afetava posições intermediárias, cujas saídas são **descartadas**.

Com duas camadas a história muda. A saída das posições intermediárias da camada 1 alimenta a camada 2, e a última posição da camada 2 as lê. Aí o futuro realmente vaza, e a loss sem máscara fica artificialmente melhor (2,1513 contra 2,2122).

E onde a máscara é absolutamente essencial: quando o modelo prevê em **todas** as posições (Capítulo 11). Ali a posição t veria literalmente o token $t+1$ que deve prever — vazamento direto e devastador.

A lição: uma proteção pode estar **correta** e ainda assim ser **inócua** no desenho atual. Removê-la e não ver diferença **não prova que é inútil** — prova que o seu teste não exercita o caminho que ela protege. É exatamente assim que proteções de segurança são removidas "porque não faziam nada".

A.4.3 E3 — Sem o embedding posicional

Configuração	Validação
com posicional	2,158917
sem posicional	2,201959
diferença	+0,0442

1 e 2. A loss piora. A atenção é **invariante a permutações**: ela calcula afinidades entre pares de posições, mas nada no mecanismo diz **qual veio antes**. Sem o embedding posicional, ana e naa viram o mesmo conjunto de tokens, e o modelo perde a capacidade de usar a **ordem** — que em linguagem é quase tudo.

A penalidade de 0,044 pode parecer modesta, mas note: para *nomes*, boa parte do sinal está em *quais* letras aparecem (estatística de caracteres), não só na ordem. Em prosa, o mesmo experimento seria muito mais destrutivo.

A.4.4 E4 — Sem a escala $1/\sqrt{\text{head_size}}$

Configuração	Treino	Validação	Peso máximo médio de atenção
com escala	2,1589	2,1517	0,406
sem escala	2,3508	2,3428	0,985

1 e 2. Sem a escala a loss piora bastante (+0,19), e a causa aparece na terceira coluna: o peso máximo por linha sobe de 0,41 para **0,985**. A distribuição de atenção virou praticamente um **one-hot** — o modelo escolhe uma posição e ignora todas as outras.

3. Softmax saturado tem gradiente quase nulo. É exatamente o mesmo efeito da tanh saturada do Capítulo 2: o modelo trava numa escolha e perde a capacidade de ajustá-la. A escala não é um detalhe de normalização — é o que mantém o mecanismo **treinável**.

A.4.5 E5 — Tamanho do contexto

block_size	Parâmetros	Treino	Validação
3	11.103	2,2194	2,2110
8	11.363	2,1589	2,1517
16	11.779	2,2122	2,1988

1. A resposta não é "mais contexto é sempre melhor". De 3 para 8 melhora; de 8 para 16 **piora**. Não é retorno decrescente — é retorno **negativo**.

Por quê? Nomes têm ~7 letras. Com block_size = 16, a imensa maioria das posições do contexto é **preenchimento** (.), e o modelo gasta capacidade processando padding em vez de sinal.

2. Somando a isso, os embeddings posicionais das posições distantes quase nunca são exercitados: recebem pouco gradiente, permanecem próximos da inicialização aleatória e injetam ruído.

A lição: o contexto deve ser dimensionado pelo **tamanho real do dado**. Contexto grande demais não é neutro — custa computação e qualidade. (No Capítulo 11, com prosa, block_size = 128 faz sentido justamente porque o texto de verdade é longo.)

3. Dois custos que crescem de formas diferentes:

- **computação:** cresce com T^2 (a matriz de afinidades é $T \times T$)
- **parâmetros:** cresce só com o embedding posicional — de 11.103 para 11.779, quase nada

É o **oposto** do MLP do Capítulo 3, onde os parâmetros cresciam linearmente com o contexto. A atenção troca custo de parâmetros por custo de computação.

A.4.6 E6 — Inspeccionando os pesos

Solução em `e6_inspecionar_atencao.py`.

O achado principal: boa parte do peso cai nos tokens de **preenchimento** (.), não nas letras. Isso **não** é bug — o softmax obriga os pesos a somarem 1, então quando o modelo não precisa trazer informação de nenhuma posição específica, ele "estaciona" o peso numa posição inofensiva. O fenômeno tem nome na literatura de LLMs: **attention sink**.

A.4.7 E7 — Multi-head attention

Cabeças	head_size	Parâmetros	Treino	Validação
1	52	11.363	2,1589	2,1517
2	26	14.119	2,0752	2,0643
4	13	14.119	2,0463	2,0367

1. Várias cabeças pequenas ganham. De 1 para 4 cabeças a validação melhora 0,115 — um ganho grande para este capítulo.

Note que 2 e 4 cabeças têm **exatamente os mesmos 14.119 parâmetros**: `head_size = n_embd // n_head`, então o total de pesos das cabeças é constante. A melhoria de 2 para 4 vem **só da organização**, não de capacidade extra.

A versão de 1 cabeça tem menos parâmetros (11.363) porque não tem a projeção de saída. A comparação limpa é entre 2 e 4 cabeças.

2. Por que várias ajudam? Uma cabeça aprende **um** critério de "onde olhar". Várias aprendem vários — uma pode seguir a vogal anterior, outra o começo da palavra, outra a última consoante. Como as saídas são concatenadas e projetadas, o modelo combina esses critérios.

3. É exatamente a peça que abre o Capítulo 5.

A.5 GABARITO DO CAPÍTULO 5 — TRANSFORMER

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Orçamento: 2.500 passos por configuração (a apostila usa 15.000).

A.5.1 E1 — Leitura de código

1. Por que `x = x + self.sa(self.ln1(x))` e não `x = self.sa(self.ln1(x))`?

A soma é a **conexão residual**. Ela muda duas coisas:

- **na ida:** a sub-camada não precisa produzir a representação inteira — só uma **correção** ao que já estava lá. Aprender um ajuste é mais fácil que reconstruir tudo.
- **na volta:** a derivada de uma soma **distribui** o gradiente igualmente. O `x +` cria um caminho pelo qual o gradiente flui até as primeiras camadas **sem ser multiplicado por nada**.

2. Por que `head_size = n_embd // n_head`? Para que a concatenação das cabeças volte a ter `n_embd`. Com `n_embd = 64` e `n_head = 4`, são 4 cabeças de 16 que concatenam de volta em 64 — **quatro cabeças pequenas custam o mesmo que uma grande**. Se cada cabeça tivesse `head_size = n_embd`, o custo seria `n_head` vezes maior.

3. Atenção vs feedforward: A **atenção comunica** — move informação entre posições. O **feedforward computa** — processa cada posição isoladamente, sem trocar nada. Um bloco faz as duas coisas: as posições conversam, depois cada uma pensa sozinha sobre o que ouviu.

A.5.2 E2 — Sem as conexões residuais

Solução em `e2_ablacoes.py`:

Configuração	Loss validação
3 blocos, residual ON	1,9029
3 blocos, residual OFF	2,1971 (+0,294)
6 blocos, residual ON	1,8865
6 blocos, residual OFF	2,7728 (+0,886)

A penalidade **triplica** ao dobrar a profundidade. E o mais revelador: **sem** residuais, ir de 3 para 6 blocos **piora** o modelo (2,197 -> 2,773); **com** residuais, melhora (1,903 -> 1,887).

Sem conexões residuais, profundidade atrapalha. Com elas, ajuda.

A.5.3 E3 — Sem LayerNorm

Blocos	lr	LayerNorm	Validação
3	1e-3	ON	1,9365
3	1e-3	OFF	1,9318
3	3e-3	ON	1,9504
3	3e-3	OFF	2,0104
8	1e-3	ON	1,9228
8	1e-3	OFF	1,9181
8	3e-3	ON	1,9709
8	3e-3	OFF	DIVERGIU

Penalidade de remover a LayerNorm:

Profundidade	lr = 1e-3	lr = 3e-3
3 blocos	-0,0047	+0,0600
8 blocos	-0,0047	divergiu

1 e 2. A resposta não é "LayerNorm sempre ajuda". Leia a tabela de penalidades:

- Com $lr = 1e-3$, remover a LayerNorm dá **-0,0047 nas duas profundidades** — ela é dispensável, e até marginalmente prejudicial, tanto com 3 quanto com 8 blocos.
- Com $lr = 3e-3$, remover custa +0,06 com 3 blocos e **destrói o treino** com 8.

Não é a profundidade que cria a necessidade — é a learning rate. A profundidade **amplifica a falha**: o mesmo lr que apenas degrada um modelo de 3 blocos aniquila um de 8.

3. Faz sentido pela teoria da Seção 4 da apostila: a LayerNorm impede que o erro de escala se acumule. Com passos pequenos as ativações não saem de escala, e não há o que corrigir. Com passos grandes elas saem — e aí o número de camadas decide se o estrago é recuperável.

A lição, que se repete no curso: uma peça pode ser **correta e necessária em geral**, mas dispensável na configuração específica que você está testando. Isso não a torna inútil — torna o seu teste insuficiente para julgá-la.

A.5.4 E4 — Profundidade e número de cabeças

n_layer	n_head	Parâmetros	Validação
1	4	53.915	1,9818
3	4	153.499	1,9365
6	4	302.875	(ver saída)

n_layer	n_head	Parâmetros	Validação
3	1	153.499	1,9618
3	4	153.499	1,9365
3	8	153.499	1,9227

1 e 2. Mais profundidade ajuda, com retorno decrescente.

3. Mudar n_head não muda o número de parâmetros — repare que as três linhas da segunda tabela têm exatamente **153.499**. Como $\text{head_size} = \text{n_embd} // \text{n_head}$, o total de pesos das cabeças é constante. O que muda é **como esses pesos são organizados**: quantos softmaxes independentes existem.

A.5.5 E5 — Uma cabeça grande vs várias pequenas

n_head	head_size	Parâmetros	Validação
1	64	153.499	1,9618
8	8	153.499	1,9227
64	1	153.499	1,9137

1 e 2. Várias cabeças pequenas ganham, e com **exatamente os mesmos parâmetros**. A melhora vem só da organização: cada cabeça se especializa numa relação diferente, e a projeção combina o que todas trouxeram.

3. Esta resposta não saiu como eu previa.

Eu esperava um limite: com `n_head = 64` cada cabeça tem dimensão 1, e um produto escalar de vetores de dimensão 1 é só a multiplicação de dois números — pareceria pobre demais para expressar afinidade.

A medição diz o contrário: 64 cabeças foi o melhor dos três.

Por que o limite não aparece aqui? Porque a tarefa é simples (nomes, contexto 8). Sessenta e quatro padrões de atenção escalares, combinados pela projeção, dão conta. O custo de cada cabeça ser pobre é compensado pela quantidade.

Na prática os modelos usam `head_size` entre 32 e 128 — mas por razões que este experimento **não exercita**: tarefas complexas onde cada cabeça precisa comparar representações ricas, e eficiência de hardware (cabeças muito pequenas geram `matmuls` pequenas, que a GPU odeia — ver Capítulo 8).

Vale saber a regra prática E saber por que ela existe. Uma regra da literatura pode não se reproduzir na sua escala.

A.5.6 E6 — Dropout

Dropout	Treino	Validação	Gap
0,0	1,9438	1,9365	-0,0073
0,1	1,9517	1,9448	-0,0069
0,3	1,9774	1,9681	-0,0093

1. O dropout atrapalha aqui, e de forma monotônica. Com 64 mil nomes, treino e validação já andam juntos — o gap é praticamente nulo. Sem overfitting para combater, o dropout só remove capacidade.

É exatamente a mesma lição do weight decay no Capítulo 7, e o padrão é o mesmo: **treino e validação pioram juntos, e o gap não muda**. Regularização que não está resolvendo nada.

2. Ele ajudaria com **poucos dados** — releia o E5 do Capítulo 3 (155 nomes: treino 0,80 contra validação 6,51).

3. Por que desligar na avaliação? Porque com dropout ativo você estaria medindo um modelo **aleatoriamente mutilado** — a loss viria pior e ruidosa, sem significar nada. O `model.eval()` desliga o dropout (e ajusta a BatchNorm, quando existe).

A.5.7 E7 — Contando os parâmetros

Com `n_embd = 64`, `n_head = 4`, `n_layer = 3`, `block_size = 8`, `vocab = 27`:

Componente	Total
token_emb (27×64)	1.728
pos_emb (8×64)	512
por bloco (qkv + proj + ff + 2 LayerNorm)	49.792
× 3 blocos	149.376
LayerNorm final	128
lm_head ($64 \times 27 + 27$)	1.755
Total	153.499

1 e 2. A fórmula reproduz exatamente o número que o código informa.

3. Quem domina: os blocos somam 149.376 de 153.499 — **97%**. E dentro de cada bloco, o **feedforward** é o maior pedaço, por causa da expansão $4\times$.

Consequências para escalar:

- Dobrar n_embd **quaduplica** o custo dos blocos (todas as matrizes são $n_e \times n_e$ ou $n_e \times 4n_e$).
- Dobrar n_layer apenas **dobra**.

É por isso que modelos grandes crescem mais em profundidade do que em largura — e por que n_embd é a decisão de arquitetura mais cara.

A.6 GABARITO DO CAPÍTULO 6 — TOKENIZATION (BPE DO ZERO)

Respostas discursivas. Os **números** vêm de `gabarito.py` (roda em ~2 minutos, sem treinar rede nenhuma).

A.6.1 E1 — Unicode e UTF-8

1. Quantos bytes ocupa "pão"?

```
'pao' : 3 caracteres, 3 bytes -> [112, 97, 111]
'pão' : 3 caracteres, 4 bytes -> [112, 195, 163, 111]
```

Mesmo número de **caracteres**, um byte a mais. O ã ocupa **2 bytes** em UTF-8 (195, 163), enquanto a ocupa 1.

2. `ord("ç")` vale 231 — isso significa que cabe em 1 byte?

Não, e este é o erro clássico do capítulo. 231 é o **code point** (a identidade Unicode do caractere), não a codificação. Em UTF-8, todo code point acima de 127 usa 2 bytes ou mais — ç vira [195, 167].

Confundir code point com byte leva a bugs sutis: você conta 3 caracteres e aloca 3 bytes.

3. Por que começar pelos bytes elimina o "desconhecido"?

Existem exatamente **256** valores de byte, e todos estão no vocabulário base. Qualquer texto — de qualquer idioma, mais emoji, mais código-fonte — é uma sequência de bytes, logo é sempre representável. Não há como aparecer algo "fora do vocabulário".

A.6.2 E2 — Tamanho do vocabulário

Vocabulário	Fusões	Tokens (treino)	Compressão	Ganho
300	44	95.043	1,578x	—
512	256	66.936	2,241x	+0,663
1024	768	52.965	2,832x	+0,591

1 e 2. A compressão melhora, com **retorno decrescente**: as primeiras 256 fusões valem +0,663 de compressão; as 512 seguintes valem só +0,591. As fusões mais frequentes são aprendidas primeiro, e as seguintes cobrem casos cada vez mais raros.

3. O custo de um vocabulário maior não está no tokenizador — está no modelo.

Duas matrizes crescem com `vocab_size`:

- a tabela de embeddings: `vocab_size × n_embd`
- a camada de saída (`lm_head`): `n_embd × vocab_size`

Dobrar o vocabulário dobra as duas. É um trade-off direto: **sequências mais curtas (bom) contra modelo maior (custo)**. Modelos reais usam entre 32 mil e 200 mil tokens, e essa escolha é uma decisão de arquitetura, não um detalhe.

A.6.3 E3 — As primeiras fusões

```
256: 'an'      261: 'il'
257: 'a\n'    262: 'on\n'
258: 'e\n'    263: 'ar'
259: 'el'     264: 'o\n'
260: 'on'     265: 'er'
```

1. Por que ('a', 'n') é a primeira? Porque `an` é a sequência de duas letras mais frequente em nomes brasileiros: **ana**, **alessandra**, **fernanda**, **luana**, **antonio**...

2. Por que tantas fusões terminam em `\n`? Quatro das dez primeiras contêm quebra de linha. O motivo é o **formato do arquivo**: um nome por linha. A quebra de linha faz parte do padrão de **terminação** dos nomes, e o tokenizador está literalmente aprendendo "como um nome acaba".

Isso revela algo importante: o tokenizador aprende o **formato do arquivo**, não só a língua. Se você mudasse para nomes separados por vírgula, ele aprenderia `,` no lugar. Formatação é dado.

3. Os tokens mais longos:

```
'ilson\n', 'erson\n', 'isson\n', 'ilene\n', 'ilton\n', 'ilane\n',
'laine\n', 'iane\n', 'ison\n', 'iele\n', 'iana\n', 'aldo\n', ...
```

São **suffixos produtivos de nomes brasileiros**. O algoritmo não tem nenhuma noção de morfologia — ele só conta pares. Estatística pura virando estrutura linguística.

A.6.4 E4 — A ordem das fusões importa

Aplicando as fusões numa ordem arbitrária em vez da ordem de aprendizado:

Texto	Ordem correta	Ordem errada	Round-trip
maria eduarda	7	8	[OK]

joao vinicius	9	9	[OK]
ana beatriz de souza	15	15	[OK]
20.000 caracteres	9.137	10.041	+9,89%

1. O round-trip continua valendo. O decode desfaz qualquer sequência de fusões válidas, então o texto volta igual. A ordem errada **não quebra** o tokenizador — só o deixa pior.

2. A compressão piora ~10% em texto de tamanho razoável. Significativo, ainda que não catastrófico.

Repare no contraste entre as duas partes da tabela — e é a lição metodológica do exercício. Nos três textos curtos, dois deram **exatamente o mesmo** número de tokens. Quem medisse só neles concluiria que "a ordem quase não importa". Só com volume o efeito de ~10% aparece de forma confiável. **Amostra pequena esconde efeito real.**

3. Por que a regra é obrigatória mesmo assim? Dois motivos:

- Uma fusão tardia pode usar um token criado por uma anterior; aplicá-la antes desmonta a cadeia.
- Sem ordem fixa, o mesmo texto poderia produzir tokens diferentes em execuções diferentes. Um tokenizador precisa ser **determinístico** — senão o modelo recebe entradas inconsistentes.

A.6.5 E5 — O imposto do português

Solução em `e5_bpe_portugues.py`. Resumo: um BPE do **mesmo tamanho** treinado em português com acentos aprende 'ação ', 'ção ', 'não ', 'são ' como tokens únicos, e gasta **53% menos tokens** (64 contra 135 em três frases).

A.6.6 E6 — Tokens fora do domínio

Texto	Bytes	Tokens	Compressão	Round-trip
nomes (mesmo domínio)	2.000	900	2,22x	[OK]
frase em português	60	50	1,20x	[OK]
japonês + emoji	17	17	1,00x	[OK]
código Python	33	29	1,14x	[OK]

1. Fora do domínio a compressão desaba para perto de 1,00x: nenhuma das fusões aprendidas se aplica, então cada byte vira um token.

2. **Mas o round-trip continua True em todos os casos** — e é essa a garantia que importa. O tokenizador **não comprime** o que não conhece, mas **nunca falha**. Japonês, emoji e código-fonte são todos representáveis, porque o vocabulário base são os 256 bytes.

3. **Um tokenizador de palavras** receberia uma palavra japonesa e não teria entrada nenhuma para ela — precisaria emitir <UNK>, **perdendo a informação de forma irreversível**. É a diferença entre "comprime mal" e "não representa".

A.6.7 E7 — Integrando com o modelo

Este exercício é mais de raciocínio que de medição.

1. **O que muda em vocab_size e nos parâmetros?** De 27 para 512 (ou 1024). As duas matrizes ligadas ao vocabulário crescem na mesma proporção: embeddings ($\text{vocab_size} \times \text{n_emdb}$) e lm_head ($\text{n_emdb} \times \text{vocab_size}$). Com o modelo do Capítulo 5 ($\text{n_emdb} = 64$), isso vai de ~3,5 mil para ~65 mil parâmetros só nessas duas.

2. **Quanto texto cabe no mesmo block_size?** Com compressão de 2,24x, uma janela de 8 tokens passa a cobrir ~18 caracteres em vez de 8. **O contexto efetivo mais que dobra sem custo nenhum** — é o principal ganho prático da tokenização.

3. **A loss é comparável? Não, e entender por quê é o ponto do exercício.**

A cross-entropy mede a incerteza **por token**. Prever entre 27 opções e prever entre 512 são tarefas de dificuldade diferente: o piso de um chute uniforme passa de $\ln(27) = 3,30$ para $\ln(512) = 6,24$. Uma loss de 2,0 significa coisas muito diferentes nos dois casos.

Para comparar de forma justa, converte-se para **bits por caractere** (BPC):

$$\text{BPC} = \text{loss_por_token} / \ln(2) / \text{caracteres_por_token}$$

Essa métrica é independente do tokenizador — e é por isso que é usada na literatura para comparar modelos com vocabulários diferentes. É exatamente a armadilha que o Capítulo 11 aponta ao aposentar o benchmark de 1,776.

A.7 GABARITO DO CAPÍTULO 7 — OPTIMIZATION

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Orçamento: 2.500 passos por configuração (a apostila usa 15.000). Onde essa redução muda a conclusão, está dito.

Aviso geral deste gabarito: quatro das oito respostas contrariam a expectativa — inclusive a minha, quando escrevi os exercícios. Todas foram corrigidas pela medição.

A.7.1 E1 — Leitura de código

1. O que a correção de viés faz? m e v começam em **zero**. No primeiro passo, $m = (1 - \beta_1) \cdot g = 0,1 \cdot g$ — dez vezes menor que o gradiente real. As médias móveis **subestimam** no início. Dividir por $(1 - \beta_1 t)$ compensa exatamente isso: no passo 1 o divisor é $0,1$, multiplicando m por 10.

Ela importa mais no início porque é lá que o viés é grande. Conforme t cresce, $\beta_1 t \rightarrow 0$, o divisor tende a 1 e a correção **desaparece sozinha**.

2. Weight decay somado ao gradiente vs aplicado ao parâmetro: No Adam original, o termo de decay entrava no gradiente e depois passava pela **divisão por $\sqrt{v}$** . Isso distorce a regularização: parâmetros com gradiente grande recebiam *menos* decay, sem nenhuma razão. O AdamW aplica o decay direto no parâmetro, fora do caminho adaptativo — daí o "W" de *decoupled*.

3. Por que $\sqrt{\text{fan_in}}$ e não fan_in ? Um neurônio soma fan_in produtos independentes. A **variância** de uma soma de n termos independentes cresce com n , logo o **desvio padrão** cresce com $\sqrt{n}$. Como queremos controlar o desvio padrão, dividimos por $\sqrt{\text{fan_in}}$.

A.7.2 E2 — O warmup é necessário?

Warmup	Validação	Norma média (100 primeiros passos)
0	1,9198	1,223
50	1,9211	1,351
200	1,9219	1,498
500	1,9226	1,645

1. Neste modelo o warmup não ajuda — e piora de forma monotônica. Gastar 500 dos 2.500 passos com learning rate reduzida é desperdiçar 20% do orçamento num modelo que não tem problema de estabilidade.

2. Cuidado com a métrica — e eu escolhi a errada.

A norma dos gradientes nos primeiros passos **aumenta** com o warmup (1,223 -> 1,645). Seria tentador ler isso como "o warmup deixa o treino mais instável". Não é.

Com warmup a learning rate começa minúscula, então o modelo demora mais para sair da região de inicialização — onde a loss é alta e os gradientes são **naturalmente maiores**. A norma alta mede **lentidão**, não instabilidade. A métrica não testava o que eu queria testar.

3. Então por que o warmup existe? Porque nos primeiros passos m e v do Adam quase não têm informação. Em modelos **grandes**, com batches grandes, um passo ruim no início pode levar a loss a um regime do qual ela não se recupera.

O warmup é um seguro contra um risco que cresce com a escala. Nesta escala, ele só custa. Isso não o torna errado — torna este modelo insuficiente para julgá-lo.

A.7.3 E3 — Calibrando o gradient clipping

Norma do gradiente sem clipping: **média 1,324, máxima 2,427**.

Clip	Validação	% de passos cortados
0,1	1,9206	100,0%
1,0	1,9205	99,6%
3,0	1,9219	0,0%
100,0	1,9219	0,0%

1. Com `clip = 3,0` (acima da norma típica de 1,32) **nada** é cortado — o comportamento correto de uma rede de segurança que não é acionada.

2. Com `clip = 0,1`, 100% dos passos são cortados: isso deixa de ser clipping e vira **normalização** do gradiente — o erro documentado na Seção 5 da apostila.

Mas seja honesto com o número: **aqui isso não prejudicou** (1,9206 contra 1,9219 sem clipping — até marginalmente melhor). Normalizar todo gradiente é um algoritmo **diferente**, não necessariamente pior. Ele só não é o que você pensa que está rodando.

O problema de configurar errado nem sempre é perder desempenho. Às vezes é **não saber qual algoritmo você está usando** — e então não conseguir explicar o resultado.

3. Com `clip = 100` nada é cortado e a loss é idêntica à de não ter clipping. Conclusão honesta: **neste modelo o clipping é dispensável**. Não há picos. Ele existe para o caso patológico.

4. O clipping altera só o **tamanho** do gradiente, não a **direção**: todos os componentes são multiplicados pelo mesmo fator. A informação de "para onde ir" é preservada.

A.7.4 E4 — O ganho certo para a GELU

Desvio padrão das ativações na 8ª camada:

Ganho	tanh	ReLU	GELU
1,000	0,2483	0,0493	0,0068
1,414 ($\sqrt{2}$)	0,5515	0,7879	0,5393
1,500	0,5892	1,2635	1,0508
1,667 (5/3)	0,6472	2,9400	2,7308
2,000	0,7253	12,6212	12,2773

1. O ganho que mantém o desvio ~1 para GELU é $\approx 1,49$.

2. Fica **entre** o da ReLU (1,414) e o da tanh (1,667), mais perto da ReLU. Faz sentido: a GELU é uma versão suave da ReLU e deixa passar um pouco menos de sinal, precisando de um ganho ligeiramente maior.

3. O PyTorch não tem entrada para GELU:

```
torch.nn.init.calculate_gain('relu') = 1.4142
torch.nn.init.calculate_gain('gelu') -> ValueError
```

Na prática quase ninguém ajusta esse ganho para GELU — porque modelos modernos usam **LayerNorm**, que re-normaliza a cada bloco e torna o valor exato bem menos crítico (a apostila mede isso na Seção 2).

A.7.5 E5 — A curva da learning rate

Solução em `e5_curva_lr.py`. Curva em U, com mínimo em **3e-3** naquele orçamento — maior que o 1e-3 da apostila, porque **a melhor learning rate depende do orçamento de passos**.

A.7.6 E6 — AdamW vs SGD no modelo de verdade

Otimizador	lr	Validação
AdamW	1e-3	1,9219
SGD	1e-3	2,6488
SGD	1e-2	2,2738
SGD	1e-1	2,0090
SGD	5e-1	1,9294
SGD + momentum	1e-2	2,0105
SGD + momentum	1e-1	1,9213

1. Com a **mesma** learning rate (1e-3), o SGD fica muito atrás: 2,65 contra 1,92. Um passo bom para o AdamW é minúsculo para o SGD.

2. Mas ajustando a learning rate, o SGD+momentum **EMPATA** com o AdamW (1,9213 contra 1,9219). O SGD puro fica logo atrás (1,9294).

Ou seja: neste modelo o AdamW **não é magicamente superior** — ele é superior **com a learning rate padrão**. A grande vantagem prática do Adam não é chegar mais longe, é chegar lá **sem que você precise caçar a learning rate certa**. Isso vale muito quando cada tentativa custa horas de treino.

3. O modelo tem parâmetros de escalas muito diferentes (embeddings, pesos de atenção, ganhos de LayerNorm, vieses). Uma única learning rate não serve para todos — é o experimento do `optimizers.py`. O Adam normaliza o passo por parâmetro e escapa desse aperto.

A.7.7 E7 — Por que o weight decay atrapalhou

Weight decay	Treino	Validação	Gap
0,00	1,9243	1,9207	-0,0036
0,01	1,9256	1,9219	-0,0037
0,10	1,9382	1,9334	-0,0048
0,50	2,0015	1,9938	-0,0077

1 e 2. Olhe as três colunas juntas. Aumentar o weight decay piora **treino e validação juntos**, e o **gap entre eles quase não muda**. Esse padrão é a assinatura de que a regularização **não está resolvendo overfitting** — ela está apenas removendo capacidade.

Se houvesse overfitting, o esperado seria: treino piora, validação **melhora**, gap diminui. Não é o que acontece.

3. Com gap praticamente nulo no baseline (-0,004), **não há overfitting para combater**. Weight decay é remédio para uma doença que este modelo não tem.

4. Ele ajudaria com **poucos dados** — releia o E5 do Capítulo 3: com 155 nomes, treino 0,80 contra validação 6,51. Lá o gap é enorme, e a regularização teria o que fazer.

A.7.8 E8 — Outros agendamentos

Agendamento	Validação
constante	1,9449
cosseno	1,9219
linear	1,9206
degraus	1,9404

Diferença entre o melhor e o pior: **0,0243**.

1 e 2. Decair ganha de não decair — mas os agendamentos não são equivalentes entre si. O linear e o cosseno ficam praticamente empatados e claramente à frente; os **degraus** ficam quase tão ruins quanto o constante.

Faz sentido: com apenas 2.500 passos, o primeiro degrau (em 50%) demora demais a chegar. Agendamentos por degrau foram feitos para treinos longos, em que cada patamar tem tempo de render.

E o cosseno ser o padrão da área é mais **convenção** do que superioridade medida — pelo menos nesta escala, o linear empata ou ganha.

3. O problema de depender de `max_steps`: o cosseno precisa saber o total de passos **de antemão**. Se você quiser continuar um treino já terminado, a curva já chegou ao fim e não há como esticá-la sem estragar o formato — a learning rate ficaria presa no mínimo.

É um incômodo real, e existem alternativas: o **WSD** (*warmup-stable-decay*) mantém a learning rate constante por tempo indeterminado e só decai no fim, permitindo decidir depois quando parar.

A.8 GABARITO DO CAPÍTULO 8 — DEVICE (CPU E GPU)

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Para exercitar a GPU, rode com o Python que tem o `torch-directml`:
`C:\dml312\Scripts\python.exe solucoes/gabarito.py`

(!) **Seus números não vão bater com os daqui, e está tudo bem.** Veja a nota sobre variabilidade no E2.

A.8.1 E1 — Latência vs vazão

1. Por que a GPU perde numa matmul de 128×128? Porque há um **custo fixo** por operação — lançar o kernel, coordenar o dispositivo, sincronizar — que não depende do tamanho do trabalho. Numa matmul minúscula, esse custo fixo é maior que o cálculo em si. A CPU faz a conta e devolve; a GPU passa a maior parte do tempo se organizando.

2. Por que a CPU fica em ~340 GFLOP/s e a GPU vai de 167 a 5.500? A CPU **satura cedo**: ela tem poucos núcleos e já os usa quase todos mesmo em matrizes pequenas. A GPU tem **milhares** de núcleos, e só os ocupa quando há trabalho para todos. Em matrizes pequenas, a maioria fica ociosa.

3. Uma tarefa com muitos if e desvios roda melhor em quê? Em CPU. Núcleos de GPU executam em grupos que compartilham a mesma instrução; quando o código diverge (uns entram no `if`, outros não), o hardware precisa executar os dois caminhos em sequência e descartar o que não vale — perdendo boa parte do paralelismo. A CPU tem previsão de desvio e execução fora de ordem justamente para esse tipo de código.

A.8.2 E2 — Encontre o seu ponto de virada

Uma execução (AMD Radeon RX 7600 via DirectML):

Tamanho	CPU (ms)	GPU (ms)	Speedup	GFLOP/s GPU
128	0,05	0,03	1,52x	136
256	0,18	0,04	4,51x	857
512	1,36	0,10	13,89x	2.745
1024	7,47	0,40	18,55x	5.332
2048	62,94	3,72	16,92x	4.619

1 e 2. A GPU ganha a partir de algumas centenas, e o speedup cresce até estabilizar em torno de 15-18x.

A.8.2.1 (!) Sobre a variabilidade — e um erro que eu cometi

Estes números não batem com os da apostila (que mediu 0,75x em 128 e 15,26x em 1024). E não porque um dos dois esteja errado.

Ao preparar este gabarito eu rodei a mesma medição três vezes e obtive **1,08x, 1,52x e 2,09x** para 128×128. A variação vinha de **outros processos disputando a CPU** — o gabarito de outro capítulo estava treinando ao fundo enquanto eu media.

Medição de tempo é sensível a:

- carga da máquina (o meu erro)
- estado térmico e frequência do processador
- até se você multiplica a @ b ou a @ a — a segunda tem melhor localidade de cache e deixa a CPU mais rápida

O que é estável, e o que você deve verificar:

- a **forma** da curva: speedup baixo nos tamanhos pequenos, crescendo até estabilizar
- a **ordem de grandeza** do ganho máximo (uma a duas dezenas de vezes)
- o fato de que a **CPU satura** e a **GPU escala**

Se quiser números comparáveis, feche tudo e rode com a máquina ociosa. É irônico e apropriado que o capítulo sobre medir corretamente tenha me pegado justamente aqui.

3. O pico de GFLOP/s medido fica bem abaixo da especificação da placa. Isso é normal: a especificação supõe uso perfeito das unidades, precisão reduzida e nenhum gargalo de memória. Os capítulos 9 e 10 atacam exatamente essa diferença.

A.8.3 E3 — A armadilha do `.item()`

Medição	ms/passo
sem ler a loss	1,18
lendo <code>.item()</code> a cada passo	1,57 (+33%)

1 e 2. Um terço a mais de tempo, e a causa são **duas coisas** que o `.item()` faz:

- **transfere** um valor da GPU para a CPU
- para poder fazê-lo, **espera** a GPU terminar tudo o que estava na fila

A segunda é a cara. Ela destrói o paralelismo entre CPU e GPU: normalmente a CPU vai enfileirando o próximo passo enquanto a GPU trabalha no atual; com o `.item()`, ela para e espera.

3. Como registrar sem pagar o preço:

- registre a cada N passos (100, por exemplo) — você precisa da **curva**, não de todos os pontos
- ou acumule os valores num tensor **na GPU** e leia só no fim

A.8.4 E4 — Tamanho do batch

Batch	ms/ passo	ms por exemplo
32	0,149	0,00466
128	0,174	0,00136
512	0,183	0,00036
2048	0,477	0,00023

1 e 2. O tempo por passo cresce **muito menos** que proporcionalmente: batch 64x maior custa só 3x mais tempo. Por isso o tempo **por exemplo** cai 20x. É a mesma lição da Seção 4 da apostila.

3. O limite é a memória da placa — você verá um erro de alocação.

4. Cuidado com a conclusão, e este é o ponto do exercício. Batch maior processa mais exemplos por segundo, mas **cada passo continua sendo um passo de gradiente**. Se você dobrar o batch sem ajustar a learning rate, o treino faz metade do progresso por exemplo visto. Releia o E5 do Capítulo 7 e a Seção 6 do Capítulo 10.

A.8.5 E5 — Meça errado de propósito

Medição	Tempo	"Speedup"
GPU sem drenar	0,054 ms	230,6x
GPU com drenar	0,460 ms	26,9x

A medição errada exagera em ~9x (e numa das execuções, em 18x).

1. Sem drenar, a GPU parece absurdamente rápida — porque estamos medindo o tempo de **enfileirar** a operação, não o de executá-la. O Python devolve o controle imediatamente.

2. **Por que a CPU não é afetada?** Porque nela a execução é **síncrona**: quando a linha termina, a conta terminou. Não existe fila.

3. **Como detectar que a medição está contaminada:**

- speedups que **não crescem de forma monotônica** com o tamanho
- valores fisicamente implausíveis (centenas de vezes)
- tempos que não mudam quando o trabalho aumenta

A.8.6 E6 — Operações que caem de volta na CPU

Solução em `e6_fallback.py`:

Otimizador	GPU (ms/passo)	Speedup	Fallback
AdamW	75,2	2,65x	<code>aten::lerp.Scalar_out</code>
SGD + momentum	32,7	5,94x	nenhum
SGD puro	30,9	6,23x	nenhum

Trocar o otimizador **mais que dobra** o ganho. Aquela única operação custava ~42 ms por passo — mais da metade do tempo total. E não aparece como erro: só como lentidão.

A.8.7 E7 — Quando vale a pena mudar para GPU?

Juntando os números da Seção 7 da apostila:

Modelo	Parâmetros	Speedup
pequeno (o do curso)	153 mil	0,30x — a GPU perde
médio	3,2 M	2,65x
grande	18,9 M	6,82x

1. O ponto de equilíbrio fica entre 153 mil e 3,2 milhões de parâmetros — mais perto do limite inferior, já que 3,2 M já dá 2,65x.

2. O custo não é só de execução. Instalar um ambiente separado, descobrir operações não suportadas (o lerp do E6) e depurar travamentos consomem **horas**. Para um treino de 6 minutos, não compensa; para um de 6 horas, compensa muito. A conta é de tempo total, não de tempo de GPU.

3. Para o modelo pequeno deste curso, a recomendação é CPU — e essa é a resposta contra-intuitiva do capítulo. "Usar GPU" não é automaticamente certo: é uma decisão que depende da escala e que **se mede**.

A.9 GABARITO DO CAPÍTULO 9 — PRECISION (TREINAR COM MENOS BITS)

Respostas discursivas. Os **números** vêm de `gabarito.py`, que roda na CPU em segundos. O **E6** depende de GPU — a resposta está abaixo, com os números que a apostila mediu.

A.9.1 E1 — Contas com bits

1. Qual formato representa 100000? E 1e-9?

- 100000 -> só o **bf16**. O fp16 estoura em 65.504.
- 1e-9 -> só o **bf16**. O fp16 zera abaixo de $\sim 6e-8$ (contando com subnormais).

Os dois casos têm a mesma causa: os 8 bits de expoente do bf16 contra os 5 do fp16.

2. Quantos valores distintos entre 1 e 2?

Com m bits de mantissa, exatamente 2^m valores. Então:

- bf16 (7 bits) -> **128** valores entre 1 e 2
- fp16 (10 bits) -> **1.024** valores
- fp32 (23 bits) -> $\sim 8,4$ milhões

3. Por que dobrar os bits de mantissa não dobra a precisão? Porque a relação é **exponencial**, não linear: cada bit a mais **dobra** o número de valores representáveis. Ir de 7 para 14 bits multiplica a resolução por $2^7 = 128$, não por 2.

A.9.2 E2 — Achando os limites na mão

fp16: estoura em $2^{15} = 3,277e+04$	(<code>finfo.max</code> = 6,550e+04)
zera em $2^{-25} = 2,980e-08$	(<code>finfo.tiny</code> = 6,104e-05)
bf16: estoura em $2^{127} = 1,701e+38$	(<code>finfo.max</code> = 3,390e+38)
zera em $2^{-134} = 4,592e-41$	(<code>finfo.tiny</code> = 1,175e-38)

1. O expoente máximo bate com o `finfo.max` — a potência de 2 exata fica um pouco abaixo do máximo, que usa a mantissa cheia ($1,111\dots 2 \times 2^{15}$).

2. Por que dá para ir muito abaixo do .tiny antes de zerar? Por causa dos números **subnormais**. Abaixo do menor valor normal, o formato abre mão de precisão para continuar representando valores cada vez menores — usa a mantissa como se o expoente estivesse travado no mínimo. É uma "rampa de saída" gradual em vez de um corte seco. Repare na distância: o fp16 tem `.tiny` = 6,1e-05 mas só zera em 3,0e-08, três ordens de grandeza abaixo.

3. O bf16 estoura e zera em expoentes muito maiores porque tem 8 bits de expoente — o mesmo alcance do fp32.

A.9.3 E3 — O epsilon na prática

x	1+x em fp32	1+x em fp16	1+x em bf16
1e-1	1,100000	1,099609	1,101562
1e-2	1,010000	1,009766	1,007812
1e-3	1,001000	1,000977	1,000000
1e-4	1,000100	1,000000	1,000000

(Os valores em negrito são somas que **não mudaram nada** — o x foi arredondado para fora.)

1. Em fp16 a soma deixa de ter efeito por volta de $x = 1e-4$ (epsilon = $9,77e-04$).
2. Em bf16 o limite é **maior** (epsilon = $7,81e-03$): ele já falha em $1e-3$. Tem 3 bits menos de mantissa, então distingue menos casas decimais.

3. **É exatamente por isso que os pesos mestres ficam em fp32.** Uma atualização típica de treino tem ordem de $1e-4$ a $1e-6$. Somada a um peso de ordem 1 em 16 bits, ela é arredondada para fora e o peso **não muda** — como a apostila mede (o peso continua em 1,000000 depois de 100 atualizações).

A.9.4 E4 — A janela do loss scaling

Pesos	Escala	% zerados	infs	Situação
0,02	1	50,3%	0	perde gradiente
0,02	1.024	0,1%	0	ok
0,02	2 ²⁰	0,0%	2	overflow
0,50	1	0,1%	0	ok
0,50	1.024	0,0%	0	ok
0,50	2 ²⁰	0,0%	1	overflow

1 e 2. Leia a tabela com cuidado — o efeito não é o que parece.

Com pesos maiores (0,50), o underflow praticamente **desaparece**: com escala 1, os zerados caem de 50,3% para 0,1%. O limite **inferior** da janela desce muito.

Mas o limite **superior** quase não se mexe: os dois casos começam a estourar por volta de 2²⁰. Então a janela **não se desloca — ela alarga**.

E faz sentido: o *underflow* depende da magnitude dos gradientes (que muda com os pesos), enquanto o *overflow* depende do teto do formato (65.504), que é **fixo**. Só o piso se move.

3. O scaler dinâmico continua se justificando, mas por um motivo mais preciso: a magnitude dos gradientes **cai** conforme o modelo converge. Uma escala fixa escolhida no início pode ficar pequena demais depois, reintroduzindo underflow justamente na fase de ajuste fino.

A.9.5 E5 — Melhorando o GradScaler

Fator	Escala final	Passos descartados
1,01	70.966	0 / 40
2,00	524.288	4 / 40
16,00	65.536	6 / 40

1. Fator **grande** (16) reage rápido mas passa do ponto: sobe demais, estoura, desce demais — e descarta 50% mais passos. Fator **pequeno** (1,01) é estável (zero descartes) mas demora a se ajustar quando a magnitude dos gradientes muda.

2. **Por que um piso é útil?** Ele evita que uma sequência de overflows derrube a escala a um valor tão baixo que **reintroduza underflow**. Sem piso, o scaler pode se auto-sabotar ao reagir de forma exagerada a um pico isolado.

3. **Por que `growth_interval = 2000`?** Porque errar para cima custa **um passo descartado**, mas errar com **frequência** faz o treino descartar passos o tempo todo. É melhor subir raramente e ficar estável do que perseguir o ótimo e viver descartando.

A.9.6 E6 — Seu hardware acelera 16 bits? (*precisa de GPU*)

Medido na máquina desta apostila (AMD Radeon RX 7600 via DirectML), matmul 1024×1024:

Dispositivo	fp32	fp16	Ganho
CPU	5,35 ms	1.951 ms	~0,003x
Radeon	0,40 ms	0,41 ms	0,98x

1. Nesta máquina, **nenhum ganho**.

2. **Isso significa que precisão reduzida é inútil? Não** — e o exercício existe para essa distinção. São **dois benefícios independentes**:

- **memória**: garantido pela aritmética (metade dos bytes), vale em qualquer hardware
- **velocidade**: depende de o hardware e o backend terem unidades de 16 bits

Aqui o segundo não aparece. Numa NVIDIA com tensor cores, aparece — tipicamente 2x a 8x.

3. Se você tem NVIDIA, compare o ganho medido com a especificação da placa. Ele raramente chega ao pico teórico, porque nem toda operação do modelo é matmul.

A.9.7 E7 — Treino em precisão mista

Na versão CPU do `gabarito.py`, um problema de brinquedo treinado 400 passos:

Modo	Loss final
fp32	0,0463
bf16	0,0463
fp16 (sem scaler)	0,0463

1 e 3. O autocast com **bf16** chega à mesma loss do fp32 — não estraga o treino. Esperado: o bf16 tem o alcance do fp32.

2. Aqui o fp16 sem scaler também deu idêntico — e isso exige honestidade sobre o que o teste não mostra.

Não significa que loss scaling seja dispensável. Significa que **este problema não provoca underflow**. Compare com o `loss_scaling.py` da apostila: lá os pesos são deliberadamente pequenos (`p.mul_(0.02)`), os gradientes ficam na casa de $1e-8$, e **50,3% deles zeram**. Aqui os pesos são os da inicialização padrão, os gradientes são saudáveis, e nada zera.

A lição metodológica: um teste que passa não prova que a proteção é desnecessária — prova que o teste não exercita o caso que ela protege. É a mesma armadilha do E2 do Capítulo 4, com a máscara causal.

Modelos profundos de verdade têm gradientes muito menores nas camadas iniciais, e é lá que o fp16 sem scaler quebra.

4. Por que a economia de memória não chega aos 50%? Porque a cópia **mestra** dos pesos e o estado do otimizador (`m` e `v` do AdamW) continuam em fp32. Só as ativações e as `matmuls` usam 16 bits.

A.10 GABARITO DO CAPÍTULO 10 — DISTRIBUTED (TREINAR EM VÁRIOS DISPOSITIVOS)

Respostas discursivas. Os **números** vêm de `gabarito.py`, medidos numa máquina de 12 núcleos.

`python solucoes/gabarito.py`

O E3 roda em **subprocesso com timeout**, porque o objetivo literal daquele exercício é fazer o programa travar — e um gabarito que trava para sempre não serve para ninguém.

A.10.1 E1 — Entendendo o coletivo

1. Quatro processos, o rank *i* tem o valor *i*.

Operação	Resultado em cada rank
<code>all_reduce</code> com SUM	$0+1+2+3 = 6$
<code>all_reduce</code> com MAX	3

O ponto do "all": **todos** terminam com o mesmo valor. Não é o rank 0 que fica com a soma — é todo mundo.

2. As quatro operações.

Operação	Quem fica com o resultado
<code>reduce</code>	um rank (o destino)
<code>all_reduce</code>	todos
<code>gather</code>	um rank recebe a lista completa
<code>all_gather</code>	todos recebem a lista completa

`reduce/all_reduce` **combinam** os valores (soma, máximo). `gather/all_gather` **empilham** sem combinar: cada rank fica com os *N* tensores separados.

3. Por que o broadcast não serve para sincronizar gradientes.

Porque o broadcast copia o valor de **um** rank para os demais — e nenhum rank tem o valor certo para transmitir. Cada um calculou o gradiente do **seu** pedaço do batch; o gradiente correto é a **média dos quatro**, que não existe em lugar nenhum antes de alguém

somá-los.

Usar broadcast do rank 0 equivaleria a jogar fora os dados dos outros três processos — todo o trabalho deles seria descartado. É exatamente o que a soma do `all_reduce` evita.

A.10.2 E2 — Escalando o número de processos

Tempo de um `all_reduce`, melhor de 3 rodadas, **duas passadas independentes** (12 núcleos):

Processos	10 mil	1 milhão	10 milhões
2	0,54 / 0,56 ms	21,0 / 15,3 ms	264,9 / 280,2 ms
4	2,35 / 1,83 ms	9,8 / 8,7 ms	93,6 / 105,8 ms
8	5,81 / 5,70 ms	13,3 / 12,7 ms	174,2 / 177,7 ms

Reprodutibilidade entre passadas: **1,02x a 1,38x**. O efeito discutido abaixo é de **2,7x** — bem acima do ruído.

1. Não há uma resposta só — ela depende do tamanho do tensor, e é essa a descoberta.

- **10 mil elementos:** o tempo **cresce ~10x** de 2 para 8 processos.
- **10 milhões:** o tempo **cai quase 3x** de 2 para 4 processos.

Sim, *cai*. Com mais processos, o all-reduce do tensor grande fica **mais rápido**.

Eu tinha escrito que o tempo sempre cresce com N, apoiado na propriedade do anel (cada processo troca ~2× o tamanho do tensor, independentemente de N). Essa propriedade é verdadeira — e não é a que domina.

A explicação é a mesma dicotomia do Capítulo 8, **latência vs vazão**:

Tamanho do tensor	O que domina	Efeito de mais processos
Tensor pequeno	latência dos N-1 saltos do anel	mais saltos -> pior
Tensor grande	vazão: transferir e somar 40 MB	trabalho dividido -> melhor

No caso grande, o anel divide o trabalho em pedaços de tamanho/N. Com N=2 cada processo lida com 20 MB por etapa; com N=4, com 10 MB. Pedaços menores cabem melhor na cache, e a **soma** — que é trabalho real, não só transferência — se espalha por mais

núcleos.

2. No esquema ingênuo (todos $\rightarrow$ rank 0 $\rightarrow$ todos), o rank 0 receberia $(N-1) \times$ tamanho: o tempo cresceria **linearmente** com N , e ele viraria um gargalo cada vez pior. E o ganho observado em $N=4$ seria **impossível** — o rank 0 é um só, e não paraleliza.

3. Onde os processos passam a competir por CPU. Nesta máquina de 12 núcleos, $N=8$ perde para $N=4$ nas colunas grandes: o tempo piora por **contenção**, não por comunicação. Numa máquina só, "mais processos" tem um teto físico, e ele fica perto do número de núcleos.

(!) **Cuidado ao generalizar.** Tudo isto foi medido em *uma* máquina, com gloo em CPU. Num cluster de verdade, com GPUs e rede dedicada, o equilíbrio entre latência e vazão é outro — mas a **pergunta a fazer** é a mesma.

A.10.3 E3 — Provoque um deadlock

Este exercício me pegou. Vale ler a previsão antes do resultado.

A.10.3.1 A previsão (errada)

A explicação clássica do deadlock em send/recv é o **buffer de socket**: o sistema operacional aceita mensagens pequenas e devolve o controle imediatamente, sem esperar ninguém receber. Sob essa teoria, tensores pequenos passariam por acidente e só os grandes travariam — o que faria deste um bug clássico de "passa no teste, quebra em produção".

Eu escrevi essa resposta antes de medir. O próprio enunciado original do exercício dava essa dica.

A.10.3.2 A medição

Tensor	Alternância par/ímpar	Resultado
1 elemento	sim	terminou
1 elemento	não	TRAVOU
100 elementos	sim	terminou
100 elementos	não	TRAVOU
5 milhões	sim	terminou
5 milhões	não	TRAVOU

Trava sempre. Até com **um único elemento**. O `dist.send` do gloo só retorna quando o `recv` correspondente foi postado do outro lado — não existe tamanho pequeno o bastante para escapar.

A explicação do buffer de socket vale para **sockets crus e para MPI**. Não vale para o gloo, que fez outra escolha de projeto.

A.10.3.3 O que aprender com isso

Sobre o gloo: a escolha dele é boa. Um deadlock que acontece *sempre* é muito melhor que um que só aparece em produção com tensores grandes. O gloo trocou "às vezes funciona" por "nunca funciona", e **falhar de forma determinística é um recurso, não um defeito** — o bug aparece no primeiro teste que você rodar, na sua máquina, com dados de brinquedo.

Sobre você (e sobre mim). Eu tinha um modelo mental correto e o apliquei a uma implementação que não o segue. O modelo estava certo *em geral* e errado *naquele caso* — e nenhuma quantidade de raciocínio teria revelado isso. Só medir revela.

É o mesmo padrão do E2 do Capítulo 4 e do E3 do Capítulo 5.

E note o modo de falha: não há exceção, não há mensagem. O programa simplesmente para — exatamente como aconteceu no desenvolvimento deste capítulo, com o hostname que resolvia para um IP público (Seção 8 da apostila).

A.10.4 E4 — A loss que engana

Solução em `e4_loss_correta.py`.

1. Por que os ranks imprimem losses diferentes se os pesos são idênticos? Porque cada rank calcula a loss no **seu** micro-batch, e os micro-batches são diferentes. Os *pesos* são idênticos (o DDP garante isso sincronizando os gradientes); os *dados* não.

2. Com `all_reduce` da loss dividido por `world_size`, os valores passam a bater entre os ranks — porque agora todos estão reportando a mesma quantidade: a loss média sobre o batch global.

3. O que aconteceria com a curva se você registrasse só o rank 0. Ela ficaria **mais ruidosa** do que o treino realmente é. A loss do rank 0 é uma amostra de batch 64; a loss global é uma amostra de batch 256. Quatro vezes mais dados, metade do ruído.

O perigo prático: você olha a curva serrilhada, conclui que o treino está instável, e sai mexendo na learning rate para consertar um problema que **existe só na medição**.

A.10.5 E5 — Batch efetivo e learning rate

300 passos, batch **local** fixo em 64:

Processos	Batch efetivo	lr	Loss global
1	64	1e-3	2,0095
2	128	1e-3	1,9417
4	256	1e-3	1,8494
2	128	1,4e-3 ($\times\sqrt{2}$)	1,9155
4	256	2,0e-3 ($\times\sqrt{4}$)	1,7846

1. Com o batch local fixo, mais processos = batch efetivo maior. Como o número de **passos** é o mesmo, o modelo vê mais dados — mas dá o mesmo número de **atualizações**. A loss melhora, porque cada gradiente é menos ruidoso.

2. **Sim, ajustar a lr melhora ainda mais.** Escalar por $\sqrt{\text{world_size}}$ recuperou parte da diferença nos dois casos (1,9417 -> 1,9155 e 1,8494 -> 1,7846). Gradiente menos ruidoso tolera — e pede — passo maior.

3. **Por que "mais GPUs" pode deixar o treino mais lento em número de passos.** Porque você processa mais exemplos por passo, mas continua andando com o **mesmo tamanho de passo**. Mais dados por atualização, mesmo número de atualizações: é preciso mais passos para chegar ao mesmo lugar. O ganho está em tempo de parede por passo, não em progresso por passo — e só se materializa se você ajustar a learning rate junto.

É a mesma armadilha do E4 do Capítulo 8 (batch maior é mais eficiente por exemplo, mas não é grátis do ponto de vista da otimização).

A.10.6 E6 — Meça o ZeRO

Estado do otimizador, por processo:

Otimizador	Processos	Normal	ZeRO-1	Redução
AdamW	2	8,54 MB	4,33 MB	2,0x

AdamW	4	8,54 MB	2,23 MB	3,8x
SGD sem momentum	2	0,00 MB	0,00 MB	—
SGD sem momentum	4	0,00 MB	0,00 MB	—

1. Sim, o estado por processo cai para $\sim 1/N$ — como a apostila mede com 4 processos.

2. E aqui está o ponto do exercício: com SGD sem momentum, o ZeRO-1 não economiza nada. Zero contra zero.

O motivo é direto: o SGD puro **não guarda estado nenhum** — ele usa o gradiente e descarta. Se o otimizador não guarda nada, não há o que fatiar. O ZeRO-1 economiza exatamente aquilo que o otimizador guarda.

(Os estágios 2 e 3, que fatiam **gradientes** e **pesos**, continuariam ajudando — eles não dependem do otimizador.)

3. Com **bf16 nos pesos** (Capítulo 9), a conta muda de forma interessante. Pesos e gradientes passam de 4 para 2 bytes por parâmetro, mas o estado do AdamW costuma ficar em fp32 por precisão. Com 8 processos:

Estratégia	Pesos	Gradientes	Estado	Total/GPU
DDP	2 GB	2 GB	8 GB	12 GB
ZeRO-1	2 GB	2 GB	1 GB	5 GB

O peso **relativo** do estado do otimizador **aumenta** quando os pesos encolhem — ou seja, precisão reduzida torna o ZeRO ainda mais valioso. As duas técnicas se reforçam.

A.10.7 E7 — Quando distribuir *não* compensa

Juntando os números medidos:

- tempo de cálculo de um passo (modelo pequeno, CPU, Capítulo 8): **$\sim 12,8$ ms**
- all-reduce de ~ 150 mil parâmetros (0,6 MB): a coluna "10 mil" do E2, escalada — fica na casa de **alguns milissegundos**

1 e 2. Para o modelo pequeno deste curso, comunicação e cálculo ficam na **mesma ordem de grandeza**. Distribuir em 4 processos dividiria o cálculo por 4 mas acrescentaria comunicação a cada passo — ganho líquido pequeno ou negativo.

3. A regra geral:

Distribuir compensa quando **tempo de cálculo por passo** >> **tempo de comunicação por passo**

O cálculo cresce com o **tamanho do modelo** e com o **batch**. A comunicação cresce só com o tamanho do **modelo** — o all-reduce troca gradientes, e há um gradiente por parâmetro, não por exemplo.

Logo: **aumentar o batch melhora a razão de graça**. É por isso que treino distribuído de verdade usa batches enormes — não é só por velocidade, é para que a comunicação valha a pena.

E é a mesma conclusão contra-intuitiva do E7 do Capítulo 8: para o modelo deste curso, a resposta certa é **não distribuir**.

A.11 GABARITO DO CAPÍTULO 11 — DATASETS (A VIRADA PARA TEXTO DE VERDADE)

Respostas discursivas. Os **números** vêm de `gabarito.py`.
`bash python solucoes/gabarito.py`

A.11.1 Sobre o orçamento de treino — leia antes dos números

O modelo da apostila leva **~18 minutos** por treino (3.000 passos). Este gabarito roda vários experimentos, então usa **400 passos**. A arquitetura é **idêntica** à da apostila (`n_embd=192`, 6 cabeças, 4 blocos, 2,2 M parâmetros) — só o número de passos muda.

Essa distinção importa. Reduzir passos é defensável para perguntas **comparativas**; trocar a arquitetura não seria, porque mudaria o objeto em estudo.

Mas "defensável" não é "seguro". A aposta implícita é que a **ordem** entre as configurações não depende do orçamento, mesmo que os valores absolutos dependam. Essa aposta **já tinha perdido duas vezes** neste curso:

Onde	Com orçamento curto	Com orçamento cheio
Cap. 3, E4	melhor <code>lr</code> = 1,0	melhor <code>lr</code> = 0,1
Cap. 7, E5	3e-3 ganha	1e-3 ganha

Nos dois casos a pergunta era sobre **dinâmica de otimização** — e o orçamento é justamente a variável que a dinâmica consome. Já as perguntas *deste* capítulo são **estruturais** (quanto contexto, quantas posições, quantos dados). Eu escrevi, aqui mesmo, que por isso elas *deveriam* ser estáveis.

A.11.1.1 E aí eu medi, e estava errado

`checagem_orcamento.py` repete o E4 com o triplo dos passos e compara os **rankings**:

Orçamento	Ranking (melhor -> pior)
400 passos	256 < 128 < 32
1.200 passos	128 < 256 < 32

O **ranking virou**. Com 400 passos, `block_size=256` parecia o melhor; com 1.200 ele perde para 128 (4,1270 contra 4,1649). O E4 não era estrutural o bastante para escapar.

Refeito com os **3.000 passos** da apostila, virou de novo — e o mesmo aconteceu com o E7:

Exercício	Com 400 passos	Com 3.000 passos
E4 (contexto)	256 < 128 < 32	32 < 128 < 256 (inverteu por completo)
E7 (mais obras)	6 < 8 < 4 < 11	11 < 8 < 6 < 4 (inverteu por completo)

Duas inversões totais, nos dois exercícios que testei. Por isso as respostas do E4 e do E7 vêm de `e4_orcamento_cheio.py` e `e7_orcamento_cheio.py` — cerca de uma hora de treino cada. Vale a hora: a alternativa era publicar respostas que a própria checagem do curso já desmentia.

E os outros exercícios deste capítulo? E1, E3 e E5 não dependem de treino (são conceituais, de tokenização e de contagem de previsões), então o orçamento não os afeta. O **E6** (*model collapse*) foi reverificado em 3.000 passos (`e6_orcamento_cheio.py`) e a conclusão **se manteve, doze vezes mais forte**: a degradação passou de +0,45 para +5,33.

Ou seja: dos três exercícios deste capítulo que dependem de treino, **dois inverteram e um se confirmou**. Não há como saber qual é qual sem medir — e essa é exatamente a razão de medir.

A regra que fica. "Esta pergunta é estrutural, logo o orçamento não importa" é uma hipótese, não um argumento. Rode a sua configuração com o triplo dos passos e veja se a ordem se mantém. Se virar, o orçamento é a variável que decide — e o resultado curto não responde nada.

A.11.2 E1 — Por que essa ordem?

1. Por que o tokenizador é treinado só no texto de treino?

Porque o BPE **aprende com os dados**: ele decide quais pares de bytes fundir contando frequências. Se ele vir o texto de validação, essas contagens incluem material que o modelo deveria não conhecer.

O que vazaria, concretamente: se `Memorial de Aires` entrasse no treino do tokenizador, ele criaria tokens para nomes e expressões daquele livro. Na hora de avaliar, o texto de validação ficaria **artificialmente mais barato de codificar** — menos tokens, cada um mais previsível — e a loss cairia sem que o modelo tivesse ficado melhor.

É um vazamento sutil porque o tokenizador não é "o modelo". Mas ele faz parte do **pipeline**, e tudo que toca o pipeline precisa respeitar a divisão.

2. Por que dividir por obra e não sorteando parágrafos?

Porque parágrafos da mesma obra **compartilham informação**. Um exemplo concreto: se um parágrafo de *Dom Casmurro* que menciona "José Dias" cair no treino e o parágrafo seguinte cair na validação, o modelo já viu o nome, o assento narrativo, o vocabulário da cena e até a construção da frase anterior. Ele acerta a validação por ter visto quase a mesma coisa — não por ter aprendido português.

Dividir por obra garante que a validação seja um texto que o modelo **nunca viu em nenhuma forma**. É a diferença entre "consegue continuar este livro" e "consegue escrever como Machado".

O E2 mede exatamente esse efeito.

3. Por que uint16 e não int64?

Porque o vocabulário tem 1.024 entradas e cabe folgado em 16 bits (que aguentam 65.536). O int64 guardaria os mesmos números em **8 bytes** em vez de 2:

Tipo	Bytes/token	621 mil tokens
int64	8	4,97 MB
uint16	2	1,24 MB

Quatro vezes menos espaço, e o ganho não é só de disco: é de **banda de memória**. Como o treino lê os dados por memmap a cada batch, um arquivo 4x menor significa 4x menos tráfego entre disco, cache e CPU. Em corpora de verdade — centenas de gigabytes — essa escolha decide se o treino é limitado pelo cálculo ou pela leitura dos dados.

(!) O limite é real: com vocabulário acima de 65.536, o uint16 **estoura silenciosamente** (os índices dão a volta). Tokenizadores modernos usam 100 mil a 200 mil tokens e precisam de uint32.

A.11.3 E2 — A divisão errada

Solução em `e2_vazamento.py`.

A.11.4 E3 — O tokenizador é do domínio

1. Os 20 tokens mais longos aprendidos:

'José Dias '	'ima Justin'	'que não '	'minha mã'	'Capitú, '
'os olhos '	'da minha '	'inha mã'	'Capitú '	'José Di'
'a minha '	'seminari'	'ação, '	'almente '	'não me '
'que era '	'Capitú'	'ação, '	'é que '	'primeir'

6 dos 20 contêm nome de personagem (José Dias, Capitú, Justin[a]). Vários outros são marcas do texto específico: 'minha mã[e]', 'seminari[o]', 'os olhos ' — o narrador de *Dom Casmurro* falando da mãe, do seminário e dos olhos de ressaca.

Ninguém programou isso. O BPE só contou pares de bytes.

2. Isso é bom ou ruim? Depende de onde você vai usá-lo.

É **ótimo para este corpus e péssimo fora dele**. Um tokenizador que gasta um único token em 'José Dias ' comprime Machado maravilhosamente — e não tem nenhuma fusão útil para um artigo de medicina, onde cada 'hemoglobina' seria fatiado em pedaços.

3. O BPE treinado em nomes, aplicado a Machado.

Mesmo trecho de prosa (109 bytes), dois BPEs de vocabulário 1.024:

BPE treinado em	Tokens	Chars/token
Machado	43	2,53
nomes (Cap. 6)	81	1,35

1,88x mais tokens para dizer a mesma coisa. O BPE de nomes aprendeu terminações como 'ilson' e 'erson', que quase não aparecem em texto corrido — e não aprendeu nada sobre que, de ou ção.

O custo é direto: 1,88x mais tokens significa 1,88x mais posições de contexto gastas para o mesmo texto, e um treino proporcionalmente mais caro para o mesmo conteúdo.

Nota de método. A primeira versão deste gabarito comparava "81 tokens no trecho" (de um tokenizador) com "2,59 chars/token no corpus inteiro" (do outro). São **grandezas diferentes**, e a comparação parecia razoável sem ser. O número acima mede os dois no mesmo trecho.

A.11.5 E4 — Tamanho do contexto

Com os 3.000 passos da apostila (e4_orcamento_cheio.py):

block_size	Treino	Validação	Gap	minutos
32	3,1947	3,8305	0,64	4,1
128	2,7816	3,9241	1,14	16,7
256	2,7419	4,0136	1,27	38,8

1. Mais contexto PIORA a loss de validação neste corpus. O melhor `block_size` é 32 — o menor dos três.

Essa é a resposta oposta à que eu tinha escrito, e a história de como ela mudou é a parte mais útil do exercício:

Orçamento	Ranking (melhor -> pior)	Vencedor
400 passos	256 < 128 < 32	256
1.200 passos	128 < 256 < 32	128
3.000 passos	32 < 128 < 256	32

A ordem se **inverteu completamente** conforme o orçamento cresceu. Com 400 passos eu concluí "mais contexto ajuda"; no orçamento de verdade, mais contexto atrapalha.

A.11.5.1 Por que — olhe a coluna do gap

O contexto maior tem loss de **treino menor** (2,74 contra 3,19) e loss de **validação maior**. Isso é overfitting, e o gap cresce monotonicamente com o contexto: 0,64 -> 1,14 -> 1,27.

A causa é um detalhe do desenho do experimento que vale internalizar. Com o **número de passos fixo**, `block_size` maior significa **mais épocas sobre o mesmo corpus**:

block_size	Tokens vistos em 3.000 passos	Épocas sobre 621 mil tokens
32	3,1 M	~5
128	12,3 M	~20
256	24,6 M	~40

Quarenta passagens por um corpus de 621 mil tokens, com um modelo de 2,2 M parâmetros, é receita de memorização. O que a tabela mede não é só "quanto contexto ajuda" — é "quanto contexto ajuda **a esta razão entre dados e capacidade**".

Com 400 passos ninguém tinha épocas suficientes para memorizar, e aí o contexto maior só mostrava a sua vantagem. O overfitting só aparece quando você treina o bastante para ele aparecer.

A.11.5.2 E a comparação com o Capítulo 4 fica mais interessante

No Capítulo 4, `block_size=16` era pior que 8 — e eu atribuí isso a nomes serem curtos. Aqui, com prosa, o contexto maior **também** perde. A explicação "nomes são curtos" era verdadeira mas insuficiente: o fator comum aos dois casos é **corpus pequeno demais para a capacidade do modelo**.

Contexto longo não é um bem em si. Ele é uma aposta: você gasta capacidade e épocas para poder olhar mais longe. A aposta paga quando há dados suficientes para sustentá-la. Os modelos de contexto enorme que você vê por aí são treinados em trilhões de tokens — não em 1,6 MB de Machado.

(!) **Uma consequência para a apostila.** O capítulo treina com `block_size=128`, que por esta medida não é o ótimo para a loss de validação. A escolha continua defensável por outro motivo: um modelo com contexto 32 **não consegue** condicionar em mais de 32 tokens na hora de gerar, por melhor que seja a sua loss média. Loss de validação mede previsão do próximo token, não coerência de texto longo. Mas é uma escolha que agora está medida, não presumida.

2. Dobrar o contexto quadruplica o tempo? Não — e a resposta surpreende.

Mudança	Contexto	Tempo (3.000 passos)	Se fosse linear	Se fosse quadrático
32 -> 128	4x	4,07x	4x	16x
128 -> 256	2x	2,32x	2x	4x

O crescimento é **quase linear**, apesar de a atenção ser $O(T^2)$. (A medição com 400 passos deu 4,13x e 2,06x — as duas concordam, e é o esperado: tempo *por passo* não depende de quantos passos você dá.)

O motivo: a atenção é **só uma parte** do custo. As camadas lineares — qkv, projeção, e o feedforward que tem 4x a largura do modelo — crescem **linearmente** com T, e neste tamanho elas dominam. O termo quadrático existe, mas só passa a mandar em contextos bem maiores.

É por isso que "atenção é quadrática" é verdade e é enganoso ao mesmo tempo: verdade sobre a fórmula, enganoso sobre onde o seu tempo está indo. Meça.

3. "Para prosa, o contexto maior ajuda mais do que ajudava para nomes. Por quê?"

A premissa da pergunta está errada, e o enunciado a colocou lá porque eu acreditava nela. Medido, o contexto maior não ajuda aqui — atrapalha.

O raciocínio que sustentava a premissa continua **correto e é insuficiente**: prosa realmente tem dependências longas (concordância, referência a personagens, estrutura de período), e nomes de 7 letras não têm nada disso. Existe, sim, informação útil a 200 tokens de distância em Machado.

O que falta ao raciocínio é a outra metade da conta. Poder usar contexto longo exige:

1. **informação útil lá atrás** — prosa tem, nomes não têm; e
2. **dados suficientes para aprender a usá-la sem memorizar o caminho**.

Este corpus atende ao item 1 e falha no item 2. O modelo tem contexto disponível e não tem exemplos bastantes para aprender a explorá-lo — então usa a capacidade extra para decorar o treino.

É o tipo de erro que este curso repete de propósito: uma explicação **verdadeira** que não é a **dominante**. A mesma coisa aconteceu no E2 do Capítulo 10, onde a propriedade do anel que eu citei era real mas não era o que mandava no tempo.

A.11.6 E5 — Prever em todas as posições

Modo	Treino	Validação	Previsões/batch	ms/ passo
todas as posições	4,4455	4,6957	4.096	365,1
só a última	6,0319	6,0564	32	360,4

1. Com `batch_size=32` e `block_size=128`: **4.096** previsões por batch contra **32**. São **128x mais sinal de treino**.

2. A **diferença é enorme**: 4,70 contra 6,06 de loss de validação, com o mesmo orçamento de passos. A versão que prevê só na última posição recebe 128x menos informação por passo.

3. E o custo é praticamente o mesmo — 365 ms contra 360 ms.

Esse é o ponto do exercício. As duas versões processam as 128 posições de qualquer jeito; a única diferença é **quantas saídas da camada final entram na loss**. A versão "só a última" faz o cálculo inteiro e **joga fora 99% dele**.

128x mais sinal, custo igual. É o tipo de mudança que quase não existe em engenharia — normalmente se paga por ganho.

Os capítulos 3 a 7 usaram a versão ineficiente **de propósito**, porque ela mantinha a métrica comparável entre capítulos. A partir daqui, não há motivo para isso.

A.11.7 E6 — Dados sintéticos e *model collapse*

[OK] **Reverificado no orçamento cheio.** Este era o último exercício do capítulo apoiado em 400 passos, num capítulo onde o E4 e o E7 tiveram a conclusão **invertida** ao passar para 3.000. Eu tinha publicado a conclusão com uma justificativa — não uma medição — de que ela deveria sobreviver. `e6_orcamento_cheio.py` mediu:

Modelo	Treino	Validação real
A (corpus de Machado)	2,7816	3,9241
B (texto gerado pelo A)	0,1149	9,2588

Degradação de +5,33, contra +0,45 com 400 passos. Perplexidade de 50,6 para **10.496**. A conclusão não só se manteve como ficou **doze vezes mais forte** — e a justificativa que eu tinha dado (o argumento informacional não depende do orçamento) estava certa.

Os números da seção abaixo continuam sendo os de 400 passos, porque é o que o `gabarito.py` roda em poucos minutos. As conclusões valem nos dois orçamentos.

Modelo	Treino	Validação real
modelo A (corpus de Machado)	4,4455	4,6957
modelo B (60 mil tokens gerados pelo A)	4,3012	5,1453

1 e 2. O modelo B fica pior na validação real — degradação de **+0,4496**. O fenômeno tem nome na literatura: *model collapse*.

A.11.7.1 O detalhe que torna isso perigoso

Olhe a coluna de treino. A loss de treino do modelo B é **menor** que a do A (4,30 contra 4,45). Se você olhasse só a curva de treino, concluiria que o B está aprendendo melhor.

E no orçamento cheio o contraste fica grotesco: **0,1149 de treino contra 9,2588 de validação**. O modelo B decorou o texto sintético quase perfeitamente — loss de treino 24 vezes menor que a do A — enquanto ficava 2,4 vezes pior no texto real. É a demonstração mais nítida deste curso de que **uma curva de treino excelente pode acompanhar um modelo destruído**.

Não está. Texto gerado por um modelo é **mais previsível** que texto humano — menos vocabulário, menos construções raras, menos surpresa. Ficar bom em prever texto fácil não é ficar bom em prever Machado.

O colapso chega disfarçado de progresso. É por isso que a validação precisa vir de dados reais e intocados — o mesmo princípio do E1 e do E2 deste capítulo.

A.11.7.2 Por que acontece

A razão é informacional. O texto gerado por A é uma **amostra** da distribuição que A aprendeu — e amostrar **perde a cauda**. Padrões raros aparecem pouco ou não aparecem, e o B aprende uma versão ainda mais concentrada. Repetindo o ciclo, a diversidade colapsa.

Nenhum modelo pode aprender dos próprios dados mais do que já sabe. A geração não cria informação nova sobre o mundo.

3. Quando dados sintéticos ajudam de verdade: quando existe uma **fonte de verdade externa** para filtrar.

Domínio	O verificador
matemática	dá para conferir a resposta
código	dá para rodar os testes
jogos	dá para saber quem ganhou

Nesses casos o modelo gera muitas tentativas e um verificador seleciona as boas. A informação que entra vem do **verificador**, não do modelo — e por isso o ciclo não colapsa. É a base do que o Capítulo 15 vai fazer.

A.11.8 E7 — Escalando o corpus

A.11.8.1 Versão do gabarito: frações do mesmo corpus

Corpus	Tokens	Treino	Validação	Gap
25%	155.283	4,3627	4,7601	0,3975
50%	310.567	4,4252	4,7330	0,3077
100%	621.134	4,4455	4,6957	0,2501

1 e 2. Mais dados melhoram a validação. Os ganhos são pequenos e da mesma ordem ao dobrar o corpus (0,027 e depois 0,037).

(!) **Não leia isto como saturação.** Estes números são de **400 passos**, e a seção seguinte mostra que nesse orçamento o modelo mal completa uma passada pelo corpus — não há épocas suficientes para o overfitting aparecer, que é justamente o mecanismo pelo qual mais dados ajudam. Com os 3.000 passos da apostila, o ganho é **muito maior e não satura**. Esta tabela serve para ver o mecanismo depressa, não para concluir sobre escala.

3. A lógica do Capítulo 3 vale em direção, não em magnitude.

O **gap** entre treino e validação encolhe monotonicamente (0,40 -> 0,31 -> 0,25), confirmando que parte do problema era memorização — exatamente como lá.

Mas compare a escala:

Capítulo	Mudança nos dados	Efeito no gap
Cap. 3	155 -> 64.000 nomes	5,7 -> ~0
Cap. 11	155 mil -> 621 mil tokens	0,40 -> 0,25

Lá o modelo tinha ~10 exemplos por parâmetro; aqui já tem centenas de milhares de tokens.

Eu ia escrever aqui que "melhorias ficam progressivamente mais caras, é a forma de toda curva de escala". A frase é verdadeira **em geral** e não descreve o que foi medido: no orçamento cheio o ganho não deu sinal de desacelerar. Ver a seção seguinte.

A.11.8.2 Versão literal: acrescentando obras de verdade

O enunciado pede para **acrescentar obras**. A lista completa de Machado no Gutenberg tem 12 (confirmado na API, não chutado) — o `prepare_data.py` usa 5. `e7_mais_obras.py` baixa as outras 7 e mede.

Duas decisões de método, que importam mais que o resultado:

1. **O tokenizador fica fixo.** Usamos o `tokenizador.pkl` já treinado, sem retreinar. Se retreinássemos, mudaríamos **duas coisas ao mesmo tempo** (mais dados e outro tokenizador) e não saberíamos a qual atribuir a diferença. Custo honesto: o tokenizador fica levemente mal-adaptado às obras novas — é o preço de isolar a variável, e é o preço certo a pagar.
2. **A validação não muda.** *Memorial de Aires* continua fora do treino. Sem isso as losses não seriam comparáveis entre configurações.

A.11.8.3 Os resultados, com os 3.000 passos da apostila

`e7_orcamento_cheio.py`:

Obras	Tokens	Treino	Validação	Gap
4	617.457	2,8302	3,9989	1,1687
6	812.973	2,9886	3,7763	0,7877
8	1.073.284	3,0770	3,7120	0,6350
11	1.427.186	3,1520	3,6712	0,5192

Perfeitamente monotônico nas três colunas, e cada uma conta um pedaço da mesma história:

- a **validação** melhora sem parar — mais dados, melhor generalização;
- o **treino piora** sem parar — corpus mais diverso é mais difícil de decorar;
- o **gap** despenca de 1,17 para 0,52 — que é a definição de menos overfitting.

1 e 2. Sim, acrescentar obras melhora, e não saturou dentro do que foi testado. Indo de 617 mil para 1,43 milhão de tokens, a validação cai 0,33. Não há sinal de platô — o que sugere que este modelo ainda está limitado por **dados**, não por capacidade.

A.11.8.4 A resposta com 400 passos era o oposto

Orçamento	Ranking (melhor -> pior)
400 passos	6 < 8 < 4 < 11 (o pior)
3.000 passos	11 (o melhor) < 8 < 6 < 4

Com o orçamento curto, a conclusão era "melhora até 6 obras e depois piora". Com o orçamento cheio, é "melhora sempre, monotonicamente". A inversão é **completa**.

E há uma lição a mais escondida aí. Eu tinha escrito uma hipótese específica para explicar por que 11 obras seria pior: *Poesias Completas* é o único livro em **verso**, e misturar poesia com prosa mudaria a distribuição. A hipótese era plausível, específica, e parecia **confirmada** pelos dados de 400 passos.

No orçamento cheio, 11 obras é a **melhor** configuração. O efeito regularizador de mais dados supera a diferença de gênero — e a minha explicação bonita estava explicando um artefato de orçamento.

Por que os dois orçamentos discordam. Com 400 passos, o modelo vê ~1,6 M de tokens: nem uma passada completa pelo corpus maior. Ninguém decora nada, e o único sinal que sobra é o de **distribuição** — daí a poesia atrapalhar. Com 3.000 passos ele vê ~12 M de tokens: 20 passadas pelo corpus pequeno contra 8,6 pelo grande. Aí o que domina é **overfitting**, e mais dados ganham com folga.

Note que o confundimento aponta na direção **oposta** à do E4: lá, a configuração "maior" fazia *mais* épocas; aqui, faz *menos*. É o mesmo mecanismo produzindo conclusões invertidas — e é por isso que a pergunta "quantas épocas cada configuração faz?" deveria vir antes de qualquer interpretação.

3. A lógica do Capítulo 3 vale, e agora com força. Lá, aumentar os dados de 155 para 64 mil nomes levou o gap de 5,7 para ~0. Aqui, dobrar o corpus leva o gap de 1,17 para 0,52. Direção idêntica, magnitude menor — e, ao contrário do que o experimento com frações do corpus sugeria, **ainda longe de saturar**.

A.12 GABARITO DO CAPÍTULO 12 — INFERENCE I: KV-CACHE

Respostas discursivas. Os **números** vêm de `gabarito.py`.

```
bash python solucoes/gabarito.py
```

Os exercícios E2 e E3 introduzem os bugs **de propósito**, em subclasses, para que você veja o modo de falha sem editar o modelo bom.

A.12.1 E1 — O que dá e o que não dá para cachear

1. Por que k e v são cacheáveis e q não é.

k e v da posição t dependem só do token em t e da sua posição — e nada disso muda quando um token novo chega em $t+1$. São propriedades **do passado**, e o passado está fechado.

q é diferente por natureza: a cada passo há um token novo fazendo uma pergunta nova ao histórico. A query é a única coisa que muda de verdade a cada iteração. Cachear query não faria sentido — ela é usada uma vez e descartada.

2. Sem máscara causal, o cache continuaria válido? Não.

Com atenção bidirecional, a representação da posição 3 depende do token 7. Quando o token 8 chega, **todas** as representações anteriores mudam — o k e o v guardados ficam obsoletos.

É uma observação que vale guardar: **o KV-cache só existe porque o modelo é causal**. Modelos tipo BERT não têm KV-cache, e não é falta de otimização — é que a otimização não existe para eles. A restrição que parecia limitação (só olhar para trás) é o que torna a geração eficiente.

3. As contas.

Caminho	Posições processadas
Ingênuo	$1 + 2 + \dots + 128 = 8.256$
Com cache	128 (uma por token)
Descartado no caminho ingênuo: $1 - 128/8.256 = 98,4\%$.	

A.12.2 E2 — Quebrando a posição de propósito

Trocando a posição absoluta por `torch.arange(T)`:

Verificação	Resultado
Levantou exceção?	não
Gerou texto?	sim , 81 tokens
Primeira divergência	token 4

```
referência : 'de vocês e vinham a vida.\n\n--Papae não ira a Deus que a convidasse...'
com o bug  : 'de vocês\nde lerguamos justa e uma dez do mariamente o a um homem...'
```

1. Nenhuma exceção, nenhum NaN, nenhum aviso. O texto sai com a mesma aparência de sempre. É o pior tipo de bug: ele não avisa.

2. Por que a divergência começa no token 4, e não no 1.

Porque o **prefill** processa o prompt inteiro de uma vez, e ali `T == T_total` — então `arange(T)` por acaso dá as posições certas. O bug só se manifesta na fase de **decode**, quando passamos um token só e ele acha que está na posição 0.

Isso é instrutivo por si: o bug se esconde exatamente na fase que o teste rápido (gerar um tokenzinho e ver se sai algo) não exercita.

3. Conseguiria dizer qual está errado sem a referência? Não.

Os dois textos são igualmente ruins a olho nu — porque um modelo de 2,2 M parâmetros já escreve mal quando está **certo**. A degradação se esconde no ruído de base.

Numa otimização de inferência, "**parece bom**" **não é evidência de nada**. É por isso que o critério do capítulo é *saída idêntica*.

A.12.3 E3 — Quebrando a máscara de propósito

Aplicando a máscara também no decode:

Verificação	Resultado
Primeira divergência	token 3
Tokens distintos na saída	54 de 80

1. Eu previ repetição. Errado.

Ao escrever este gabarito eu afirmei que o texto "degenera em repetição, porque todo token passa a ser gerado a partir do mesmo contexto". A contagem desmente: 54 tokens distintos em 80. A amostragem continua estocástica — o que se perdeu foi o **contexto**, não a diversidade.

A.12.3.1 O que o modelo virou — medido, não suposto

O teste certo não é olhar o texto. É perguntar: *de que a previsão ainda depende?* Duas histórias com o **mesmo primeiro token** e 19 tokens do meio completamente diferentes:

Modelo	Logits idênticos?	Diferença máxima
com o bug	sim	0,00
correto	não	6,22

Bit a bit idênticos. O modelo passou a prever a partir de (primeiro token, token atual, posição) e **mais nada**.

Ele virou, literalmente, o modelo do Capítulo 1: um **bigrama**. Toda a maquinaria de atenção continua rodando, consumindo tempo, e não transporta informação nenhuma.

2. Por que só o primeiro token sobrevive.

No decode $\tau == 1$, então `tril[:1, :T_total]` seleciona a **primeira** linha da matriz triangular: `[1, 0, 0, ..., 0]`. Ela permite olhar apenas a posição 0 — daí o primeiro token continuar influenciando enquanto todo o resto desaparece.

3. Eu também previ que este bug seria mais fácil de notar. Errado de novo.

O raciocínio era que ele "falha de forma escandalosa". Compare os dois textos: são igualmente ruins.

A lição dos dois exercícios juntos. Em modelo pequeno, nenhum dos dois bugs se denuncia pela leitura. Um deles rebaixa um Transformer a um bigrama e **você não percebe olhando**. O teste de equivalência não é burocracia — é o único instrumento que funciona.

E note que em modelo grande a situação é pior, não melhor: um GPT de verdade com esses bugs ainda escreveria português fluente. A fluência vem da camada de saída; a coerência é que vem do contexto.

A.12.4 E4 — Meça o ganho

`benchmark_cache.py`, máquina ociosa:

Tokens	Ingênuo	Com cache	Speedup	ms/token ing.	ms/token cache
16	0,070 s	0,045 s	1,53x	4,3	2,8
32	0,122 s	0,085 s	1,44x	3,8	2,7
64	0,272 s	0,170 s	1,60x	4,3	2,7
128	0,906 s	0,469 s	1,93x	7,1	3,7

A.12.4.1 O número que contradiz a Seção 1 da apostila

A apostila mostra que o caminho ingênuo desperdiça **98,4%** das posições processadas. Seria natural esperar um ganho de dezenas de vezes.

O ganho medido é 1,93x.

Não é erro de medição nem cache mal implementado. A Seção 2 do mesmo benchmark explica:

Medição	Custo
Prefill de 64 tokens de uma vez	5,66 ms -> 0,09 ms por token
Decode de 1 token com cache	3,30 ms
Razão	37x

Processar uma *posição* custa 0,09 ms. Processar uma *chamada* custa ~3,3 ms. A diferença é **custo fixo**: laço Python, despacho do PyTorch, alocação de tensores, leitura dos pesos das 4 camadas.

E os dois caminhos fazem o **mesmo número de chamadas** — uma por token gerado. Ambos pagam $128 \times \sim 3,3 \text{ ms} \approx 420 \text{ ms}$ só de overhead. O cache elimina o trabalho *dentro* de cada chamada, que neste modelo é a menor parte da conta.

98% do cálculo eliminado, 1,93x de ganho. É a lição do Capítulo 8 reaparecendo: otimizar o que não domina não acelera. O KV-cache é indispensável — mas o *ganho* que ele entrega depende de o cálculo ser o gargalo, e num modelo de 2,2 M parâmetros na CPU ele não é.

Onde o ganho aparece de verdade: **modelos grandes** (o trabalho por posição cresce), **contextos longos** (o desperdício quadrático cresce) e **GPU** (o custo fixo por chamada é diluído). Num 7B com contexto de milhares de tokens, a diferença é entre viável e inviável — mas isso é uma extrapolação, não algo medido aqui.

A.12.4.2 As respostas

1. O ms/token do ingênuo cresce, mas só com clareza no último ponto (4,3 -> 3,8 -> 4,3 -> 7,1). Até 64 tokens fica praticamente plano: o custo fixo domina e o contexto ainda é curto. O do cache também sobe um pouco no fim (2,7 -> 3,7), pela leitura de um cache maior — ou seja, "constante" é aproximação, não fato.

2. O speedup cresce, devagar e não de forma monotônica (1,53 -> 1,44 -> 1,60 -> 1,93). A queda em 32 tokens está dentro do ruído. A tendência é de crescimento, pelo motivo certo: o custo do ingênuo cresce com o contexto e o do cache quase não.

3. Para um único token o cache não ajuda — é um prefill nos dois casos. Ele começa a pagar do segundo token em diante, e não há comprimento em que atrapalhe: a alocação do cache não aparece no relógio neste tamanho.

A.12.5 E5 — A conta de memória

1. A fórmula, conferida contra `bytes_do_cache()`:

$$2 \times (K + V) \times n_{\text{layer}} \times \text{batch} \times n_{\text{head}} \times T \times \text{head_size} \times \text{bytes}$$

Para o modelo do curso com $T=128$: **786.432 bytes** (768 KB) — manual e método batem.

2. Um 7B em bf16, contexto 8.192:

Grandeza	Valor
Cache por usuário	4,3 GB
Placa de 80 GB - 14 GB de pesos	66 GB livres
Usuários simultâneos	15

Quinze. Uma placa de topo de linha, um modelo que cabe folgado nela, e quinze conversas ao mesmo tempo.

3. A reescrita que muda a leitura.

Como $n_head \times head_size = n_embd$:

$$2 \times n_layer \times batch \times T \times n_embd \times bytes$$

O cache não depende de quantas cabeças você usa, nem do tamanho delas — depende da **largura** e da **profundidade** do modelo. Redistribuir n_embd entre mais ou menos cabeças não muda nada no custo de servir.

O que muda é **quebrar a simetria**: se as cabeças de Q forem muitas e as de K/V forem poucas, o produto $n_head_kv \times head_size$ fica menor que n_embd . É precisamente isso que MQA e GQA fazem — e é por isso que elas são a intervenção óbvia nesta fórmula.

A.12.6 E6 — Cache com batch

Modo	Tempo
16 sequências de 64 tokens em paralelo	0,53 s
16 sequências de 64 tokens separadas	4,78 s
ganho	9,1x

1. Nada muda no código. A dimensão de batch já está na forma do cache (B , n_head , T , hs) e todas as operações são em lote — bastou passar um prompt com B linhas. Que a mudança seja nula é um bom sinal sobre a implementação.

2. O ganho é grande pela razão do Capítulo 8: o decode é limitado por **memória**, não por cálculo. Ler os pesos custa o mesmo para 1 ou para 16 sequências, então o custo se dilui.

É por isso que serviço de inferência agrupa requisições de **usuários diferentes** no mesmo batch. Não é para economizar — é que a alternativa desperdiça quase tudo.

3. Comprimentos diferentes são o problema de verdade. Se uma sequência termina no token 10 e outra no 200, o batch inteiro roda até a mais longa acabar: as posições já terminadas gastam cálculo à toa e o cache delas continua ocupando memória.

Estratégia	Como funciona
Preenchimento com máscara	simples e desperdiçador
Continuous batching	retira a sequência pronta e põe outra da fila no lugar

Paged attention (vLLM)	aloca o cache em páginas, sem reservar o pior caso por sequência
-------------------------------	--

A.12.7 E7 — Multi-query attention (desafio)

MQA improvisada sobre os pesos treinados (média das 6 cabeças de K/V):

Configuração	Loss de validação	Cache (T=128)
Multi-head original	3,9357	768 KB
MQA improvisada	5,2697 (+1,3339)	128 KB (6x menor)

1 e 2. O cache cai por um fator de `n_head` — 6x aqui, **32x** num 7B. É a maior economia disponível nesta fórmula.

3. E a loss desaba. +1,33 de loss é um modelo destruído, não degradado.

Não há surpresa: o modelo foi **treinado** com 6 cabeças de K/V independentes, e cada cabeça de Q aprendeu a perguntar para a **sua**. Substituir as seis por uma média destrói essa correspondência.

MQA não é uma otimização que se liga na hora de servir. É uma decisão de arquitetura. O modelo precisa nascer assim (ou passar por *uptraining* com dados). Llama 2 70B e Mistral usam GQA porque foram projetados com ela.

A lição geral vale além deste exercício, e fecha o capítulo:

"Economiza memória" e "dá para aplicar depois" são afirmações **independentes**. O KV-cache é gratuito porque **não muda a conta** — é a mesma matemática, reorganizada. A MQA não é, porque muda. Toda vez que alguém oferecer uma otimização, a primeira pergunta é: *isso altera o resultado?*

A.13 GABARITO DO CAPÍTULO 13 — INFERENCE II: QUANTIZATION

Respostas discursivas. Os **números** vêm de `gabarito.py`.
`bash python solucoes/gabarito.py`

A.13.1 E1 — A aritmética

1. Tensor em $[-3, 3]$, simétrica de 8 bits.

```
escala = max|x| / 127 = 3/127 = 0,023622
q(1,5) = round(1,5 / 0,023622) = round(63,5) = 64      (ou 63, conforme o
recuperado = 64 × 0,023622 = 1,51181                  desempate do round)
```

Erro de 0,0118 — cerca de **0,8%** do valor, e no máximo meia escala (0,0118) para qualquer entrada. O erro máximo de quantização é sempre **metade do passo**.

2. Por que $[-127, 127]$ e não $[-128, 127]$.

Para manter a **simetria**. Se usássemos -128, o intervalo representável seria assimétrico e o mapeamento $q = x/escala$ deixaria de ser simétrico em torno de zero: +3 viraria 127 e -3 viraria -128, valores com magnitudes diferentes.

Custa 1 nível em 256 — **0,4%** da resolução — e evita um caso especial em toda conta subsequente. É uma troca boa, e é o que PyTorch faz por padrão.

3. Tensor em $[10, 11]$.

Com simétrica, $escala = 11/127 = 0,0866$. Mas o *intervalo útil* dos dados tem largura 1, então cabem apenas $1/0,0866 \approx 12$ níveis — de 256 disponíveis. Os outros 244 representam valores entre -11 e +10, que nunca ocorrem.

Erro esperado: metade do passo sobre um valor típico de 10,5, ou seja $0,0433/10,5 \approx 0,4\%$. Parece pouco, mas a *resolução dentro do intervalo real* é péssima.

A assimétrica usaria $escala = (11-10)/255 = 0,0039$ — **22x mais fina** — e representaria o mesmo intervalo com todos os 256 níveis.

(!) Com a ressalva do E2: como $[10, 11]$ **não contém o zero**, é preciso estender o intervalo para $[0, 11]$, e aí a escala vira $11/255 = 0,0431$. Ainda **2x melhor** que a simétrica, mas não 22x. A extensão custa caro justamente nos casos mais deslocados.

A.13.2 E2 — A armadilha do zero-point

Distribuição `randn(1000) + 5`, valores em $[1, 24 ; 8, 63]$:

Variante	Erro
simétrica	0,393%
assimétrica sem estender o zero	1,007% <- pior
assimétrica com estender o zero	0,192%

1. Com o bug, a simétrica ganha — o oposto do que a teoria diz. Esse é justamente o sintoma que denuncia o problema: quando um método que *deveria* ser melhor sai pior, há bug antes de haver descoberta.

2. O zero-point ideal é -170,7 e não cabe em $[-128, 127]$.

A razão é conceitual, não numérica: o zero-point é *o inteiro que representa o float 0,0*. Se os dados vão de 1,24 a 8,63, o zero está **fora** do intervalo — e representá-lo exigiria um inteiro que o tipo não tem.

3. A cadeia causal completa, e vale segui-la inteira:

```
dados em [1,24 ; 8,63], que não contêm o zero
-> zero-point ideal = qmin - xmin/escala cai fora de [-128, 127]
-> o clamp o move para -128
-> o mapeamento inteiro se desloca em relação ao pretendido
-> q = x/escala + zero estoura o teto para os valores grandes
-> 12 de 1000 valores saturam em +127 e perdem a informação
-> o erro medido dispara
```

A correção — estender o intervalo para conter o zero — é o que PyTorch, TFLite e ONNX fazem. Ela custa resolução e garante duas coisas: o zero-point cabe, e o float 0,0 tem um inteiro exato (o que importa para *padding* e máscaras).

A.13.3 E3 — A métrica que mente

Matriz 8×64 com uma linha 100x maior, quantização int8:

Métrica	Per-tensor	Per-channel
erro relativo global (norma)	1,48%	0,52%
... só nas 7 linhas normais	56,15%	0,60%
mediana do erro por elemento	100,00%	0,67%

E o número que resolve a questão: **65,4%** dos pesos das linhas normais viraram **zero exato**.

1. Por que os dois primeiros números são tão diferentes. O erro global é ~1,5%; o das linhas normais é ~56%. A norma global é dominada pela linha grande, que quantiza bem — ela tem valores grandes o bastante para usar toda a resolução disponível.

2. A conta. $||w||$ soma **quadrados**. A linha 0 tem valores 100x maiores, logo contribui **10.000x** mais que cada uma das outras para o denominador. O denominador de erro_relativo é praticamente só ela; o numerador (o erro) está espalhado por toda a matriz. Resultado: o erro das linhas pequenas acaba dividido por um número que não tem nada a ver com elas.

3. Uma métrica que denuncia sem desagregar: a mediana do erro relativo por elemento.

```
((rec - orig).abs() / orig.abs()).median()
```

Ela não tem denominador global, então nenhum elemento pode esconder os outros. Na tabela acima ela marca **100%** — mais da metade dos pesos perdeu todo o seu valor.

Alternativa igualmente boa: **a fração de pesos que virou zero exato**. Ela transforma "degradação" em "apagamento", que é o que de fato aconteceu.

Isto vale muito além da quantização. Sempre que os dados forem heterogêneos em escala, uma média ponderada pela magnitude vai contar a história do subconjunto grande.

A.13.4 E4 — Onde fica o precipício

Bits	Erro no peso	Piora da loss	Perplexidade
fp32	—	—	51,3
8	0,69%	-0,0001	51,3
6	2,84%	+0,0021	51,4
4	12,55%	+0,0636	54,6
3	29,31%	+0,4657	81,7
2	80,84%	+3,5298	1.749,3

1. Uma não prevê a outra, e é o ponto do exercício.

O erro no peso cresce de forma **suave e regular** — dobra a cada bit. A piora da loss é **desprezível até 4 bits e depois dispara**. São curvas de formatos completamente diferentes.

A razão: a rede tem folga. Perturbações pequenas nos pesos são absorvidas — até deixarem de ser pequenas.

Corolário prático: **nunca escolha o número de bits olhando o erro de reconstrução**. Ele é fácil de medir e não responde à pergunta.

2. O joelho está entre 4 e 3 bits. A piora salta de +0,064 para +0,466 — **7x**. E de 3 para 2 o modelo é destruído: a perplexidade sai de 82 para **1.749**.

3. Para servir este modelo eu escolheria int8, e a justificativa precisa dos **dois** eixos:

- **qualidade:** de graça (a loss não muda na quarta casa)
- **tamanho:** 4x menor, o que ataca o gargalo estabelecido no Capítulo 12

int4 também é defensável — custa 0,06 de loss e dá 8x. A escolha entre os dois depende de quanta memória você precisa economizar. O que **não** é defensável é escolher por um eixo só.

A.13.5 E5 — Quantizando os embeddings também

Configuração	Tamanho	Loss	Custo
int8, só Linear	2,89 MB	3,9371	—
int8, + embeddings	2,20 MB	3,9371	+0,0000
int4, só Linear	1,90 MB	4,0008	—
int4, + embeddings	1,10 MB	4,0457	+0,0449

1. Em int8 é literalmente de graça: 24% menor, loss idêntica até a quarta casa. Em int4 o custo aparece — +0,045, o que quase dobra a penalidade total do int4 (0,064 -> 0,109).

2. Por que a resposta não é "no que tem a maior fração".

Vale onde a fração é grande **o bastante para mover o total**. Num 7B, os embeddings são ~2% dos parâmetros: quantizá-los muda o tamanho em 2%, o que não compensa nem o risco de qualidade nem a complexidade. Aqui são 10%, e a economia de 24% é visível.

A pergunta é sempre *"quanto isso move o número que me importa"*, não *"qual é maior"*.

3. A camada de saída merece cuidado extra, por uma razão estrutural: ela produz os **logits**, e a softmax é sensível a diferenças pequenas entre eles. Um erro que seria inofensivo numa camada intermediária pode **reordenar** os tokens mais prováveis.

Muitos esquemas de produção deixam a camada de saída — e a de entrada — em precisão maior justamente por isso.

A.13.6 E6 — O que a quantização faz com a velocidade

Configuração	Gerar 64 tokens
float32	0,129 s
int8	0,184 s (0,70x)

1. O float32 é mais rápido. A quantização, desta forma, **custa** tempo.

2. Onde o tempo vai. No forward da LinearQuantizada:

```
F.linear(x, desquantizar_simetrica(self.q, self.escala), self.bias)
```

A desquantização reconstrói a matriz inteira em float32 — conversão de tipo mais multiplicação pela escala, sobre **todos** os pesos, a **cada** chamada. Depois disso a matmul é exatamente a mesma de antes. Trabalho extra, mesmo trabalho antigo.

3. O que precisaria mudar. Um kernel que multiplicasse $\text{int8} \times \text{int8}$ acumulando em int32, sem nunca materializar a matriz em float. Não se faz em PyTorch puro porque o operador não existe — `F.linear` espera floats. É preciso descer para C++/CUDA, ou usar llama.cpp, bitsandbytes, ONNX Runtime, que já fizeram isso.

"Menor" e "mais rápido" são independentes. A primeira decorre da representação; a segunda exige que alguém escreva o kernel.

A.13.7 E7 — Quantização por grupos

Matriz 256×256 com as 32 primeiras colunas 50x maiores, tudo em **int4**:

Esquema	Erro global	Erro nas colunas normais	bits/peso reais
---------	-------------	--------------------------	-----------------

per-channel (1 escala/linha)	11,23%	100,00%	4,06
grupos de 128	10,51%	65,90%	4,12
grupos de 64	10,13%	38,57%	4,25
grupos de 32	9,93%	9,69%	4,50

A.13.7.1 Eu caí na armadilha do meu próprio E3

Na primeira versão deste exercício eu medi **só o erro global** e conclui que os grupos quase não valiam a pena: de 11,2% para 9,9%, contra 11% mais bits — praticamente empate.

Errado, e pelo motivo que o E3 acabou de ensinar. A norma global é dominada pelas 32 colunas outlier, que quantizam bem em qualquer esquema.

Olhando a coluna que importa — as **colunas normais, que são 87,5% da matriz** — o quadro é outro: o per-channel em int4 as **apaga por completo** (100% de erro), e grupos de 32 reduzem isso para 9,69%. **Dez vezes melhor, por 11% mais bits.**

1 e 2. Grupos menores reduzem o erro onde ele existe, porque cada escala cobre menos variação. Com G=32 o bloco de outliers cai inteiro num grupo só e **para de contaminar os demais** — que é exatamente o objetivo.

3. A conta de bits por peso reais, que costuma ser esquecida:

```
bits_reais = bits + (bits_da_escala / G)
```

Com int4 e G=64, guardando a escala em fp16: $4 + 16/64 = 4,25$ bits por peso. Com G=32: **4,5 bits**. Ou seja, "int4" com grupos pequenos não é 4 bits — é até **12% mais**.

É esse compromisso que os nomes dos formatos GGUF expõem: Q4_0, Q4_K_M e afins diferem no tamanho do grupo e em quantos bits gastam com metadados. Escolher entre eles é escolher um ponto nesta curva — e a escolha só faz sentido se você medir o erro **onde ele está**.

A.14 GABARITO DO CAPÍTULO 14 — FINETUNING I: SFT

Respostas discursivas. Os **números** vêm de `gabarito.py` e de `e2_quando_a_mascara_importa.py`.
`bash python solucoes/gabarito.py`

A.14.1 E1 — Tokens especiais e vocabulário

1. Por que um id novo, e não a string `###FIM###`.

Porque o texto do usuário pode conter a string. Se alguém escrever `###FIM###` no meio de uma pergunta, o seu parser corta ali — e o modelo, treinado a tratar aquilo como fronteira, obedece.

Um token especial é uma garantia **estrutural**: não existe sequência de caracteres que o tokenizador transforme no id 1026. A fronteira não pode ser falsificada pela entrada.

É a base da defesa contra *prompt injection* por delimitador. Quando você lê que um modelo "confundiu a instrução do sistema com o texto do usuário", quase sempre há uma fronteira que era texto quando deveria ser estrutura.

2. Quantos parâmetros novos, com `n_embd = 192`.

Matriz	Parâmetros
tabela de embeddings	3 576 × 192
camada de saída (pesos)	3 576 × 192
camada de saída (bias)	3 3
total	1.155

Confirmado pelo código: 2.196.352 -> 2.197.507. Um aumento de **0,05%**.

3. Zeros e valores aleatórios grandes.

Zeros na camada de saída não são neutros. Um logit zero é um valor específico dentro de uma distribuição já treinada, e para um modelo convergido a maioria dos logits reais é *negativa* — então o token novo nasceria **mais provável** que muitos tokens legítimos. O modelo passaria os primeiros passos consertando isso.

Valores aleatórios grandes são piores: o token novo domina a softmax, a loss explode nos primeiros passos, e o gradiente que corrige isso perturba o resto da camada.

A **média** das linhas existentes evita os dois: o token novo nasce estatisticamente indistinguível dos outros e se diferencia conforme aprende.

A.14.2 E2 — Quando a máscara importa

A.14.2.1 A medição

Pedido/resposta	% de pedido	Com máscara	Sem máscara	Penalidade
24 / ~34	42%	3,9964	3,9898	-0,0066
64 / ~20	72%	3,9450	3,9586	+0,0136
104 / ~12	87%	3,9588	3,9752	+0,0165

1. A explicação padrão é que treinar no pedido "gasta capacidade". O que ela **não** diz, e deveria, é que a loss sem máscara é uma **média sobre as posições** — então o tamanho relativo do pedido decide quanto ele dilui o sinal da resposta. Toda a dependência está aí.

2. A **penalidade aparece a partir de ~70% de pedido**, e o sinal **inverte**: com pedido curto, mascarar é marginalmente pior.

Repetindo a configuração de pedido longo com **seis sementes**:

Semente	Com máscara	Sem máscara	Diferença
1337	3,9588	3,9752	+0,0165
42	3,9737	3,9796	+0,0058
2024	3,9389	3,9690	+0,0301
7	3,9537	3,9860	+0,0322
99	3,9607	3,9867	+0,0261
555	3,9536	3,9624	+0,0088

6 de 6 na mesma direção, média +0,0199 ± 0,0112.

3. Nulo e negativo pedem reações diferentes.

No caso de pedido curto o resultado foi **negativo**, não nulo: mascarar saiu *pior* (-0,0066). Um resultado nulo autorizaria "não detectei diferença"; um negativo consistente autorizaria "a recomendação está invertida aqui". Este é pequeno demais para a segunda afirmação — está dentro do que a semente move.

A.14.2.2 O erro de análise que quase enterrou o resultado

Com as três primeiras sementes eu comparei a diferença entre variantes (0,0175) com a variação da **mesma** variante entre sementes (0,0348) e concluí: "*o ruído supera o efeito*".

O critério estava errado para o desenho. As duas variantes rodam com a **mesma semente** — é um experimento **pareado**, e nele a variação que a semente causa afeta os dois lados e **se cancela na diferença**.

Critério	Números	Veredito
não pareado	ruído 0,0348 > efeito 0,0199	"não estabelecido"
pareado	6/6 na mesma direção, média > desvio	estabelecido

Mesmos dados, conclusões opostas, decididas só pela estatística escolhida.

É o segundo caso do curso em que a **medição estava certa e a leitura não**. O outro é o Capítulo 13, onde uma métrica global marcava 1,3% de erro enquanto 48% das linhas estavam destruídas.

A.14.2.3 E a magnitude

+0,0199 sobre uma loss de 3,95 é 0,5%. O efeito é real e pequeno. "Estatisticamente detectável" e "importante na prática" não são a mesma coisa, e este número é o primeiro sem ser o segundo — *nesta escala*. Em SFT real, com prompt de sistema longo e histórico de conversa, a proporção é muito mais desfavorável que 87%.

A.14.3 E3 — Dados de tamanho fixo

Com TAM_RESPOSTA = 40 fixo, todos os 8.000 exemplos ficam com exatamente 66 tokens:

Modelo	Taxa de parada	Comprimento mediano
--------	----------------	---------------------

SFT (com máscara)	100%	40
SFT (sem máscara)	100%	40

1 e 2. A taxa de parada é 100% e não significa nada. A métrica que separa as duas versões é o **comprimento mediano**: cravado em 40, sem variância.

O modelo aprendeu a **contar até 40**, não a concluir. Era a única regra consistente nos dados — e ele a aprendeu perfeitamente.

Com a correção (resposta até o fim de frase), os cumprimentos passam a variar de 39 a 82 tokens, 44 valores distintos, e a mediana medida vira 34 — próxima mas não igual à do treino, que é o que se espera de um modelo que está lendo o texto.

3. Regularidades acidentais num conjunto real:

Regularidade	O que o modelo aprende
toda resposta começa com "Claro!"	a dizer "Claro!"
respostas de um anotador são mais longas	o estilo daquele anotador
exemplos difíceis foram descartados na curadoria	a nunca dizer "não sei"
perguntas de código sempre vêm com bloco markdown	a formatar mesmo quando não cabe
O modelo aprende a regularidade que os seus dados de fato contêm , não a que você pretendia ensinar. Antes de treinar, procure o que é acidentalmente constante.	

A.14.4 E4 — A learning rate do finetuning

400 passos, 8.000 exemplos:

lr	Loss na resposta	Taxa de parada	Loss no texto puro
3e-5	4,2448	0%	3,9517
3e-4	3,9749	97%	4,0372
1e-3	4,0181	100%	4,1464
(modelo-base)	—	0%	3,9104

1. Não são as três. Com $3e-5$ a taxa de parada é **0%** — passos pequenos demais não chegam a instalar um comportamento que não existia; só ajustam o que já estava lá.

E a melhor loss está no **meio**, não num extremo. Learning rate boa não é a maior nem a menor.

2. Olhe a última coluna. Ela mede a loss no texto corrido do Capítulo 11, sem formato de instrução — a capacidade original do modelo. O modelo-base marca 3,9104, e a coluna se afasta **monotonicamente** conforme a learning rate cresce: 3,9517 -> 4,0372 -> 4,1464.

O modelo está trocando a competência antiga pela nova, e o câmbio piora com o tamanho do passo.

3. Catastrophic forgetting. O finetuning vê um conjunto pequeno e homogêneo, e o gradiente empurra **todos** os pesos na direção que serve àquele conjunto. Nada no objetivo preserva o que foi aprendido antes.

A única proteção embutida é dar passos pequenos e poucos. É por isso que a learning rate de finetuning é tipicamente 3 a 10 vezes menor que a do pré-treino — não é cautela, é o mecanismo.

Note que os dois eixos apontam para lados opostos, e o $3e-4$ do capítulo cai onde eles se equilibram: melhor loss na tarefa nova, com metade do custo em esquecimento que o $1e-3$ cobra.

A.14.5 E5 — Quantos exemplos bastam?

Exemplos	Loss na resposta	Taxa de parada
500	4,8662	100%
2.000	4,1922	100%
8.000	3,9749	97%

1. As duas métricas se separam, e essa é a resposta.

A **taxa de parada satura em 500** — já é 100%. O formato é barato de instalar: três tokens especiais e uma convenção de onde cada um entra.

A **loss não satura**: 4,8662 -> 4,1922 -> 3,9749, quase **0,9** de melhora.

São coisas diferentes: uma mede se o modelo respeita o **formato**, a outra se ele escreve uma resposta **boa**. Só a primeira é barata.

2. Por isso a comparação com o Capítulo 11 precisa ser dividida. Para o **formato**, a curva satura cedo — ao contrário de lá. Para a **qualidade** da resposta, a curva se parece com a de lá, porque a tarefa volta a ser modelar texto.

3. Resultados como o do **LIMA** ("mil exemplos bem escolhidos bastam") falam do primeiro eixo. Se o que você quer é instalar **comportamento** — formato, tom, recusa, uso de ferramenta — mil exemplos variados bastam mesmo, e a medição acima mostra por quê: é pouca informação a transmitir.

O que mil exemplos **não** compram é capacidade de gerar conteúdo melhor. Essa vem do pré-treino, e nenhum SFT a substitui.

A.14.6 E6 — O *alignment tax*

Modelo	Loss no texto corrido do Cap. 11
modelo-base	3,9104
depois do SFT (3e - 4)	4,0372 (+0,1268)

1 e 2. O SFT **piorou** a capacidade de modelar português comum em +0,127 de loss. Não é catastrófico, e não é zero.

3. É o ***alignment tax***: o custo, na competência original, de instalar um comportamento novo. Ele **não é inevitável**, e há três formas conhecidas de reduzi-lo:

Técnica	Como ataca
misturar dados de pré-treino no conjunto de SFT	o gradiente continua vendo texto comum
LoRA / congelar camadas	limita quanto os pesos originais podem se mover
lr menor, menos passos	o que o E4 mede

Todas atacam a mesma causa: **nada no objetivo do SFT pede que o modelo lembre do que sabia**.

A.14.7 E7 — Conversa com vários turnos

1. A máscara passa a ter **várias faixas** — uma por resposta, em vez de uma só:

```
<|pedido|> A <|resposta|> B <|fim|> <|pedido|> C <|resposta|> D <|fim|>
\_ máscara \_/\_ treina \_/ \_ máscara \_/\_ treina \_/
```

Todos os turnos do usuário ficam mascarados, inclusive os do meio.

2. O `<|fim|>` tem de vir no fim de CADA resposta, e dá para decidir isso sem treinar.

Se ele aparecesse só no fim da conversa, a geração não teria onde parar ao terminar um turno — o modelo continuaria e **inventaria a fala do usuário**. É exatamente o comportamento do modelo-base que o capítulo inteiro combateu, só que agora dentro do formato.

3. É por isso que modelos reais usam um token de fim **por turno** (`<|im_end|>` e equivalentes), e não um marcador único de conversa.

E note que isso torna a conta do E2 **mais** relevante, não menos: numa conversa longa o histórico mascarado cresce e as respostas não. A fração de posições que seriam desperdiçadas sem máscara aumenta a cada turno — o regime desfavorável do E2 é o regime normal de um chat.

A.15 GABARITO DO CAPÍTULO 15 — FINETUNING II: RL

Respostas discursivas. Os **números** vêm de `experimento.py` e de `e4_dial_do_kl.py`.

```
bash python experimento.py # ~2h30 (3 configurações)
python solucoes/e4_dial_do_kl.py 0.1 0.5 # ~50 min por
beta
```

A.15.1 E1 — SFT e RL, lado a lado

1. De onde vem o sinal.

Método	Fonte do sinal
SFT	um alvo : existe um texto de referência, e a loss mede a distância até ele
RL	um juiz : não há texto de referência, e a loss mede se o que saiu pontuou bem

A diferença é de *fonte*, não de algoritmo. Os dois fazem descida de gradiente sobre os mesmos pesos.

2. Por que o RL precisa gerar.

Porque a recompensa é uma função da **saída**, e a saída não existe antes de o modelo produzi-la. No SFT o alvo já está no disco; aqui é preciso rodar o modelo até o fim para ter o que avaliar.

O custo é grande e é o motivo de este capítulo ser o mais lento do curso: cada passo de treino contém **48 passos de geração** — e com dois modelos, política e referência. Um passo de SFT é um forward e um backward; um passo de RL é 96 forwards mais um backward.

É por isso que RLHF é caro de um jeito que finetuning não é, e por que técnicas como o DPO — que eliminam a geração — ganharam tração tão rápido.

3. Por que o baseline não muda o gradiente esperado.

Porque $E[R - \text{média}(R)] = 0$. Subtrair uma constante que não depende da ação não altera a esperança do gradiente:

$$E[(R - b) \cdot \nabla \log P] = E[R \cdot \nabla \log P] - \underbrace{b \cdot E[\nabla \log P]}_{= 0}$$

O último termo é zero porque $E[\nabla \log P] = \nabla E[1] = 0$ — a soma das probabilidades é sempre 1, então o gradiente dela é nulo.

O que muda é a **variância**. Sem baseline, uma recompensa de 0,9 e outra de 0,8 são as duas "boas" e as duas são reforçadas com força parecida. Com baseline, a de 0,9 é reforçada e a de 0,8 é **penalizada** — o sinal passa a ser sobre qual é melhor, não sobre qual é positiva.

A.15.2 E2 — Escreva uma recompensa ruim

A resposta é sua, mas há um padrão que quase sempre aparece e vale nomear.

Toda recompensa da forma "meça **X** como proxy de **Y**" tem uma região onde X e Y andam juntos — normalmente a região onde estão os dados naturais — e uma região onde eles se descolam. O RL vai direto para a segunda, porque é lá que X é maior.

Recompensa	X (medido)	Y (pretendido)	Onde descola
pontos	fração de tokens de pontuação	frases bem pontuadas	' . '
comprimento mínimo	número de tokens	resposta completa	repetição, enrolação
"contém a palavra da pergunta"	sobreposição léxica	responde ao que foi perguntado	eco do enunciado
"poucas repetições"	tokens distintos	fluência	palavras raras aleatórias

3. O exercício pede para identificar o X e o Y da sua própria recompensa. Se você não conseguir separá-los, provavelmente escreveu Y como se fosse mensurável — e é justamente esse o momento em que uma recompensa ruim nasce parecendo razoável.

Note que **nenhuma** das quatro acima está "com bug". Todas medem exatamente o que dizem. O erro está em confundir a medida com a intenção.

A.15.3 E3 — O baseline importa mesmo?

1 e 2. Sem baseline, o aprendizado continua acontecendo — só que mais devagar e com a curva de recompensa muito mais ruidosa.

O motivo é o do E1: o gradiente esperado é o mesmo, então a direção média está certa. O que piora é a **variância** de cada estimativa individual. Com recompensas todas

positivas, cada passo reforça tudo que foi amostrado, e o sinal útil — "isto foi melhor que aquilo" — fica submerso.

3. Recompensa sempre negativa é o caso que expõe o problema.

Se $R \in [-1, 0]$, então sem baseline **toda** ação é penalizada, sempre. O modelo empurra para baixo a probabilidade de tudo que gera, inclusive do que gerou a recompensa menos ruim. Ele aprende — porque as coisas piores são penalizadas mais — mas por um caminho absurdo: reduzindo a probabilidade de tudo, e diferenciando só pelas taxas.

Isso é o argumento para **normalizar recompensas** (subtrair a média, dividir pelo desvio). A escala e o deslocamento da sua recompensa não deveriam mudar o algoritmo, e sem normalização eles mudam.

A.15.4 E4 — Onde fica a faixa útil do β

Recompensa pontos (mal especificada), 400 passos, partindo de loss 4,0784:

β	Recompensa	KL	Loss real	Custo	Regime
0,00	0,058 -> 0,996	14,87	4,8211	+0,7427	hackeia por completo — ' . '
0,02	0,058 -> 0,958	11,63	4,3963	+0,3179	hackeia quase igual — ' . '
0,10	0,055 -> 0,218	1,41	4,0832	+0,0048	aprende sem custo
0,50	0,053 -> 0,064	0,31	4,0810	+0,0026	mal se move
2,00	0,053 -> 0,055	0,23	4,0800	+0,0016	congelado

1. A faixa útil existe, é $\beta \approx 0,1$ — e é um ponto só.

Em $\beta = 0,10$ a recompensa quadruplica (0,055 -> 0,218), a KL cai de 14,87 para 1,41, e o custo em português é **+0,005**, indistinguível de zero. A resposta volta a ser texto.

De um lado dele, em $\beta = 0,02$, o modelo hackeia quase completo. Do outro, em $\beta = 0,5$, ele paralisa: a recompensa sobe 0,011 em 400 passos. **Um fator de cinco para cada lado**, e a janela fecha.

Isso é desconfortável, e é o resultado. O β não é um parâmetro que você ajusta "mais ou menos" — é uma escolha entre três regimes qualitativamente diferentes, e só um deles serve.

2. A transição entre 0,02 e 0,10 é abrupta, e o motivo é estrutural.

A recompensa efetiva é $R - \beta \cdot KL$. O hacking total rende $R \approx 1$ e custa $KL \approx 15$. Então:

β	Ganho do hack	Custo do hack	Compensa?
0,02	+1,0	-0,30	sim, muito
0,10	+1,0	-1,50	não

O β não modula suavemente *quanto* o modelo hackeia — ele decide se hackear **vale a pena**. É uma mudança de sinal numa desigualdade, e por isso a transição parece um degrau em vez de uma rampa.

É a mesma forma do precipício entre 4 e 3 bits no Capítulo 13: um parâmetro contínuo produzindo um comportamento com limiar.

3. Custo zero não é sucesso — e as duas últimas linhas provam.

Em $\beta = 0,5$ o custo é **+0,0026**, quase metade do custo em $\beta = 0,10$. Pela coluna do custo, seria a melhor configuração da tabela.

Olhe a recompensa: **0,053 -> 0,064**. Em 400 passos, o modelo aprendeu 0,011. Ele não foi protegido; foi **impedido de aprender**. Em $\beta = 2,0$ isso fica explícito — KL de 0,23, e a política praticamente não se moveu do ponto de partida.

É o mesmo formato de erro que o Capítulo 13 encontrou na quantização: uma métrica que parece boa porque o sistema **não está fazendo nada**. Lá era o erro global escondendo linhas zeradas; aqui é o custo baixo escondendo um modelo que não treinou.

Por isso a tabela precisa das **duas** colunas. Otimizar só o custo leva ao β infinito, que é exatamente o mesmo que não treinar — e tem custo zero.

A.15.5 E5 — A recompensa não é o objetivo

1. As ordens são opostas.

Por recompensa (melhor -> pior)	Por loss em Machado (melhor -> pior)
pontos sem freio (0,996)	(partida, sem RL) — 4,0784
pontos com freio (0,958)	comprimento com freio — 4,1701
comprimento com freio (0,743)	pontos com freio — 4,3963
(partida) — sem recompensa	pontos sem freio — 4,8211

O **campeão** de recompensa é o **pior** modelo. Não é coincidência nem azar: ele é o pior justamente **porque** foi o melhor em otimizar a coisa errada.

2. O que um relatório só com a recompensa diria.

Diria: "o RL funcionou — a recompensa subiu 17 vezes". Cada palavra verdadeira, e a conclusão que o leitor tira é falsa.

O engano é específico: a recompensa **sempre sobe**. Ela mede se o otimizador está funcionando, não se o modelo melhorou. Reportá-la sozinha é reportar que o gradiente descendo — informação sobre o algoritmo, disfarçada de informação sobre o resultado.

3. Duas métricas que um projeto real precisa acompanhar:

Métrica	O que detecta que a recompensa não detecta
KL da referência	que a política está se afastando — o mecanismo, antes do dano
Desempenho numa tarefa que o RL não vê (aqui, loss em Machado)	que a capacidade original está sendo consumida

Uma terceira, quando há gente disponível: **avaliação humana em amostra**, justamente porque ela não pode ser otimizada diretamente pelo modelo. É por isso que laboratórios mantêm avaliações humanas caras mesmo tendo métricas automáticas baratas — a métrica barata é o que o modelo está otimizando, então ela é a que menos serve para julgá-lo.

A.15.6 E6 — Conserte a recompensa

1 e 2. Há consertos que ajudam: penalizar pontuação repetida, exigir um mínimo de tokens não-pontuação entre os pontos, medir a fração de *frases bem formadas* em vez de *caracteres de pontuação*.

Cada um fecha o buraco que você viu. E o modelo procura o próximo.

3. Não existe recompensa que resista a qualquer otimização, e isso não é pessimismo — é a estrutura do problema.

Se a recompensa é uma função computável da saída, e você otimiza contra ela com força suficiente, o modelo encontra o máximo dessa função. A pergunta nunca é "esta recompensa tem buracos?" — tem — e sim "**o máximo dela é aceitável?**".

Isso reorganiza o que fazer:

Estratégia	Por que funciona (ou não)
escrever recompensa melhor	corrida sem fim — fecha um buraco, abre outro
limitar a otimização (KL, poucos passos)	compra tempo , não resolve — é o E4
recompensa verificável	muda o jogo: se o máximo é "a resposta está certa", hackear = acertar

A terceira é a razão de a área ter migrado para **matemática, código e tarefas com verificador**. Não é que esses domínios sejam mais importantes — é que neles a recompensa não é um proxy. Quando o teste passa, o teste passou.

E é a mesma conclusão do E6 do Capítulo 11, chegando por outro caminho: dados gerados pelo próprio modelo ajudam **quando existe um verificador externo**. Lá era sobre dados, aqui é sobre recompensa, e o critério é o mesmo.

A.15.7 E7 — DPO (desafio)

1 e 2. A implementação é direta e vale pelo contraste: o DPO não gera durante o treino — ele consome pares prontos, com uma loss que se parece com uma cross-entropy. Um passo de DPO custa o mesmo que um passo de SFT, contra os 96 forwards de um passo de REINFORCE.

3. O DPO não evita reward hacking, e a razão está no enunciado do exercício.

Você rotulou os pares com a **mesma recompensa ruim**. O DPO vai otimizar a preferência que recebeu, com a mesma obediência literal — e a preferência recebida diz que

respostas com mais pontuação são melhores.

Trocar o algoritmo não conserta a especificação. O DPO elimina o modelo de recompensa explícito e o custo da geração; **não** elimina a distância entre o que você mediu e o que você queria.

Se a sua preferência vier de humanos, o problema muda de lugar mas não desaparece: humanos preferem respostas longas, confiantes e bem formatadas — e um modelo otimizado contra essa preferência aprende a ser longo, confiante e bem formatado. Inclusive quando está errado.

A.16 GABARITO DO CAPÍTULO 16 — DEPLOYMENT

Respostas discursivas. Os **números** vêm de `carga.py`, rodado contra os dois servidores.

```
bash python servidor.py # terminal 1 python carga.py #
terminal 2
```

A.16.1 E1 — TTFT e latência

1. O que o streaming melhora.

O **tempo até o usuário ver alguma coisa**, e nada mais. O servidor faz o mesmo trabalho, o texto é o mesmo, o total é o mesmo.

O que muda é a experiência: sem streaming, uma tela parada durante toda a geração e depois tudo de uma vez. Com streaming, a primeira palavra quase imediatamente e a leitura acompanha a produção.

É uma otimização de **latência percebida**, não de latência. A distinção parece acadêmica até você usar os dois.

2. Por que num LLM sim e num classificador não.

Porque a geração é **autorregressiva**: a saída é uma sequência produzida em ordem, e cada token existe antes do seguinte. Há o que transmitir aos poucos.

Um classificador produz **uma** saída, no fim de um único forward. Não existe "primeira metade da etiqueta". Não há nada para transmitir antes de terminar.

O streaming não é uma técnica que se aplica a qualquer modelo — ele é uma consequência de a saída ser sequencial.

3. Sem o `flush()`.

O sistema operacional acumula os bytes num buffer e envia quando ele enche ou quando a conexão fecha. O resultado prático: o cliente recebe tudo de uma vez, no fim — exatamente o comportamento do endpoint não-streaming.

E o código **continuará parecendo streaming**: o laço escreve token por token, o Transfer-Encoding: chunked está lá, tudo tem cara de certo. O defeito só aparece medindo o TTFT.

É o mesmo padrão do Capítulo 12: um bug que não levanta exceção e só é detectável por medição.

A.16.2 E2 — Meça o seu próprio TTFT

Tokens pedidos	TTFT (stream)	Total (stream)	TTFT/total	Total (sem stream)
40	0,085 s	0,131 s	65,0%	0,117 s
200	0,084 s	0,872 s	9,6%	0,762 s

1 e 2. O TTFT é praticamente constante; o total é que cresce.

0,085 s com 40 tokens, 0,084 s com 200. Faz sentido: o primeiro token custa um prefill mais um passo de decode, e isso não depende de quantos tokens virão depois.

Então a razão TTFT/total **despenca** conforme a resposta cresce: de 65% para 9,6%. O streaming melhora pouco em respostas curtas e muito em respostas longas — e a tendência é mecânica, não empírica.

3. A extrapolação, e ela muda a leitura do capítulo.

Um modelo real a 30 tok/s, 500 tokens:

Medida	Sem streaming	Com streaming
tempo até ver algo	~17 s	~0,3 s
razão TTFT/total	100%	~1,8%

Dezessete segundos de tela parada é o que separa "está quebrado" de "está pensando". A magnitude de 30% medida no capítulo é pequena **porque o modelo é rápido demais para o efeito aparecer** — não porque o efeito seja pequeno.

Esta é a mesma armadilha do Capítulo 12, onde o KV-cache rendeu 1,93x em vez das dezenas que a teoria sugeria: **a escala do curso comprime os ganhos**. O mecanismo é o que se aprende; a magnitude vem da escala em que você o aplica.

A.16.3 E3 — Por que threads não bastam

Clientes	Latência mediana	p95	Vazão total
1	0,15 s	0,15 s	268 tok/s
2	0,20 s	0,27 s	301 tok/s
4	0,32 s	0,50 s	320 tok/s
8	0,54 s	0,96 s	334 tok/s

1. Não cresce proporcionalmente — cresce 25% para 8x mais clientes. Se escalasse, os 334 seriam ~2.100.

2. Tirar a trava não resolve, e provavelmente piora.

Sem ela, oito threads chamam o modelo simultaneamente. Mas o PyTorch já usa **todos os núcleos** internamente numa única matmul — então as threads passam a disputar os mesmos núcleos, com troca de contexto e contenção de cache por cima.

O resultado típico é vazão igual ou pior, e latência mais irregular. **Meça antes de opinar** — este exercício existe para isso.

3. Por que agrupar resolve, e threads não.

O Capítulo 12 estabeleceu que o decode é limitado por **memória**: para produzir um token é preciso ler **todos** os pesos do modelo. Lá isso foi medido — gerar 16 sequências em paralelo custou 9,1x menos que 16 gerações separadas.

A diferença é o que se compartilha:

Estratégia	Leituras dos pesos, para 8 clientes
8 threads independentes	8 (cada uma lê tudo)
1 lote de 8	1 (todos usam a mesma leitura)

Threads dividem o *tempo* de acesso ao modelo. O batching divide o *custo* de cada acesso. São coisas diferentes, e só a segunda ataca o gargalo real.

A.16.4 E4 — O tamanho do lote

Oito clientes, 40 tokens cada:

-- lote	Latência mediana	Vazão	Ganho sobre lote 1
1	0,62 s	288 tok/s	—
4	0,47 s	514 tok/s	1,8x
16	0,47 s	673 tok/s	2,3x

1. A vazão mais que dobra de 1 para 16, e a latência **melhora** (0,62 -> 0,47 s). Não há troca aqui: mais lote é melhor nos dois eixos, até o ponto em que satura.

2. --lote 1 deveria reproduzir o servidor sem batching. Não reproduz exatamente — 288 tok/s contra 334 do servidor.py.

O que mais mudou: no servidor_batch.py toda requisição passa por **duas filas** (queue.Queue) e é servida por um thread que não é o do HTTP. Isso custa sincronização e troca de contexto por token. Com lote 1 você paga esse custo sem receber nada em troca.

Vale registrar como leitura de tabela: --lote 1 **não** é o controle limpo que parece. Ele isola o efeito do agrupamento, mas carrega a sobrecarga da arquitetura de filas.

3. O ganho satura, e a resposta é a mesma do E4 do Capítulo 8.

De 4 para 16 o ganho é bem menor que de 1 para 4. O que limita é o mesmo: em algum ponto o lote deixa de ser **limitado por memória** e passa a ser **limitado por cálculo** — a matmul fica grande o bastante para ocupar os núcleos, e dali em diante dobrar o lote dobra o tempo.

A diferença é que aqui há um segundo limite, que o Capítulo 8 não tinha: o **KV-cache cresce com o lote** (Capítulo 12). Em escala real, é a memória que fecha a conta antes do cálculo.

A.16.5 E5 — A espera que custou caro

1. A conta bate.

Com a espera de 8 ms por passo, a latência foi de 0,15 s (servidor simples) para **0,62 s**. A diferença é 0,47 s, contra os $40 \times 8 \text{ ms} = 0,32 \text{ s}$ previstos — a mesma ordem, com o resto vindo da sobrecarga das filas medida no E4.

2. Sim, existe um cenário em que ela ajuda: chegadas espaçadas.

Se as requisições chegam de forma dispersa — uma a cada 5 ms, digamos — esperar um pouco antes de começar junta várias num lote maior. Sem espera, a primeira requisição começa sozinha e as seguintes perdem a carona.

É um compromisso real, e tem nome em sistemas de fila: **esperar aumenta a latência de quem já chegou para aumentar a vazão de todos**. Serviços de verdade expõem isso como um parâmetro (`max_waiting_time`, ou equivalente).

O erro da minha versão não foi ter espera — foi **esperar a cada passo de decode** em vez de só na formação do lote. Esperar uma vez custa 8 ms; esperar 40 vezes custa 320.

3. A regra:

Nunca bloqueie quando há trabalho a fazer. Bloqueie apenas quando estiver ocioso.

Ela aparece em qualquer laço de serviço. Um exemplo fora daqui: um consumidor de fila de mensagens que faz `poll(timeout=100ms)` mesmo tendo mensagens no buffer local — ele adiciona 100 ms a cada ciclo por nada. O padrão correto é drenar o que já está disponível sem bloquear, e só então esperar.

A.16.6 E6 — O p95 conta o que a média esconde

1. Por que diferentes num caso e iguais no outro.

Servidor	Mediana	p95
com trava	0,54 s	0,96 s
com batching	0,42 s	0,42 s

Com a trava existe uma **fila**: as requisições são atendidas em sequência, então a primeira espera pouco e a oitava espera sete gerações. A distribuição tem cauda porque as posições na fila são diferentes.

Com batching **todos avançam no mesmo passo** — cada iteração produz um token para o lote inteiro. Não há posição na fila, então não há cauda.

p95 = mediana é a assinatura de um sistema onde ninguém espera pelos outros. Vale procurar por isso em qualquer serviço que você meça: quando os dois números se separam, há uma fila em algum lugar.

2. Média 200 ms, p99 de 4 s.

Uma em cada cem requisições demora **vinte vezes** a média. Para o usuário isso não é "às vezes um pouco mais lento" — é o sistema travando de vez em quando, sem padrão aparente.

E numa página que faz 10 chamadas, a chance de pelo menos uma cair no p99 é de ~10%. A cauda que parecia rara vira comum quando as requisições se acumulam.

Reportar só a média esconde precisamente o que o usuário lembra.

3. Outra métrica de cauda para um serviço de LLM:

O **p95 do TTFT**, separado do p95 da latência total. São coisas diferentes: um servidor pode ter TTFT estável e latência total com cauda (respostas de tamanhos muito variados), ou o contrário (fila na admissão). Só medindo os dois dá para saber onde está o problema.

Uma segunda: a **taxa de requisições recusadas ou expiradas**. Ela é zero até o sistema saturar, e depois é a métrica mais importante que existe — e não aparece em nenhuma estatística de latência, porque as requisições que falharam não têm latência.

A.16.7 E7 — Backpressure (desafio)

1 e 2. Com fila ilimitada, a latência cresce sem limite: a 64 clientes cada um espera o lote inteiro várias vezes, e as requisições mais antigas provavelmente já expiraram do lado do cliente — o servidor gasta trabalho gerando respostas que ninguém vai ler.

O limite deve sair da sua medição: escolha o N tal que a latência do último da fila ainda seja aceitável para o seu caso.

3. Por que recusar pode ser melhor que atender devagar.

Três razões, e a terceira é a que menos se pensa:

- **Um 429 é acionável.** O cliente sabe imediatamente que deve tentar depois, ou ir para outra réplica. Uma resposta que demora 40 s não informa nada — ele fica esperando.
- **Trabalho desperdiçado prejudica quem está sendo atendido.** Gerar tokens para um cliente que já desistiu consome exatamente os mesmos recursos de quem ainda está lá.
- **Sem limite, a degradação é global.** Com fila ilimitada, sobrecarga não faz alguns falharem — faz **todos** ficarem lentos. Recusar concentra o dano em quem chegou por último, em vez de espalhá-lo por todos.

É a mesma escolha que o E6 revela na tabela: um sistema pode falhar de forma **concentrada e visível** ou **difusa e silenciosa**. A primeira é quase sempre melhor de

operar — e é o mesmo argumento que o Capítulo 12 fez sobre o globo travar sempre em vez de às vezes.

A.17 GABARITO DO CAPÍTULO 17 — MULTIMODAL

Respostas discursivas. Os **números** vêm de `gabarito.py`, que roda com **600 passos** por variante (a apostila usa 1.500) — são vários treinos.

```
bash python solucoes/gabarito.py
```

A.17.1 E1 — Por que quantizar

1. Compressão.

Medida	Pixels	Tokens	Razão
posições	784	49	16x
bits	$784 \times 8 = 6.272$	$49 \times 7 = 343$	18x

Em posições e em bits a compressão é parecida — o que se ganha em quantidade de símbolos, gasta-se um pouco em cada símbolo ser mais "caro" (7 bits contra 8). O ganho real é o primeiro: **o que limita um Transformer é o comprimento da sequência**, não a largura do vocabulário. Reduzir 784 para 49 é a diferença entre caber e não caber.

2. Por que não usar os vetores contínuos direto.

Porque a **saída** do modelo é uma distribuição sobre um conjunto finito. Um Transformer autorregressivo termina numa softmax: ele escolhe entre K opções, amostra uma, e realimenta.

Com vetores contínuos não há o que amostrar — você teria de prever um vetor de 32 números reais, e a loss viraria um erro quadrático em vez de uma cross-entropy. Isso é possível (existem modelos assim) e muda o objetivo de "modelar uma distribuição" para "prever a média", que produz saídas borradas.

A quantização não é só compressão: é o que permite **amostrar**.

3. A analogia com o BPE, e onde ela quebra.

Onde funciona: ambos aprendem um alfabeto de pedaços recorrentes a partir dos dados, ambos reduzem o comprimento da sequência, e ambos são específicos do domínio em que foram treinados (o E3 do Capítulo 11 mediu isso para texto).

Onde quebra: o BPE é **exato** — decodificar desfaz o encode perfeitamente. O VQ-VAE **perde informação**: a reconstrução é aproximada, e o erro de 0,0039 é justamente essa perda. Um é compressão sem perdas de símbolos discretos; o outro é compressão com perdas de um sinal contínuo.

A.17.2 E2 — O straight-through estimator

Variante	Erro de reconstrução	Códigos usados
com straight-through	0,0069	67
sem straight-through	0,0756	1

1. Sem o straight-through, tudo colapsa. O erro fica 11x pior — e, mais revelador, **um único código** é usado para o dataset inteiro.

O decoder ainda aprende: ele recebe gradiente normalmente. O que não aprende é o **encoder** — o argmin corta a cadeia, nada diz a ele que vetores produzir, e ele fica essencialmente na inicialização. Com o encoder parado, todos os vetores caem perto do mesmo lugar, e a quantização vira uma constante.

2. O que a linha faz.

$$z_q = z + (z_q - z).detach()$$

Fase	Valor
forward	$z + z_q - z = z_q$ — o vetor quantizado, exato
backward	$d/dz [z + \text{constante}] = 1$ — passa direto para o encoder

O `.detach()` congela o *nó no grafo*, não o número. É contabilidade de derivadas, não aritmética: o valor calculado é idêntico, e só o caminho do gradiente muda.

3. Por que uma aproximação funciona.

O encoder recebe o gradiente do decoder como se a quantização não existisse. Isso é falso — mas é **quase** verdade, porque z e z_q são próximos por construção.

E são próximos justamente porque o *commitment loss* os mantém assim. As duas peças se sustentam: a aproximação é boa exatamente na medida em que a outra perda faz o seu trabalho.

A.17.3 E3 — As duas perdas

BETA_COMP	Erro	Códigos usados
0,00	0,0755	1
0,25 (<i>o do artigo</i>)	0,0069	67
2,00	0,0046	105

1. Com BETA_COMP = 0 o resultado é catastrófico — e é o mesmo colapso do E2, por outra causa. Sem nada que o prenda, o encoder foge para uma região que o codebook não alcança, e todo vetor cai no mesmo código.

2. E aqui eu estava errado.

Eu tinha escrito que BETA_COMP alto prenderia o encoder e pioraria a representação. Medido, **2,0 é o melhor dos três nos dois eixos**: erro 0,0046 contra 0,0069, e 105 códigos usados contra 67.

O valor 0,25 é o do artigo original do VQ-VAE, e eu o tratei como se fosse um ótimo demonstrado. Nesta configuração — MNIST, codebook de 128, 600 passos — não é. Um *commitment* mais forte mantém o encoder perto do codebook, o que faz mais códigos serem alcançáveis, o que melhora tudo.

A lição não é "use 2,0". É que **um hiperparâmetro herdado de um artigo foi ajustado para outra configuração** — outro dataset, outro tamanho, outro orçamento. Vale medir na sua.

É o mesmo padrão do Capítulo 11, onde a melhor learning rate mudou conforme o orçamento de treino.

3. O papel de cada perda:

Perda	O que ela move
perda_codebook	o codebook , em direção ao encoder
perda_commit	o encoder , em direção ao codebook

São os dois lados do mesmo encontro, e cada uma move um lado.

Removendo a perda_codebook, os vetores do codebook só se moveriam pelo gradiente que chega via decoder — muito mais fraco. Na prática ele ficaria quase congelado na inicialização, e o encoder teria de se contorcer para caber num alfabeto aleatório.

A.17.4 E4 — Códigos mortos

Codebook	Usados	Fração	Erro
32	32	100%	0,0070
64	55	86%	0,0065
128	67	52%	0,0069
256	57	22%	0,0066

1. A fração usada cai, e o erro quase não melhora.

De 32 para 256 códigos — oito vezes mais capacidade nominal — o erro vai de 0,0070 para 0,0066. Praticamente nada. **Um codebook maior não vira mais capacidade; vira mais códigos mortos.**

O mecanismo é de realimentação: para ser escolhido, um código precisa estar perto de algum vetor do encoder; para chegar perto, precisa ser escolhido e receber gradiente. Quem nasce longe nunca entra no jogo.

É o mesmo formato do colapso do E2 e do E3, e vale notar que os três aparecem por caminhos diferentes: sem gradiente no encoder, sem commitment, ou por inicialização azarada. O VQ-VAE tem várias maneiras de degenerar para "poucos códigos fazem tudo".

2 e 3. A correção, e o que ela realmente melhora.

Reinicializar códigos mortos sobre vetores do batch quase sempre **aumenta muito a fração usada e melhora pouco o erro de reconstrução.**

Isso não a torna inútil — torna-a mal avaliada pela estatística de uso. O ganho real aparece depois: um codebook bem aproveitado dá ao Transformer um vocabulário mais informativo, e é lá que se deve medir.

Cuidado com a métrica que melhora sem que o sistema tenha melhorado no que importa. É o mesmo tipo de armadilha do Capítulo 15, onde $\beta = 0,5$ tinha o menor custo em português — porque o modelo não estava aprendendo nada.

A.17.5 E5 — O mesmo Transformer

1. Zero linhas do modelo mudaram.

```
from modelo import GPT
```

É a mesma classe, byte por byte, que escreveu prosa nos capítulos 11 a 15. O que muda é a config:

Config	vocab_size	block_size	n_embd
Capítulo 11 (texto)	1.024	128	192
Capítulo 17 (imagem)	128	49	128

2. A comparação de perplexidade não é justa, e entender por que vale mais que o número.

Modelo	Perplexidade	Vocabulário	Normalizada
Capítulo 11	51,7	1.024	5,0%
Capítulo 17	3,9	128	3,0%

Perplexidade é "entre quantas opções o modelo está efetivamente escolhendo". Comparar 3,9 com 51,7 ignora que os vocabulários diferem 8x.

E mesmo normalizada a comparação é fraca, porque as **tarefas** são diferentes: prever o próximo pedaço de um dígito manuscrito é bem mais restrito que prever a próxima palavra de Machado. Perplexidades só se comparam dentro da mesma tarefa e do mesmo tokenizador.

3. A ordem de varredura é arbitrária para imagem, e cria um problema real.

O modelo gera da esquerda para a direita, de cima para baixo — porque é assim que se achatou o mapa 7×7. Mas imagem não tem ordem natural, e a consequência é concreta: **o modelo desenha a parte de cima sem saber o que virá embaixo, e não pode voltar atrás**. Um traço mal começado no topo condena o dígito inteiro.

Texto tem ordem natural (o tempo); imagem não tem. Alternativas:

Abordagem	Ideia
ordem aleatória	o modelo recebe a posição junto com o token e pode gerar em qualquer ordem
múltiplas escalas	gera uma versão grosseira primeiro, refina depois

difusão

abandona a autorregressão: refina a imagem
inteira de uma vez, várias vezes

A terceira é a que dominou geração de imagem, e a razão é exatamente esta: ela não precisa escolher uma ordem.

A.17.6 E6 — Condicionar pelo dígito

O caminho é o mesmo do Capítulo 14: reserve ids novos acima do vocabulário existente (aqui, 128 a 137 para os dez dígitos), expanda a tabela de embeddings e a camada de saída, e ponha o rótulo como primeiro token da sequência.

3. O que mudaria para condicionar por uma legenda de verdade.

Estruturalmente, **quase nada** — e é esse o ponto. Em vez de um token de condição, você põe a legenda tokenizada pelo BPE do Capítulo 6:

```
<|texto|> um dígito sete inclinado <|imagem|> t1 t2 ... t49
```

O modelo aprende a prever os tokens de imagem condicionados aos de texto, com a mesma loss de sempre.

O que muda de verdade é o **dado**: você precisa de pares imagem-legenda, e é por isso que este capítulo usa o rótulo do MNIST em vez de legendas. Dez rótulos existem; um corpus pareado útil começa em dezenas de gigabytes.

A arquitetura de um "texto -> imagem" não é o obstáculo. O obstáculo são os dados pareados — a mesma conclusão que o Capítulo 11 tirou sobre texto, em escala maior.

A.17.7 E7 — Um modelo, duas modalidades (desafio)

1 e 2. Juntar os vocabulários funciona mecanicamente: ids 0-1023 para texto, 1024-1151 para imagem, e um GPT treinado em sequências dos dois tipos. Espere alguma interferência — a capacidade é dividida entre duas distribuições sem relação.

3. A pergunta que importa: o modelo aprende alguma relação entre as modalidades? Não.

E a razão está no objetivo de treino. O modelo é recompensado por prever o próximo token **dentro de cada sequência**. Se nenhuma sequência contém texto e imagem, nada no gradiente jamais liga um token de texto a um token de imagem — as duas metades do vocabulário nunca coocorrem.

O resultado é **um modelo com duas personalidades**, não um modelo multimodal. Ele modela duas distribuições que por acaso compartilham pesos.

Para que aprendesse, seria preciso o que este capítulo não tem: **sequências que misturem as duas modalidades**. É por isso que dados pareados são o recurso caro em multimodalidade — e por que "juntar os tokenizadores" resolve a parte fácil do problema.

Vale terminar o curso com isto: a arquitetura quase nunca é o obstáculo. Do Capítulo 1 ao 17, o que mudou a qualidade foi sempre **dados, escala e medição honesta** — nunca uma ideia arquitetural esperta.

APENDICE B — FONTES PRIMÁRIAS POR CAPÍTULO

Este apêndice associa cada capítulo aos trabalhos em que as técnicas apresentadas foram originalmente propostas. Serve a dois propósitos: permitir ao leitor ir à fonte quando a exposição didática não bastar, e deixar explícito que nada neste material é invenção do autor — a contribuição está na ordem, na implementação e na medição, não nos métodos.

Cap.	Tema	Fontes primárias
1	Modelo bigrama	SHANNON (1948); JURAFSKY; MARTIN (2024)
2	Diferenciação automática	KARPATHY (2023)
3	<i>N</i> -grama neural	BENGIO <i>et al.</i> (2003)
4	Atenção	BAHDANAU; CHO; BENGIO (2015); VASWANI <i>et al.</i> (2017)
5	<i>Transformer</i>	VASWANI <i>et al.</i> (2017); RADFORD <i>et al.</i> (2018, 2019); BA; KIROS; HINTON (2016); XIONG <i>et al.</i> (2020)
6	Tokenização (BPE)	GAGE (1994); SENNRICH; HADDOW; BIRCH (2016)
7	Otimização	KINGMA; BA (2015); LOSHCHILOV; HUTTER (2019); GLOROT; BENGIO (2010); HE <i>et al.</i> (2015)
8	Dispositivo e desempenho	PASZKE <i>et al.</i> (2019)
9	Precisão reduzida	MICIKEVICIUS <i>et al.</i> (2018)
10	Treinamento distribuído	LI <i>et al.</i> (2020); RAJBHANDARI <i>et al.</i> (2020)
11	Dados e escala	KAPLAN <i>et al.</i> (2020); HOFFMANN <i>et al.</i> (2022); ASSIS (1881, 1891, 1899)

12	Inferência e <i>KV-cache</i>	SHAZEER (2019); POPE <i>et al.</i> (2023)
13	Quantização	JACOB <i>et al.</i> (2018); DETTMERS <i>et al.</i> (2022); FRANTAR <i>et al.</i> (2023)
14	Ajuste fino supervisionado	WEI <i>et al.</i> (2022); OUYANG <i>et al.</i> (2022)
15	Aprendizado por reforço	WILLIAMS (1992); SCHULMAN <i>et al.</i> (2017); STIENNON <i>et al.</i> (2020); OUYANG <i>et al.</i> (2022); DEEPSEEK-AI (2025)
16	Servimento e produção	YU <i>et al.</i> (2022); KWON <i>et al.</i> (2023)
17	Multimodalidade	OORD; VINYALS; KAVUKCUOGLU (2017); BENGIO; LÉONARD; COURVILLE (2013); RAMESH <i>et al.</i> (2021); ESSER; ROMBACH; OMMER (2021); DOSOVITSKIY <i>et al.</i> (2021); HO; JAIN; ABBEEL (2020); LECUN; CORTES; BURGES (1998)

B.1 Sobre o programa do curso

A sequência dos dezessete capítulos segue o programa do **LLM101n**, de Andrej Karpathy e da Eureka Labs (KARPATHY, 2024). A escolha dos tópicos e sua ordem são daquele programa; a redação, a implementação, os experimentos e todos os números apresentados são deste material.

B.2 Sobre os dados

Uso	Origem
Nomes próprios (cap. 1 e 3)	IBGE, Censo Demográfico 2010
<i>Corpus</i> literário (cap. 6, 11-16)	Machado de Assis, via Project Gutenberg, domínio público
Dígitos manuscritos (cap. 17)	MNIST (LECUN; CORTES; BURGES, 1998)

Todas as três fontes são de uso livre. A escolha não foi apenas jurídica: um material didático cujos dados o leitor não pode baixar e reexecutar não é reprodutível, e a reprodutibilidade é uma premissa declarada desta apostila.